



软件源代码缺陷检测 TDengine 检测报告

苏州棱镜七彩信息科技有限公司

2026-04-17 21:26:36



2004

2010

2012

2006

CHART SEGMENT TITLE

CHART SEGMENT TITLE

报告声明

棱镜七彩是苏州棱镜七彩信息科技有限公司的简称，PrismSAST 是棱镜七彩具有自主知识产权的软件源代码安全缺陷检测平台的简称。

本检测报告由棱镜七彩软件源代码安全缺陷检测平台（PrismSAST）采用智能检测引擎进行自动化检测，并对检测结果经过人工复核后产生的检测报告。棱镜七彩保留对检测报告中各款项的解读权利，但并不意味着对检测结果中的缺陷或安全漏洞可能引起的任何直接经济损失或任何间接损失承担责任。

本检测报告的最终解释权归属棱镜七彩。

16 C/C++缺陷类型-返回值问题

16.1 分项汇总表

序号	C/C++缺陷类型-返回值问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4729	有返回值的函数必须通过返回语句返回	58	严重
2	WK4732	函数返回值的类型必须与定义一致	252	严重
3	WK4733	具有返回值的函数,其返回值如果不被使用,调用是应有void说明	1690	严重

16.2 缺陷/安全漏洞详解

16.2.1 严重类

16.2.1.1 WK4729 有返回值的函数必须通过返回语句返回

缺陷数量： 58 个		
缺陷描述： 有返回值的函数必须通过返回语句返回。		
修复意见： 无		
1	缺陷发生位置： /source/dnode/vnode/src/bse/bseTable.c	行号： 1533
步骤描述:		

	有返回值的函数必须通过返回语句返回	
	代码片段: 1531: } 1532: 1533: int32_t tableReaderIterNext(STableReaderIter *pIter, uint8_t **pValue, int32_t *len) { 1534: int32_t code = 0; 1535: int32_t lino = 0;	
	污染路径: 无	
2	缺陷发生位置:	行号:
	/source/libs/index/src/index.c	574
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 572: } 573: 574: static int32_t idxMayMergeTempToFinalRsIt(SArray* result, TFileValue* tfv, SIdxTRslt* tr) { 575: int32_t code = 0; 576: int32_t sz = taosArrayGetSize(result);	
3	污染路径: 无	
	缺陷发生位置:	行号:
	/source/util/src/tcompare.c	198
	步骤描述: 有返回值的函数必须通过返回语句返回	
4	代码片段: 196: int32_t compareDoubleValDesc(const void *pLeft, const void *pRight) { return compareDoubleVal(pRight, pLeft); } 197: 198: int32_t compareLenPrefixedStr(const void *pLeft, const void *pRight) { 199: int32_t len1 = varDataLen(pLeft); 200: int32_t len2 = varDataLen(pRight);	
	污染路径: 无	
	缺陷发生位置:	行号:
	/source/os/src/osString.c	204
	步骤描述:	

	有返回值的函数必须通过返回语句返回	
	代码片段: 202: } 203: } 204: int32_t taosStr2int16(const char *str, int16_t *val) { 205: OS_PARAM_CHECK(str); 206: OS_PARAM_CHECK(val);	
	污染路径: 无	
5	缺陷发生位置: /source/os/src/osSemaphore.c	行号: 413
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 411: } 412: 413: int32_t tsem_post(tsem_t* psem) { 414: OS_PARAM_CHECK(psem); 415: if (sem_post(psem) == 0) {	
	污染路径: 无	
6	缺陷发生位置: /source/dnode/mnode/impl/src/mndProfile.c	行号: 182
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 180: 181: 182: static SConnObj *mndCreateConn(SMnode *pMnode, const char *user, const char* tokenName, int8_t connType, SIpAddr *pAddr, 183: int32_t pid, const char *app, int64_t startTime, const char *sVer) { 184: SProfileMgmt *pMgmt = &pMnode->profileMgmt;	
	污染路径: 无	
7	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 606
	步骤描述: 有返回值的函数必须通过返回语句返回	

	代码片段: 604: static bool isCharStart(char c) { return strcasecmp(tsCharset, "UTF-8") == 0 ? ((c & 0xC0) != 0x80) : true; } 605: 606: static int32_t trimHelper(char *orgStr, char *remStr, int32_t orgLen, int32_t remLen, bool trimLeft, bool isNchar) { 607: if (trimLeft) { 608: int32_t pos = 0;	
	污染路径: 无	
8	缺陷发生位置: /source/libs/new-stream/src/dataSinkMgr.c	行号: 1025
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1023: } 1024: 1025: int32_t checkAndMoveMemCache(bool forWrite) { 1026: int32_t code = TSDB_CODE_SUCCESS; 1027: if (!g_slidigGrpMemList.enabled && g_pDataSinkManager.usedMemSize > tsStreamBufferSizeBytes - g_pDataSinkManager.memAlterSize) {	
	污染路径: 无	
9	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReadUtil.c	行号: 76
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 74: } 75: 76: int32_t uidComparFunc(const void* p1, const void* p2) { 77: uint64_t pu1 = *(const uint64_t*)p1; 78: uint64_t pu2 = *(const uint64_t*)p2;	
	污染路径: 无	
10	缺陷发生位置: /source/client/src/clientTmq.c	行号: 2839
	步骤描述:	

	有返回值的函数必须通过返回语句返回	
	代码片段: 2837: } 2838: 2839: const char* tmq_err2str(int32_t err) { 2840: if (err == 0) { 2841: return "success";	
	污染路径: 无	
11	缺陷发生位置: /source/dnode/mnode/impl/src/mndDb.c	行号: 1222
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1220: 1221: #ifndef TD_ENTERPRISE 1222: int32_t mndCheckDbDnodeList(SMnode *pMnode, char *db, char *dnodeListStr, SArray *dnodeList) { 1223: if (dnodeListStr[0] != 0) { 1224: terrno = TSDB_CODE_OPS_NOT_SUPPORT;	
	污染路径: 无	
12	缺陷发生位置: /source/util/src/tversion.c	行号: 73
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 71: } 72: 73: int32_t taosCheckVersionCompatible(int32_t clientVer, int32_t serverVer, int32_t comparedSegments) { 74: switch (comparedSegments) { 75: case 4:	
	污染路径: 无	
13	缺陷发生位置: /source/client/src/clientSmlJson.c	行号: 384
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段:	

	<pre> 382: } 383: 384: static int64_t smlParseTSFromJSON(SSmlHandle *info, cJSON *timestamp) { 385: // Timestamp must be the first KV to parse 386: int32_t toPrecision = info->currSTableMeta ? info->currSTableMeta->tableInfo.precision : TSDB_TIME_PRECISION_NANO; </pre>	
	污染路径: 无	
14	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 1510
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: <pre> 1508: } 1509: 1510: int32_t bpSetTableNameTags(BindData *data, int32_t tblIdx, char *tblName, TAOS_STMT *stmt) { 1511: int32_t code = 0; 1512: if (gCurCase->bindTagNum > 0) { </pre>	
	污染路径: 无	
15	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 1663
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: <pre> 1661: } 1662: 1663: int32_t bpSetTableNameTags(BindData *data, int32_t tblIdx, char *tblName, TAOS_STMT2 *stmt) { 1664: int32_t code = 0; 1665: if (gCurCase->bindTagNum > 0) { </pre>	
	污染路径: 无	
16	缺陷发生位置: /source/dnode/vnode/src/tq/tqSink.c	行号: 327
	步骤描述: 有返回值的函数必须通过返回语句返回	

	代码片段: 325: } 326: 327: int32_t tsAscendingSortFn(const void* p1, const void* p2) { 328: SRow* pRow1 = *(SRow**)p1; 329: SRow* pRow2 = *(SRow**)p2;	
	污染路径: 无	
17	缺陷发生位置: /source/os/src/osDir.c	行号: 386
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 384: } 385: 386: int32_t taosRealPath(char *dirname, char *realPath, int32_t maxlen) { 387: OS_PARAM_CHECK(dirname); 388:	
	污染路径: 无	
18	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqSubs.c	行号: 186
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 184: } 185: 186: static int ttqSubAdd_shared(struct tmqtt *context, const char *sub, uint8_t qos, uint32_t identifier, int options, 187: struct tmqtt__subhier *subhier, const char *sharename) { 188: struct tmqtt__subleaf *newleaf;	
	污染路径: 无	
19	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmTransport.c	行号: 122
	步骤描述: 有返回值的函数必须通过返回语句返回	

	代码片段: 120: return; 121: } 122: static int32_t dmIsForbiddenIp(int8_t forbidden, char *user, SIpAddr *clientIp) { 123: if (IP_FORBIDDEN_CHECK_WHITE_LIST(forbidden)) { 124: dError("User:%s host:%s not in ip white list or in block white list", user, IP_ADDR_STR(clientIp)); 	
	污染路径: 无	
20	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqSubs.c	行号: 116
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 114: } 115: 116: static int subs__process(struct tmqtt_subhier *hier, const char *source_id, const char *topic, uint8_t qos, int retain, 117: struct tmqtt_msg_store *stored) { 118: int rc = 0; 	
	污染路径: 无	
21	缺陷发生位置: /source/dnode/mnode/impl/src/mndVgroup.c	行号: 3323
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 3321: } 3322: 3323: int32_t mndBuildRaftAlterVgroupAction(SMnode *pMnode, STrans *pTrans, SDbObj *pOldDb, SDbObj *pNewDb, SVgObj *pVgroup, 3324: SArray *pArray) { 3325: int32_t code = 0; 	
	污染路径: 无	
22	缺陷发生位置: /source/libs/executor/src/scanoperator.c	行号: 2784
	步骤描述:	

	有返回值的函数必须通过返回语句返回	
	代码片段: 2782: } 2783: 2784: int32_t colldComparFn(const void* param1, const void* param2) { 2785: int32_t p1 = *(int32_t*)param1; 2786: int32_t p2 = *(int32_t*)param2;	
	污染路径: 无	
23	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmTransport.c	行号: 383
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 381: } 382: 383: static inline int32_t dmSendReq(const SEpSet *pEpSet, SRpcMsg *pMsg) { 384: int32_t code = 0; 385: SDnode *pDnode = dmInstance();	
	污染路径: 无	
24	缺陷发生位置: /source/dnode/mnode/impl/src/mndSubscribe.c	行号: 1420
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1418: } 1419: 1420: static int32_t mndProcessDropCgroupReq(SRpcMsg *pMsg) { 1421: if (pMsg == NULL) { 1422: return TSDB_CODE_INVALID_PARA;	
	污染路径: 无	
25	缺陷发生位置: /source/dnode/mnode/impl/src/mndSma.c	行号: 628
	步骤描述: 有返回值的函数必须通过返回语句返回	

	代码片段: 626: } 627: 628: static int32_t mndGetSma(SMnode *pMnode, SUserIndexReq *indexReq, SUserIndexRsp *rsp, bool *exist) { 629: int32_t code = -1; 630: SSmaObj *pSma = NULL;	
	污染路径: 无	
26	缺陷发生位置: /tools/src/pub.c	行号: 81
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 79: 80: // get comn mode, if invalid exit app 81: int8_t getConnMode(char *arg) { 82: // compare 83: if (strcasecmp(arg, STR_NATIVE) == 0 strcasecmp(arg, "0") == 0) {	
	污染路径: 无	
27	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmWorker.c	行号: 216
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 214: } 215: 216: static bool vmDataSpaceSufficient(SVnodeObj *pVnode) { 217: STfs *pTfs = pVnode->pImpl->pTfs; 218: if (pTfs) {	
	污染路径: 无	
28	缺陷发生位置: /source/libs/planner/src/planOptimizer.c	行号: 3865
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段:	

	3863: } 3864: 3865: static SNodeList* partTagsGetFuncs(SLogicNode* pNode) { 3866: if (QUERY_NODE_LOGIC_PLAN_PARTITION == nodeType(pNode)) { 3867: return NULL;	
	污染路径: 无	
29	缺陷发生位置: /source/util/src/tcompare.c	行号: 267
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 265: } 266: 267: int32_t compareLenBinaryVal(const void *pLeft, const void *pRight) { 268: int32_t len1 = varDataLen(pLeft); 269: int32_t len2 = varDataLen(pRight);	
	污染路径: 无	
30	缺陷发生位置: /source/dnode/mnode/impl/src/mndStream.c	行号: 977
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 975: 976: 977: static int32_t mndProcessStartStreamReq(SRpcMsg *pReq) { 978: SMnode *pMnode = pReq->info.node; 979: SStreamObj *pStream = NULL;	
	污染路径: 无	
31	缺陷发生位置: /source/dnode/mnode/impl/src/mndRole.c	行号: 160
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段:	

	cannot be determined 419: * - ZSTD_CONTENTSIZE_ERROR if an error occurred (e.g. invalid magic number, srcSize too small) */ 420: unsigned long long ZSTD_getFrameContentSize(const void *src, size_t srcSize) 421: { 422: #if defined(ZSTD_LEGACY_SUPPORT) && (ZSTD_LEGACY_SUPPORT >= 1)	
	污染路径: 无	
35	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeOpen.c	行号: 50
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 48: } 49: 50: static int32_t vnodeMkDir(STfs *pTfs, const char *path) { 51: if (pTfs) { 52: return tfsMkdirRecur(pTfs, path);	
	污染路径: 无	
36	缺陷发生位置: /source/libs/sync/src/syncRaftLog.c	行号: 203
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 201: // if not log, return 0 202: // if error, return SYNC_TERM_INVALID 203: SyncTerm raftLogLastTerm(struct SSyncLogStore* pLogStore) { 204: SSyncLogStoreData* pData = pLogStore->data; 205: SWal* pWal = pData->pWal;	
	污染路径: 无	
37	缺陷发生位置: /source/libs/executor/src/executil.c	行号: 3722
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段:	

	3720: } 3721: 3722: static int32_t orderbyGroupIdComparFn(const void* p1, const void* p2) { 3723: STableKeyInfo* pInfo1 = (STableKeyInfo*)p1; 3724: STableKeyInfo* pInfo2 = (STableKeyInfo*)p2;	
	污染路径: 无	
38	缺陷发生位置: /source/libs/executor/src/executil.c	行号: 235
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 233: } 234: 235: int32_t resultrowComparAsc(const void* p1, const void* p2) { 236: SResKeyPos* pp1 = *(SResKeyPos**)p1; 237: SResKeyPos* pp2 = *(SResKeyPos**)p2;	
	污染路径: 无	
39	缺陷发生位置: /source/libs/index/src/indexTfile.c	行号: 1012
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1010: return -1; 1011: } 1012: static int tfileWriteFooter(TFileWriter* write) { 1013: char buf[sizeof(FILE_MAGIC_NUMBER) + 1] = {0}; 1014: void* pBuf = (void*)buf;	
	污染路径: 无	
40	缺陷发生位置: /source/common/src/tvariant.c	行号: 494
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 492: } 493:	

	494: int32_t taosVariantCompare(const SVariant *p1, const SVariant *p2) { 495: if (p1->nType == TSDB_DATA_TYPE_NULL && p2->nType == TSDB_DATA_TYPE_NULL) { 496: return 0; 	
	污染路径: 无	
41	缺陷发生位置: /source/dnode/mnode/impl/src/mndStream.c	行号: 880
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 878: } 879: 880: static int32_t mndProcessStopStreamReq(SRpcMsg *pReq) { 881: SMnode *pMnode = pReq->info.node; 882: SStreamObj *pStream = NULL; 	
	污染路径: 无	
42	缺陷发生位置: /source/dnode/mnode/impl/src/mndXnode.c	行号: 3974
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 3972: } 3973: 3974: static size_t taosCurlWriteData(char *pCont, size_t contLen, size_t nmemb, void *userdata) { 3975: SCurlResp *pRsp = userdata; 3976: if (contLen == 0 nmemb == 0 pCont == NULL) { 	
	污染路径: 无	
43	缺陷发生位置: /source/os/src/osSemaphore.c	行号: 331
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 329: } 330:	

	331: int32_t tsem_init(tsem_t* psem, int flags, unsigned int count) { 332: if (sem_init(psem, flags, count) == 0) { 333: return 0;	
	污染路径: 无	
44	缺陷发生位置: /source/os/src/osString.c	行号: 172
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 170: } 171: 172: int32_t taosStr2int64(const char *str, int64_t *val) { 173: if (str == NULL val == NULL) { 174: return TSDB_CODE_INVALID_PARA;	
	污染路径: 无	
45	缺陷发生位置: /source/os/src/osSemaphore.c	行号: 422
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 420: } 421: 422: int32_t tsem_destroy(tsem_t* sem) { 423: OS_PARAM_CHECK(sem); 424: if (sem_destroy(sem) == 0) {	
	污染路径: 无	
46	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCacheRead.c	行号: 421
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 419: } 420: 421: static int32_t tsdbCacheQueryReseek(void* pQHandle) { 422: int32_t code = 0; 423: SCacheRowsReader* pReader = pQHandle;	

	污染路径: 无	
47	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 1194
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1192: } 1193: 1194: int32_t taosGetDiskSize(char *dataDir, SDiskSize *diskSize) { 1195: OS_PARAM_CHECK(dataDir); 1196: OS_PARAM_CHECK(diskSize);	
	污染路径: 无	
48	缺陷发生位置: /source/dnode/mnode/impl/src/mndTrans.c	行号: 1908
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1906: } 1907: 1908: static int32_t mndTransExecuteActions(SMnode *pMnode, STrans *pTrans, SArray *pArray, bool topHalf, bool notSend) { 1909: int32_t numOfActions = taosArrayGetSize(pArray); 1910: int32_t code = 0;	
	污染路径: 无	
49	缺陷发生位置: /source/libs/monitorfw/src/taos_linked_list.c	行号: 223
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 221: } 222: 223: taos_linked_list_compare_t taos_linked_list_compare(taos_linked_list_t *self, void *item_a, void *item_b) { 224: TAOS_TEST_PARA(self != NULL);	

	225: if (self->compare_fn) {	
	污染路径: 无	
50	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 317
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 315: } SLastColV0; 316: 317: static int32_t tsdbCacheDeserializeV0(char const *value, SLastCol *pLastCol) { 318: SLastColV0 *pLastColV0 = (SLastColV0 *)value; 319:	
	污染路径: 无	
51	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqSubs.c	行号: 263
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 261: } 262: 263: static int ttqSubAdd_normal(struct tmqtt *context, const char *sub, uint8_t qos, uint32_t identifier, int options, 264: struct tmqtt__subhier *subhier) { 265: struct tmqtt__subleaf *newleaf = NULL;	
	污染路径: 无	
52	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbUtil.c	行号: 742
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 740: 741: // SRowMerger =====	
	742: int32_t tsdbRowMergerAdd(SRowMerger *pMerger, TSDBROW *pRow, STSchema *pTSchema) {	

	743: int32_t code = 0; 744: TSDBKEY key = TSDBROW_KEY(pRow);	
	污染路径: 无	
53	缺陷发生位置: /source/util/src/thashutil.c	行号: 173
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 171: 172: 173: TAOS_NO_SANITIZE_ADDR_UB 174: uint32_t taosFloatHash(const char *key, uint32_t UNUSED_PARAM(len)) { 175: float f = GET_FLOAT_VAL(key);	
	污染路径: expanded from macro 'TAOS_NO_SANITIZE_ADDR_UB'	
54	缺陷发生位置: /source/libs/parser/src/parAstCreator.c	行号: 1279
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1277: } 1278: 1279: static int32_t addParamToLogicConditionNode(SLogicConditionNode* pCond, SNode* pParam) { 1280: if (QUERY_NODE_LOGIC_CONDITION == nodeType(pParam) && pCond->condType == ((SLogicConditionNode*)pParam)->condType && 1281: ((SLogicConditionNode*)pParam)->condType != LOGIC_COND_TYPE_NOT) {	
	污染路径: 无	
55	缺陷发生位置: /source/os/src/osDir.c	行号: 139
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 137: }	

	138: 139: int32_t taosMkDir(const char *dirname) { 140: if (taosDirExist(dirname)) return 0; 141: #ifdef WINDOWS	
	污染路径: 无	
56	缺陷发生位置: /source/dnode/mnode/sdb/src/sdbRaw.c	行号: 19
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 17: #include "sdb.h" 18: 19: int32_t sdbGetIdFromRaw(SSdb *pSdb, SSdbRaw *pRaw) { 20: EKeyType keytype = pSdb->keyTypes[pRaw->type]; 21: if (keytype == SDB_KEY_INT32) {	
	污染路径: 无	
57	缺陷发生位置: /source/os/src/osString.c	行号: 234
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 232: } 233: 234: int32_t taosStr2UInt64(const char *str, uint64_t *val) { 235: if (str == NULL val == NULL) { 236: return TSDB_CODE_INVALID_PARA;	
	污染路径: 无	
58	缺陷发生位置: /source/client/src/clientTmq.c	行号: 2889
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 2887: } 2888: 2889: const char* tmq_get_db_name(TAOS_RES* res) { 2890: if (res == NULL) { 2891: return NULL;	

	污染路径: 无

16.2.1.2 WK4732 函数返回值的类型必须与定义一致

缺陷数量： 252 个		
缺陷描述： 函数返回值的类型必须与定义一致。		
修复意见： 无		
1	缺陷发生位置： /contrib/TSZ/zstd/compress/huf_compress.c	行号： 106
	步骤描述： 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段： 104: { unsigned const maxCount = HIST_count_simple(count, &maxSymbolValue, weightTable, wtSize); /* never fails */ 105: if (maxCount == wtSize) return 1; /* only a single symbol in src : rle */ 106: if (maxCount == 1) return 0; /* each symbol present maximum once => not compressible */ 107: } 108:	
	污染路径： 无	
2	缺陷发生位置： /contrib/TSZ/zstd/legacy/zstd_v01.c	行号： 475
	步骤描述： 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'short'.	
	代码片段： 473: static short FSE_abs(short a) 474: { 475: return a<0? -a : a; 476: } 477:	
	污染路径：	

	无	
3	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 93
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'const char *'.	
	代码片段: <pre> 91: unsigned ZDICT_isError(size_t errorCode) { return ERR_isError(errorCode); } 92: 93: const char* ZDICT_getErrorName(size_t errorCode) { return ERR_getErrorName(errorCode); } 94: 95: unsigned ZDICT_getDictID(const void* dictBuffer, size_t dictSize) </pre>	
	污染路径: 无	
4	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 78
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 76: for (char c = *cmd++; c != 0; c = *cmd++) { 77: if (c != ' ' && c != '\t' && c != ';') { 78: return false; 79: } 80: } </pre>	
	污染路径: expanded from macro 'false'	
5	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 368
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'unsigned long long'，定义类型为'U64'.	
	代码片段: <pre> 366: return __builtin_bswap64(in); 367: #else 368: return ((in << 56) & 0xff00000000000000ULL) 369: ((in << 40) & 0x00ff000000000000ULL) 370: ((in << 24) & 0x0000ff0000000000ULL) </pre>	
	污染路径:	

	无	
6	缺陷发生位置: /tools/taos-tools/src/toolsString.c	行号: 75
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 73: for (int i = 0; i < len; i++) { 74: if (!isdigit(input[i])) 75: return false; 76: } 77: </pre>	
	污染路径: expanded from macro 'false'	
7	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 2360
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'uint32_t'.	
	代码片段: <pre> 2358: errorPrint("taos_stmt_add_batch() failed! reason: %s\n", 2359: taos_stmt_errstr(pThreadInfo->conn- >stmt)); 2360: return 0; 2361: } 2362: if(delay3) { </pre>	
	污染路径: 无	
8	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 660
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'.	
	代码片段: <pre> 658: MEM_STATIC unsigned BITv07_endOfDStream(const BITv07_DStream_t* DStream) 659: { 660: return ((DStream->ptr == DStream->start) && (DStream->bitsConsumed == sizeof(DStream- >bitContainer)*8)); 661: } 662: </pre>	

	污染路径: 无	
9	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 127
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 125: #endif 126: 127: MEM_STATIC unsigned MEM_32bits(void) { return sizeof(void*)==4; } 128: MEM_STATIC unsigned MEM_64bits(void) { return sizeof(void*)==8; } 129:	
	污染路径: 无	
10	缺陷发生位置: /contrib/TSZ/zstd/compress/zstd_compress.c	行号: 133
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 131: #else 132: (void) cctx; 133: return 0; 134: #endif 135: }	
	污染路径: 无	
11	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 209
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned long long', 定义类型为'U64'.	
	代码片段: 207: return __builtin_bswap64(in); 208: #else 209: return ((in << 56) & 0xff00000000000000ULL) 210: ((in << 40) & 0x00ff000000000000ULL) 211: ((in << 24) & 0x0000ff0000000000ULL)	
	污染路径:	

	无	
12	缺陷发生位置: /source/util/src/tenv.c	行号: 50
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'unsigned long'，定义类型为'int'.	
	代码片段: 48: 49: *p = '\0'; 50: return strlen(cfgNameStr); 51: } 52:	
	污染路径: 无	
13	缺陷发生位置: /contrib/TSZ/zstd/compress/huf_compress.c	行号: 101
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: 99: 100: /* init conditions */ 101: if (wtSize <= 1) return 0; /* Not compressible */ 102: 103: /* Scan input and build symbol stats */	
	污染路径: 无	
14	缺陷发生位置: /contrib/TSZ/zstd/compress/fse_compress.c	行号: 444
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: 442: } } } 443: 444: return 0; 445: } 446:	
	污染路径: 无	
15	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 492
	步骤描述:	

	函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'.	
	代码片段: 490: MEM_STATIC unsigned BIT_endOfDStream(const BIT_DStream_t* DStream) 491: { 492: return ((DStream->ptr == DStream->start) && (DStream->bitsConsumed == sizeof(DStream-> bitContainer)*8)); 493: } 494:	
	污染路径: 无	
16	缺陷发生位置: /source/common/src/dmRepair.c	行号: 11
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 9: #include "dmRepair.h" 10: 11: TD_WEAK bool dmRepairFlowEnabled() { return false; } 12: TD_WEAK bool dmRepairNodeTypelsVnode() { return false; } 13: TD_WEAK bool dmRepairModelsForce() { return false; }	
17	污染路径: expanded from macro 'false'	
	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 351
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'.	
	代码片段: 349: return (unsigned) r; 350: # elif defined(__GNUC__) && (__GNUC__ >= 3) /* Use GCC Intrinsic */ 351: return 31 - __builtin_clz (val); 352: # else /* Software version */ 353: static const unsigned DeBruijnClz[32] = { 0, 9, 1, 10, 13, 21, 2, 29, 11, 14, 16, 18, 22, 25, 3, 30, 8, 12, 20, 28, 15, 17, 24, 7, 19, 27, 23, 6, 26, 5, 4, 31 };	
	污染路径: 无	

18	缺陷发生位置: /source/os/src/osSocket.c	行号: 202
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int8_t'.	
	代码片段: 200: if (-1 == fd) { // exception 201: errno = TAOS_SYSTEM_ERROR(ERRNO); 202: return 0; 203: } 204:	
	污染路径: 无	
19	缺陷发生位置: /contrib/TSZ/zstd/deprecated/zbuff_decompress.c	行号: 74
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 72: * Tool functions 73: *****/ 74: size_t ZBUFF_recommendedDInSize(void) { return ZSTD_DStreamInSize(); } 75: size_t ZBUFF_recommendedDOutSize(void) { return ZSTD_DStreamOutSize(); }	
	污染路径: 无	
20	缺陷发生位置: /contrib/TSZ/zstd/compress/fse_compress.c	行号: 180
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 178: #endif 179: 180: return 0; 181: } 182:	
	污染路径: 无	
21	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 164
	步骤描述:	

	函数返回值类型必须与定义一致，实际返回为'long'，定义类型为'size_t'.	
	代码片段: 162: 163: ip += (bitCount+7)>>3; 164: return ip-istart; 165: } 166:	
	污染路径: 无	
22	缺陷发生位置: /tools/taos-tools/src/benchCsv.c	行号: 86
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'long'.	
	代码片段: 84: 85: static long csvValidateParamTsInterval(const char* csv_ts_interval) { 86: if (!csv_ts_interval *csv_ts_interval == '\0') return -1; 87: 88: char* endptr;	
	污染路径: 无	
23	缺陷发生位置: /tools/taos-tools/src/benchCsv.c	行号: 100
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'long'.	
	代码片段: 98: if (*endptr == '\0' 99: *(endptr + 1) != '\0') { 100: return -1; 101: } 102:	
	污染路径: 无	
24	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 652
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'.	
	代码片段: 650: return static_cast<unsigned>(i);	

	<pre> 651: } 652: return 0; 653: } 654: </pre>	
	污染路径: 无	
25	缺陷发生位置: /source/dnode/mnode/impl/src/mndXnode.c	行号: 3980
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'。	
	代码片段: <pre> 3978: pRsp->data = NULL; 3979: uError("curl response is received, len:%" PRId64, pRsp->dataLen); 3980: return 0; 3981: } 3982: </pre>	
	污染路径: 无	
26	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 533
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'。	
	代码片段: <pre> 531: return (unsigned) r; 532: # elif defined(__GNUC__) && (__GNUC__ >= 3) /* Use GCC Intrinsic */ 533: return 31 - __builtin_clz (val); 534: # else /* Software version */ 535: static const unsigned DeBruijnClz[32] = { 0, 9, 1, 10, 13, 21, 2, 29, 11, 14, 16, 18, 22, 25, 3, 30, 8, 12, 20, 28, 15, 17, 24, 7, 19, 27, 23, 6, 26, 5, 4, 31 }; </pre>	
	污染路径: 无	
27	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1902
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'long'，定义类型为'int'。	
	代码片段: <pre> 1900: // create topic <topic_name> as select </pre>	

	1901: if (strncasecmp(p, "create topic ", 13) == 0) { 1902: return as_pos_end - p; 1903: } 1904:	
	污染路径: 无	
28	缺陷发生位置: /source/libs/monitorfw/src/taos_metric_formatter.c	行号: 168
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'long'，定义类型为'int'.	
	代码片段: 166: memcpy(vgroupid, start, len); 167: vgroupid[len] = '\0'; 168: return strtol(vgroupid, NULL, 10); 169: } 170: }	
	污染路径: 无	
29	缺陷发生位置: /source/util/src/mpDirect.c	行号: 78
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'void *'.	
	代码片段: 76: MP_CHECK_QUOTA(pPool, pJob, oSize); 77: 78: return taosStrdupi(ptr); 79: } 80:	
	污染路径: 无	
30	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 576
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'.	
	代码片段: 574: static const char* ERR_strings[] = { ERROR_LIST(ERROR_GENERATE_STRING) }; 575: 576: ERR_STATIC unsigned ERR_isError(size_t code) { return (code > ERROR(maxCode)); }	

	577: 578: ERR_STATIC const char* ERR_getErrorName(size_t code)	
	污染路径: 无	
31	缺陷发生位置: /contrib/TSZ/zstd/compress/hist.c	行号: 58
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 56: 57: memset(count, 0, (maxSymbolValue+1) * sizeof(*count)); 58: if (srcSize==0) { *maxSymbolValuePtr = 0; return 0; } 59: 60: while (ip<end) {	
	污染路径: 无	
32	缺陷发生位置: /source/libs/scalar/src/scifunc.c	行号: 476
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'uint8_t'.	
	代码片段: 474: uint8_t getCharLen(const unsigned char *str) { 475: if (strcasecmp(tsCharset, "UTF-8") != 0) { 476: return 1; 477: } 478: if ((str[0] & 0x80) == 0) {	
	污染路径: 无	
33	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 471
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 469: #ifndef FSE_COMMONDEFS_ONLY 470: 471: static unsigned FSE_isError(size_t code) { return (code > (size_t)(-FSE_ERROR_maxCode)); } 472: 473: static short FSE_abs(short a)	

	污染路径: 无	
34	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 8923
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 8921: break; 8922: } 8923: return 1; 8924: } 8925:	
	污染路径: 无	
35	缺陷发生位置: /contrib/TSZ/zstd/compress/huf_compress.c	行号: 120
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 118: CHECK_F(FSE_buildCTable_wksp(CTable, norm, maxSymbolValue, tableLog, scratchBuffer, sizeof(scratchBuffer))); 119: { CHECK_V_F(cSize, FSE_compress_usingCTable(op, oend - op, weightTable, wtSize, CTable)); 120: if (cSize == 0) return 0; /* not enough space for compressed data */ 121: op += cSize; 122: }	
	污染路径: 无	
36	缺陷发生位置: /source/libs/scalar/src/scifunc.c	行号: 479
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'uint8_t'.	
	代码片段: 477: } 478: if ((str[0] & 0x80) == 0) { 479: return 1; 480: } else if ((str[0] & 0xE0) == 0xC0) { 481: return 2;	

	污染路径: 无	
37	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 290
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'.	
	代码片段: 288: 289: MEM_STATIC unsigned MEM_32bits(void) { return sizeof(size_t)==4; } 290: MEM_STATIC unsigned MEM_64bits(void) { return sizeof(size_t)==8; } 291: 292: MEM_STATIC unsigned MEM_isLittleEndian(void)	
	污染路径: 无	
38	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 265
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'long'，定义类型为'int'.	
	代码片段: 263: l++; 264: if (name == NULL) 265: return l - long_options; 266: else 267: return -1;	
	污染路径: 无	
39	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 1737
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'long'，定义类型为'int'.	
	代码片段: 1735: } 1736: 1737: return orig - SA; 1738: } 1739:	

	污染路径: 无	
40	缺陷发生位置: /source/libs/executor/test/joinTests.cpp	行号: 667
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int64_t*', 定义类型为'void*'.	
	代码片段: 665: return &INT_FILTER_VALUE; 666: case TSDB_DATA_TYPE_BIGINT: 667: return &BIGINT_FILTER_VALUE; 668: default: 669: return NULL;	
	污染路径: 无	
41	缺陷发生位置: /tools/taos-tools/src/dumpUtil.c	行号: 87
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 85: } 86: 87: return false; 88: } 89:	
	污染路径: expanded from macro 'false'	
42	缺陷发生位置: /source/client/src/clientSmlJson.c	行号: 417
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 415: return TSDB_CODE_TSC_VALUE_OUT_OF_RANGE; 416: } else { 417: return convertTimePrecision(timeDouble, fromPrecision, toPrecision); 418: } 419: } else if (cJSON_IsObject(timestamp)) {	
	污染路径: 无	
43	缺陷发生位置:	行号:

	/source/os/src/osTime.c	350
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'char *'.	
	代码片段: 348: return ((char *)bp); 349: #else 350: return strtptime(buf, fmt, tm); 351: #endif 352: }	
	污染路径: 无	
44	缺陷发生位置: /source/dnode/mnode/impl/src/mndDef.c	行号: 106
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'void *'.	
	代码片段: 104: } 105: 106: static void *topicNameDup(void *p) { return taosStrdup((char *)p); } 107: 108: int32_t tNewSMqConsumerObj(int64_t consumerId, char *cgroup, int8_t updateType, char *topic, SCMSubscribeReq *subscribe,	
	污染路径: 无	
45	缺陷发生位置: /contrib/TSZ/zstd/compress/zstd_compress.c	行号: 123
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 121: ZSTD_freeCCtxContent(cctx); 122: ZSTD_free(cctx, cctx->customMem); 123: return 0; 124: } 125:	
	污染路径: 无	
46	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 98

	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 96: { 97: if (dictSize < 8) return 0; 98: if (MEM_readLE32(dictBuffer) != ZSTD_MAGIC_DICTIONARY) return 0; 99: return MEM_readLE32((const char*)dictBuffer + 4); 100: }	
	污染路径: 无	
47	缺陷发生位置: /source/libs/executor/test/joinTests.cpp	行号: 665
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int32_t *', 定义类型为'void *'.	
	代码片段: 663: return &TIMESTAMP_FILTER_VALUE; 664: case TSDB_DATA_TYPE_INT: 665: return &INT_FILTER_VALUE; 666: case TSDB_DATA_TYPE_BIGINT: 667: return &BIGINT_FILTER_VALUE;	
	污染路径: 无	
48	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqPacketDataTypes.c	行号: 198
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 196: return 4; 197: } else { 198: return 5; 199: } 200: }	
	污染路径: 无	
49	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 6079
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'uint8_t'.	
	代码片段:	

	6077: } 6078: 6079: static uint8_t calcPrecision(uint8_t lp, uint8_t rp) { return (lp > rp ? rp : lp); } 6080: 6081: static uint8_t calcJoinTablePrecision(SJoinTableNode* pJoinTable) {	
	污染路径: 无	
50	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 109
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 107: 108: MEM_STATIC unsigned MEM_32bits(void) { return sizeof(void*)==4; } 109: MEM_STATIC unsigned MEM_64bits(void) { return sizeof(void*)==8; } 110: 111: MEM_STATIC unsigned MEM_isLittleEndian(void)	
	污染路径: 无	
51	缺陷发生位置: /contrib/TSZ/zstd/deprecated/zbuff_compress.c	行号: 55
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int *'.	
	代码片段: 53: ZBUFF_CCtx* ZBUFF_createCCtx(void) 54: { 55: return ZSTD_createCStream(); 56: } 57:	
	污染路径: 无	
52	缺陷发生位置: /source/os/src/osSystem.c	行号: 279
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned long', 定义类型为'int'.	
	代码片段:	

	277: } 278: 279: return strlen(buf); 280: } 281:	
	污染路径: 无	
53	缺陷发生位置: /source/common/src/cos.c	行号: 1979
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'long'.	
	代码片段: 1977: } 1978: void s3EvictCache(const char *path, long object_size) { } 1979: long s3Size(const char *object_name) { return 0; } 1980: int32_t s3GetObjectsByPrefix(const char *prefix, const char *path) { return 0; } 1981: int32_t s3GetObjectToFile(const char *object_name, const char *fileName) { return 0; }	
	污染路径: 无	
54	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 289
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'.	
	代码片段: 287: #endif 288: 289: MEM_STATIC unsigned MEM_32bits(void) { return sizeof(size_t)==4; } 290: MEM_STATIC unsigned MEM_64bits(void) { return sizeof(size_t)==8; } 291:	
	污染路径: 无	
55	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 8915
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	

	代码片段: 8913: switch (precision) { 8914: case TSDB_TIME_PRECISION_MILLI: 8915: return 1; 8916: case TSDB_TIME_PRECISION_MICRO: 8917: return 1000;	
	污染路径: 无	
56	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 197
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: 195: } 196: 197: return 0; 198: } 199:	
	污染路径: 无	
57	缺陷发生位置: /contrib/TSZ/zstd/deprecated/zbuff_common.c	行号: 26
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'const char *'.	
	代码片段: 24: /*! ZBUFF_getErrorName() : 25: * provides error code string from function result (useful for debugging) */ 26: const char* ZBUFF_getErrorName(size_t errorCode) { return ERR_getErrorName(errorCode); }	
	污染路径: 无	
58	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 81
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 79: } 80: } 81: return true;	

	82: } 83:	
	污染路径: expanded from macro 'true'	
59	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 388
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 386: while (true) { 387: if (0 == pBind[i].buffer_type) { 388: return false; 389: } 390:	
	污染路径: expanded from macro 'false'	
60	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 1039
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 1037: MEM_STATIC unsigned FSEv05_endOfDState(const FSEv05_DState_t* DStatePtr) 1038: { 1039: return DStatePtr->state == 0; 1040: } 1041:	
	污染路径: 无	
61	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 58
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 56: 57: FORCE_INLINE bool isRest(int32_t iface) { 58: return REST_IFACE == iface SML_REST_IFACE == iface; 59: } 60:	

	污染路径: 无	
62	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 861
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 859: return (unsigned) r; 860: # elif defined(__GNUC__) && (__GNUC__ >= 3) /* Use GCC Intrinsic */ 861: return 31 - __builtin_clz (val); 862: # else /* Software version */ 863: static const unsigned DeBruijnClz[32] = { 0, 9, 1, 10, 13, 21, 2, 29, 11, 14, 16, 18, 22, 25, 3, 30, 8, 12, 20, 28, 15, 17, 24, 7, 19, 27, 23, 6, 26, 5, 4, 31 };	
	污染路径: 无	
63	缺陷发生位置: /source/libs/scalar/src/scifunc.c	行号: 485
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'uint8_t'.	
	代码片段: 483: return 3; 484: } else if ((str[0] & 0xF8) == 0xF0) { 485: return 4; 486: } else { 487: return 1;	
	污染路径: 无	
64	缺陷发生位置: /tools/taos-tools/src/benchInsertMix.c	行号: 389
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 387: mix->bufCnt[type] += 1; 388: 389: return true; 390: } 391:	

	污染路径: expanded from macro 'true'	
65	缺陷发生位置: /contrib/TSZ/zstd/compress/huf_compress.c	行号: 72
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 70: unsigned HUF_optimalTableLog(unsigned maxTableLog, size_t srcSize, unsigned maxSymbolValue) 71: { 72: return FSE_optimalTableLog_internal(maxTableLog, srcSize, maxSymbolValue, 1); 73: } 74:	
	污染路径: 无	
66	缺陷发生位置: /contrib/TSZ/sz/src/sz.c	行号: 103
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 101: { 102: printf("Error: dataType can only be SZ_FLOAT, SZ_DOUBLE .\n"); 103: return 0; 104: } 105:	
	污染路径: 无	
67	缺陷发生位置: /source/libs/decimal/src/detail/intx/int128.hpp	行号: 242
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 240: // but compilers (GCC8, Clang7) 241: // have problem with properly optimizing subtraction. 242: return (x.hi < y.hi) ((x.hi == y.hi) & (x.lo < y.lo)); 243: } 244:	

	污染路径: 无	
68	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 771
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 769: MEM_STATIC unsigned BIT_endOfDStream(const BIT_DStream_t* DStream) 770: { 771: return ((DStream->ptr == DStream->start) && (DStream->bitsConsumed == sizeof(DStream-> bitContainer)*8)); 772: } 773:	
	污染路径: 无	
69	缺陷发生位置: /source/libs/decimal/src/detail/intx/int128.hpp	行号: 79
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned long', 定义类型为'_Bool'.	
	代码片段: 77: #endif 78: 79: constexpr explicit operator bool() const noexcept { return hi lo; } 80: 81: /// Explicit converting operator for all builtin integral types.	
	污染路径: 无	
70	缺陷发生位置: /contrib/libmqt/mtq/src/mtqPacketDataTypes.c	行号: 190
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 188: unsigned int packet__varint_bytes(uint32_t word) { 189: if (word < 128) { 190: return 1; 191: } else if (word < 16384) { 192: return 2;	

	污染路径: 无	
71	缺陷发生位置: /source/libs/new-stream/src/dataSinkCache.c	行号: 147
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 145: } 146: if (g_slidigGrpMemList.waitMoveMemSize > DS_MEM_SIZE_RESERVED) { 147: return true; 148: } 149:	
	污染路径: expanded from macro 'true'	
72	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-fmtstream.c	行号: 459
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'ssize_t'.	
	代码片段: 457: 458: if (! _argp_fmtstream_ensure (fs, size_guess)) 459: return -1; 460: 461: va_start (args, fmt);	
	污染路径: 无	
73	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 47
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 45: 46: /*=== Version ===*/ 47: unsigned FSE_versionNumber(void) { return FSE_VERSION_NUMBER; } 48: 49:	

	污染路径: expanded from macro 'FSE_VERSION_NUMBER'	
74	缺陷发生位置: /utils/test/c/tmqDemo.c	行号: 203
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: <pre> 201: if ((pDir = taosOpenDir(dir)) == NULL) { 202: fprintf(stderr, "Cannot open dir: %s\n", dir); 203: return -1; 204: } 205: </pre>	
	污染路径: 无	
75	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 633
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: <pre> 631: return (unsigned) r; 632: # elif defined(__GNUC__) && (__GNUC__ >= 3) /* Use GCC Intrinsic */ 633: return 31 - __builtin_clz (val); 634: # else /* Software version */ 635: static const unsigned DeBruijnClz[32] = { 0, 9, 1, 10, 13, 21, 2, 29, 11, 14, 16, 18, 22, 25, 3, 30, 8, 12, 20, 28, 15, 17, 24, 7, 19, 27, 23, 6, 26, 5, 4, 31 }; </pre>	
	污染路径: 无	
76	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 583
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: <pre> 581: } 582: 583: return false; 584: } 585: </pre>	
	污染路径:	

	expanded from macro 'false'	
77	缺陷发生位置: /tools/src/pub.c	行号: 86
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int8_t'.	
	代码片段: 84: return CONN_MODE_NATIVE; 85: } else if (strcasecmp(arg, STR_WEBSOCKET) == 0 strcasecmp(arg, "1") == 0) { 86: return CONN_MODE_WEBSOCKET; 87: } else { 88: fprintf(stderr, "invalid input %s for option -Z, only support: %s or %s\r\n", arg, STR_NATIVE, STR_WEBSOCKET);	
	污染路径: expanded from macro 'CONN_MODE_WEBSOCKET'	
78	缺陷发生位置: /source/util/src/tutil.c	行号: 80
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 78: if (z[j] == 0) { 79: z[0] = 0; 80: return 0; 81: } 82:	
	污染路径: 无	
79	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 354
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 352: return (unsigned) r; 353: # elif defined(__GNUC__) && (__GNUC__ >= 3) /* Use GCC Intrinsic */ 354: return 31 - __builtin_clz (val); 355: # else /* Software version */ 356: static const unsigned DeBruijnClz[32] = { 0, 9, 1, 10, 13, 21, 2, 29, 11, 14, 16, 18, 22, 25, 3, 30, 8, 12, 20, 28, 15, 17, 24, 7, 19, 27, 23, 6, 26, 5, 4, 31 };	

	污染路径: 无	
80	缺陷发生位置: /source/dnode/vnode/src/bse/bseTable.c	行号: 1097
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'long', 定义类型为'int'.	
	代码片段: 1095: p = taosDecodeVariantl64(p, &pMeta->range.sseq); 1096: p = taosDecodeVariantl64(p, &pMeta->range.eseq); 1097: return p - buf; 1098: } 1099: int32_t metaBlockAdd(SBlock *p, SMetaBlock *pBlk) {	
	污染路径: 无	
81	缺陷发生位置: /source/os/src/osSystem.c	行号: 244
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'void **', 定义类型为'int'.	
	代码片段: 242: if (!isCommandAllowed(cmd)) { 243: terrno = TSDB_CODE_INVALID_PARA; 244: return NULL; 245: } 246:	
	污染路径: expanded from macro 'NULL'	
82	缺陷发生位置: /source/util/src/tenv.c	行号: 79
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned long', 定义类型为'int'.	
	代码片段: 77: } 78: } 79: return strlen(cfgStr); 80: }	
	污染路径: 无	
83	缺陷发生位置: /source/libs/function/src/builtinsimpl.c	行号: 8279
	步骤描述:	

	函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'double'.	
	代码片段: 8277: int64_t duration = pRateInfo->lastKey - pRateInfo->firstKey; 8278: if (duration == 0) { 8279: return 0; 8280: } 8281:	
	污染路径: 无	
84	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1896
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'long'，定义类型为'int'.	
	代码片段: 1894: // create stream <stream_name> as select 1895: if (strncasecmp(p, "create stream ", 14) == 0) { 1896: return as_pos_end - p; 1897: ; 1898: }	
	污染路径: 无	
85	缺陷发生位置: /source/os/src/osFile.c	行号: 214
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'void *'，定义类型为'int'.	
	代码片段: 212: tstrncpy(s, path, sizeof(s)); 213: if (taosMulMkDir(taosDirName(s)) != 0) { 214: return NULL; 215: } 216: fp = taosOpenFile(path, tdFileOptions);	
	污染路径: expanded from macro 'NULL'	
86	缺陷发生位置: /contrib/TSZ/zstd/common/mem.h	行号: 94
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'.	
	代码片段: 92: #endif	

	93: 94: MEM_STATIC unsigned MEM_32bits(void) { return sizeof(size_t)==4; } 95: MEM_STATIC unsigned MEM_64bits(void) { return sizeof(size_t)==8; } 96:	
	污染路径: 无	
87	缺陷发生位置: /source/util/src/tcompression.c	行号: 1008
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'long', 定义类型为'int32_t'.	
	代码片段: 1006: } 1007: 1008: return nelements * longBytes; 1009: } 1010:	
	污染路径: 无	
88	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqStrings.c	行号: 72
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'const char *'.	
	代码片段: 70: // If ttq_errno is greater than 127, 71: // a mqtt5_return_code error was used 72: return tmqtt_reason_string(ttq_errno); 73: } else { 74: return "Unknown error.";	
	污染路径: 无	
89	缺陷发生位置: /source/libs/new-stream/test/dataSinkTest.cpp	行号: 1062
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'_Bool', 定义类型为'int'.	
	代码片段: 1060: 1061: taos_cleanup(); 1062: return ret ret2;	

	1063: }	
	污染路径: 无	
90	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 895
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 893: return true; 894: } 895: return false; 896: } 897: }	
	污染路径: expanded from macro 'false'	
91	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 136
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'.	
	代码片段: 134: #endif 135: 136: MEM_STATIC unsigned MEM_32bits(void) { return sizeof(void*)==4; } 137: MEM_STATIC unsigned MEM_64bits(void) { return sizeof(void*)==8; } 138:	
	污染路径: 无	
92	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 137
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'.	
	代码片段: 135: #endif 136: 137: MEM_STATIC unsigned MEM_32bits(void) { return sizeof(void*)==4; } 138: MEM_STATIC unsigned MEM_64bits(void) { return sizeof(void*)==8; }	

	139:	
	污染路径: 无	
93	缺陷发生位置: /tools/taos-tools/src/benchInsertMix.c	行号: 131
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 129: bool needExecDel(SMixRatio* mix) { 130: if (mix->genCnt[MDEL] == 0 mix->doneCnt[MDEL] >= mix->genCnt[MDEL]) { 131: return false; 132: } 133:	
	污染路径: expanded from macro 'false'	
94	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 114
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 112: constexpr bool operator==(const uint<N>& a, const uint<N>& b) noexcept 113: { 114: return (a.lo == b.lo) & (a.hi == b.hi); 115: } 116:	
	污染路径: 无	
95	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 2703
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 2701: } 2702: 2703: return true; 2704: } 2705:	

	污染路径: expanded from macro 'true'	
96	缺陷发生位置: /source/common/src/dmRepair.c	行号: 24
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 22: TD_WEAK bool dmRepairNeedTsdbRepair(int32_t vnodeId) { 23: TAOS_UNUSED(vnodeId); 24: return false; 25: } 26:	
	污染路径: expanded from macro 'false'	
97	缺陷发生位置: /source/util/src/tcompression.c	行号: 325
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int8_t'.	
	代码片段: 323: return compressL2LevelDict[alg].lvl[2]; 324: } 325: return 1; 326: } 327:	
	污染路径: 无	
98	缺陷发生位置: /contrib/TSZ/zstd/compress/zstd_compress.c	行号: 140
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 138: size_t ZSTD_sizeof_CCtx(const ZSTD_CCtx* cctx) 139: { 140: if (cctx==NULL) return 0; /* support sizeof on NULL */ 141: return sizeof(*cctx) + cctx->workSpaceSize 142: + ZSTD_sizeof_CDict(cctx->cdictLocal)	
	污染路径: 无	
99	缺陷发生位置:	行号:

	/tools/taos-tools/src/benchUtil.c	898
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: <pre> 896: } 897: } 898: return false; 899: } 900: </pre>	
	污染路径: expanded from macro 'false'	
100	缺陷发生位置: /contrib/TSZ/zstd/common/error_private.h	行号: 56
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: <pre> 54: #define ZSTD_ERROR(name) ((size_t)-PREFIX(name)) 55: 56: ERR_STATIC unsigned ERR_isError(size_t code) { return (code > ERROR(maxCode)); } 57: 58: ERR_STATIC ERR_enum ERR_getErrorCode(size_t code) { if (!ERR_isError(code)) return (ERR_enum)0; return (ERR_enum) (0-code); } </pre>	
	污染路径: 无	
101	缺陷发生位置: /contrib/TSZ/zstd/common/zstd_internal.h	行号: 218
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'U32'.	
	代码片段: <pre> 216: return (unsigned)r; 217: # elif defined(__GNUC__) && (__GNUC__ >= 3) /* GCC Intrinsic */ 218: return 31 - __builtin_clz(val); 219: # else /* Software version */ 220: static const U32 DeBruijnClz[32] = { 0, 9, 1, 10, 13, 21, 2, 29, 11, 14, 16, 18, 22, 25, 3, 30, 8, 12, 20, 28, 15, 17, 24, 7, 19, 27, 23, 6, 26, 5, 4, 31 }; </pre>	
	污染路径:	

	无	
102	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 523
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'.	
	代码片段: <pre> 521: else /* invalid */ 522: { 523: return 0; 524: } 525: </pre>	
	污染路径: 无	
103	缺陷发生位置: /contrib/TSZ/sz/src/sz.c	行号: 129
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: <pre> 127: return r1*sizeof(float); 128: } 129: return 0; 130: } 131: else if(dataType == SZ_DOUBLE) </pre>	
	污染路径: 无	
104	缺陷发生位置: /tools/taos-tools/src/toolstime.c	行号: 162
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: <pre> 160: dig = (num[i] - '0'); 161: } else { 162: return 0; 163: } 164: ret += dig * base; </pre>	
	污染路径: 无	
105	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 396
	步骤描述:	

	函数返回值类型必须与定义一致，实际返回为'long long'，定义类型为'int64_t'.	
	代码片段: <pre> 394: struct timeval systemTime; 395: taosGetTimeOfDay(&systemTime); 396: return (int64_t)systemTime.tv_sec * 1000LL + (int64_t)systemTime.tv_usec / 1000; 397: } 398: </pre>	
	污染路径: 无	
106	缺陷发生位置: /contrib/TSZ/zstd/deprecated/zbuff_compress.c	行号: 146
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: <pre> 144: * Tool functions 145: *****/ 146: size_t ZBUFF_recommendedCInSize(void) { return ZSTD_CStreamInSize(); } 147: size_t ZBUFF_recommendedCOutSize(void) { return ZSTD_CStreamOutSize(); } </pre>	
	污染路径: 无	
107	缺陷发生位置: /source/os/src/osFile.c	行号: 185
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: <pre> 183: if (code != 0) { 184: terrno = code; 185: return -1; 186: } 187: </pre>	
	污染路径: 无	
108	缺陷发生位置: /source/dnode/vnode/src/bse/bseTable.c	行号: 996
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'long'，定义类型为'int'.	
	代码片段:	

	994: p = taosDecodeVariantl64(p, &pHandle->range.sseq); 995: p = taosDecodeVariantl64(p, &pHandle->range.eseq); 996: return p - buf; 997: } 998:	
	污染路径: 无	
109	缺陷发生位置: /source/dnode/mnode/impl/src/mndXnode.c	行号: 3990
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: 3988: if (pRsp->data == NULL) { 3989: uError("failed to prepare recv buffer for post rsp, len:%d, code:%s", (int32_t)size + 1, tstrerror(errno)); 3990: return 0; // return the recv length, if failed, return 0 3991: } 3992: } else {	
	污染路径: 无	
110	缺陷发生位置: /tools/taos-tools/src/dumpUtil.c	行号: 239
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'long'.	
	代码片段: 237: // move end 238: if (fseek(fp, 0, SEEK_END) != 0) { 239: return -1; 240: } 241: // get	
	污染路径: 无	
111	缺陷发生位置: /source/os/src/osSocket.c	行号: 229
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int8_t'.	
	代码片段: 227: TAOS_SKIP_ERROR(taosCloseSocket(&pSocket)); 228: 229: return 1;	

	230: } 231:	
	污染路径: 无	
112	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 1251
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 1249: } 1250: 1251: return false; 1252: } 1253:	
	污染路径: expanded from macro 'false'	
113	缺陷发生位置: /source/libs/executor/test/executorTests.cpp	行号: 71
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'long'，定义类型为'int32_t'.	
	代码片段: 69: SDummyInputInfo* pInfo = static_cast<SDummyInputInfo*>(pOperator->info); 70: if (pInfo->current >= pInfo->totalPages) { 71: return NULL; 72: } 73:	
	污染路径: expanded from macro 'NULL'	
114	缺陷发生位置: /tools/taos-tools/src/toolstime.c	行号: 647
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 645: for (int i = 0; i < seg_len; ++i) { 646: if (*c == 'Z' *c == 'z' *c == '+' *c == '-') { 647: return true; 648: } 649: c--;	

	污染路径: expanded from macro 'true'	
115	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 57
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: <pre> 55: } 56: else 57: return 0; 58: } 59: </pre>	
	污染路径: 无	
116	缺陷发生位置: /source/dnode/mnode/impl/src/mndXnode.c	行号: 3996
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: <pre> 3994: if (p == NULL) { 3995: uError("failed to prepare recv buffer for post rsp, len:%d, code:%s", (int32_t)size + 1, tstrerror(terno)); 3996: return 0; // return the recv length, if failed, return 0 3997: } 3998: </pre>	
	污染路径: 无	
117	缺陷发生位置: /tools/rocks-reader/rreader.cpp	行号: 113
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned long', 定义类型为'int32_t'.	
	代码片段: <pre> 111: pLastCol->colVal.value.nData = pLastColV0- >colVal.value.nData; 112: pLastCol->colVal.value.pData = (uint8_t *) (&pLastColV0[1]); 113: return sizeof(SLastColV0) + pLastColV0- >colVal.value.nData; 114: } else { 115: pLastCol->colVal.value.val = pLastColV0- </pre>	

	>colVal.value.val;	
	污染路径: 无	
118	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeStream.c	行号: 171
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 169: if (gid == (uint64_t)-1) return true; 170: } 171: return false; 172: } 173:	
	污染路径: expanded from macro 'false'	
119	缺陷发生位置: /contrib/TSZ/zstd/common/mem.h	行号: 95
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'.	
	代码片段: 93: 94: MEM_STATIC unsigned MEM_32bits(void) { return sizeof(size_t)==4; } 95: MEM_STATIC unsigned MEM_64bits(void) { return sizeof(size_t)==8; } 96: 97: MEM_STATIC unsigned MEM_isLittleEndian(void)	
	污染路径: 无	
120	缺陷发生位置: /tools/rocks-reader/rreader.cpp	行号: 116
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'unsigned long'，定义类型为'int32_t'.	
	代码片段: 114: } else { 115: pLastCol->colVal.value.val = pLastColV0->colVal.value.val; 116: return sizeof(SLastColV0); 117: } 118: }	

	污染路径: 无	
121	缺陷发生位置: /source/libs/executor/test/joinTests.cpp	行号: 663
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int64_t *', 定义类型为'void *'.	
	代码片段: 661: switch (type) { 662: case TSDB_DATA_TYPE_TIMESTAMP: 663: return &TIMESTAMP_FILTER_VALUE; 664: case TSDB_DATA_TYPE_INT: 665: return &INT_FILTER_VALUE;	
	污染路径: 无	
122	缺陷发生位置: /source/client/src/clientSml.c	行号: 167
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 165: int64_t unit = smlToMilli[fromPrecision - TSDB_TIME_PRECISION_HOURS]; 166: if (tsInt64 != 0 && unit > INT64_MAX / tsInt64) { 167: return -1; 168: } 169: tsInt64 *= unit;	
	污染路径: 无	
123	缺陷发生位置: /source/os/src/osDir.c	行号: 482
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'void *', 定义类型为'int'.	
	代码片段: 480: if (dirname == NULL) { 481: terrno = TSDB_CODE_INVALID_PARA; 482: return NULL; 483: } 484:	
	污染路径: expanded from macro 'NULL'	

124	缺陷发生位置: /contrib/TSZ/zstd/common/mem.h	行号: 250
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'U32'.	
	代码片段: 248: MEM_STATIC U32 MEM_readLE24(const void* memPtr) 249: { 250: return MEM_readLE16(memPtr) + (((const BYTE*)memPtr)[2] << 16); 251: } 252:	
	污染路径: 无	
125	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 990
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 988: MEM_STATIC unsigned BITv06_endOfDStream(const BITv06_DStream_t* DStream) 989: { 990: return ((DStream->ptr == DStream->start) && (DStream->bitsConsumed == sizeof(DStream-> bitContainer)*8)); 991: } 992:	
	污染路径: 无	
126	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 49
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 47: constexpr explicit operator bool() const noexcept 48: { 49: return static_cast<bool>(lo) static_cast<bool>(hi); 50: } 51:	
	污染路径: 无	

127	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 141
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'uint8_t'.	
	代码片段: 139: } 140: if (key2 == NULL) 141: return FUNTYPE_NONE; 142: 143: char* key3 = strstr(key2, "+");	
	污染路径: expanded from macro 'FUNTYPE_NONE'	
128	缺陷发生位置: /test/new_test_framework/script/api/dbTableRoute.c	行号: 113
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'long long'，定义类型为'int64_t'.	
	代码片段: 111: struct timeval systemTime; 112: rtGetTimeOfDay(&systemTime); 113: return (int64_t)systemTime.tv_sec * 1000LL + (int64_t)systemTime.tv_usec/1000; 114: } 115:	
	污染路径: 无	
129	缺陷发生位置: /source/os/src/osDir.c	行号: 528
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'void *'，定义类型为'int'.	
	代码片段: 526: if (pDir == NULL) { 527: errno = TSDB_CODE_INVALID_PARA; 528: return NULL; 529: } 530: #ifdef WINDOWS	
	污染路径: expanded from macro 'NULL'	
130	缺陷发生位置: /source/os/src/osFile.c	行号: 205
	步骤描述:	

	函数返回值类型必须与定义一致，实际返回为'void *'，定义类型为'int'.	
	代码片段: 203: if (path == NULL) { 204: terrno = TSDB_CODE_INVALID_PARA; 205: return NULL; 206: } 207: TdFilePtr fp = taosOpenFile(path, tdFileOptions);	
	污染路径: expanded from macro 'NULL'	
131	缺陷发生位置: /contrib/TSZ/zstd/common/bitstream.h	行号: 273
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: 271: BIT_addBitsFast(bitC, 1, 1); /* endMark */ 272: BIT_flushBits(bitC); 273: if (bitC->ptr >= bitC->endPtr) return 0; /* overflow detected */ 274: return (bitC->ptr - bitC->startPtr) + (bitC->bitPos > 0); 275: }	
	污染路径: 无	
132	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqPacketDataTypes.c	行号: 196
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'.	
	代码片段: 194: return 3; 195: } else if (word < 268435456) { 196: return 4; 197: } else { 198: return 5;	
	污染路径: 无	
133	缺陷发生位置: /source/os/src/osSocket.c	行号: 224
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int8_t'.	
	代码片段:	

	222: terrno = TAOS_SYSTEM_ERROR(ERRNO); 223: TAOS_SKIP_ERROR(taosCloseSocket(&pSocket)); 224: return 0; 225: } 226:	
	污染路径: 无	
134	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 462
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: 460: 461: DTableH->fastMode = (U16)noLarge; 462: return 0; 463: } 464:	
	污染路径: 无	
135	缺陷发生位置: /contrib/TSZ/sz/src/sz.c	行号: 138
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: 136: return r1*sizeof(double); 137: } 138: return 0; 139: } 140: else	
	污染路径: 无	
136	缺陷发生位置: /contrib/TSZ/zstd/deprecated/zbuff_decompress.c	行号: 75
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: 73: *****/ 74: size_t ZBUFF_recommendedDInSize(void) { return ZSTD_DStreamInSize(); } 75: size_t ZBUFF_recommendedDOutSize(void) { return	

	ZSTD_DStreamOutSize(); }	
	污染路径: 无	
137	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 130
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 128: 129: MEM_STATIC unsigned MEM_32bits(void) { return sizeof(size_t)==4; } 130: MEM_STATIC unsigned MEM_64bits(void) { return sizeof(size_t)==8; } 131: 132: MEM_STATIC unsigned MEM_isLittleEndian(void)	
	污染路径: 无	
138	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 879
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 877: bool searchBArray(BArray *pArray, const char *field_name, int32_t name_len, uint8_t field_type) { 878: if (pArray == NULL field_name == NULL) { 879: return false; 880: } 881:	
	污染路径: expanded from macro 'false'	
139	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeStream.c	行号: 749
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 747: static bool isAlteredTable(int8_t action, ETableType tbType) { 748: if (action == TSDB_ALTER_TABLE_UPDATE_MULTI_TABLE_TAG_VAL && tbType == TSDB_CHILD_TABLE) {	

	749: return true; 750: } else if (action == TSDB_ALTER_TABLE_UPDATE_CHILD_TABLE_TAG_VAL && tbType == TSDB_SUPER_TABLE) { 751: return true;	
	污染路径: expanded from macro 'true'	
140	缺陷发生位置: /source/os/src/osLocale.c	行号: 72
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'char *'.	
	代码片段: 70: } 71: 72: return taosStrdup(charsetstr); 73: } 74:	
	污染路径: 无	
141	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 170
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: 168: cell->nbBits = 0; 169: 170: return 0; 171: } 172:	
	污染路径: 无	
142	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 97
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'.	
	代码片段: 95: unsigned ZDICT_getDictID(const void* dictBuffer, size_t dictSize) 96: { 97: if (dictSize < 8) return 0;	

	98: if (MEM_readLE32(dictBuffer) != ZSTD_MAGIC_DICTIONARY) return 0; 99: return MEM_readLE32((const char*)dictBuffer + 4);	
	污染路径: 无	
143	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 3485
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: 3483: printErrCmdCodeStr(cmd, code, res); 3484: closeBenchConn(conn); 3485: return -1; 3486: } 3487: TAOS_ROW row = NULL;	
	污染路径: 无	
144	缺陷发生位置: /source/libs/decimal/test/decimalTest.cpp	行号: 272
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'double'.	
	代码片段: 270: } 271: DEFINE_REAL_OP(+); 272: DEFINE_REAL_OP(-); 273: DEFINE_REAL_OP(*); 274: DEFINE_REAL_OP(/);	
	污染路径: expanded from macro 'DEFINE_REAL_OP'	
145	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 854
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: 852: mtctx->jobIDMask = nbJobs - 1; 853: } 854: return 0; 855: } 856:	

	污染路径: 无	
146	缺陷发生位置: /contrib/TSZ/sz/src/iniparser.c	行号: 457
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'double'.	
	代码片段: 455: str = iniparser_getstring(d, key, INI_INVALID_KEY); 456: if (str==INI_INVALID_KEY) return notfound ; 457: return atof(str); 458: } 459:	
	污染路径: 无	
147	缺陷发生位置: /source/util/src/tenv.c	行号: 20
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'char'.	
	代码片段: 18: #include "tenv.h" 19: 20: static char toLowChar(char c) { return (c > 'Z' c < 'A' ? c : (c - 'A' + 'a')); } 21: 22: int32_t taosEnvNameToCfgName(const char *envNameStr, char *cfgNameStr, int32_t cfgNameMaxLen) {	
	污染路径: 无	
148	缺陷发生位置: /source/client/test/clientMonitorTests.cpp	行号: 145
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 143: int64_t fileSize = 0; 144: if (taosStatFile(path, &fileSize, NULL, NULL) < 0) { 145: return -1; 146: } 147:	
	污染路径: 无	

149	缺陷发生位置: /contrib/TSZ/zstd/compress/huf_compress.c	行号: 105
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: <pre> 103: /* Scan input and build symbol stats */ 104: { unsigned const maxCount = HIST_count_simple(count, &maxSymbolValue, weightTable, wtSize); /* never fails */ 105: if (maxCount == wtSize) return 1; /* only a single symbol in src : rle */ 106: if (maxCount == 1) return 0; /* each symbol present maximum once => not compressible */ 107: }</pre>	
	污染路径: 无	
150	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 945
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: <pre> 943: size_t ZSTDMT_freeCCtx(ZSTDMT_CCtx* mtctx) 944: { 945: if (mtctx==NULL) return 0; /* compatible with free on NULL */ 946: POOL_free(mtctx->factory); /* stop and free worker threads */ 947: ZSTDMT_releaseAllJobResources(mtctx); /* release job resources into pools first */</pre>	
	污染路径: 无	
151	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 259
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'long'，定义类型为'size_t'.	
	代码片段: <pre> 257: } } 258: 259: return op-ostart; 260: } 261:</pre>	

	污染路径: 无	
152	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 1241
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 1239: if ((strcmp(dbName, "information_schema") == 0) 1240: (strcmp(dbName, "performance_schema") == 0)) { 1241: return true; 1242: } 1243: } else if (g_majorVersionOfClient == 2) {	
	污染路径: expanded from macro 'true'	
153	缺陷发生位置: /source/client/src/clientSml.c	行号: 161
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 159: int64_t tsInt64 = taosStr2Int64(value, &endPtr, 10); 160: if (unlikely(value + len != endPtr)) { 161: return -1; 162: } 163:	
	污染路径: 无	
154	缺陷发生位置: /source/client/src/clientImpl.c	行号: 73
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'char *'.	
	代码片段: 71: (void)snprintf(key, sizeof(key), "%s:%s:%s:%d", user, auth, ip, port); 72: } 73: return taosStrdup(key); 74: } 75:	
	污染路径:	

	无	
155	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 2664
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 2662: *pkCur = 0; 2663: *pkCnt = 0; 2664: return true; 2665: } else { 2666: // add one </pre>	
	污染路径: expanded from macro 'true'	
156	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 574
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'.	
	代码片段: <pre> 572: static const char* ERR_strings[] = { ERROR_LIST(ERROR_GENERATE_STRING) }; 573: 574: ERR_STATIC unsigned ERR_isError(size_t code) { return (code > ERROR(maxCode)); } 575: 576: ERR_STATIC const char* ERR_getErrorName(size_t code) </pre>	
	污染路径: 无	
157	缺陷发生位置: /tools/taos-tools/src/toolstime.c	行号: 153
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: <pre> 151: dig = num[i] - 'A' + 10; 152: } else { 153: return 0; 154: } 155: ret += dig * base; </pre>	
	污染路径: 无	
158	缺陷发生位置:	行号:

	/contrib/TSZ/zstd/legacy/zstd_v02.c	137
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 135: 136: MEM_STATIC unsigned MEM_32bits(void) { return sizeof(void*)==4; } 137: MEM_STATIC unsigned MEM_64bits(void) { return sizeof(void*)==8; } 138: 139: MEM_STATIC unsigned MEM_isLittleEndian(void)	
	污染路径: 无	
159	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 413
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 411: 412: if (pBind[i].buffer_type == dataType) { 413: return true; 414: } 415:	
	污染路径: expanded from macro 'true'	
160	缺陷发生位置: /contrib/TSZ/zstd/common/bitstream.h	行号: 165
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 163: return (unsigned) r; 164: # elif defined(__GNUC__) && (__GNUC__ >= 3) /* Use GCC Intrinsic */ 165: return 31 - __builtin_clz (val); 166: # else /* Software version */ 167: static const unsigned DeBruijnClz[32] = { 0, 9, 1, 10, 13, 21, 2, 29,	
	污染路径: 无	
161	缺陷发生位置:	行号:

	/contrib/TSZ/zstd/common/bitstream.h	274
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'long', 定义类型为'size_t'.	
	代码片段: 272: BIT_flushBits(bitC); 273: if (bitC->ptr >= bitC->endPtr) return 0; /* overflow detected */ 274: return (bitC->ptr - bitC->startPtr) + (bitC->bitPos > 0); 275: } 276:	
	污染路径: 无	
162	缺陷发生位置: /contrib/TSZ/zstd/common/mem.h	行号: 206
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned long long', 定义类型为'U64'.	
	代码片段: 204: return __builtin_bswap64(in); 205: #else 206: return ((in << 56) & 0xff00000000000000ULL) 207: ((in << 40) & 0x00ff000000000000ULL) 208: ((in << 24) & 0x0000ff0000000000ULL)	
	污染路径: 无	
163	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 3462
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 3460: SBenchConn* conn = initBenchConn(database->dbName); 3461: if (NULL == conn) { 3462: return -1; 3463: } 3464: char cmd[SHORT_1K_SQL_BUFF_LEN] = "\0";	
	污染路径: 无	
164	缺陷发生位置:	行号:

	/tools/taos-tools/src/benchInsertMix.c	134
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 132: } 133: 134: return true; 135: } 136:	
	污染路径: expanded from macro 'true'	
165	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 398
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 396: errorPrint("Input %s, time format error!\n", 397: g_args.humanStartTime); 398: return -1; 399: } 400: } else {	
	污染路径: 无	
166	缺陷发生位置: /source/client/src/clientFirewall.c	行号: 622
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 620: int64_t fsize = 0; 621: int64_t mtime = 0; 622: if (path == NULL path[0] == 0) return -1; 623: if (taosStatFile(path, &fsize, &mtime, NULL) < 0) return -1; 624: return mtime;	
	污染路径: 无	
167	缺陷发生位置: /contrib/TSZ/zstd/deprecated/zbuff_common.c	行号: 23
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	

	代码片段: 21: /*! ZBUFF_isError() : 22: * tells if a return value is an error code */ 23: unsigned ZBUFF_isError(size_t errorCode) { return ERR_isError(errorCode); } 24: /*! ZBUFF_getErrorName() : 25: * provides error code string from function result (useful for debugging) */	
	污染路径: 无	
168	缺陷发生位置: /tools/taos-tools/src/benchInsertMix.c	行号: 379
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 377: if(mix->bufCnt[type] >= mix->capacity[type]) { 378: // no space to save 379: return false; 380: } 381:	
	污染路径: expanded from macro 'false'	
169	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 124
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'char *'，定义类型为'void *'.	
	代码片段: 122: void* memcpy(void* dest, const void* src, size_t n){ 123: 124: return ((char*) memcpy(dest, src, n)) + n; 125: } 126:	
	污染路径: 无	
170	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 1245
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段:	

	<pre> 1243: } else if (g_majorVersionOfClient == 2) { 1244: if (strcmp(dbName, "log") == 0) { 1245: return true; 1246: } 1247: } else { </pre>	
	污染路径: expanded from macro 'true'	
171	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 957
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: <pre> 955: ZSTD_free(mtctx->roundBuff.buffer, mtctx->cMem); 956: ZSTD_free(mtctx, mtctx->cMem); 957: return 0; 958: } 959: </pre>	
	污染路径: 无	
172	缺陷发生位置: /source/os/src/osFile.c	行号: 128
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: <pre> 126: if (from == NULL to == NULL) { 127: terrno = TSDB_CODE_INVALID_PARA; 128: return -1; 129: } 130: #ifdef WINDOWS </pre>	
	污染路径: 无	
173	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqPacketDataTypes.c	行号: 194
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'.	
	代码片段: <pre> 192: return 2; 193: } else if (word < 2097152) { 194: return 3; 195: } else if (word < 268435456) { </pre>	

	196: return 4;	
	污染路径: 无	
174	缺陷发生位置: /tools/taos-tools/src/toolsString.c	行号: 78
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 76: } 77: 78: return true; 79: } 80:	
	污染路径: expanded from macro 'true'	
175	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 922
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'.	
	代码片段: 920: MEM_STATIC unsigned FSE_endOfDState(const FSE_DState_t* DStatePtr) 921: { 922: return DStatePtr->state == 0; 923: } 924:	
	污染路径: 无	
176	缺陷发生位置: /source/util/src/tutil.c	行号: 309
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: 307: dig = (num[i] - '0'); 308: } else { 309: return 0; 310: } 311: ret += dig * base;	
	污染路径:	

	无	
177	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 409
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 407: while (true) { 408: if (0 == pBind[i].buffer_type) { 409: return false; 410: } 411:	
	污染路径: expanded from macro 'false'	
178	缺陷发生位置: /source/client/src/clientSmlTelnet.c	行号: 39
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 37: if (unlikely(!data)) { 38: smlBuildInvalidDataMsg(&info->msgBuf, "timestamp can not be null", NULL); 39: return -1; 40: } 41: if (unlikely(len == 1 && data[0] == '0')) {	
	污染路径: 无	
179	缺陷发生位置: /source/os/src/osSystem.c	行号: 239
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'void **', 定义类型为'int'.	
	代码片段: 237: if (cmd == NULL) { 238: terrno = TSDB_CODE_INVALID_PARA; 239: return NULL; 240: } 241:	
	污染路径: expanded from macro 'NULL'	
180	缺陷发生位置: /source/util/src/tutil.c	行号: 300

	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 298: dig = num[i] - 'A' + 10; 299: } else { 300: return 0; 301: } 302: ret += dig * base;	
	污染路径: 无	
181	缺陷发生位置: /source/os/src/osSystem.c	行号: 249
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'void *', 定义类型为'int'.	
	代码片段: 247: if (!sanitizeCommand(cmd)) { 248: errno = TSDB_CODE_INVALID_PARA; 249: return NULL; 250: } 251:	
	污染路径: expanded from macro 'NULL'	
182	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeStream.c	行号: 756
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 754: return true; 755: } 756: return false; 757: } 758:	
	污染路径: expanded from macro 'false'	
183	缺陷发生位置: /tools/taos-tools/src/benchLog.c	行号: 104
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 102: }	

	103: } 104: return true; 105: } 106:	
	污染路径: expanded from macro 'true'	
184	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 2667
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 2665: } else { 2666: // add one 2667: return false; 2668: } 2669: }	
	污染路径: expanded from macro 'false'	
185	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 83
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'uint8_t'.	
	代码片段: 81: // check valid 82: if (expr == NULL multiple == NULL addend == NULL base == NULL) { 83: return FUNTYPE_NONE; 84: } 85:	
	污染路径: expanded from macro 'FUNTYPE_NONE'	
186	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 381
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'long long'，定义类型为'int64_t'.	
	代码片段: 379: struct timeval systemTime; 380: taosGetTimeOfDay(&systemTime); 381: return (int64_t)systemTime.tv_sec * 1000000LL + (int64_t)systemTime.tv_usec;	

	382: } 383:	
	污染路径: 无	
187	缺陷发生位置: /contrib/TSZ/zstd/compress/fse_compress.c	行号: 425
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 423: for (s=0; ToDistribute > 0; s = (s+1)% (maxSymbolValue+1)) 424: if (norm[s] > 0) { ToDistribute--; norm[s]++; } 425: return 0; 426: } 427:	
	污染路径: 无	
188	缺陷发生位置: /contrib/TSZ/zstd/common/bitstream.h	行号: 208
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 206: bitC->endPtr = bitC->startPtr + dstCapacity - sizeof(bitC->bitContainer); 207: if (dstCapacity <= sizeof(bitC->bitContainer)) return ERROR(dstSize_tooSmall); 208: return 0; 209: } 210:	
	污染路径: 无	
189	缺陷发生位置: /source/client/src/clientSmlTelnet.c	行号: 53
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 51: if (unlikely(ts == -1)) { 52: smlBuildInvalidDataMsg(&info->msgBuf, "invalid timestamp", data); 53: return -1;	

	54: } 55: return ts;	
	污染路径: 无	
190	缺陷发生位置: /source/util/src/tdigest.c	行号: 248
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'double'.	
	代码片段: 246: 247: double tdigestQuantile(TDigest *t, double q) { 248: if (t == NULL) return 0; 249: 250: int32_t i;	
	污染路径: 无	
191	缺陷发生位置: /contrib/TSZ/zstd/compress/zstd_compress.c	行号: 34
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 32: *****/ 33: size_t ZSTD_compressBound(size_t srcSize) { 34: return ZSTD_COMPRESSBOUND(srcSize); 35: } 36:	
	污染路径: 无	
192	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeSimpleTest.cpp	行号: 41
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'char **', 定义类型为'void *'.	
	代码片段: 39: } 40: 41: void* sdbGetRowObj(SSdbRow* pRow) { return pRow ? pRow->data : nullptr; } 42: 43: int sdbSetRawInt32(SSdbRaw* pRaw, int pos, int32_t val) {	

	污染路径: 无	
193	缺陷发生位置: /source/dnode/mnode/impl/src/mndXnode.c	行号: 4013
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 4011: pRsp->dataLen = 0; 4012: uError("failed to malloc curl response"); 4013: return 0; 4014: } 4015: }	
	污染路径: 无	
194	缺陷发生位置: /source/libs/function/src/builtinsimpl.c	行号: 7414
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'double'.	
	代码片段: 7412: static double twa_get_area(SPoint1 s, SPoint1 e) { 7413: if (e.key == INT64_MAX s.key == INT64_MIN) { 7414: return 0; 7415: } 7416:	
	污染路径: 无	
195	缺陷发生位置: /source/os/src/osFile.c	行号: 198
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 196: 197: terrno = code; 198: return -1; 199: #endif 200: }	
	污染路径: 无	
196	缺陷发生位置:	行号:

	/tools/taos-tools/src/benchJsonOpt.c	88
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'uint8_t'.	
	代码片段: <pre> 86: size_t len = strlen(expr); 87: if (len == 0 len > 100) { 88: return FUNTYPE_NONE; 89: } 90: </pre>	
	污染路径: expanded from macro 'FUNTYPE_NONE'	
197	缺陷发生位置: /tools/src/pub.c	行号: 138
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'uint16_t'.	
	代码片段: <pre> 136: uint16_t defaultPort(int8_t connMode, char *dsn) { 137: // port 0 is default 138: return 0; 139: 140: /* </pre>	
	污染路径: 无	
198	缺陷发生位置: /source/os/src/osSocket.c	行号: 217
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int8_t'.	
	代码片段: <pre> 215: if (taosSetSockOpt(pSocket, SOL_SOCKET, SO_REUSEADDR, (void *)&reuse, sizeof(reuse)) < 0) { 216: TAOS_SKIP_ERROR(taosCloseSocket(&pSocket)); 217: return 0; 218: } 219: </pre>	
	污染路径: 无	
199	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 375
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'long long', 定义类型为'int64_t'.	

	代码片段: 373: struct timeval systemTime; 374: taosGetTimeOfDay(&systemTime); 375: return (int64_t)systemTime.tv_sec * 1000LL + (int64_t)systemTime.tv_usec/1000; 376: } 377:	
	污染路径: 无	
200	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeStream.c	行号: 159
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 157: static bool ignoreMetaChange(int64_t tableListVer, int64_t ver) { 158: stDebug("%s tableListVer:%" PRIu64 " ver:%" PRIu64, __func__, tableListVer, ver); 159: return tableListVer >= ver; 160: } 161:	
	污染路径: 无	
201	缺陷发生位置: /contrib/test/tdev/src/main.c	行号: 108
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'long', 定义类型为'uint64_t'.	
	代码片段: 106: gettimeofday(&tv, NULL); 107: 108: return tv.tv_sec * 1000000 + tv.tv_usec; 109: } 110:	
	污染路径: 无	
202	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 128
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段:	

	126: 127: MEM_STATIC unsigned MEM_32bits(void) { return sizeof(void*)==4; } 128: MEM_STATIC unsigned MEM_64bits(void) { return sizeof(void*)==8; } 129: 130: MEM_STATIC unsigned MEM_isLittleEndian(void)	
	污染路径: 无	
203	缺陷发生位置: /source/libs/new-stream/src/dataSinkCache.c	行号: 153
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 151: (pSlidingGrpMgr->usedMemSize > 152: g_slidigGrpMemList.waitMoveMemSize / taosHashGetSize(g_slidigGrpMemList.pSlidingGrpList))) { 153: return true; 154: } 155: return false;	
	污染路径: expanded from macro 'true'	
204	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 8917
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: 8915: return 1; 8916: case TSDB_TIME_PRECISION_MICRO: 8917: return 1000; 8918: case TSDB_TIME_PRECISION_NANO: 8919: return 1000000;	
	污染路径: 无	
205	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 77
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'char'，定义类型为'int'.	
	代码片段:	

	<pre> 75: { 76: if (__fs->p < __fs->end _argp_fmtstream_ensure (__fs, 1)) 77: return *__fs->p++ = __ch; 78: else 79: return EOF; </pre>	
	污染路径: 无	
206	缺陷发生位置: /source/os/src/osTime.c	行号: 489
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'time_t'.	
	代码片段: <pre> 487: time_t taosTimeGm(struct tm *tmp) { 488: if (tmp == NULL) { 489: return -1; 490: } 491: #ifdef WINDOWS </pre>	
	污染路径: 无	
207	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1882
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'long'，定义类型为'int'.	
	代码片段: <pre> 1880: } 1881: 1882: return p1 - p; 1883: } 1884: </pre>	
	污染路径: 无	
208	缺陷发生位置: /source/common/src/dmRepair.c	行号: 14
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 12: TD_WEAK bool dmRepairNodeTypesVnode() { return false; } 13: TD_WEAK bool dmRepairModelsForce() { return false; } </pre>	

	14: TD_WEAK bool dmRepairHasBackupPath() { return false; } 15: TD_WEAK const char *dmRepairBackupPath() { return ""; } 16:	
	污染路径: expanded from macro 'false'	
209	缺陷发生位置: /contrib/TSZ/zstd/deprecated/zbuff_compress.c	行号: 147
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 145: *****/ 146: size_t ZBUFF_recommendedCInSize(void) { return ZSTD_CStreamInSize(); } 147: size_t ZBUFF_recommendedCOutSize(void) { return ZSTD_CStreamOutSize(); }	
	污染路径: 无	
210	缺陷发生位置: /contrib/TSZ/zstd/deprecated/zbuff_decompress.c	行号: 22
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int *'.	
	代码片段: 20: ZBUFF_DCtx* ZBUFF_createDCtx(void) 21: { 22: return ZSTD_createDStream(); 23: } 24:	
	污染路径: 无	
211	缺陷发生位置: /source/libs/decimal/test/decimalTest.cpp	行号: 273
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'double'.	
	代码片段: 271: DEFINE_REAL_OP(+); 272: DEFINE_REAL_OP(-); 273: DEFINE_REAL_OP(*); 274: DEFINE_REAL_OP(/); 275:	

	污染路径: expanded from macro 'DEFINE_REAL_OP'	
212	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 129
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 127: #endif 128: 129: MEM_STATIC unsigned MEM_32bits(void) { return sizeof(size_t)==4; } 130: MEM_STATIC unsigned MEM_64bits(void) { return sizeof(size_t)==8; } 131:	
	污染路径: 无	
213	缺陷发生位置: /source/client/src/clientSmlTelnet.c	行号: 48
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 46: smlBuildInvalidDataMsg(&info->msgBuf, 47: "timestamp precision can only be seconds(10 digits) or milli seconds(13 digits)", data); 48: return -1; 49: } 50: int64_t ts = smlGetTimeValue(data, len, fromPrecision, toPrecision);	
	污染路径: 无	
214	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 757
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 755: return (unsigned) r; 756: # elif defined(__GNUC__) && (__GNUC__ >= 3) /* Use GCC Intrinsic */ 757: return 31 - __builtin_clz (val); 758: # else /* Software version */ 759: static const unsigned DeBruijnClz[32] = { 0, 9, 1, 10,	

	13, 21, 2, 29, 11, 14, 16, 18, 22, 25, 3, 30, 8, 12, 20, 28, 15, 17, 24, 7, 19, 27, 23, 6, 26, 5, 4, 31 };	
	污染路径: 无	
215	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 513
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 511: TAOS_ROW row = taos_fetch_row(tres); 512: if (row == NULL) { 513: return 0; 514: } 515:	
	污染路径: 无	
216	缺陷发生位置: /source/util/src/tcompression.c	行号: 200
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned long', 定义类型为'int32_t'.	
	代码片段: 198: if (ret == Z_OK) { 199: output[0] = 1; 200: return dstLen + 1; 201: } else { 202: output[0] = 0;	
	污染路径: 无	
217	缺陷发生位置: /contrib/TSZ/zstd/common/bitstream.h	行号: 451
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 449: MEM_STATIC unsigned BIT_endOfDStream(const BIT_DStream_t* DStream) 450: { 451: return ((DStream->ptr == DStream->start) && (DStream->bitsConsumed == sizeof(DStream-> bitContainer)*8)); 452: }	

	453:	
	污染路径: 无	
218	缺陷发生位置: /source/os/src/osThread.c	行号: 763
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'pthread_t', 定义类型为'int'.	
	代码片段: 761: } 762: 763: TdThread taosThreadSelf(void) { return pthread_self(); } 764: 765: int32_t taosThreadSetCancelState(int32_t state, int32_t *oldstate) {	
	污染路径: 无	
219	缺陷发生位置: /contrib/TSZ/zstd/compress/hist.c	行号: 106
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 104: memset(count, 0, maxSymbolValue + 1); 105: *maxSymbolValuePtr = 0; 106: return 0; 107: } 108: if (!maxSymbolValue) maxSymbolValue = 255; /* 0 == default */	
	污染路径: 无	
220	缺陷发生位置: /source/libs/new-stream/src/dataSinkCache.c	行号: 140
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 138: bool shouldWriteSlidingGrpMemList(SSlidingGrpMgr* pSlidingGrpMgr) { 139: if (pSlidingGrpMgr->usedMemSize < (1 * 1024 * 1024) && g_slidigGrpMemList.waitMoveMemSize < DS_MEM_SIZE_RESERVED) { 140: return false;	

	141: } 142: int64_t size = taosHashGetSize(g_slidigGrpMemList.pSlidingGrpList);	
	污染路径: expanded from macro 'false'	
221	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 655
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned char'.	
	代码片段: 653: 654: fail: 655: return 0; 656: } 657:	
	污染路径: 无	
222	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqPacketDataTypes.c	行号: 192
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'.	
	代码片段: 190: return 1; 191: } else if (word < 16384) { 192: return 2; 193: } else if (word < 2097152) { 194: return 3;	
	污染路径: 无	
223	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 2645
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 2643: // fixed count value is max 2644: if (stbInfo->repeat_ts_max == 0){ 2645: return true; 2646: } 2647:	

	污染路径: expanded from macro 'true'	
224	缺陷发生位置: /source/os/src/osString.c	行号: 868
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'float'.	
	代码片段: 866: if (str == NULL) { 867: errno = TSDB_CODE_INVALID_PARA; 868: return 0; 869: } 870: float tmp = strtod(str, pEnd);	
	污染路径: 无	
225	缺陷发生位置: /source/os/src/osTime.c	行号: 503
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'time_t'.	
	代码片段: 501: return local - offset; 502: #else 503: return timegm(tmp); 504: #endif 505: }	
	污染路径: 无	
226	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 138
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 136: 137: MEM_STATIC unsigned MEM_32bits(void) { return sizeof(void*)==4; } 138: MEM_STATIC unsigned MEM_64bits(void) { return sizeof(void*)==8; } 139: 140: MEM_STATIC unsigned MEM_isLittleEndian(void)	
	污染路径: 无	

227	缺陷发生位置: /contrib/TSZ/zstd/compress/hist.c	行号: 44
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 42: 43: /* --- Error management --- */ 44: unsigned HIST_isError(size_t code) { return ERR_isError(code); } 45: 46: /*- *****	
	污染路径: 无	
228	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqProperty.c	行号: 233
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 231: 232: unsigned int property__get_length(const tmqtt_property *property) { 233: if (!property) return 0; 234: 235: switch (property->identifier) {	
	污染路径: 无	
229	缺陷发生位置: /source/libs/scalar/src/scifunc.c	行号: 481
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'uint8_t'.	
	代码片段: 479: return 1; 480: } else if ((str[0] & 0xE0) == 0xC0) { 481: return 2; 482: } else if ((str[0] & 0xF0) == 0xE0) { 483: return 3;	
	污染路径: 无	
230	缺陷发生位置:	行号:

	/tools/taos-tools/src/benchData.c	350
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 348: MAX_PATH_LEN - offset - len, 349: "%s", pos + 4); 350: return true; 351: } 352:	
	污染路径: expanded from macro 'true'	
231	缺陷发生位置: /source/util/src/tjson.c	行号: 438
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'const char *'.	
	代码片段: 436: } 437: 438: const char* tjsonGetError() { return cJSON_GetErrorPtr(); } 439: 440: void tjsonDeleteItemFromObject(const SJson* pjson, const char* pName) {	
	污染路径: 无	
232	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 108
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 106: #endif 107: 108: MEM_STATIC unsigned MEM_32bits(void) { return sizeof(void*)==4; } 109: MEM_STATIC unsigned MEM_64bits(void) { return sizeof(void*)==8; } 110:	
	污染路径: 无	
233	缺陷发生位置:	行号:

	/tools/taos-tools/src/toolstime.c		188
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.		
	代码片段: 186: int32_t totalLen = i; 187: if (totalLen <= 0) { 188: return -1; 189: } 190:		
	污染路径: 无		
234	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c		行号: 342
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'unsigned long'，定义类型为'int'.		
	代码片段: 340: } else { 341: pLastCol->colVal.value.val = pLastColV0->colVal.value.val; 342: return sizeof(SLastColV0); 343: } 344: }		
	污染路径: 无		
235	缺陷发生位置: /tools/taos-tools/src/benchCsv.c		行号: 95
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'long'.		
	代码片段: 93: endptr == csv_ts_interval 94: num <= 0) { 95: return -1; 96: } 97:		
	污染路径: 无		
236	缺陷发生位置: /contrib/TSZ/zstd/compress/hist.c		行号: 166
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'unsigned int'，定义类型为'size_t'.		

	代码片段: 164: { 165: if (sourceSize < 1500) /* heuristic threshold */ 166: return HIST_count_simple(count, maxSymbolValuePtr, source, sourceSize); 167: return HIST_count_parallel_wksp(count, maxSymbolValuePtr, source, sourceSize, 0, workSpace); 168: }	
	污染路径: 无	
237	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 218
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 216: static unsigned FSE_32bits(void) 217: { 218: return sizeof(void*)==4; 219: } 220:	
	污染路径: 无	
238	缺陷发生位置: /source/common/src/ttime.c	行号: 1816
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'long', 定义类型为'int'.	
	代码片段: 1814: const char *l, *r; 1815: for (l = fmt[0], r = s;; l++, r++) { 1816: if (*l == '\0') return fmt - arr; 1817: if (*r == '\0' tolower(*l) != tolower(*r)) break; 1818: }	
	污染路径: 无	
239	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 415
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 413: precision, 0)) {	

	<pre> 414: errorPrint("Input %s, time format error!\n", g_args.humanEndTime); 415: return -1; 416: } 417: } else { </pre>	
	污染路径: 无	
240	缺陷发生位置: /source/util/src/tcompression.c	行号: 270
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'unsigned long'，定义类型为'int32_t'.	
	代码片段: <pre> 268: } 269: output[0] = 1; 270: return len + 1; 271: } 272: int32_t l2DecompressImpl_xz(const char *const input, const int32_t compressedSize, char *const output, </pre>	
	污染路径: 无	
241	缺陷发生位置: /contrib/TSZ/zstd/compress/fse_compress.c	行号: 418
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: <pre> 416: if (count[s] > maxC) { maxV=s; maxC=count[s]; } 417: norm[maxV] += (short)ToDistribute; 418: return 0; 419: } 420: </pre>	
	污染路径: 无	
242	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 1834
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'long'，定义类型为'int'.	
	代码片段: <pre> 1832: } 1833: </pre>	

	1834: return orig - SA; 1835: } 1836:	
	污染路径: 无	
243	缺陷发生位置: /tools/taos-tools/src/dumpUtil.c	行号: 71
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 69: // rpc error 70: if (code >= TSDB_CODE_RPC_BEGIN && code <= TSDB_CODE_RPC_END) { 71: return true; 72: } 73:	
	污染路径: expanded from macro 'true'	
244	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 349
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned int'.	
	代码片段: 347: return (unsigned) r; 348: # elif defined(__GNUC__) && (GCC_VERSION >= 304) /* GCC Intrinsic */ 349: return 31 - __builtin_clz (val); 350: # else /* Software version */ 351: static const unsigned DeBruijnClz[32] = { 0, 9, 1, 10, 13, 21, 2, 29, 11, 14, 16, 18, 22, 25, 3, 30, 8, 12, 20, 28, 15, 17, 24, 7, 19, 27, 23, 6, 26, 5, 4, 31 };	
	污染路径: 无	
245	缺陷发生位置: /source/os/src/osString.c	行号: 859
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'double'.	
	代码片段: 857: if (str == NULL) { 858: terrno = TSDB_CODE_INVALID_PARA;	

	859: return 0; 860: } 861: double tmp = strtod(str, pEnd);	
	污染路径: 无	
246	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 1024
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: 1022: return ERROR(parameter_unsupported); 1023: } 1024: return 0; 1025: } 1026:	
	污染路径: 无	
247	缺陷发生位置: /source/libs/new-stream/src/dataSinkCache.c	行号: 155
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 153: return true; 154: } 155: return false; 156: } 157:	
	污染路径: expanded from macro 'false'	
248	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 159
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 157: // It also should make the function smaller, but no proper benchmark has 158: // been done. 159: return (a.hi < b.hi) ((a.hi == b.hi) & (a.lo < b.lo)); 160: } 161:	

	污染路径: 无	
249	缺陷发生位置: /source/libs/scalar/src/scifunc.c	行号: 483
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'uint8_t'.	
	代码片段: 481: return 2; 482: } else if ((str[0] & 0xF0) == 0xE0) { 483: return 3; 484: } else if ((str[0] & 0xF8) == 0xF0) { 485: return 4;	
	污染路径: 无	
250	缺陷发生位置: /source/os/src/osMemory.c	行号: 403
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'char *'.	
	代码片段: 401: #endif 402: 403: return tstrdup(ptr); 404: #endif 405: }	
	污染路径: 无	
251	缺陷发生位置: /contrib/TSZ/zstd/decompress/zstd_decompress.c	行号: 176
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 174: size_t ZSTD_sizeof_DCtx (const ZSTD_DCtx* dctx) 175: { 176: if (dctx==NULL) return 0; /* support sizeof NULL */ 177: return sizeof(*dctx) 178: + ZSTD_sizeof_DDict(dctx->ddictLocal)	
	污染路径: 无	
252	缺陷发生位置:	行号:

	/contrib/TSZ/zstd/dictBuilder/zdict.c	91
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 89: * Helper functions 90: *****/ 91: unsigned ZDICT_isError(size_t errorCode) { return ERR_isError(errorCode); } 92: 93: const char* ZDICT_getErrorName(size_t errorCode) { return ERR_getErrorName(errorCode); }	
	污染路径: 无	

16.2.1.3 WK4733 具有返回值的函数,其返回值如果不被使用, 调用是应有 void 说明

缺陷数量: 1690 个		
缺陷描述: 具有返回值的函数,其返回值如果不被使用,调用时应有 void 说明。		
修复意见: 无		
1	缺陷发生位置: /source/libs/sync/src/syncEnv.c	行号: 40
	步骤描述: 不使用返回值的函数'sError'应使用'void'说明.	
	代码片段: 38: gNodeRefId = taosOpenRef(200, (RefFp)syncNodeClose); 39: if (gNodeRefId < 0) { 40: sError("failed to init node rset"); 41: syncCleanUp(); 42: return TSDB_CODE_SYN_WRONG_REF;	
	污染路径: 函数'sError'的定义点.	
2	缺陷发生位置: /source/libs/metrics/test/metricsTest.cpp	行号: 244

	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 242: for (int32_t i = 200; i <= 205; i++) { 243: char dbname[32]; 244: snprintf(dbname, sizeof(dbname), "test_db_%d", i - 199); 245: bool exists = CheckVgroupMetricsExist(i, 123456789, 1, "localhost:6030", dbname); 246: ASSERT_TRUE(exists);	
	污染路径: 无	
3	缺陷发生位置: /source/dnode/mnode/impl/src/mndVgroup.c	行号: 449
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 447: if (code < 0) { 448: terrno = TSDB_CODE_APP_ERROR; 449: taosMemoryFree(pReq); 450: mError("vgId:%d, failed to serialize create vnode req,since %s", createReq.vgId, terrstr()); 451: return NULL;	
	污染路径: 函数'taosMemoryFree'的定义点.	
4	缺陷发生位置: /source/libs/transport/src/transCli.c	行号: 753
	步骤描述: 不使用返回值的函数'TAOS_UNUSED'应使用'void'说明.	
	代码片段: 751: cliMayUpdateFqdnCache(pThrd->fqdn2ipCache, conn->dstAddr); 752: tTrace("%s conn:%p, failed to connect %s since conn timeout", CONN_GET_INST_LABEL(conn), conn, conn->dstAddr); 753: TAOS_UNUSED(transUnrefCliHandle(conn)); 754: } 755:	
	污染路径: 函数'TAOS_UNUSED'的定义点.	

	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 172: r = taos_counter_delete(tsInsertCounter, keys[i]); 173: if (r) { 174: dError("failed to delete monitor sample key:%s", keys[i]); 175: } 176: }	
	污染路径: 函数'dError'的定义点.	
9	缺陷发生位置: /source/libs/monitorfw/src/taos_metric_formatter_custom.c	行号: 146
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 144: } 145: 146: taosMemoryFreeClear(arr); 147: taosMemoryFreeClear(keyvalue); 148: taosMemoryFreeClear(keyvalues);	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
10	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttCtx.c	行号: 293
	步骤描述: 不使用返回值的函数'bndInfo'应使用'void'说明.	
	代码片段: 291: } 292: 293: bndInfo("subscribed topic: %s.", topic_name); 294: 295: if (context->topic_list) {	
	污染路径: 函数'bndInfo'的定义点.	
11	缺陷发生位置: /source/libs/tmqtt/tools/src/topic-producer.c	行号: 100
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段:	

	<pre> 98: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x %x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 99: taos_errstr(NULL)); 100: taos_cleanup(); 101: return NULL; 102: }</pre>	
	污染路径: 函数'taos_cleanup'的定义点.	
12	缺陷发生位置: /source/libs/transport/src/thttp.c	行号: 908
	步骤描述: 不使用返回值的函数'tError'应使用'void'说明.	
	代码片段: <pre> 906: int err = uv_loop_init(http->loop); 907: if (err != 0) { 908: tError("http-report failed init uv, reason:%s", uv_strerror(err)); 909: code = TSDB_CODE_THIRDPARTY_ERROR; 910: goto _ERROR;</pre>	
	污染路径: 函数'tError'的定义点.	
13	缺陷发生位置: /docs/examples/c/with_reqid_demo.c	行号: 83
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 81: // close & clean 82: taos_close(taos); 83: taos_cleanup(); 84: return 0; 85: // ANCHOR_END: with_reqid</pre>	
	污染路径: 函数'taos_cleanup'的定义点.	
14	缺陷发生位置: /source/libs/new-stream/src/streamRunner.c	行号: 737
	步骤描述: 不使用返回值的函数'stDebugL'应使用'void'说明.	
	代码片段: <pre> 735: if (stDebugFlag & DEBUG_DEBUG) { 736: if (pBlock == NULL pBlock->info.rows == 0) {</pre>	

	737: stDebugL("output projection block is null or has no rows"); 738: return; 739: }	
	污染路径: 函数'stDebugL'的定义点.	
15	缺陷发生位置: /source/libs/executor/src/tsort.c	行号: 215
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 213: void* pTuple = createTuple(colNum, tupleLen); 214: if (!pTuple) { 215: taosMemoryFree(t); 216: return termno; 217: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
16	缺陷发生位置: /test/new_test_framework/script/api/stmt2-memleak.c	行号: 40
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 38: void createCtb(TAOS* taos, const char* tbname) { 39: char* tmp = (char*)malloc(sizeof(char) * 100); 40: sprintf(tmp, "create table db.%s using db.stb tags(0, 'after')", tbname); 41: do_query(taos, tmp); 42: }	
	污染路径: 无	
17	缺陷发生位置: /contrib/lemon/lemon.c	行号: 1029
	步骤描述: 不使用返回值的函数'SetAdd'应使用'void'说明.	
	代码片段: 1027: rp->lhsStart = 1; 1028: newcfp = Configlist_addbasis(rp,0); 1029: SetAdd(newcfp->fws,0); 1030: }	

	污染路径: 函数'computeRangeSize_float'的定义点.	
21	缺陷发生位置: /source/util/src/tunit.c	行号: 33
	步骤描述: 不使用返回值的函数'SET_ERRNO'应使用'void'说明.	
	代码片段: <pre> 31: int64_t int64Val = taosStr2Int64(str, &endPtr, 0); 32: if (*endPtr == '.' ERRNO == ERANGE) { 33: SET_ERRNO(0); 34: val = taosStr2Double(str, &endPtr); 35: useDouble = true; </pre>	
	污染路径: 函数'SET_ERRNO'的定义点.	
22	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1951
	步骤描述: 不使用返回值的函数'HUF_decodeStreamX2'应使用'void'说明.	
	代码片段: <pre> 1949: HUF_decodeStreamX2(op1, &bitD1, opStart2, dt, dtLog); 1950: HUF_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 1951: HUF_decodeStreamX2(op3, &bitD3, opStart4, dt, dtLog); 1952: HUF_decodeStreamX2(op4, &bitD4, oend, dt, dtLog); 1953: </pre>	
	污染路径: 函数'HUF_decodeStreamX2'的定义点.	
23	缺陷发生位置: /contrib/test/traft/join_into_vgroup/node_leader10002.c	行号: 67
	步骤描述: 不使用返回值的函数'printRaftState'应使用'void'说明.	
	代码片段: <pre> 65: struct raft *r = getRaft(&raftEnv, 100); 66: assert(r != NULL); 67: printRaftState(r); 68: 69: // submit value </pre>	

	污染路径: 函数'printRaftState'的定义点.	
24	缺陷发生位置: /examples/c/asyncdemo.c	行号: 127
	步骤描述: 不使用返回值的函数'gettimeofday'应使用'void'说明.	
	代码片段: <pre> 125: } 126: 127: gettimeofday(&systemTime, NULL); 128: for (i = 0; i < numOfTables; ++i) 129: tableList[i].timeStamp = (time_t)(systemTime.tv_sec) * 1000 + systemTime.tv_usec / 1000; </pre>	
	污染路径: 无	
25	缺陷发生位置: /utils/test/c/tmq_ts6115.c	行号: 141
	步骤描述: 不使用返回值的函数'create_topic'应使用'void'说明.	
	代码片段: <pre> 139: 140: pollStart = false; 141: create_topic(); 142: taosThreadCreate(&(thread1), &thattr, dropTopicThreadFunc, NULL); 143: taosThreadCreate(&(thread2), &thattr, consumeThreadFunc, NULL); </pre>	
	污染路径: 函数'create_topic'的定义点.	
26	缺陷发生位置: /source/os/src/osEnv.c	行号: 78
	步骤描述: 不使用返回值的函数'tstrncpy'应使用'void'说明.	
	代码片段: <pre> 76: tstrncpy(tsOsName, "Darwin", sizeof(tsOsName)); 77: #else 78: tstrncpy(tsOsName, "Linux", sizeof(tsOsName)); 79: #endif 80: if (configDir[0] == 0) { </pre>	
	污染路径:	

	函数'tstrncpy'的定义点.	
27	缺陷发生位置: /utils/test/c/tmq_td32526.c	行号: 184
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 182: taos_free_result(tmqmessage); 183: } else { 184: ASSERT(taos_errno(NULL) == 0); 185: break; 186: }	
	污染路径: 函数'ASSERT'的定义点.	
28	缺陷发生位置: /contrib/test/traft/join_into_vgroup/node_leader10002.c	行号: 139
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 137: // for test: submit value every second 138: while (1) { 139: sleep(1); 140: submitValue(); 141: }	
	污染路径: 无	
29	缺陷发生位置: /utils/test/c/tmq_td38404.c	行号: 52
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 50: char* result = tmq_get_json_meta(msg); 51: printf("meta result: %s\n", result); 52: ASSERT(strcmp(result, result1) == 0 strcmp(result, result2) == 0); 53: tmq_free_json_meta(result); 54: }	
	污染路径: 函数'ASSERT'的定义点.	
30	缺陷发生位置: /source/libs/new-stream/src/streamMgmt.c	行号: 837

	步骤描述: 不使用返回值的函数'streamReleaseTask'应使用'void'说明.	
	代码片段: 835: } 836: 837: streamReleaseTask(taskAddr); 838: 839: return code;	
	污染路径: 函数'streamReleaseTask'的定义点.	
31	缺陷发生位置: /source/client/src/clientEnv.c	行号: 83
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 81: pRequest->self = taosAddRef(clientReqRefPool, pRequest); 82: if (pRequest->self < 0) { 83: tscError("failed to add ref to request"); 84: code = terrno; 85: return code;	
32	污染路径: 函数'tscError'的定义点.	
	缺陷发生位置: /tools/taos-tools/src/benchMain.c	行号: 44
	步骤描述: 不使用返回值的函数'write'应使用'void'说明.	
33	代码片段: 42: if (sem_post(&g_arguments->cancelSem) != 0) { 43: const char* msg = "sem_post failed in signal handler\n"; 44: write(STDERR_FILENO, msg, strlen(msg)); 45: } 46:	
	污染路径: 无	
	缺陷发生位置: /source/libs/executor/src/aggregateoperator.c	行号: 180
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	

	代码片段: 178: cleanupExprSuppWithoutFilter(&pInfo->scalarExprSup); 179: cleanupGroupResInfo(&pInfo->groupResInfo); 180: taosMemoryFreeClear(param); 181: } 182:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
34	缺陷发生位置: /source/libs/metrics/test/metricsTest.cpp	行号: 459
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 457: // Verify metrics exist (check the last added one) 458: char dbname[32]; 459: snprintf(dbname, sizeof(dbname), "load_test_db_%d", iterations - 1); 460: bool exists = CheckVgroupMetricsExist(vgId, 123456789, 1, "localhost:6030", dbname); 461: ASSERT_TRUE(exists);	
	污染路径: 无	
35	缺陷发生位置: /source/libs/executor/src/groupcacheoperator.c	行号: 92
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 90: } 91: qDebug("%s", buf); 92: taosMemoryFree(buf); 93: } 94:	
	污染路径: 函数'taosMemoryFree'的定义点.	
36	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmWorker.c	行号: 362
	步骤描述: 不使用返回值的函数'streamReleaseTask'应使用'void'说明.	
	代码片段: 360: end:	

	361: taosArrayDestroy(pResList); 362: streamReleaseTask(taskAddr); 363: 364: STREAM_PRINT_LOG_END(code, lino);	
	污染路径: 函数'streamReleaseTask'的定义点.	
37	缺陷发生位置: /source/libs/executor/src/dataInserter.c	行号: 77
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 75: if (p == NULL) return; 76: SSubmitTbDataSendInfo* pSendInfo = p; 77: taosMemoryFree(pSendInfo->msg.pData); 78: taosMemoryFree(pSendInfo); 79: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
38	缺陷发生位置: /tools/taos-tools/src/benchCommandOpt.c	行号: 585
	步骤描述: 不使用返回值的函数'pthread_join'应使用'void'说明.	
	代码片段: 583: pthread_create(&read_id, NULL, queryNtableAggrFunc, pThreadInfo); 584: } 585: pthread_join(read_id, NULL); 586: // REST 587: if (REST_IFACE != g_arguments->iface) {	
	污染路径: 无	
39	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmlnt.c	行号: 743
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 741: taosHashInit(TSDB_MIN_VNODES, taosGetDefaultHashFunction(TSDB_DATA_TYPE_INT), true, HASH_ENTRY_LOCK); 742: if (pMgmt->runngingHash == NULL) {	

	743: dError("failed to init vnode hash since %s", terrstr()); 744: return TSDB_CODE_OUT_OF_MEMORY; 745: }	
	污染路径: 函数'dError'的定义点.	
40	缺陷发生位置: /source/common/src/msg/xnode.c	行号: 152
	步骤描述: 不使用返回值的函数'ENCODESQL'应使用'void'说明.	
	代码片段: 150: 151: TAOS_CHECK_EXIT(tStartEncode(&encoder)); 152: ENCODESQL(); 153: 154: TAOS_CHECK_EXIT(tEncodeI32(&encoder, pReq->urlLen));	
	污染路径: 函数'ENCODESQL'的定义点.	
41	缺陷发生位置: /source/libs/new-stream/src/streamCheckpoint.c	行号: 221
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 219: *dataLen = 0; 220: } 221: taosMemoryFree(encryptedData); 222: (void)taosCloseFile(&pFile); 223: STREAM_PRINT_LOG_END(code, lino);	
	污染路径: 函数'taosMemoryFree'的定义点.	
42	缺陷发生位置: /source/libs/new-stream/src/stream.c	行号: 66
	步骤描述: 不使用返回值的函数'streamTimerCleanUp'应使用'void'说明.	
	代码片段: 64: stInfo("stream cleanup start"); 65: stTriggerTaskEnvCleanup(); 66: streamTimerCleanUp(); 67: smUndeployAllTasks(); 68: destroyDataSinkMgr();	

	污染路径: 函数'streamTimerCleanUp'的定义点.	
43	缺陷发生位置: /source/dnode/mnode/impl/src/mndDnode.c	行号: 2049
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 2047: 2048: if (taosArrayPush(fqdns, &fqdn) == NULL) { 2049: mError("failed to fqdn into array, but continue at this time"); 2050: } 2051: sdbRelease(pSdb, pObj);	
	污染路径: 函数'mError'的定义点.	
44	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeModule.c	行号: 31
	步骤描述: 不使用返回值的函数'monInitVnode'应使用'void'说明.	
	代码片段: 29: TAOS_CHECK_RETURN(wallInit(stopDnodeFp)); 30: 31: monInitVnode(); 32: 33: return 0;	
	污染路径: 函数'monInitVnode'的定义点.	
45	缺陷发生位置: /utils/test/c/tmqSim.c	行号: 320
	步骤描述: 不使用返回值的函数'pPrint'应使用'void'说明.	
	代码片段: 318: #if 1 319: pPrint("%s configDir:%s %s", GREEN, configDir, NC); 320: pPrint("%s dbName:%s %s", GREEN, g_stConflInfo.dbName, NC); 321: pPrint("%s cdbName:%s %s", GREEN, g_stConflInfo.cdbName, NC); 322: pPrint("%s consumeDelay:%d %s", GREEN, g_stConflInfo.consumeDelay, NC);	

	污染路径: 函数'pPrint'的定义点.	
46	缺陷发生位置: /source/os/src/osFile.c	行号: 195
	步骤描述: 不使用返回值的函数'TAOS_SKIP_ERROR'应使用'void'说明.	
	代码片段: <pre> 193: if (pFileTo != NULL) TAOS_SKIP_ERROR(taosCloseFile(&pFileTo)); 194: /* coverity[+retval] */ 195: TAOS_SKIP_ERROR(taosRemoveFile(to)); 196: 197: terrno = code; </pre>	
	污染路径: 函数'TAOS_SKIP_ERROR'的定义点.	
47	缺陷发生位置: /docs/examples/c/connect_example.c	行号: 28
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 26: fprintf(stderr, "Failed to set user ip, ErrCode: 0x%x, ErrMessage: %s.\n", taos_errno(taos), taos_errstr(taos)); 27: taos_close(taos); 28: taos_cleanup(); 29: return -1; 30: } </pre>	
	污染路径: 函数'taos_cleanup'的定义点.	
48	缺陷发生位置: /source/libs/parser/src/parlInsertUtil.c	行号: 1280
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: <pre> 1278: pColSchema->name, tDataTypes[pColSchema->type].name, pColSchema->bytes, tDataTypes[*fields].name, 1279: *(int32_t*)(fields + sizeof(int8_t))); 1280: uError("checkSchema %s", buf); 1281: } 1282: return TSDB_CODE_INVALID_PARA; </pre>	

	污染路径: 函数'uError'的定义点.	
49	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqMux.c	行号: 135
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 133: int ttqMuxDelete(struct tmqtt *context) { 134: #ifndef WITH_EPOLL 135: UNUSED(context); 136: return -1; 137: #else	
	污染路径: 函数'UNUSED'的定义点.	
50	缺陷发生位置: /source/libs/executor/src/externalwindowoperator.c	行号: 190
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 188: } else if (pCtx->pCCtx) { 189: extWinDestroyCGrpCtx(pCtx->pCCtx); 190: taosMemoryFree(pCtx->pCCtx); 191: } 192: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
51	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeQuery.c	行号: 887
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 885: if (tDeserializeSVSubTablesReq(pMsg->pCont, pMsg->contLen, &req)) { 886: code = ternno; 887: qError("tDeserializeSVSubTablesReq failed"); 888: goto _return; 889: }	
	污染路径: 函数'qError'的定义点.	

52	缺陷发生位置： /contrib/libmqtt/ttq/src/ttqSessionExpiry.c	行号： 60
	步骤描述： 不使用返回值的函数'DL_INSERT_INORDER'应使用'void'说明.	
	代码片段： 58: context->expiry_list_item = item; 59: 60: DL_INSERT_INORDER(expiry_list, item, session_expiry_cmp); 61: 62: return TTQ_ERR_SUCCESS;	
	污染路径： 函数'DL_INSERT_INORDER'的定义点.	
53	缺陷发生位置： /source/libs/wal/test/walMetaTest.cpp	行号： 429
	步骤描述： 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段： 427: for (i = 0; i < 100; i++) { 428: char newStr[100]; 429: sprintf(newStr, "%s-%d", ranStr, i); 430: int len = strlen(newStr); 431: code = walAppendLog(pWal, i, 0, syncMeta, newStr, len, NULL);	
	污染路径： 无	
54	缺陷发生位置： /source/libs/transport/src/transSvr.c	行号： 317
	步骤描述： 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段： 315: } 316: taosHashCleanup(pWhiteList); 317: taosMemoryFree(pWhite); 318: } 319:	
	污染路径： 函数'taosMemoryFree'的定义点.	
55	缺陷发生位置： /source/libs/catalog/src/ctgRemote.c	行号： 790

	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 788: 789: ctgFreeBatch(&newBatch); 790: taosMemoryFree(msg); 791: 792: return code;	
	污染路径: 函数'taosMemoryFree'的定义点.	
56	缺陷发生位置: /test/new_test_framework/script/api/stmt2-performance.c	行号: 135
	步骤描述: 不使用返回值的函数'clock_gettime'应使用'void'说明.	
	代码片段: 133: } 134: 135: clock_gettime(CLOCK_MONOTONIC, &end); // 获取开始 时间 TAOS_STMT2_BINDV bindv; 136: bind_time += ((double)(end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9); 137:	
	污染路径: 无	
57	缺陷发生位置: /docs/examples/c/async_query_example.c	行号: 141
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 139: taos_free_result(res); 140: taos_close(_taos); 141: taos_cleanup(); 142: } 143: }	
	污染路径: 函数'taos_cleanup'的定义点.	
58	缺陷发生位置: /source/util/src/tcompare.c	行号: 1369
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	

	代码片段: 1367: void regexCacheFree(void *ppUsingRegex) { 1368: regfree(&*(UsingRegex **)ppUsingRegex)->pRegex); 1369: taosMemoryFree(*(UsingRegex **)ppUsingRegex); 1370: } 1371:	
	污染路径: 函数'taosMemoryFree'的定义点.	
59	缺陷发生位置: /source/util/src/tlockfree.c	行号: 77
	步骤描述: 不使用返回值的函数'taosMsleep'应使用'void'说明.	
	代码片段: 75: oLatch = atomic_load_32(pLatch); 76: if (oLatch == TD_RWLATCH_WRITE_FLAG) break; 77: taosMsleep(1); 78: } 79: }	
	污染路径: 函数'taosMsleep'的定义点.	
60	缺陷发生位置: /source/libs/executor/src/mergeoperator.c	行号: 576
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 574: } 575: 576: taosMemoryFreeClear(param); 577: } 578:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
61	缺陷发生位置: /source/libs/new-stream/src/dataSinkMgr.c	行号: 265
	步骤描述: 不使用返回值的函数'stError'应使用'void'说明.	
	代码片段: 263: } 264: if (code != 0) { 265: stError("failed to create stream task data sink	

	manager, cleanMode:%d, err: 0x%0x", cleanMode, code); 266: return code; 267: } else {	
	污染路径: 函数'stError'的定义点.	
62	缺陷发生位置: /source/dnode/mnode/impl/src/mndDnode.c	行号: 1343
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 1341: rsp.dnodeList = taosArrayInit(5, sizeof(SDNodeAddr)); 1342: if (NULL == rsp.dnodeList) { 1343: mError("failed to alloc epSet while process dnode list req"); 1344: code = terrno; 1345: goto _OVER;	
	污染路径: 函数'mError'的定义点.	
63	缺陷发生位置: /tools/shell/src/shellUtil.c	行号: 121
	步骤描述: 不使用返回值的函数'taosMsleep'应使用'void'说明.	
	代码片段: 119: fflush(stdout); 120: if (code == TSDB_SRV_STATUS_NETWORK_OK && shell.args.is_startup) { 121: taosMsleep(1000); 122: } else { 123: break;	
	污染路径: 函数'taosMsleep'的定义点.	
64	缺陷发生位置: /source/util/test/lrucacheTest.cpp	行号: 86
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 84: for (int32_t i = 0; i < 10; i++) { 85: char key[64]; 86: snprintf(key, sizeof(key), "key_%d", i); 87: LRUHandle* handle = taosLRUCacheLookup(cache1,	

	key, strlen(key)); 88: if (handle) {	
	污染路径: 无	
65	缺陷发生位置: /tools/lastqps-test/taos_query_a_simple.c	行号: 165
	步骤描述: 不使用返回值的函数'taos_query_a_func'应使用'void'说明.	
	代码片段: 163: if (atomic_load(¶m->current_query_nums) < param->max_query_nums) { 164: atomic_fetch_add(¶m->current_query_nums, 1); 165: taos_query_a_func(param->taos, param->sql, queryCallback, param); 166: } else { 167: usleep(50);	
	污染路径: 函数'taos_query_a_func'的定义点.	
66	缺陷发生位置: /source/util/src/tref.c	行号: 532
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 530: pSet->fp = NULL; 531: 532: taosMemoryFreeClear(pSet->nodeList); 533: taosMemoryFreeClear(pSet->lockedBy); 534:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
67	缺陷发生位置: /source/libs/wal/test/walMetaTest.cpp	行号: 482
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 480: do { 481: char newStr[100]; 482: sprintf(newStr, "%s-%d", ranStr, 0); 483: sprintf(newStr, "%s-%d", ranStr, 0);	

	484: int len = strlen(newStr);	
	污染路径: 无	
68	缺陷发生位置: /utils/test/c/tmq_taosx_ci.c	行号: 1225
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 1223: raw.raw_type = 2; 1224: int32_t code = tmq_write_raw((TAOS*)1, raw); 1225: ASSERT(code); 1226: const char* err = tmq_err2str(code); 1227: char* tmp = strstr(err, "Invalid parameters,detail:taos:");	
	污染路径: 函数'ASSERT'的定义点.	
69	缺陷发生位置: /source/util/src/tunit.c	行号: 30
	步骤描述: 不使用返回值的函数'SET_ERRNO'应使用'void'说明.	
	代码片段: 28: char* endPtr; 29: bool useDouble = false; 30: SET_ERRNO(0); 31: int64_t int64Val = taosStr2Int64(str, &endPtr, 0); 32: if (*endPtr == '.' ERRNO == ERANGE) {	
	污染路径: 函数'SET_ERRNO'的定义点.	
70	缺陷发生位置: /source/os/test/osTimeTests.cpp	行号: 389
	步骤描述: 不使用返回值的函数'unsetenv'应使用'void'说明.	
	代码片段: 387: std::cout << "Test completed with " << errors.load() << " errors" << std::endl; 388: 389: unsetenv("TZ"); 390: tzset(); 391: }	

	污染路径: 无	
71	缺陷发生位置: /source/libs/index/src/indexFstFile.c	行号: 34
	步骤描述: 不使用返回值的函数'TAOS_UNUSED'应使用'void'说明.	
	代码片段: 32: static void deleteDataBlockFromLRU(const void* key, size_t keyLen, void* value, void* ud) { 33: TAOS_UNUSED(ud); 34: TAOS_UNUSED(key); 35: TAOS_UNUSED(keyLen); 36: taosMemoryFree(value);	
	污染路径: 函数'TAOS_UNUSED'的定义点.	
72	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 2073
	步骤描述: 不使用返回值的函数'HUFv05_decodeStreamX2'应使用'void'说明.	
	代码片段: 2071: /* finish bitStreams one by one */ 2072: HUFv05_decodeStreamX2(op1, &bitD1, opStart2, dt, dtLog); 2073: HUFv05_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 2074: HUFv05_decodeStreamX2(op3, &bitD3, opStart4, dt, dtLog); 2075: HUFv05_decodeStreamX2(op4, &bitD4, oend, dt, dtLog);	
	污染路径: 函数'HUFv05_decodeStreamX2'的定义点.	
73	缺陷发生位置: /source/dnode/mnode/sdb/src/sdbFile.c	行号: 72
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 70: } 71: 72: mInfo("sdb upgrade success"); 73: return 0; 74: }	

	污染路径: 函数'mInfo'的定义点.	
74	缺陷发生位置: /source/libs/sync/src/syncSnapshot.c	行号: 63
	步骤描述: 不使用返回值的函数'TAOS_RETURN'应使用'void'说明.	
	代码片段: 61: (void)taosThreadMutexInit(&pBuf->mutex, NULL); 62: *ppBuf = pBuf; 63: TAOS_RETURN(0); 64: } 65:	
	污染路径: 函数'TAOS_RETURN'的定义点.	
75	缺陷发生位置: /contrib/test/traft/cluster/node10000_restart.c	行号: 108
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 106: // for test: submit a value every second 107: while (1) { 108: sleep(1); 109: submitValue(); 110: }	
	污染路径: 无	
76	缺陷发生位置: /source/common/src/msg/tmsg.c	行号: 1024
	步骤描述: 不使用返回值的函数'DECODESQL'应使用'void'说明.	
	代码片段: 1022: TAOS_CHECK_EXIT(tDecodeI64(&decoder, &pReq->suid)); 1023: 1024: DECODESQL(); 1025: 1026: tEndDecode(&decoder);	
	污染路径: 函数'DECODESQL'的定义点.	

77	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbDataFileRAW.c	行号: 106
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 104: tsdbError("vgId:%d %s failed at %s:%d since %s", TD_VID(config->tsdb->pVnode), __func__, __FILE__, lino, 105: tstrerror(code)); 106: taosMemoryFree(writer); 107: writer = NULL; 108: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
78	缺陷发生位置: /source/libs/scheduler/src/scheduler.c	行号: 29
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 27: int32_t schedulerInit() { 28: if (schMgmt.jobRef >= 0) { 29: qError("scheduler already initialized"); 30: return TSDB_CODE_QRY_INVALID_INPUT; 31: }	
	污染路径: 函数'qError'的定义点.	
79	缺陷发生位置: /source/dnode/mnode/impl/src/mndInfoSchema.c	行号: 127
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 125: STableMetaRsp *pMeta = taosHashGet(pMnode->infosMeta, tbName, strlen(tbName)); 126: if (NULL == pMeta) { 127: mError("invalid information schema table name:%s", tbName); 128: code = TSDB_CODE_PAR_TABLE_NOT_EXIST; 129: TAOS_RETURN(code);	
	污染路径: 函数'mError'的定义点.	
80	缺陷发生位置:	行号:

	/utils/test/c/tmqOffset.c	48
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 46: printf("subkey:%s, type:%d, uid/version:%"PRId64", ts: %"PRId64"\n", 47: offset.subKey, offset.val.type, offset.val.uid, offset.val.ts); 48: taosMemoryFree(memBuf); 49: } 50:	
	污染路径: 函数'taosMemoryFree'的定义点.	
81	缺陷发生位置: /source/client/src/clientFirewall.c	行号: 175
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 173: pRoot = cJSON_Parse(pBuf); 174: } 175: taosMemoryFree(pBuf); 176: } 177: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
82	缺陷发生位置: /source/client/wrapper/src/clientJniConnector.c	行号: 130
	步骤描述: 不使用返回值的函数'jniDebug'应使用'void'说明.	
	代码片段: 128: 129: atomic_store_32(&__init, 2); 130: jniDebug("native method register finished"); 131: } 132:	
	污染路径: 函数'jniDebug'的定义点.	
83	缺陷发生位置: /source/libs/executor/src/groupcacheoperator.c	行号: 91
	步骤描述:	

	不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: <pre> 89: offset += snprintf(buf + offset, bufSize - offset, " %" PRId64, pGrpCacheOperator- >execInfo.pDownstreamBlkNum[i]); 90: } 91: qDebug("%s", buf); 92: taosMemoryFree(buf); 93: }</pre>	
	污染路径: 函数'qDebug'的定义点.	
84	缺陷发生位置: /source/libs/index/src/indexCache.c	行号: 413
	步骤描述: 不使用返回值的函数'indexError'应使用'void'说明.	
	代码片段: <pre> 411: 412: if (taosThreadCondInit(&cache->finished, NULL) != 0) { 413: indexError("failed to create cond for index cache"); 414: taosMemoryFree(cache); 415: return NULL;</pre>	
	污染路径: 函数'indexError'的定义点.	
85	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 1998
	步骤描述: 不使用返回值的函数'HUFv07_decodeStreamX2'应使用'void'说明.	
	代码片段: <pre> 1996: 1997: /* finish bitStreams one by one */ 1998: HUFv07_decodeStreamX2(op1, &bitD1, opStart2, dt, dtLog); 1999: HUFv07_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 2000: HUFv07_decodeStreamX2(op3, &bitD3, opStart4, dt, dtLog);</pre>	
	污染路径: 函数'HUFv07_decodeStreamX2'的定义点.	
86	缺陷发生位置: /source/libs/new-stream/src/streamHb.c	行号: 116

	步骤描述: 不使用返回值的函数'stTrace'应使用'void'说明.	
	代码片段: 114: SEpSet epSet = {0}; 115: 116: stTrace("stream hb begin"); 117: 118: TAOS_CHECK_EXIT(streamHbBuildRequestMsg(&reqMsg, &skipHb));	
	污染路径: 函数'stTrace'的定义点.	
87	缺陷发生位置:	行号:
	/contrib/libmqtt/ttq/src/ttqSessionExpiry.c	111
	步骤描述: 不使用返回值的函数'ttqCxtDisconnect'应使用'void'说明.	
	代码片段: 109: context->will_delay_interval = 0; 110: // will_delay_remove(context); 111: ttqCxtDisconnect(context); 112: } 113: }	
88	污染路径: 函数'ttqCxtDisconnect'的定义点.	
	缺陷发生位置:	行号:
	/source/client/wrapper/src/wrapperFunc.c	97
	步骤描述: 不使用返回值的函数'taosDriverEnvInit'应使用'void'说明.	
89	代码片段: 95: 96: static void taos_init_driver_env(void) { 97: taosDriverEnvInit(); 98: } 99:	
	污染路径: 函数'taosDriverEnvInit'的定义点.	
	缺陷发生位置:	行号:
89	/source/libs/tdb/src/db/tdbPager.c	866
	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	

	代码片段: 864: code = tdbTbPopFreePage(pPager->pEnv->pFreeDb, pPgno, pTxn); 865: if (code) { 866: tdbError("tdb/remove-free-page: pop failed with ret: %d.", code); 867: } 868:	
	污染路径: 函数'tdbError'的定义点.	
90	缺陷发生位置: /source/dnode/mnode/sdb/src/sdbHash.c	行号: 293
	步骤描述: 不使用返回值的函数'mTrace'应使用'void'说明.	
	代码片段: 291: int32_t sdbWriteWithoutFree(SSdb *pSdb, SSdbRaw *pRaw) { 292: if (pRaw->type == SDB_CFG) { 293: mTrace("sdb write cfg"); 294: } 295: SHashObj *hash = sdbGetHash(pSdb, pRaw->type);	
	污染路径: 函数'mTrace'的定义点.	
91	缺陷发生位置: /source/dnode/bnode/src/bnode.c	行号: 47
	步骤描述: 不使用返回值的函数'bndInfo'应使用'void'说明.	
	代码片段: 45: } 46: 47: bndInfo("Bnode opened."); 48: 49: return TSDB_CODE_SUCCESS;	
	污染路径: 函数'bndInfo'的定义点.	
92	缺陷发生位置: /source/dnode/mnode/impl/src/mndCluster.c	行号: 475
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段:	

	473: 474: int32_t mndProcessConfigClusterRsp(SRpcMsg *pRsp) { 475: mInfo("config rsp from cluster"); 476: return 0; 477: }	
	污染路径: 函数'mInfo'的定义点.	
93	缺陷发生位置: /source/client/src/clientTmq.c	行号: 343
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 341: } 342: if (conf->token) { 343: taosMemoryFree(conf->token); 344: } 345: taosMemoryFree(conf);	
	污染路径: 函数'taosMemoryFree'的定义点.	
94	缺陷发生位置: /docs/examples/c-ws-new/tmq_demo.c	行号: 73
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 71: } 72: taos_free_result(pRes); 73: sleep(1); 74: } 75: fprintf(stdout, "Prepare data thread exit\n");	
	污染路径: 无	
95	缺陷发生位置: /source/libs/tdb/src/db/tdbPage.c	行号: 47
	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	
	代码片段: 45: 46: if (!xMalloc) { 47: tdbError("tdb/page-create: null xMalloc."); 48: return TSDB_CODE_INVALID_PARA;	

	49: }	
	污染路径: 函数'tdbError'的定义点.	
96	缺陷发生位置: /source/client/src/clientMonitor.c	行号: 146
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 144: SMsgSendInfo* pInfo = taosMemoryCalloc(1, sizeof(SMsgSendInfo)); 145: if (pInfo == NULL) { 146: tscError("sendReport failed, out of memory send info"); 147: code = terrno; 148: goto FAILED;	
	污染路径: 函数'tscError'的定义点.	
97	缺陷发生位置: /source/libs/new-stream/src/streamTriggerTask.c	行号: 5284
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 5282: code = stTriggerTaskParseCheckpoint(pTask, buf, len); 5283: if (code != TSDB_CODE_SUCCESS) { 5284: taosMemoryFree(buf); 5285: QUERY_CHECK_CODE(code, lino, _end); 5286: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
98	缺陷发生位置: /docs/examples/c/stmt2_insert_demo.c	行号: 293
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 291: insertData(taos); 292: taos_close(taos); 293: taos_cleanup(); 294: return 0; 295: }	

	污染路径: 函数'taos_cleanup'的定义点.	
99	缺陷发生位置: /source/libs/parser/src/parAstCreator.c	行号: 1092
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 1090: } 1091: 1092: taosMemoryFree(hint); 1093: return pHintList; 1094: _err:	
	污染路径: 函数'taosMemoryFree'的定义点.	
100	缺陷发生位置: /docs/examples/c/insert_data_demo.c	行号: 70
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 68: // close & clean 69: taos_close(taos); 70: taos_cleanup(); 71: return 0; 72: // ANCHOR_END: insert_data	
	污染路径: 函数'taos_cleanup'的定义点.	
101	缺陷发生位置: /source/libs/executor/src/timesliceoperator.c	行号: 1593
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 1591: taosVariantDestroy(&pInfo->pFillCollInfo[i].fillVal); 1592: } 1593: taosMemoryFree(pInfo->pFillCollInfo); 1594: } 1595: nodesDestroyNode(pInfo->pWin);	
	污染路径: 函数'taosMemoryFree'的定义点.	
102	缺陷发生位置:	行号:

	/source/common/src/cos_cp.c		12
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.		
	代码片段: <pre> 10: TdFilePtr fd = taosOpenFile(cp_path, TD_FILE_WRITE TD_FILE_CREATE /* TD_FILE_TRUNC*/ TD_FILE_WRITE_THROUGH); 11: if (!fd) { 12: uError("%s Failed to open %s", __func__, cp_path); 13: TAOS_CHECK_RETURN(terrno); 14: }</pre>		
	污染路径: 函数'uError'的定义点.		
103	缺陷发生位置: /source/common/src/msg/tmsg.c		行号: 834
	步骤描述: 不使用返回值的函数'ENCODESQL'应使用'void'说明.		
	代码片段: <pre> 832: TAOS_CHECK_EXIT(tEncodeI64(&encoder, pReq- >keep)); 833: 834: ENCODESQL(); 835: 836: TAOS_CHECK_EXIT(tEncodeI8(&encoder, pReq- >virtualStb));</pre>		
	污染路径: 函数'ENCODESQL'的定义点.		
104	缺陷发生位置: /source/libs/tdb/src/db/tdbBtree.c		行号: 124
	步骤描述: 不使用返回值的函数'tdbOsFree'应使用'void'说明.		
	代码片段: <pre> 122: if (ret < 0) { 123: tdbAbort(pEnv, txn); 124: tdbOsFree(pBt); 125: return ret; 126: }</pre>		
	污染路径: 函数'tdbOsFree'的定义点.		
105	缺陷发生位置:		行号:

	/source/dnode/mnode/impl/src/mndStreamTransAct.c	66
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 64: if (code != TSDB_CODE_SUCCESS) { 65: mError("failed to create stop all streams trans, code: %s", tstrerror(code)); 66: taosMemoryFree(pBuf); 67: } 68:	
	污染路径: 函数'taosMemoryFree'的定义点.	
106	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeAsync.c	行号: 222
	步骤描述: 不使用返回值的函数'setThreadName'应使用'void'说明.	
	代码片段: 220: } 221: 222: setThreadName(async->label); 223: 224: for (;;) {	
	污染路径: 函数'setThreadName'的定义点.	
107	缺陷发生位置: /source/libs/parser/src/parInsertSml.c	行号: 120
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 118: uError("%s failed at %d since %s", __func__, lino, tstrerror(code)); 119: } 120: taosMemoryFree(pUcs4); 121: return code; 122: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
108	缺陷发生位置: /source/libs/decimal/src/detail/intx/int128.hpp	行号: 190
	步骤描述:	

	不使用返回值的函数'operator--'应使用'void'说明.	
	代码片段: <pre> 188: { 189: auto ret = x; 190: --x; 191: return ret; 192: }</pre>	
	污染路径: 函数'operator--'的定义点.	
109	缺陷发生位置: /docs/examples/c-ws-new/with_reqid_demo.c	行号: 28
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 26: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x %x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 27: taos_errstr(NULL)); 28: taos_cleanup(); 29: return -1; 30: }</pre>	
	污染路径: 函数'taos_cleanup'的定义点.	
110	缺陷发生位置: /source/libs/tdb/test/tdbTest.cpp	行号: 155
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 153: 154: for (int iData = 1; iData <= nData; iData++) { 155: sprintf(key, "key%d", iData); 156: sprintf(val, "value%d", iData); 157: ret = tdbTbInsert(pDb, key, strlen(key), val, strlen(val), txn);</pre>	
	污染路径: 无	
111	缺陷发生位置: /source/libs/executor/src/externalwindowoperator.c	行号: 590
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段:	

	588: SMergeAlignedExternalWindowOperator* pMAExtW = (SMergeAlignedExternalWindowOperator*)pOperator; 589: destroyExternalWindowOperatorInfo(pMAExtW->pExtW); 590: taosMemoryFreeClear(pMAExtW); 591: } 592:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
112	缺陷发生位置: /source/client/src/clientEnv.c	行号: 400
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 398: int32_t code = taosVersionStrToInt(td_version, &rpclnit.compatibilityVer); 399: if (TSDB_CODE_SUCCESS != code) { 400: tscError("invalid version string."); 401: return code; 402: }	
	污染路径: 函数'tscError'的定义点.	
113	缺陷发生位置: /test/new_test_framework/script/api/insert_stb.c	行号: 30
	步骤描述: 不使用返回值的函数'gettimeofday'应使用'void'说明.	
	代码片段: 28: static int64_t currTimeInUs() { 29: struct timeval start_time; 30: gettimeofday(&start_time, NULL); 31: return (start_time.tv_sec) * 1000000 + (start_time.tv_usec); 32: }	
	污染路径: 无	
114	缺陷发生位置: /source/util/src/tdes.c	行号: 89
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段:	

	87: 88: char* decode = taosDesImp(keyStr, temp, len, DECRYPTION_MODE); 89: taosMemoryFree(temp); 90: 91: return decode;	
	污染路径: 函数'taosMemoryFree'的定义点.	
115	缺陷发生位置: /source/dnode/vnode/src/bse/bseMgt.c	行号: 469
	步骤描述: 不使用返回值的函数'bseClose'应使用'void'说明.	
	代码片段: 467: _err: 468: if (code != 0) { 469: bseClose(p); 470: bseError("vgld:%d failed to open bse at line %d since %s", BSE_VGID(p), lino, tstrerror(code)); 471: }	
	污染路径: 函数'bseClose'的定义点.	
116	缺陷发生位置: /source/dnode/mnode/impl/src/mndStreamErrorInjection.c	行号: 39
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 37: exit(-1); 38: } else if (pAction->msgType == TDMT_STREAM_CONSEN_CHKPT) { // pAction->msgType == TDMT_STREAM_CONSEN_CHKPT 39: mError(40: "***sleep 5s and core dump, following tasks will not recv consen-checkpoint info, so the tasks will " 41: "not started***");	
	污染路径: 函数'mError'的定义点.	
117	缺陷发生位置: /source/libs/new-stream/src/streamRunner.c	行号: 679
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	

	代码片段: 677: ST_TASK_ELOG("%s failed to get trigger calc params for index:%d, size:%d", __FUNCTION__, index, 678: (int32_t)pExec->runtimeInfo.funcInfo.pStreamPesudoFuncVals->size); 679: taosMemoryFreeClear(pContent); 680: return TSDB_CODE_MND_STREAM_INTERNAL_ERROR; 681: }	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
118	缺陷发生位置: /test/new_test_framework/script/api/stmt2-blob-performance.c	行号: 76
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 74: for (int i = 0; i < CTB_NUMS; i++) { 75: tbs[i] = (char*)malloc(sizeof(char) * 20); 76: sprintf(tbs[i], "ctb_%d", i); 77: createCtb(taos, tbs[i]); 78: }	
119	污染路径: 无	
	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncIndexMgrDebug.c	行号: 30
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
120	代码片段: 28: printf("syncIndexMgrPrint2 len:%" PRIu64 " %s %s \n", (uint64_t)strlen(serialized), s, serialized); 29: fflush(NULL); 30: taosMemoryFree(serialized); 31: } 32:	
	污染路径: 函数'taosMemoryFree'的定义点.	
	缺陷发生位置: /tools/shell/src/shellMain.c	行号: 130
	步骤描述: 不使用返回值的函数'taos_set_hb_quit'应使用'void'说明.	
	代码片段:	

	128: 129: // kill heart-beat thread when quit 130: taos_set_hb_quit(1); 131: 132: #ifndef TD_ASTR	
	污染路径: 函数'taos_set_hb_quit'的定义点.	
121	缺陷发生位置: /test/new_test_framework/script/api/stmt2-performance.c	行号: 36
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 34: void createCtb(TAOS* taos, const char* tname) { 35: char* tmp = (char*)malloc(sizeof(char) * 100); 36: sprintf(tmp, "create table db.%s using db.stb tags(0, 'after')", tname); 37: do_query(taos, tmp); 38: }	
	污染路径: 无	
122	缺陷发生位置: /source/libs/qworker/src/qwUtil.c	行号: 284
	步骤描述: 不使用返回值的函数'taosDisableMemPoolUsage'应使用'void'说明.	
	代码片段: 282: taosEnableMemPoolUsage(ctx->memPoolSession); 283: qDestroyTask(otaskHandle); 284: taosDisableMemPoolUsage(); 285: 286: (void)atomic_add_fetch_64(&qQueryMgmt.stat.taskExecDestroyNum, 1);	
	污染路径: 函数'taosDisableMemPoolUsage'的定义点.	
123	缺陷发生位置: /source/libs/catalog/src/ctgDbg.c	行号: 279
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 277: }	

	278: 279: qError("invalid debug option:%s", option); 280: 281: return TSDB_CODE_CTG_INTERNAL_ERROR;	
	污染路径: 函数'qError'的定义点.	
124	缺陷发生位置: /source/dnode/mnode/impl/src/mndArbGroup.c	行号: 447
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 445: char arbToken[TSDB_ARB_TOKEN_SIZE]; 446: if ((code = mndGetArbToken(pMnode, arbToken)) != 0) { 447: mError("arbgroup:0, failed to get arb token for arb-hb timer"); 448: plter = taosHashIterate(pDnodeHash, NULL); 449: while (plter) {	
	污染路径: 函数'mError'的定义点.	
125	缺陷发生位置: /source/dnode/vnode/src/bse/bseSnapshot.c	行号: 206
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 204: taosArrayDestroy(p->pFileSet); 205: bseRawFileWriterClose(p->pWriter, 0); 206: taosMemoryFree(p); 207: } 208:	
	污染路径: 函数'taosMemoryFree'的定义点.	
126	缺陷发生位置: /source/libs/metrics/test/metricsTest.cpp	行号: 236
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 234: 235: char dbname[32]; 236: snprintf(dbname, sizeof(dbname), "test_db_%d", i -	

	199); 237: int32_t code = addWriteMetrics(i, 1, 123456789, "localhost:6030", dbname, &rawMetrics); 238: ASSERT_EQ(code, TSDB_CODE_SUCCESS);	
	污染路径: 无	
127	缺陷发生位置: /source/os/src/osEnv.c	行号: 50
	步骤描述: 不使用返回值的函数'taosSeedRand'应使用'void'说明.	
	代码片段: 48: int32_t code = TSDB_CODE_SUCCESS; 49: 50: taosSeedRand(taosSafeRand()); 51: taosGetSystemLocale(tsLocale, tsCharset); 52: (void)taosGetSystemTimezone(tsTimezoneStr);	
	污染路径: 函数'taosSeedRand'的定义点.	
128	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRetention.c	行号: 427
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 425: TARRAY2_DESTROY(&rtner.fopArr, NULL); 426: (void)tsdbRemoveRetentionMonitorTask(pTsdB, &rtArg->taskid); 427: taosMemoryFree(arg); 428: if (code) { 429: tsdbError("vgId:%d %s failed at %s:%d since %s", TD_VID(pTsdB->pVnode), __func__, __FILE__, lino, tstrerror(code));	
	污染路径: 函数'taosMemoryFree'的定义点.	
129	缺陷发生位置: /source/dnode/mnode/impl/src/mndMain.c	行号: 105
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 103: 104: if (tSerializeSMTimerMsg(pReq, contLen, &timerReq) <	

	<pre> 0) { 105: mError("failed to serialize timer msg since %s", terrstr()); 106: } 107: *pContLen = contLen; </pre>	
	污染路径: 函数'mError'的定义点.	
130	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttLogging.c	行号: 325
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: <pre> 323: va_end(va); 324: 325: UNUSED(rc); 326: } 327: </pre>	
	污染路径: 函数'UNUSED'的定义点.	
131	缺陷发生位置: /docs/examples/c-ws-new/sml_insert_demo.c	行号: 70
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 68: code, taos_errstr(result)); 69: taos_close(taos); 70: taos_cleanup(); 71: return -1; 72: } </pre>	
	污染路径: 函数'taos_cleanup'的定义点.	
132	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeSimpleTest.cpp	行号: 223
	步骤描述: 不使用返回值的函数'sdbSetRawInt64'应使用'void'说明.	
	代码片段: <pre> 221: sdbSetRawInt32(pRaw, dataPos, obj.statusLen); 222: dataPos += 4; 223: sdbSetRawInt64(pRaw, dataPos, obj.createTime); 224: dataPos += 8; </pre>	

	225: sdbSetRawInt64(pRaw, dataPos, obj.updateTime);	
	污染路径: 函数'sdbSetRawInt64'的定义点.	
133	缺陷发生位置: /source/libs/new-stream/src/streamUtil.c	行号: 789
	步骤描述: 不使用返回值的函数'cJSON_free'应使用'void'说明.	
	代码片段: 787: _end: 788: if (temp != NULL) { 789: cJSON_free(temp); 790: } 791: if (obj != NULL) {	
	污染路径: 函数'cJSON_free'的定义点.	
134	缺陷发生位置: /source/libs/executor/src/operator.c	行号: 190
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 188: 189: if (pOperator == NULL) { 190: qError("invalid operator, failed to find tableScanOperator %s", id); 191: return TSDB_CODE_STREAM_INTERNAL_ERROR; 192: }	
	污染路径: 函数'qError'的定义点.	
135	缺陷发生位置: /source/util/test/decompressTest.cpp	行号: 531
	步骤描述: 不使用返回值的函数'origData'应使用'void'说明.	
	代码片段: 529: ONE_STAGE_COMP, nullptr, 0); 530: ASSERT_EQ(cnt, compData.size() - 1); 531: EXPECT_EQ(origData, decompData); 532: 533: taosGetSystemInfo();	
	污染路径:	

	无	
136	缺陷发生位置： /source/libs/executor/src/tsort.c	行号： 347
	步骤描述： 不使用返回值的函数'tsortDestroySortHandle'应使用'void'说明.	
	代码片段： 345: qError("%s failed at line %d since %s", __func__, lino, tstrerror(code)); 346: if (pSortHandle) { 347: tsortDestroySortHandle(pSortHandle); 348: } 349: return code;	
	污染路径： 函数'tsortDestroySortHandle'的定义点.	
137	缺陷发生位置： /utils/test/c/sml_test.c	行号： 126
	步骤描述： 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段： 124: printf("%s result:%s\n", __FUNCTION__, taos_errstr(pRes)); 125: int code = taos_errno(pRes); 126: ASSERT(code == 0); 127: taos_free_result(pRes); 128:	
	污染路径： 函数'ASSERT'的定义点.	
138	缺陷发生位置： /source/dnode/mnode/impl/src/mndSubscribe.c	行号： 206
	步骤描述： 不使用返回值的函数'MND_TMQ_NULL_CHECK'应使用'void'说明.	
	代码片段： 204: tlen += sizeof(SMsgHead); 205: buf = taosMemoryMalloc(tlen); 206: MND_TMQ_NULL_CHECK(buf); 207: SMsgHead *pMsgHead = (SMsgHead *)buf; 208: pMsgHead->contLen = htonl(tlen);	
	污染路径： 函数'MND_TMQ_NULL_CHECK'的定义点.	
139	缺陷发生位置：	行号：

	/source/libs/sync/test/sync_test_lib/src/syncRaftStoreDebug.c	72
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 70: printf("raftStorePrint2 len:%d %s %s \n", (int32_t)strlen(serialized), s, serialized); 71: fflush(NULL); 72: taosMemoryFree(serialized); 73: } 74: void raftStoreLog(SRaftStore *pObj) { </pre>	
140	污染路径: 函数'taosMemoryFree'的定义点.	
	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 578
	步骤描述: 不使用返回值的函数'pw'应使用'void'说明.	
141	代码片段: <pre> 576: k = t.hi; 577: } 578: pw[num_words - 1] += uw[num_words - j - 1] * vw[j] + k; 579: } 580: return p; </pre>	
	污染路径: 无	
	缺陷发生位置: /source/libs/transport/src/transComm.c	行号: 175
142	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 173: pHead->comp = 0; 174: } 175: taosMemoryFree(buf); 176: 177: int64_t elapse = taosGetTimestampMs() - start; </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
142	缺陷发生位置: /source/dnode/mgmt/mgmt_qnode/src/qmlnt.c	行号: 67
	步骤描述:	

	不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 65: 66: if ((code = udfcOpen()) != 0) { 67: dError("qnode can not open udfc"); 68: qmClose(pMgmt); 69: return code;	
	污染路径: 函数'dError'的定义点.	
143	缺陷发生位置: /source/dnode/mgmt/mgmt_dnode/src/dmHandle.c	行号: 145
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: 143: contLen = tSerializeRetrieveWhiteListReq(pHead, contLen, &req); 144: if (contLen < 0) { 145: rpcFreeCont(pHead); 146: dError("failed to serialize datetime white list request since:%s", tstrerror(contLen)); 147: return;	
	污染路径: 函数'rpcFreeCont'的定义点.	
144	缺陷发生位置: /source/libs/executor/src/groupoperator.c	行号: 88
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 86: 87: cleanupBasicInfo(&pInfo->binfo); 88: taosMemoryFreeClear(pInfo->keyBuf); 89: taosArrayDestroy(pInfo->pGroupCols); 90: taosArrayDestroyEx(pInfo->pGroupColVals, freeGroupKey);	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
145	缺陷发生位置: /source/dnode/snode/src/snode.c	行号: 265
	步骤描述: 不使用返回值的函数'streamReleaseTask'应使用'void'说明.	

	taosMemoryMalloc(sizeof(SStmQNode)); 339: if (NULL == pNode) { 340: taosMemoryFreeClear(param); 341: mstsError("%s failed at line %d, error:%s", __FUNCTION__, __LINE__, tstrerror(errno)); 342: return;	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
149	缺陷发生位置: /test/new_test_framework/script/api/whiteListTest.c	行号: 140
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 138: dropUsers(taos); 139: taos_close(taos); 140: taos_cleanup(); 141: } 142:	
	污染路径: 函数'taos_cleanup'的定义点.	
150	缺陷发生位置: /utils/test/c/tmq_td37436.c	行号: 36
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 34: char* result = tmq_get_json_meta(msg); 35: printf("meta result: %s\n", result); 36: ASSERT(strcmp(result, result1) == 0 strcmp(result, result2) == 0); 37: tmq_free_json_meta(result); 38: } else {	
	污染路径: 函数'ASSERT'的定义点.	
151	缺陷发生位置: /source/util/test/timerTest.cpp	行号: 212
	步骤描述: 不使用返回值的函数'adjustSystemTime'应使用'void'说明.	
	代码片段: 210: 211: // Restore time	

	212: adjustSystemTime(-3600LL * 1000); 213: 214: taosTmrCleanUp(ctrl);	
	污染路径: 函数'adjustSystemTime'的定义点.	
152	缺陷发生位置: /source/dnode/mnode/impl/src/mndBnode.c	行号: 224
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 222: 223: if ((code = mndTransAppendUndoAction(pTrans, &action)) != 0) { 224: taosMemoryFree(pReq); 225: TAOS_RETURN(code); 226: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
153	缺陷发生位置: /test/new_test_framework/script/api/stmtBatchTest.c	行号: 225
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 223: 224: taosMemoryFree(v->ts); 225: taosMemoryFree(v->br); 226: taosMemoryFree(v->nr); 227: taosMemoryFree(v);	
	污染路径: 函数'taosMemoryFree'的定义点.	
154	缺陷发生位置: /test/new_test_framework/script/api/apitest.c	行号: 101
	步骤描述: 不使用返回值的函数'taos_print_row'应使用'void'说明.	
	代码片段: 99: while ((row = taos_fetch_row(res))) { 100: char temp[256] = {0}; 101: taos_print_row(temp, row, fields, num_fields); 102: puts(temp); 103: nRows++;	

	污染路径: 无	
158	缺陷发生位置: /source/libs/tdb/src/db/tdbPager.c	行号: 989
	步骤描述: 不使用返回值的函数'tmemory_barrier'应使用'void'说明.	
	代码片段: 987: } 988: 989: tmemory_barrier(); 990: 991: pPage->pPager = pPager;	
	污染路径: 函数'tmemory_barrier'的定义点.	
159	缺陷发生位置: /source/dnode/mnode/impl/src/mndBnode.c	行号: 372
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 370: 371: if ((code = mndTransAppendRedoAction(pTrans, &action)) != 0) { 372: taosMemoryFree(pReq); 373: TAOS_RETURN(code); 374: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
160	缺陷发生位置: /source/libs/nodes/src/nodesUtilFuncs.c	行号: 157
	步骤描述: 不使用返回值的函数'taosMemFreeClear'应使用'void'说明.	
	代码片段: 155: int32_t code = callocNodeChunk(g_pNodeAllocator, NULL); 156: if (TSDB_CODE_SUCCESS != code) { 157: taosMemFreeClear(*pOut); 158: return code; 159: }	
	污染路径: 函数'taosMemFreeClear'的定义点.	

161	缺陷发生位置: /source/dnode/mnode/impl/src/mndXnode.c	行号: 1103
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 1101: } 1102: if (pContStr != NULL) { 1103: taosMemoryFree(pContStr); 1104: } 1105: if (pJson != NULL) {	
	污染路径: 函数'taosMemoryFree'的定义点.	
162	缺陷发生位置: /tools/taos-tools/src/benchQuery.c	行号: 270
	步骤描述: 不使用返回值的函数'autoSleep'应使用'void'说明.	
	代码片段: 268: // sleep 269: if (interval > 0) { 270: autoSleep(interval, et - st); 271: } 272:	
	污染路径: 函数'autoSleep'的定义点.	
163	缺陷发生位置: /source/client/src/clientMsgHandler.c	行号: 169
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 167: if (taosHashPut(appInfo.pInstMapByClusterId, &connectRsp.clusterId, LONG_BYTES, &pTscObj->pAppInfo, 168: POINTER_BYTES) != 0) { 169: tscError("failed to put appInfo into appInfo.pInstMapByClusterId"); 170: } else { 171: #ifdef USE_MONITOR	
	污染路径: 函数'tscError'的定义点.	
164	缺陷发生位置: /source/libs/catalog/src/ctgDbg.c	行号: 579

	步骤描述: 不使用返回值的函数'CTG_API_ENTER'应使用'void'说明.	
	代码片段: 577: } 578: 579: CTG_API_ENTER(); 580: 581: SCtgCacheStat cache;	
	污染路径: 函数'CTG_API_ENTER'的定义点.	
165	缺陷发生位置: /source/libs/executor/src/tlinearhash.c	行号: 281
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 279: terrno = TSDB_CODE_NO_DISKSPACE; 280: (void) printf("tHash Init failed since %s, tempDir:%s", terrstr(), tsTempDir); 281: taosMemoryFree(pHashObj); 282: return NULL; 283: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
166	缺陷发生位置: /source/dnode/mnode/impl/src/mndXnode.c	行号: 791
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 789: mError("xnode:%s, failed to create since %s, line:%d", createReq.url, tstrerror(code), lino); 790: } 791: taosMemoryFreeClear(pToken); 792: mndReleaseXnode(pMnode, pObj); 793: tFreeSMCreateXnodeReq(&createReq);	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
167	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmMgmt.c	行号: 47
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	

	代码片段: 45: 46: int32_t dmlnitDnode(SDnode *pDnode) { 47: dDebug("start to create dnode"); 48: int32_t code = -1; 49: char path[PATH_MAX + 100] = {0};	
	污染路径: 函数'dDebug'的定义点.	
168	缺陷发生位置: /source/libs/index/src/indexTfile.c	行号: 349
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 347: 348: if (hasjson) { 349: taosMemoryFree(p); 350: } 351: fstSliceDestroy(&key);	
	污染路径: 函数'taosMemoryFree'的定义点.	
169	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/gnu-test-skeleton.c	行号: 203
	步骤描述: 不使用返回值的函数'mallopt'应使用'void'说明.	
	代码片段: 201: 202: /* Make uses of freed and uninitialized memory known. */ 203: mallopt (M_PERTURB, 42); 204: 205: #ifdef STDOUT_UNBUFFERED	
	污染路径: 无	
170	缺陷发生位置: /source/dnode/mnode/impl/src/mndMain.c	行号: 112
	步骤描述: 不使用返回值的函数'mTrace'应使用'void'说明.	
	代码片段: 110:	

	111: static void mndPullupTrans(SMnode *pMnode) { 112: mTrace("pullup trans msg"); 113: int32_t contLen = 0; 114: void *pReq = mndBuildTimerMsg(&contLen);	
	污染路径: 函数'mTrace'的定义点.	
171	缺陷发生位置: /docs/examples/c-ws-new/async_demo.c	行号: 96
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 94: printf("success to connect to server\n"); 95: 96: sprintf(sql, "drop database if exists %s", db); 97: queryDB(taos, sql); 98:	
	污染路径: 无	
172	缺陷发生位置: /source/util/test/trefTest.c	行号: 85
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 83: } 84: 85: void myfree(void *p) { taosMemoryFree(p); } 86: 87: void *openRefSpace(void *param) {	
	污染路径: 函数'taosMemoryFree'的定义点.	
173	缺陷发生位置: /source/dnode/mgmt/exe/dmMain.c	行号: 175
	步骤描述: 不使用返回值的函数'dlInfo'应使用'void'说明.	
	代码片段: 173: } 174: #endif 175: dlInfo("shut down signal is %d", signum); 176: #if !defined(WINDOWS) && !defined(TD_ASTR)A 177: if (sigInfo != NULL) {	

	污染路径: 函数'dlInfo'的定义点.	
174	缺陷发生位置: /docs/examples/c/sml_insert_demo.c	行号: 134
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 132: // close & clean 133: taos_close(taos); 134: taos_cleanup(); 135: return 0; 136: // ANCHOR_END: schemaless	
	污染路径: 函数'taos_cleanup'的定义点.	
175	缺陷发生位置: /source/libs/executor/src/groupoperator.c	行号: 1267
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 1265: destroyDiskbasedBuf(plInfo->pBuf); 1266: taosArrayDestroy(plInfo->pOrderInfoArr); 1267: taosMemoryFreeClear(param); 1268: } 1269:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
176	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqSessionExpiry.c	行号: 105
	步骤描述: 不使用返回值的函数'DL_FOREACH_SAFE'应使用'void'说明.	
	代码片段: 103: struct tmqtt *context; 104: 105: DL_FOREACH_SAFE(expiry_list, item, tmp) { 106: context = item->context; 107: ttqSessionExpiryRemove(context);	
	污染路径: 函数'DL_FOREACH_SAFE'的定义点.	
177	缺陷发生位置:	行号:

	/contrib/TSZ/zstd/legacy/zstd_v02.c		1790
	步骤描述: 不使用返回值的函数'HUF_decodeStreamX2'应使用'void'说明.		
	代码片段: 1788: HUF_decodeStreamX2(op1, &bitD1, opStart2, dt, dtLog); 1789: HUF_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 1790: HUF_decodeStreamX2(op3, &bitD3, opStart4, dt, dtLog); 1791: HUF_decodeStreamX2(op4, &bitD4, oend, dt, dtLog); 1792:		
	污染路径: 函数'HUF_decodeStreamX2'的定义点.		
178	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-fmtstream.c		行号: 354
	步骤描述: 不使用返回值的函数'putc'应使用'void'说明.		
	代码片段: 352: if (nl > fs->buf) 353: fwrite (fs->buf, 1, nl - fs->buf, fs->stream); 354: putc ('\n', fs->stream); 355: 356: len += buf - fs->buf;		
	污染路径: 无		
179	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeStream.c		行号: 259
	步骤描述: 不使用返回值的函数'STREAM_PRINT_LOG_END'应使用'void'说明.		
	代码片段: 257: nodesDestroyList(groupNew); 258: qStreamDestroyTableList(pTableListInfo); 259: STREAM_PRINT_LOG_END(code, lino); 260: return code; 261: }		
	污染路径: 函数'STREAM_PRINT_LOG_END'的定义点.		

180	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSnapshot.c	行号: 464
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 462: __func__, __FILE__, lineno, tstrerror(code), sver, ever, type); 463: tsdbTFileSetRangeArrayDestroy(&reader[0]->fsrArr); 464: taosMemoryFree(reader[0]); 465: reader[0] = NULL; 466: } else {	
	污染路径: 函数'taosMemoryFree'的定义点.	
181	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1952
	步骤描述: 不使用返回值的函数'HUF_decodeStreamX2'应使用'void'说明.	
	代码片段: 1950: HUF_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 1951: HUF_decodeStreamX2(op3, &bitD3, opStart4, dt, dtLog); 1952: HUF_decodeStreamX2(op4, &bitD4, oend, dt, dtLog); 1953: 1954: /* check */	
	污染路径: 函数'HUF_decodeStreamX2'的定义点.	
182	缺陷发生位置: /source/dnode/mnode/impl/src/mndTrans.c	行号: 889
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 887: pTrans->pRpcArray == NULL) { 888: terno = TSDB_CODE_OUT_OF_MEMORY; 889: mError("failed to create transaction since %s", terrstr()); 890: mndTransDrop(pTrans); 891: return NULL;	
	污染路径:	

	函数'mError'的定义点.	
183	缺陷发生位置: /source/libs/function/src/detail/tminmaxavx.c	行号: 119
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 117: return TSDB_CODE_SUCCESS; 118: #else 119: uError("unable run %s without avx2 instructions", __func__); 120: return TSDB_CODE_OPS_NOT_SUPPORT; 121: #endif	
	污染路径: 函数'uError'的定义点.	
184	缺陷发生位置: /examples/c/demo.c	行号: 108
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 106: 107: // query the records 108: sprintf(qstr, "SELECT * FROM m1"); 109: result = taos_query(taos, qstr); 110: if (result == NULL taos_errno(result) != 0) {	
	污染路径: 无	
185	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 948
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 946: // if malloc need free 947: if (tmp->free && tmp->word) taosMemoryFree(tmp->word); 948: taosMemoryFree(tmp); 949: } 950: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
186	缺陷发生位置:	行号:

	/source/dnode/mnode/impl/src/mndShow.c	282
	步骤描述: 不使用返回值的函数'mDebug'应使用'void'说明.	
	代码片段: 280: int32_t size = 0; 281: int32_t rowsRead = 0; 282: mDebug("mndProcessRetrieveSysTableReq start"); 283: SRetrieveTableReq retrieveReq = {0}; 284: TAOS_CHECK_RETURN(tDeserializeSRetrieveTableReq(pReq->pCont, pReq->contLen, &retrieveReq));	
	污染路径: 函数'mDebug'的定义点.	
187	缺陷发生位置: /source/libs/tdb/test/tdbTest.cpp	行号: 182
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 180: for (int i = 1; i <= nData; i++) { 181: sprintf(key, "key%d", i); 182: sprintf(val, "value%d", i); 183: 184: ret = tdbTbGet(pDb, key, strlen(key), &pVal, &vLen);	
	污染路径: 无	
188	缺陷发生位置: /source/dnode/mnode/impl/src/mndScan.c	行号: 438
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 436: 437: if ((code = mndTransAppendRedoAction(pTrans, &action)) != 0) { 438: taosMemoryFree(pReq); 439: TAOS_RETURN(code); 440: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
189	缺陷发生位置:	行号:

	/contrib/libmqtt/ttq/src/ttqNet.c	972
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 970: return ttq->ssl; 971: #else 972: UNUSED(ttq); 973: return NULL; 974: #endif	
	污染路径: 函数'UNUSED'的定义点.	
190	缺陷发生位置: /source/dnode/mnode/impl/src/mndInstance.c	行号: 121
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 119: code = sdbWrite(pMnode->pSdb, pRaw); 120: if (code != TSDB_CODE_SUCCESS) { 121: mError("instance:%s, failed to upsert into sdb since %s", id, tstrerror(code)); 122: } 123: return code;	
	污染路径: 函数'mError'的定义点.	
191	缺陷发生位置: /source/dnode/vnode/src/meta/metaSma.c	行号: 126
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 124: } 125: 126: taosMemoryFree(pVal); 127: return 0; 128:	
	污染路径: 函数'taosMemoryFree'的定义点.	
192	缺陷发生位置: /source/dnode/mgmt/mgmt_qnode/src/qmWorker.c	行号: 143
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	

	代码片段: 141: tSingleWorkerCleanup(&pMgmt->queryWorker); 142: tSingleWorkerCleanup(&pMgmt->fetchWorker); 143: dDebug("qnode workers are closed"); 144: }	
	污染路径: 函数'dDebug'的定义点.	
193	缺陷发生位置: /source/libs/transport/src/tmsgcb.c	行号: 64
	步骤描述: 不使用返回值的函数'tError'应使用'void'说明.	
	代码片段: 62: #if 1 63: if (rpcSendResponse(pMsg) != 0) { 64: tError("failed to send response"); 65: } 66: #else	
	污染路径: 函数'tError'的定义点.	
194	缺陷发生位置: /source/libs/planner/src/planOptimizer.c	行号: 543
	步骤描述: 不使用返回值的函数'planError'应使用'void'说明.	
	代码片段: 541: _return: 542: if (code) { 543: planError("%s failed since %s", __func__, tstrerror(code)); 544: } 545: nodesDestroyList(info.pSdrFuncs);	
	污染路径: 函数'planError'的定义点.	
195	缺陷发生位置: /test/new_test_framework/script/api/stmtQuery.c	行号: 126
	步骤描述: 不使用返回值的函数'taos_stmt_close'应使用'void'说明.	
	代码片段: 124: 125: 126: taos_stmt_close(stmt);	

	127: 128: return 0;	
	污染路径: 函数'taos_stmt_close'的定义点.	
196	缺陷发生位置: /source/libs/function/src/builtins.c	行号: 593
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 591: if (factor->valuedouble < 0 factor->valuedouble == 0 factor->valuedouble == 1) { 592: (void)snprintf(errMsg, msgLen, "%s", msg8); 593: taosMemoryFree(intervals); 594: cJSON_Delete(binDesc); 595: return TSDB_CODE_FUNC_HISTOGRAM_ERROR;	
	污染路径: 函数'taosMemoryFree'的定义点.	
197	缺陷发生位置: /source/client/test/clientBITests.cpp	行号: 82
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 80: for(int i = 0; i < 10; i++) { 81: char str[512] = {0}; 82: sprintf(str, "insert into sub_t0 values(now+%da, %d)", i, i); 83: pRes = taos_query(pConn, str); 84: ASSERT_EQ(taos_errno(pRes), TSDB_CODE_SUCCESS);	
	污染路径: 无	
198	缺陷发生位置: /source/libs/transport/src/transComm.c	行号: 241
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 239: if (decompLen != oriLen) { 240: tError("msgLen:%d, originLen:%d, decompLen:%d", len, oriLen, decompLen); 241: taosMemoryFree(buf); 242: return TSDB_CODE_INVALID_MSG;	

	243: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
199	缺陷发生位置: /source/libs/decimal/test/decimalTest.cpp	行号: 239
	步骤描述: 不使用返回值的函数'r'应使用'void'说明.	
	代码片段: 237: 238: DEFINE_OPERATOR(+, OP_TYPE_ADD); 239: DEFINE_OPERATOR(-, OP_TYPE_SUB); 240: DEFINE_OPERATOR(*, OP_TYPE_MULTI); 241: DEFINE_OPERATOR(/, OP_TYPE_DIV);	
	污染路径: expanded from macro 'DEFINE_OPERATOR'	
200	缺陷发生位置: /docs/examples/c-ws-new/async_demo.c	行号: 99
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 97: queryDB(taos, sql); 98: 99: sprintf(sql, "create database %s", db); 100: queryDB(taos, sql); 101:	
	污染路径: 无	
201	缺陷发生位置: /contrib/TSZ/sz/src/sz.c	行号: 199
	步骤描述: 不使用返回值的函数'gettimeofday'应使用'void'说明.	
	代码片段: 197: double elapsed; 198: struct timeval costEnd; 199: gettimeofday(&costEnd, NULL); 200: elapsed = ((costEnd.tv_sec*1000000+costEnd.tv_usec)- (costStart.tv_sec*1000000+costStart.tv_usec))/1000000.0; 201: totalCost += elapsed;	

	函数'uError'的定义点.	
205	缺陷发生位置: /source/dnode/vnode/src/bse/bseSnapshot.c	行号: 97
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 95: bseError("vgId:%d failed to close table pWriter since %s", BSE_VGID((SBse *)p->pBse), tsterror(terrno)); 96: } 97: taosMemoryFree(p); 98: 99: return;	
	污染路径: 函数'taosMemoryFree'的定义点.	
206	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 3134
	步骤描述: 不使用返回值的函数'parserError'应使用'void'说明.	
	代码片段: 3132: 3133: if (LIST_LENGTH(pCxt->pSubQueries) <= 0) { 3134: parserError("unexpected empty subQ list got"); 3135: return TSDB_CODE_PAR_INTERNAL_ERROR; 3136: }	
	污染路径: 函数'parserError'的定义点.	
207	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncSnapshotDebug.c	行号: 42
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 40: s = syncUtilPrintBin2((char *) (pSender->pCurrentBlock), pSender->blockLen); 41: cJSON_AddStringToObject(pRoot, "pCurrentBlock2", s); 42: taosMemoryFree(s); 43: } 44:	
	污染路径: 函数'taosMemoryFree'的定义点.	
208	缺陷发生位置:	行号:

	/source/dnode/mgmt/mgmt_snode/src/smlnt.c	97
	步骤描述: 不使用返回值的函数'dlInfo'应使用'void'说明.	
	代码片段: 95: if (pDir == NULL) { 96: // Checkpoint directory doesn't exist yet, just mark as encrypted 97: dlInfo("checkpoint directory doesn't exist, marking as encrypted"); 98: } else { 99: // Iterate through checkpoint files and re-encrypt each one	
	污染路径: 函数'dlInfo'的定义点.	
209	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 5697
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 5695: static void freeSchemaFunc(void* param) { 5696: void** p = (void**)param; 5697: taosMemoryFreeClear(*p); 5698: } 5699:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
210	缺陷发生位置: /docs/examples/c/connect_example.c	行号: 51
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 49: // close & clean 50: taos_close(taos); 51: taos_cleanup(); 52: }	
	污染路径: 函数'taos_cleanup'的定义点.	
211	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 2156
	步骤描述:	

	不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 2154: recordSchema->num_fields = size; 2155: recordSchema->fields = calloc(1, sizeof(FieldStruct) * size); 2156: ASSERT(recordSchema->fields); 2157: 2158: for (i = 0; i < size; i++) {	
	污染路径: 函数'ASSERT'的定义点.	
212	缺陷发生位置: /source/dnode/mnode/impl/src/mndTelem.c	行号: 215
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 213: mError("%s failed to send telemetry report, line %d since %s", __func__, line, tstrerror(code)); 214: } 215: taosMemoryFree(pCont); 216: return code; 217: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
213	缺陷发生位置: /source/libs/decimal/src/detail/wideInteger.cpp	行号: 49
	步骤描述: 不使用返回值的函数'operator/='应使用'void'说明.	
	代码片段: 47: intx::uint128 *pX = (intx::uint128*)pLeft; 48: const intx::uint128 *pY = (const intx::uint128*)pRight; 49: *pX /= *pY; 50: /* __uint128_t *px = (__uint128_t*)pLeft; 51: const __uint128_t *py = (__uint128_t*)pRight;	
	污染路径: 函数'operator/='的定义点.	
214	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeSvr.c	行号: 102
	步骤描述: 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段:	

	100: if (tDecode132v(pCoder, NULL) < 0) { 101: code = TSDB_CODE_INVALID_MSG; 102: TSDB_CHECK_CODE(code, lino, _exit); 103: } 104:	
	污染路径: 函数'TSDB_CHECK_CODE'的定义点.	
215	缺陷发生位置: /source/libs/executor/src/scanoperator.c	行号: 544
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 542: taosMemoryFree((void*)pVal->pName); 543: taosMemoryFree(pVal->pTags); 544: taosMemoryFree(pVal); 545: } 546:	
	污染路径: 函数'taosMemoryFree'的定义点.	
216	缺陷发生位置: /source/dnode/mnode/impl/src/mndTrans.c	行号: 296
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 294: if (array != NULL) { 295: if (taosHashPut(redoGroupActions, &pAction->groupId, sizeof(int32_t), &array, sizeof(SArray *)) < 0) { 296: mInfo("failed put action into redo group actions"); 297: return TSDB_CODE_INTERNAL_ERROR; 298: } }	
	污染路径: 函数'mInfo'的定义点.	
217	缺陷发生位置: /source/dnode/mgmt/mgmt_dnode/src/dmWorker.c	行号: 34
	步骤描述: 不使用返回值的函数'taosMsleep'应使用'void'说明.	
	代码片段: 32: 33: while (1) { 34: taosMsleep(50);	

	35: if (pMgmt->pData->dropped pMgmt->pData->stopped) break; 36:	
	污染路径: 函数'taosMsleep'的定义点.	
218	缺陷发生位置: /Utils/test/c/tmqSim.c	行号: 169
	步骤描述: 不使用返回值的函数'taosLocalTime'应使用'void'说明.	
	代码片段: 167: time_t tTime = taosGetTimestampSec(); 168: struct tm tm; 169: taosLocalTime(&tTime, &tm, NULL, 0, NULL); 170: sprintf(timeString, "%d-%02d-%02d %02d:%02d:%02d", tm.tm_year + 1900, tm.tm_mon + 1, tm.tm_mday, tm.tm_hour, tm.tm_min, tm.tm_sec); 171:	
	污染路径: 函数'taosLocalTime'的定义点.	
219	缺陷发生位置: /Utils/test/c/tmq_offset_test.c	行号: 217
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 215: 216: ASSERT(numOfAssign1 == 2); 217: ASSERT(numOfAssign1 == numOfAssign2); 218: ASSERT(numOfAssign1 == numOfAssign3); 219:	
	污染路径: 函数'ASSERT'的定义点.	
220	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmlnt.c	行号: 750
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 748: taosHashInit(TSDB_MIN_VNODES, taosGetDefaultHashFunction(TSDB_DATA_TYPE_INT), true, HASH_ENTRY_LOCK); 749: if (pMgmt->closedHash == NULL) {	

	750: dError("failed to init vnode closed hash since %s", terrstr()); 751: return TSDB_CODE_OUT_OF_MEMORY; 752: }	
	污染路径: 函数'dError'的定义点.	
221	缺陷发生位置: /source/common/src/msg/tmsg.c	行号: 951
	步骤描述: 不使用返回值的函数'DECODESQL'应使用'void'说明.	
	代码片段: 949: } 950: 951: DECODESQL(); 952: 953: if (!tDecodelsEnd(&decoder)) {	
	污染路径: 函数'DECODESQL'的定义点.	
222	缺陷发生位置: /source/dnode/bnode/src/bnode.c	行号: 24
	步骤描述: 不使用返回值的函数'bndError'应使用'void'说明.	
	代码片段: 22: *pBnode = taosMemoryCalloc(1, sizeof(SBnode)); 23: if (NULL == *pBnode) { 24: bndError("calloc SBnode failed"); 25: code = terrno; 26: TAOS_RETURN(code);	
	污染路径: 函数'bndError'的定义点.	
223	缺陷发生位置: /contrib/test/cos/main.c	行号: 104
	步骤描述: 不使用返回值的函数'cos_get_hmac_sha1_hexdigest'应使用'void'说明.	
	代码片段: 102: fmt_str = cos_create_buf(p, 1024); 103: 104: cos_get_hmac_sha1_hexdigest(sign_key, secret_key, sizeof(secret_key) - 1, time_str, sizeof(time_str) - 1); 105: char *pstr = apr_pstrndup(p, (char *)sign_key,	

	sizeof(sign_key)); 106: cos_warn_log("sign_key: %s", pstr);	
	污染路径: 函数'cos_get_hmac_sha1_hexdigest'的定义点.	
224	缺陷发生位置: /source/os/test/osStringTests.cpp	行号: 303
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 301: // 测试超出范围的值 302: char large_num[50]; 303: snprintf(large_num, sizeof(large_num), "%d", INT16_MAX); 304: result = taosStr2int16(large_num, &val); 305: ASSERT_EQ(result, 0);	
	污染路径: 无	
225	缺陷发生位置: /source/util/src/tref.c	行号: 484
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 482: rsetId); 483: (*pSet->fp)(pNode->p); 484: taosMemoryFree(pNode); 485: 486: taosDecRsetCount(pSet);	
	污染路径: 函数'taosMemoryFree'的定义点.	
226	缺陷发生位置: /source/util/test/lrucacheTest.cpp	行号: 71
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 69: for (int32_t i = 0; i < numItems; i++) { 70: char key[64]; 71: snprintf(key, sizeof(key), "key_%d", i); 72: int32_t value = i; 73:	

	污染路径: 无	
227	缺陷发生位置: /utils/test/c/tmq_taosx_ci.c	行号: 1228
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 1226: const char* err = tmq_err2str(code); 1227: char* tmp = strstr(err, "Invalid parameters,detail:taos:"); 1228: ASSERT(tmp != NULL); 1229: } 1230:	
	污染路径: 函数'ASSERT'的定义点.	
228	缺陷发生位置: /tools/shell/src/shellNettest.c	行号: 113
	步骤描述: 不使用返回值的函数'shellExit'应使用'void'说明.	
	代码片段: 111: } 112: 113: void shellNettestHandler(int32_t signum, void *sigInfo, void *context) { shellExit(); } 114: 115: static void shellWorkAsServer() {	
	污染路径: 函数'shellExit'的定义点.	
229	缺陷发生位置: /source/libs/monitorfw/src/taos_collector_registry.c	行号: 287
	步骤描述: 不使用返回值的函数'taos_free'应使用'void'说明.	
	代码片段: 285: char * item_str = tjsonToString(item); 286: r = taos_string_builder_add_str(self->string_builder_batch, item_str); 287: taos_free(item_str); 288: if (r) goto _OVER;; 289:	
	污染路径:	

	函数'taos_free'的定义点.	
230	缺陷发生位置: /source/libs/monitorfw/src/taos_collector.c	行号: 43
	步骤描述: 不使用返回值的函数'TAOS_LOG'应使用'void'说明.	
	代码片段: 41: if (self->metrics == NULL) { 42: if (taos_collector_destroy(self) != 0) { 43: TAOS_LOG("taos_collector_destroy failed"); 44: } 45: return NULL;	
	污染路径: 函数'TAOS_LOG'的定义点.	
231	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftEntryDebug.c	行号: 41
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 39: s = syncUtilPrintBin((char*)(pEntry->data), pEntry->dataLen); 40: cJSON_AddStringToObject(pRoot, "data", s); 41: taosMemoryFree(s); 42: 43: s = syncUtilPrintBin2((char*)(pEntry->data), pEntry->dataLen);	
	污染路径: 函数'taosMemoryFree'的定义点.	
232	缺陷发生位置: /source/libs/tss/src/fs.c	行号: 139
	步骤描述: 不使用返回值的函数'taosMemFree'应使用'void'说明.	
	代码片段: 137: } 138: 139: taosMemFree(ss); 140: TAOS_RETURN(TSDB_CODE_FAILED); 141: }	
	污染路径: 函数'taosMemFree'的定义点.	
233	缺陷发生位置:	行号:

	/source/util/src/theap.c	332
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 330: q->queue = createPriorityQueue(fn, deleteFn, param); 331: if (q->queue == NULL) { 332: taosMemoryFree(q); 333: return NULL; 334: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
234	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncIndexMgrDebug.c	行号: 43
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 41: char *serialized = syncIndexMgr2Str(pObj); 42: sTrace("syncIndexMgrLog2 len:%" PRIu64 " %s %s", (uint64_t)strlen(serialized), s, serialized); 43: taosMemoryFree(serialized); 44: } 45: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
235	缺陷发生位置: /source/libs/function/src/detail/tminmaxavx.c	行号: 185
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 183: return TSDB_CODE_SUCCESS; 184: #else 185: uError("unable run %s without avx2 instructions", __func__); 186: return TSDB_CODE_OPS_NOT_SUPPORT; 187: #endif	
	污染路径: 函数'uError'的定义点.	
236	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smlnt.c	行号: 225
	步骤描述:	

	不使用返回值的函数'tmsgReportStartup'应使用'void'说明.	
	代码片段: 223: return code; 224: } 225: tmsgReportStartup("snode-worker", "initialized"); 226: 227: if ((code = udfcOpen()) != 0) {	
	污染路径: 函数'tmsgReportStartup'的定义点.	
237	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smWorker.c	行号: 85
	步骤描述: 不使用返回值的函数'streamReleaseTask'应使用'void'说明.	
	代码片段: 83: } 84: if (taskAddr != NULL) { 85: streamReleaseTask(taskAddr); 86: } 87: pRsp->reqType = pMsg->msgType;	
	污染路径: 函数'streamReleaseTask'的定义点.	
238	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeOpen.c	行号: 355
	步骤描述: 不使用返回值的函数'vInfo'应使用'void'说明.	
	代码片段: 353: 354: void vnodeDestroy(int32_t vgld, const char *path, STfs *pTfs, int32_t nodeId) { 355: vInfo("path:%s is removed while destroy vnode", path); 356: if (tfsRmdir(pTfs, path) < 0) { 357: vError("failed to remove path:%s since %s", path, strerror(errno));	
	污染路径: 函数'vInfo'的定义点.	
239	缺陷发生位置: /utils/test/c/tmq_write_raw_test.c	行号: 93
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段:	

	<pre> 91: int32_t ret = tmq_write_raw(pConn, raw); 92: printf("write raw data: %s\n", tmq_err2str(ret)); 93: ASSERT(ret == 0); 94: 95: tmq_free_raw(raw); </pre>	
	污染路径: 函数'ASSERT'的定义点.	
240	缺陷发生位置: /source/libs/executor/src/executorInt.c	行号: 211
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: <pre> 209: sizeof(SResultRowPosition)); 210: if (code != TSDB_CODE_SUCCESS) { 211: qError("%s failed at line %d since %s", __func__, __LINE__, tstrerror(code)); 212: pTaskInfo->code = code; 213: return NULL; </pre>	
	污染路径: 函数'qError'的定义点.	
241	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 2074
	步骤描述: 不使用返回值的函数'HUFv05_decodeStreamX2'应使用'void'说明.	
	代码片段: <pre> 2072: HUFv05_decodeStreamX2(op1, &bitD1, opStart2, dt, dtLog); 2073: HUFv05_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 2074: HUFv05_decodeStreamX2(op3, &bitD3, opStart4, dt, dtLog); 2075: HUFv05_decodeStreamX2(op4, &bitD4, oend, dt, dtLog); 2076: </pre>	
	污染路径: 函数'HUFv05_decodeStreamX2'的定义点.	
242	缺陷发生位置: /source/libs/metrics/test/metricsTest.cpp	行号: 95
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	

	代码片段: 93: 94: char dbname[32]; 95: snprintf(dbname, sizeof(dbname), "load_test_db_%d", i); 96: int32_t code = addWriteMetrics(vgId, 1, 123456789, "localhost:6030", dbname, &rawMetrics); 97: ASSERT_EQ(code, TSDB_CODE_SUCCESS);	
	污染路径: 无	
243	缺陷发生位置: /source/libs/decimal/src/decimal.c	行号: 718
	步骤描述: 不使用返回值的函数'TAOS_STRNCAT'应使用'void'说明.	
	代码片段: 716: decimal128Abs(©); 717: extractDecimal128Digits(©, segments, &digitNum); 718: TAOS_STRNCAT(buf2, "-", 2); 719: } else { 720: extractDecimal128Digits(pDec, segments, &digitNum);	
	污染路径: 函数'TAOS_STRNCAT'的定义点.	
244	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmlnt.c	行号: 120
	步骤描述: 不使用返回值的函数'dlInfo'应使用'void'说明.	
	代码片段: 118: 119: if (!option.deploy) { 120: dlInfo("mnode start to deploy"); 121: pMgmt->pData->dnodeld = 1; 122: mmBuildOptionForDeploy(pMgmt, plInput, &option);	
	污染路径: 函数'dlInfo'的定义点.	
245	缺陷发生位置: /docs/examples/c/tmq_demo.c	行号: 87
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段:	

	<pre> 85: } 86: taos_free_result(pRes); 87: sleep(1); 88: } 89: fprintf(stdout, "Prepare data thread exit\n"); </pre>	
	污染路径: 无	
246	缺陷发生位置: /source/libs/monitor/src/monFramework.c	行号: 316
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: <pre> 314: void monGenVgroupInfoTable(SMonInfo *pMonitor){ 315: if(taos_collector_registry_deregister_metric(TABLES_NUM) != 0){ 316: uError("failed to delete metric "TABLES_NUM); 317: } 318: </pre>	
	污染路径: 函数'uError'的定义点.	
247	缺陷发生位置: /source/libs/tfs/src/tfs.c	行号: 235
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: <pre> 233: tfsInitFile(pTfs, pFile, diskId, rname); 234: 235: taosMemoryFreeClear(rname); 236: return buf; 237: } </pre>	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
248	缺陷发生位置: /test/new_test_framework/script/api/stmtQuery.c	行号: 111
	步骤描述: 不使用返回值的函数'taos_stmt_bind_param'应使用'void'说明.	
	代码片段: <pre> 109: printf("Successfully execute taos_stmt_prepare\n"); 110: </pre>	

	111: taos_stmt_bind_param(stmt, params); 112: taos_stmt_add_batch(stmt); 113:	
	污染路径: 函数'taos_stmt_bind_param'的定义点.	
249	缺陷发生位置: /source/client/src/clientStmt2.c	行号: 1940
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 1938: pCbParam->code = code; 1939: if (tsem_post(&pCbParam->sem) != 0) { 1940: tscError("failed to post semaphore"); 1941: } 1942: }	
	污染路径: 函数'tscError'的定义点.	
250	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 1021
	步骤描述: 不使用返回值的函数'prepareColData'应使用'void'说明.	
	代码片段: 1019: for (int b = 0; b < (allRowNum / gCurCase->bindRowNum); b++) { 1020: for (int c = 0; c < gCurCase->bindColNum; ++c) { 1021: prepareColData(false, BP_BIND_COL, data, b * gCurCase->bindColNum + c, b * gCurCase->bindRowNum, c); 1022: } 1023: }	
	污染路径: 函数'prepareColData'的定义点.	
251	缺陷发生位置: /source/libs/transport/test/cliBench.c	行号: 169
	步骤描述: 不使用返回值的函数'tlInfo'应使用'void'说明.	
	代码片段: 167: 168: tlInfo("client is initialized"); 169: tlInfo("threads:%d msgSize:%d requests:%d", appThreads, msgSize, numOfReqs);	

	170: 171: int64_t now = taosGetTimestampUs();	
	污染路径: 函数'tInfo'的定义点.	
252	缺陷发生位置: /utlis/test/c/get_db_name_test.c	行号: 42
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 40: code = taos_get_current_db(taos, NULL, 0, &required); 41: ASSERT(code != 0); 42: ASSERT(required == 7); 43: 44: char database[10] = {0};	
	污染路径: 函数'ASSERT'的定义点.	
253	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmTransport.c	行号: 572
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: 570: } 571: 572: dDebug("dnode rpc status client is initialized"); 573: return 0; 574: }	
	污染路径: 函数'dDebug'的定义点.	
254	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 1092
	步骤描述: 不使用返回值的函数'argp_fmtstream_set_lmargin'应使用'void'说明.	
	代码片段: 1090: argp_fmtstream_putc (pest->stream, '\n'); 1091: indent_to (pest->stream, uparams.header_col); 1092: argp_fmtstream_set_lmargin (pest->stream, uparams.header_col); 1093: argp_fmtstream_set_wmargin (pest->stream, uparams.header_col);	

	1094: argp_fmtstream_puts (pest->stream, fstr);	
	污染路径: 函数'argp_fmtstream_set_lmargin'的定义点.	
255	缺陷发生位置: /source/libs/decimal/src/detail/intx/int128.hpp	行号: 650
	步骤描述: 不使用返回值的函数'operator-='应使用'void'说明.	
	代码片段: 648: { 649: ++q.hi; 650: r -= d; 651: } 652:	
	污染路径: 函数'operator-='的定义点.	
256	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftStoreDebug.c	行号: 65
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 63: printf("raftStorePrint len:%d %s \n", (int32_t)strlen(serialized), serialized); 64: fflush(NULL); 65: taosMemoryFree(serialized); 66: } 67:	
	污染路径: 函数'taosMemoryFree'的定义点.	
257	缺陷发生位置: /source/client/src/clientFirewall.c	行号: 214
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 212: // Better approach: store original pattern in description 213: if (taosHashPut(existingPatterns, desc, strlen(desc), &i, sizeof(i)) != 0) { 214: tscError("sql security: failed to add existing pattern to hash"); 215: }	

	216: }	
	污染路径: 函数'tscError'的定义点.	
258	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeCommit.c	行号: 193
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 191: pInfo->config.syncCfg.replicaNum, pInfo->config.syncCfg.myIndex, pInfo->config.syncCfg.changeVersion); 192: } 193: taosMemoryFree(data); 194: TAOS_RETURN(code); 195: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
259	缺陷发生位置: /source/client/wrapper/jni/windows/win32/bridge/AccessBridgeCalls.c	行号: 78
	步骤描述: 不使用返回值的函数'PrintDebugString'应使用'void'说明.	
	代码片段: 76: LOAD_FP(SetJavaShutdown, SetJavaShutdownFP, "setJavaShutdownFP"); 77: LOAD_FP(SetFocusGained, SetFocusGainedFP, "setFocusGainedFP"); 78: LOAD_FP(SetFocusLost, SetFocusLostFP, "setFocusLostFP"); 79: 80: LOAD_FP(SetCaretUpdate, SetCaretUpdateFP, "setCaretUpdateFP");	
	污染路径: expanded from macro 'LOAD_FP' 函数'PrintDebugString'的定义点. expanded from macro 'LOAD_FP'	
260	缺陷发生位置: /tools/taos-tools/src/toolstime.c	行号: 867
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段:	

	865: res = localtime_r(timep, result); 866: if (res == NULL && buf != NULL) { 867: sprintf(buf, "NaN"); 868: } 869: #endif	
	污染路径: 无	
261	缺陷发生位置: /source/dnode/vnode/src/meta/metaOpen.c	行号: 957
	步骤描述: 不使用返回值的函数'metaError'应使用'void'说明.	
	代码片段: 955: __compar_fn_t func = getComparFunc(pTagIdxKey1->type, 0); 956: if (func == NULL) { 957: metaError("meta/open: %s", terrstr()); 958: return TSDB_CODE_FAILED; 959: }	
	污染路径: 函数'metaError'的定义点.	
262	缺陷发生位置: /source/libs/function/src/builtinsimpl.c	行号: 712
	步骤描述: 不使用返回值的函数'SUM_RES_INC_ISUM'应使用'void'说明.	
	代码片段: 710: 711: if (IS_SIGNED_NUMERIC_TYPE(type) type == TSDB_DATA_TYPE_BOOL) { 712: SUM_RES_INC_ISUM(pDBuf, SUM_RES_GET_ISUM(pSBuf)); 713: } else if (IS_UNSIGNED_NUMERIC_TYPE(type)) { 714: SUM_RES_INC_USUM(pDBuf, SUM_RES_GET_USUM(pSBuf));	
	污染路径: 函数'SUM_RES_INC_ISUM'的定义点.	
263	缺陷发生位置: /tools/shell/src/shellNettest.c	行号: 93
	步骤描述: 不使用返回值的函数'rpcClose'应使用'void'说明.	
	代码片段:	

	<pre> 91: _OVER: 92: if (clientRpc != NULL) { 93: rpcClose(clientRpc); 94: } 95: if (rpcRsp.pCont != NULL) { </pre>	
	污染路径: 函数'rpcClose'的定义点.	
264	缺陷发生位置: /source/dnode/mnode/sdb/src/sdbFile.c	行号: 38
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: <pre> 36: static int32_t sdbDeployData(SSdb *pSdb) { 37: int32_t code = 0; 38: mInfo("start to deploy sdb"); 39: 40: for (int32_t i = SDB_MAX - 1; i >= 0; --i) { </pre>	
	污染路径: 函数'mInfo'的定义点.	
265	缺陷发生位置: /source/dnode/mnode/sdb/src/sdb.c	行号: 43
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: <pre> 41: sdbCleanup(pSdb); 42: terrno = TSDB_CODE_OUT_OF_MEMORY; 43: mError("failed to init sdb since %s", terrstr()); 44: return NULL; 45: } </pre>	
	污染路径: 函数'mError'的定义点.	
266	缺陷发生位置: /contrib/test/traft/cluster/node10000.c	行号: 112
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: <pre> 110: // for test: submit a value every second 111: while (1) { 112: sleep(1); 113: submitValue(); </pre>	

	114: }	
	污染路径: 无	
267	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1359
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 1357: // free previous malloc 1358: if (word1->free && word1->word) { 1359: taosMemoryFree(word1->word); 1360: } 1361:	
	污染路径: 函数'taosMemoryFree'的定义点.	
268	缺陷发生位置: /source/dnode/mnode/impl/src/mndStb.c	行号: 1223
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 1221: mInfo("trans:%d, add create stb to redo action", pTrans->id); 1222: if ((code = mndTransAppendRedoAction(pTrans, &action)) != 0) { 1223: taosMemoryFree(pReq); 1224: TAOS_RETURN(code); 1225: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
269	缺陷发生位置: /test/new_test_framework/script/api/stmt2-performance.c	行号: 76
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 74: for (int i = 0; i < CTB_NUMS; i++) { 75: tbs[i] = (char*)malloc(sizeof(char) * 20); 76: sprintf(tbs[i], "ctb_%d", i); 77: createCtb(taos, tbs[i]); 78: }	

	污染路径: 无	
270	缺陷发生位置: /source/util/src/tjson.c	行号: 206
	步骤描述: 不使用返回值的函数'TAOS_STRCPY'应使用'void'说明.	
	代码片段: 204: return TSDB_CODE_SUCCESS; 205: } 206: TAOS_STRCPY(pVal, p); 207: return TSDB_CODE_SUCCESS; 208: }	
	污染路径: 函数'TAOS_STRCPY'的定义点.	
271	缺陷发生位置: /utils/test/c/blob_test.c	行号: 41
	步骤描述: 不使用返回值的函数'taosMemFree'应使用'void'说明.	
	代码片段: 39: void freeBindData(char ***table_name, TAOS_STMT2_BIND ***tags, TAOS_STMT2_BIND ***params) { 40: for (int i = 0; i < NUM_OF_SUB_TABLES; i++) { 41: taosMemFree((*table_name)[i]); 42: for (int j = 0; j < 2; j++) { 43: taosMemFree((*tags)[i][j].buffer);	
	污染路径: 函数'taosMemFree'的定义点.	
272	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 546
	步骤描述: 不使用返回值的函数'varDataSetLen'应使用'void'说明.	
	代码片段: 544: } 545: 546: varDataSetLen(output, resLen); 547: } 548:	
	污染路径: 函数'varDataSetLen'的定义点.	
273	缺陷发生位置:	行号:

	/source/util/src/tconfig.c	69
	步骤描述: 不使用返回值的函数'cfgCleanup'应使用'void'说明.	
	代码片段: <pre> 67: if (code != 0) { 68: uError("failed to init config, since %s ,at line %d", tstrerror(code), lino); 69: cfgCleanup(pCfg); 70: } 71: </pre>	
	污染路径: 函数'cfgCleanup'的定义点.	
274	缺陷发生位置: /utils/test/c/get_db_name_test.c	行号: 46
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: <pre> 44: char database[10] = {0}; 45: code = taos_get_current_db(taos, database, 3, &required); 46: ASSERT(code != 0); 47: ASSERT(required == 7); 48: ASSERT(strcpy(database, "sm")); </pre>	
	污染路径: 函数'ASSERT'的定义点.	
275	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttLogging.c	行号: 255
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: <pre> 253: if (log_timestamp_format) { 254: struct tm *ti = NULL; 255: UNUSED(get_time(&ti)); 256: log_line_pos = strftime(log_line, sizeof(log_line), log_timestamp_format, ti); 257: if (log_line_pos == 0) { </pre>	
	污染路径: 函数'UNUSED'的定义点.	
276	缺陷发生位置: /source/dnode/mnode/impl/src/mndTopic.c	行号: 978

	步骤描述: 不使用返回值的函数'MND_TMQ_NULL_CHECK'应使用'void'说明.	
	代码片段: 976: 977: sql = taosMemoryMalloc(strlen(pTopic->sql) + VARSTR_HEADER_SIZE); 978: MND_TMQ_NULL_CHECK(sql); 979: STR_TO_VARSTR(sql, pTopic->sql); 980:	
	污染路径: 函数'MND_TMQ_NULL_CHECK'的定义点.	
277	缺陷发生位置: /source/common/src/tglobal.c	行号: 2374
	步骤描述: 不使用返回值的函数'TAOS_CHECK_RETURN'应使用'void'说明.	
	代码片段: 2372: static int32_t cfgInitWrapper(SConfig **pCfg) { 2373: if (*pCfg == NULL) { 2374: TAOS_CHECK_RETURN(cfgInit(pCfg)); 2375: } 2376: TAOS_RETURN(TSDB_CODE_SUCCESS);	
278	污染路径: 函数'TAOS_CHECK_RETURN'的定义点.	
	缺陷发生位置: /contrib/TSZ/sz/src/iniparser.c	行号: 255
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 253: seclen = (int)strlen(s); 254: fprintf(f, "\n[%s]\n", s); 255: sprintf(keym, "%s:", s); 256: for (j=0 ; j<d->size ; j++) { 257: if (d->key[j]==NULL)	
279	污染路径: 函数'sprintf'的定义点.	
	缺陷发生位置: /docs/examples/c-ws-new/stmt2_insert_demo.c	行号: 63
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段:	

	61: // Allocate and assign table name 62: (*table_name)[i] = (char *)malloc(20 * sizeof(char)); 63: sprintf((*table_name)[i], "d_bind_%d", i); 64: 65: // Allocate memory for tags data	
	污染路径: 无	
280	缺陷发生位置: /source/libs/executor/src/virtualtablescanoperator.c	行号: 815
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 813: pInfo->pSortCtxList = NULL; 814: } 815: taosMemoryFreeClear(param); 816: } 817:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
281	缺陷发生位置: /source/libs/function/src/builtins.c	行号: 587
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 585: if (start->valuedouble == 0) { 586: (void)snprintf(errMsg, msgLen, "%s", msg7); 587: taosMemoryFree(intervals); 588: cJSON_Delete(binDesc); 589: return TSDB_CODE_FUNC_HISTOGRAM_ERROR;	
	污染路径: 函数'taosMemoryFree'的定义点.	
282	缺陷发生位置: /source/util/src/tarray.c	行号: 208
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 206: if (NULL == pArray) { 207: terrno = TSDB_CODE_INVALID_PARA; 208: uError("failed to return value from array of null ptr"); 209: return NULL;	

	210: }	
	污染路径: 函数'uError'的定义点.	
283	缺陷发生位置: /docs/examples/c-ws-new/stmt_insert_demo.c	行号: 193
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 191: insertData(taos); 192: taos_close(taos); 193: taos_cleanup(); 194: }	
	污染路径: 函数'taos_cleanup'的定义点.	
284	缺陷发生位置: /source/libs/tdb/src/db/tdbPage.c	行号: 52
	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	
	代码片段: 50: 51: if (!TDB_IS_PGSIZE_VLD(pageSize)) { 52: tdbError("tdb/page-create: invalid pageSize: %d.", pageSize); 53: return TSDB_CODE_INVALID_PARA; 54: }	
	污染路径: 函数'tdbError'的定义点.	
285	缺陷发生位置: /source/client/src/clientSml.c	行号: 230
	步骤描述: 不使用返回值的函数'PROCESS_SLASH_IN_MEASUREMENT'应使用'void'说明.	
	代码片段: 228: (void)memcpy(measure, currElement->measure, measureLen); 229: if (currElement->measureEscaped) { 230: PROCESS_SLASH_IN_MEASUREMENT(measure, measureLen); 231: } 232: smlStrReplace(measure, measureLen);	

	污染路径: 函数'PROCESS_SLASH_IN_MEASUREMENT'的定义点.	
286	缺陷发生位置: /source/dnode/vnode/src/meta/metaTtl.c	行号: 279
	步骤描述: 不使用返回值的函数'metaError'应使用'void'说明.	
	代码片段: 277: int code = TSDB_CODE_SUCCESS; 278: if ((code = tdbTbTraversal(pOldTtlIdx, &cvData, ttlMgrConvertOneEntry)) != TSDB_CODE_SUCCESS) { 279: metaError("failed to convert since %s", tstrerror(code)); 280: } 281:	
	污染路径: 函数'metaError'的定义点.	
287	缺陷发生位置: /source/dnode/vnode/src/tq/tqScan.c	行号: 283
	步骤描述: 不使用返回值的函数'tqDebug'应使用'void'说明.	
	代码片段: 281: TSDB_CHECK_CODE(code, lino, END); 282: *pBatchMetaRsp = *tmp; 283: tqDebug("tmqsnap task get meta"); 284: break; 285: }	
	污染路径: 函数'tqDebug'的定义点.	
288	缺陷发生位置: /source/dnode/vnode/src/bse/bseTable.c	行号: 1425
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 1423: void blockWrapperCleanup(SBlockWrapper *p) { 1424: if (p->data != NULL) { 1425: taosMemoryFree(p->data); 1426: p->data = NULL; 1427: }	
	污染路径: 函数'taosMemoryFree'的定义点.	

289	缺陷发生位置: /source/dnode/mnode/impl/src/mndStreamTrans.c	行号: 109
	步骤描述: 不使用返回值的函数'mstsError'应使用'void'说明.	
	代码片段: 107: SSdbRaw *pCommitRaw = mndStreamActionEncode(pStream); 108: if (pCommitRaw == NULL) { 109: mstsError("failed to encode stream since %s", terrstr()); 110: mndTransDrop(pTrans); 111: return terrno;	
	污染路径: 函数'mstsError'的定义点.	
290	缺陷发生位置: /source/common/src/tencrypt.c	行号: 515
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 513: } 514: if (plainContent != NULL) { 515: taosMemoryFree(plainContent); 516: } 517: if (code != 0) {	
	污染路径: 函数'taosMemoryFree'的定义点.	
291	缺陷发生位置: /source/dnode/mnode/impl/src/mndEncryptAlgr.c	行号: 505
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 503: pTrans = mndTransCreate(pMnode, TRN_POLICY_RETRY, TRN_CONFLICT_NOHING, NULL, "create-enc-algr"); 504: if (pTrans == NULL) { 505: mError("failed to create since %s", terrstr()); 506: code = terrno; 507: goto _OVER;	
	污染路径: 函数'mError'的定义点.	
292	缺陷发生位置:	行号:

	/source/dnode/vnode/src/tsdb/tsdbScan.c	303
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 301: static void scanArgDestroy(SScanArg *arg) { 302: if (arg == NULL) return; 303: taosMemoryFree(arg); 304: } 305:	
	污染路径: 函数'taosMemoryFree'的定义点.	
293	缺陷发生位置: /source/libs/parser/src/parInsertSql.c	行号: 1288
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 1286: } 1287: 1288: taosMemoryFree(tmpTokenBuf); 1289: } else { 1290: if (pToken->n + VARSTR_HEADER_SIZE > bytes) {	
	污染路径: 函数'taosMemoryFree'的定义点.	
294	缺陷发生位置: /tools/taos-tools/src/benchMain.c	行号: 250
	步骤描述: 不使用返回值的函数'pthread_cancel'应使用'void'说明.	
	代码片段: 248: 249: #ifdef LINUX 250: pthread_cancel(spId); 251: pthread_join(spId, NULL); 252: #endif	
	污染路径: 无	
295	缺陷发生位置: /source/common/src/tencrypt.c	行号: 471
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段:	

	469: } 470: 471: taosMemoryFree(fileContent); 472: fileContent = NULL; 473:	
	污染路径: 函数'taosMemoryFree'的定义点.	
296	缺陷发生位置: /test/new_test_framework/script/api/stmtTest.c	行号: 26
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 24: void check_result(TAOS *taos, int id, int expected) { 25: char sql[256] = {0}; 26: sprintf(sql, "select * from t%d", id); 27: TAOS_RES *result; 28: result = taos_query(taos, sql);	
	污染路径: 无	
297	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 932
	步骤描述: 不使用返回值的函数'mempcpy'应使用'void'说明.	
	代码片段: 930: #endif 931: 932: mempcpy (mempcpy (entries, hol->entries, 933: hol->num_entries * sizeof (struct hol_entry)), 934: more->entries,	
	污染路径: 函数'mempcpy'的定义点.	
298	缺陷发生位置: /source/libs/executor/src/externalwindowoperator.c	行号: 4003
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 4001: int32_t numCols = taosArrayGetSize(pBlock->pDataBlock);	

	4002: if (numCols < 2) { 4003: qError("%s invalid external-window block: expected at least 2 columns, got %d", __func__, numCols); 4004: TAOS_CHECK_EXIT(TSDB_CODE_INVALID_PARA); 4005: }	
	污染路径: 函数'qError'的定义点.	
299	缺陷发生位置: /docs/examples/c/stmt2_insert_demo.c	行号: 283
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 281: (void)fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x%x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 282: taos_errstr(NULL)); 283: taos_cleanup(); 284: exit(EXIT_FAILURE); 285: }	
	污染路径: 函数'taos_cleanup'的定义点.	
300	缺陷发生位置: /source/dnode/mnode/impl/src/mndStreamMgmt.c	行号: 2747
	步骤描述: 不使用返回值的函数'mstsInfo'应使用'void'说明.	
	代码片段: 2745: atomic_store_8(&pStream->stopped, 2); 2746: 2747: mstsInfo("set stream %s stopped by user", streamName); 2748: 2749: _exit:	
	污染路径: 函数'mstsInfo'的定义点.	
301	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 1085
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 1083: taosMemoryFree(keys_list[0]);	

	1084: 1085: taosMemoryFree(keys_list); 1086: taosMemoryFree(keys_list_sizes); 1087: taosMemoryFree(values_list);	
	污染路径: 函数'taosMemoryFree'的定义点.	
302	缺陷发生位置: /source/util/src/tlog.c	行号: 348
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 346: tsLogObj.logHandle = NULL; 347: } 348: taosMemoryFreeClear(tsLogOutput); 349: } 350:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
303	缺陷发生位置: /source/dnode/mnode/impl/src/mndConsumer.c	行号: 539
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 537: void *pltem = *(void **)param; 538: if (pltem != NULL) { 539: taosMemoryFree(pltem); 540: } 541: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
304	缺陷发生位置: /docs/examples/c/line_example.c	行号: 43
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 41: taos_free_result(res); 42: taos_close(taos); 43: taos_cleanup(); 44: } 45: // output:	

	污染路径: 函数'taos_cleanup'的定义点.	
305	缺陷发生位置: /test/new_test_framework/script/api/whiteListTest.c	行号: 135
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 133: while (nWhiteListVerNotified < 10) { 134: printf("white list update notified %d times\n", nWhiteListVerNotified); 135: sleep(1); 136: } 137: printf("succeed in getting white list nofication. %d times\n", nWhiteListVerNotified);	
	污染路径: 无	
306	缺陷发生位置: /source/dnode/vnode/src/meta/metaTtl.c	行号: 282
	步骤描述: 不使用返回值的函数'metaInfo'应使用'void'说明.	
	代码片段: 280: } 281: 282: metaInfo("ttlMgr convert end."); 283: TAOS_RETURN(code); 284: }	
	污染路径: 函数'metaInfo'的定义点.	
307	缺陷发生位置: /source/dnode/vnode/src/meta/metaEntry2.c	行号: 102
	步骤描述: 不使用返回值的函数'tdbFreeClear'应使用'void'说明.	
	代码片段: 100: tsterror(code)); 101: tDecoderClear(&decoder); 102: tdbFreeClear(value); 103: return code; 104: }	
	污染路径:	

	函数'tdbFreeClear'的定义点.	
308	缺陷发生位置: /source/libs/transport/src/transTLS.c	行号: 220
	步骤描述: 不使用返回值的函数'tlInfo'应使用'void'说明.	
	代码片段: 218: int8_t ready = atomic_load_8(&p->tlsLoading) ? 0 : 1; 219: if (ready) { 220: tlInfo("transport instance is ready to reload tls config"); 221: } else { 222: tlInfo("transport instance is not ready to reload tls config");	
	污染路径: 函数'tlInfo'的定义点.	
309	缺陷发生位置: /source/dnode/mnode/impl/src/mndTrans.c	行号: 970
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 968: int32_t pos = 0; 969: if (taosHashPut(pTrans->groupActionPos, &pAction->groupId, sizeof(int32_t), &pos, sizeof(int32_t)) < 0) { 970: mInfo("failed put action into group action pos"); 971: return TSDB_CODE_INTERNAL_ERROR; 972: }	
	污染路径: 函数'mInfo'的定义点.	
310	缺陷发生位置: /source/libs/tdb/src/db/tdbPage.c	行号: 108
	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	
	代码片段: 106: 107: if (TDB_DESTROY_PAGE_LOCK(pPage) != 0) { 108: tdbError("tdb/page-destroy: destroy page lock failed."); 109: } 110:	
	污染路径: 函数'tdbError'的定义点.	

311	缺陷发生位置: /source/client/src/clientMonitorSlow.c	行号: 74
	步骤描述: 不使用返回值的函数'tscLog'应使用'void'说明.	
	代码片段: 72: (void)releaseTscObj(rid); 73: } else { 74: tscLog("slowQueryLog, not found rid"); 75: } 76: }	
	污染路径: 函数'tscLog'的定义点.	
312	缺陷发生位置: /source/client/src/clientStmt2.c	行号: 2190
	步骤描述: 不使用返回值的函数'STMT2_ELOG_E'应使用'void'说明.	
	代码片段: 2188: 2189: if (pStmt->sql.bindRowFormat) { 2190: STMT2_ELOG_E("can't mix bind row format and bind column format"); 2191: STMT_ERR_RET(TSDB_CODE_TSC_STMT_API_ERROR); 2192: }	
	污染路径: 函数'STMT2_ELOG_E'的定义点.	
313	缺陷发生位置: /source/dnode/mnode/impl/src/mndUser.c	行号: 192
	步骤描述: 不使用返回值的函数'TAOS_RETURN'应使用'void'说明.	
	代码片段: 190: 191: (void)taosThreadRwlockInit(&userCache.rw, NULL); 192: TAOS_RETURN(0); 193: } 194:	
	污染路径: 函数'TAOS_RETURN'的定义点.	
314	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 614

	步骤描述: 不使用返回值的函数'exponent'应使用'void'说明.	
	代码片段: 612: result *= base; 613: base = sqr(base); 614: exponent >>= 1; 615: } 616: return result;	
	污染路径: 无	
315	缺陷发生位置: /source/dnode/vnode/src/bse/bseMgt.c	行号: 231
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 229: if (code != 0) { 230: bseError("vgld:%d, failed to read current at line %d since %s", BSE_VGID(pBse), lino, tsterror(code)); 231: taosMemoryFree(pCurrent); 232: } 233: return code;	
	污染路径: 函数'taosMemoryFree'的定义点.	
316	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSet2.c	行号: 33
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 31: if (lvl[0] != NULL) { 32: TARRAY2_DESTROY(lvl[0]->fobjArr, tsdbSttLvlClearFObj); 33: taosMemoryFree(lvl[0]); 34: lvl[0] = NULL; 35: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
317	缺陷发生位置: /source/libs/transport/test/ivrBench.c	行号: 171
	步骤描述: 不使用返回值的函数'tError'应使用'void'说明.	

	代码片段: 169: void *pRpc = rpcOpen(&rpclnit); 170: if (pRpc == NULL) { 171: tError("failed to start RPC server"); 172: return -1; 173: }	
	污染路径: 函数'tError'的定义点.	
318	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqSendPublish.c	行号: 172
	步骤描述: 不使用返回值的函数'ttq_free'应使用'void'说明.	
	代码片段: 170: rc = packet__alloc(packet); 171: if (rc) { 172: ttq_free(packet); 173: return rc; 174: }	
	污染路径: 函数'ttq_free'的定义点.	
319	缺陷发生位置: /source/dnode/mnode/impl/src/mndTelem.c	行号: 208
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 206: 207: code = taosSendTelemReport(&pMgmt->addrMgt, tsTelemUri, tsTelemPort, pCont, strlen(pCont), HTTP_FLAT); 208: taosMemoryFree(pCont); 209: return code; 210:	
	污染路径: 函数'taosMemoryFree'的定义点.	
320	缺陷发生位置: /source/libs/executor/src/dataDeleter.c	行号: 162
	步骤描述: 不使用返回值的函数'taosFreeQitem'应使用'void'说明.	
	代码片段: 160: code = allocBuf(pDeleter, pInput, pBuf); 161: if (code) {	

	162: taosFreeQitem(pBuf); 163: return code; 164: }	
	污染路径: 函数'taosFreeQitem'的定义点.	
321	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqDb.c	行号: 315
	步骤描述: 不使用返回值的函数'ttq_free'应使用'void'说明.	
	代码片段: 313: 314: mqttt_property_free_all(&item->properties); 315: ttq_free(item); 316: } 317:	
	污染路径: 函数'ttq_free'的定义点.	
322	缺陷发生位置: /utls/test/c/get_db_name_test.c	行号: 41
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 39: int required = 0; 40: code = taos_get_current_db(taos, NULL, 0, &required); 41: ASSERT(code != 0); 42: ASSERT(required == 7); 43:	
	污染路径: 函数'ASSERT'的定义点.	
323	缺陷发生位置: /source/client/src/clientMsgHandler.c	行号: 46
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 44: if (NEED_CLIENT_RM_TBLMETA_REQ(pRequest->type)) { 45: if (removeMeta(pRequest->pTscObj, pRequest->targetTableList, IS_VIEW_REQUEST(pRequest->type)) != 0) { 46: tscError("failed to remove meta data for table"); 47: } 48: }	

	污染路径: 函数'tscError'的定义点.	
324	缺陷发生位置: /source/util/src/tlosertree.c	行号: 97
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 95: int32_t code = 0; 96: if (!(idx <= pTree->totalSources - 1 && idx >= pTree->numOfSources && pTree->totalSources >= 2)) { 97: uError("losertree failed at: %s:%d", __func__, __LINE__); 98: return TSDB_CODE_INVALID_PARA; 99: }	
	污染路径: 函数'uError'的定义点.	
325	缺陷发生位置: /source/libs/monitor/src/monMain.c	行号: 157
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 155: } 156: } else { 157: uError("failed to set insert counter, already set"); 158: } 159: }	
	污染路径: 函数'uError'的定义点.	
326	缺陷发生位置: /utils/test/c/tmq_token.c	行号: 148
	步骤描述: 不使用返回值的函数'taosSsleep'应使用'void'说明.	
	代码片段: 146: taosSsleep(5); 147: alter_token_disable(); 148: taosSsleep(5); 149: alter_token_enable(); 150: taosSsleep(5);	
	污染路径: 函数'taosSsleep'的定义点.	

	/source/client/src/clientRawBlockWrite.c	251
	步骤描述: 不使用返回值的函数'uDebug'应使用'void'说明.	
	代码片段: 249: 250: end: 251: RAW_LOG_END 252: return code; 253: }	
	污染路径: expanded from macro 'RAW_LOG_END' 函数'uDebug'的定义点. expanded from macro 'RAW_LOG_START'	
331	缺陷发生位置: /source/util/test/trefTest.c	行号: 43
	步骤描述: 不使用返回值的函数'taosUsleep'应使用'void'说明.	
	代码片段: 41: pSpace->rid[id] = taosAddRef(pSpace->rsetld, pSpace->p[id]); 42: } 43: taosUsleep(100); 44: } 45:	
	污染路径: 函数'taosUsleep'的定义点.	
332	缺陷发生位置: /utils/test/c/tmq_ts6115.c	行号: 77
	步骤描述: 不使用返回值的函数'taosMsleep'应使用'void'说明.	
	代码片段: 75: } 76: taos_free_result(pRes); 77: taosMsleep(200); 78: } 79: tmq_consumer_close(tmq);	
	污染路径: 函数'taosMsleep'的定义点.	
333	缺陷发生位置: /source/libs/new-stream/src/dataSinkCache.c	行号: 376

	步骤描述: 不使用返回值的函数'stError'应使用'void'说明.	
	代码片段: 374: _end: 375: if (code != TSDB_CODE_SUCCESS) { 376: stError("failed to encode data since %s, lineno:%d", tstrerror(code), lino); 377: } 378: return code;	
	污染路径: 函数'stError'的定义点.	
334	缺陷发生位置: /source/libs/index/src/index.c	行号: 242
	步骤描述: 不使用返回值的函数'indexError'应使用'void'说明.	
	代码片段: 240: } 241: if (taosThreadMutexUnlock(&index->mtx) != 0) { 242: indexError("failed to unlock index mutex"); 243: } 244:	
	污染路径: 函数'indexError'的定义点.	
335	缺陷发生位置: /tests/taosc_test/taoscTest.cpp	行号: 172
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 170: 171: const char* table_name = "taosc_test_table1"; 172: sprintf(sql, "select * from taosc_test_db.%s;", table_name); 173: TAOS_RES* res = taos_query(taos, sql); 174: ASSERT_TRUE(res != nullptr);	
	污染路径: 无	
336	缺陷发生位置: /source/dnode/bnode/src/bnode.c	行号: 53
	步骤描述: 不使用返回值的函数'mqttMgmtStopMqtttd'应使用'void'说明.	

	代码片段: 51: 52: void bndClose(SBnode *pBnode) { 53: mqttMgmtStopMqtttd(); 54: 55: taosMemoryFree(pBnode);	
	污染路径: 函数'mqttMgmtStopMqtttd'的定义点.	
337	缺陷发生位置: /source/libs/planner/src/planPhysiCreator.c	行号: 94
	步骤描述: 不使用返回值的函数'TAOS_STRNCAT'应使用'void'说明.	
	代码片段: 92: TAOS_STRNCAT(*ppKey, ".", 2); 93: TAOS_STRNCAT(*ppKey, pCol->refTableName, TSDB_TABLE_NAME_LEN); 94: TAOS_STRNCAT(*ppKey, ".", 2); 95: TAOS_STRNCAT(*ppKey, pCol->refColName, TSDB_COL_NAME_LEN); 96: *pLen = taosHashBinary(*ppKey, strlen(*ppKey), TSDB_TABLE_FNAME_LEN + 1 + TSDB_COL_NAME_LEN + 1 + extraBufLen);	
	污染路径: 函数'TAOS_STRNCAT'的定义点.	
338	缺陷发生位置: /source/libs/catalog/src/ctgCache.c	行号: 132
	步骤描述: 不使用返回值的函数'ctgTrace'应使用'void'说明.	
	代码片段: 130: *pCache = NULL; 131: CTG_CACHE_NHIT_INC(CTG_CI_DB, 1); 132: ctgTrace("db:%s, db not in cache", dbFName); 133: return TSDB_CODE_SUCCESS; 134: }	
	污染路径: 函数'ctgTrace'的定义点.	
339	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/gnu-test-skeleton.c	行号: 158
	步骤描述:	

	不使用返回值的函数'nanosleep'应使用'void'说明.	
	代码片段: <pre> 156: ts.tv_sec = 0; 157: ts.tv_nsec = 100000000; 158: nanosleep (&ts, NULL); 159: } 160: if (killed != 0 && killed != pid) </pre>	
	污染路径: 无	
340	缺陷发生位置: /source/libs/tfs/src/tfs.c	行号: 146
	步骤描述: 不使用返回值的函数'TAOS_RETURN'应使用'void'说明.	
	代码片段: <pre> 144: int32_t tfsGetDisksAtLevel(STfs *pTfs, int32_t level) { 145: if (level < 0 level >= pTfs->nlevel) { 146: TAOS_RETURN(0); 147: } 148: </pre>	
	污染路径: 函数'TAOS_RETURN'的定义点.	
341	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbDataFileRAW.c	行号: 202
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 200: tsdbDataFileRAWWriterDoClose(writer[0]); 201: } 202: taosMemoryFree(writer[0]); 203: writer[0] = NULL; 204: </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
342	缺陷发生位置: /contrib/TSZ/zstd/decompress/huf_decompress.c	行号: 968
	步骤描述: 不使用返回值的函数'assert'应使用'void'说明.	
	代码片段: <pre> 966: { 967: assert(dstSize > 0); </pre>	

	968: assert(dstSize <= 128*1024); 969: /* decoder timing evaluation */ 970: { U32 const Q = (cSrcSize >= dstSize) ? 15 : (U32) (cSrcSize * 16 / dstSize); /* Q < 16 */	
	污染路径: 函数'assert'的定义点.	
343	缺陷发生位置: /source/common/src/msg/streamMsg.c	行号: 3133
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 3131: _exit: 3132: if (buf != NULL) { 3133: taosMemoryFree(buf); 3134: } 3135: return code;	
	污染路径: 函数'taosMemoryFree'的定义点.	
344	缺陷发生位置: /source/libs/parser/src/parTokenizer.c	行号: 995
	步骤描述: 不使用返回值的函数'taosHashCleanup'应使用'void'说明.	
	代码片段: 993: void* m = keywordHashTable; 994: if (m != NULL && atomic_val_compare_exchange_ptr(&keywordHashTable, m, 0) == m) { 995: taosHashCleanup(m); 996: } 997: }	
	污染路径: 函数'taosHashCleanup'的定义点.	
345	缺陷发生位置: /test/new_test_framework/script/api/sameReqidTest.c	行号: 123
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 121: taos_free_result(res); 122: 123: usleep(1000000);	

	124: } 125:	
	污染路径: 无	
346	缺陷发生位置: /utils/test/c/tmq_td35698.c	行号: 57
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 55: int32_t ret = tmq_write_raw(pConn, raw); 56: printf("write raw data: %s\n", tmq_err2str(ret)); 57: ASSERT(ret == 0); 58: 59: tmq_free_raw(raw);	
	污染路径: 函数'ASSERT'的定义点.	
347	缺陷发生位置: /docs/examples/c-ws/connect_example.c	行号: 8
	步骤描述: 不使用返回值的函数'ws_enable_log'应使用'void'说明.	
	代码片段: 6: 7: int main() { 8: ws_enable_log("debug"); 9: char *dsn = "ws://localhost:6041"; 10: WS_TAOS *taos = ws_connect(dsn);	
	污染路径: 函数'ws_enable_log'的定义点.	
348	缺陷发生位置: /source/client/src/clientSml.c	行号: 227
	步骤描述: 不使用返回值的函数'SML_CHECK_NULL'应使用'void'说明.	
	代码片段: 225: int measureLen = currElement->measureLen; 226: char *measure = (char *)taosMemoryMalloc(measureLen); 227: SML_CHECK_NULL(measure); 228: (void)memcpy(measure, currElement->measure, measureLen); 229: if (currElement->measureEscaped) {	

	污染路径: 函数'SML_CHECK_NULL'的定义点.	
349	缺陷发生位置: /source/libs/qworker/src/qwDbg.c	行号: 295
	步骤描述: 不使用返回值的函数'taosSsleep'应使用'void'说明.	
	代码片段: 293: static int32_t ignoreTime = 0; 294: if (++ignoreTime > 10) { 295: taoSsleep(taosRand() % 20); 296: } 297: }	
	污染路径: 函数'taosSsleep'的定义点.	
350	缺陷发生位置: /source/libs/catalog/src/ctgUtil.c	行号: 41
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 39: taosArrayDestroy(pParam->msgIdx); 40: 41: taosMemoryFree(param); 42: } 43:	
	污染路径: 函数'taosMemoryFree'的定义点.	
351	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/bug-argp1.c	行号: 25
	步骤描述: 不使用返回值的函数'argp_parse'应使用'void'说明.	
	代码片段: 23: { 24: int i; 25: argp_parse (&test_argp, argc, argv, 0, &i, NULL); 26: return 0; 27: }	
	污染路径: 无	

352	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 2592
	步骤描述: 不使用返回值的函数'tmpGeometry'应使用'void'说明.	
	代码片段: 2590: case TSDB_DATA_TYPE_GEOMETRY: { 2591: char *buf = (char *)benchCalloc(col->length + 1, 1, false); 2592: tmpGeometry(buf, stbInfo->iface, col, 0); 2593: tools_cJSON_AddStringToObject(record, "value", buf); 2594: tmfree(buf);	
	污染路径: 函数'tmpGeometry'的定义点.	
353	缺陷发生位置: /tools/rocks-reader/rreader.cpp	行号: 29
	步骤描述: 不使用返回值的函数"应使用'void'说明.	
	代码片段: 27: typedef int64_t tb_uid_t; 28: 29: struct SValue { 30: int8_t type; 31: union {	
	污染路径: 函数"的定义点.	
354	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 1086
	步骤描述: 不使用返回值的函数'DEBUGLOG'应使用'void'说明.	
	代码片段: 1084: { 1085: ZDICT_cover_params_t params; 1086: DEBUGLOG(3, "ZDICT_trainFromBuffer"); 1087: memset(¶ms, 0, sizeof(params)); 1088: params.d = 8;	
	污染路径: 函数'DEBUGLOG'的定义点.	
355	缺陷发生位置: /source/dnode/vnode/src/meta/metaTable2.c	行号: 136

	步骤描述: 不使用返回值的函数'tdbFreeClear'应使用'void'说明.	
	代码片段: <pre> 134: } 135: pReq->uid = *(tb_uid_t *)value; 136: tdbFreeClear(value); 137: 138: code = metaGetInfo(pMeta, pReq->uid, &info, NULL); </pre>	
	污染路径: 函数'tdbFreeClear'的定义点.	
356	缺陷发生位置: /source/libs/decimal/src/detail/wideInteger.cpp	行号: 57
	步骤描述: 不使用返回值的函数'operator%='应使用'void'说明.	
	代码片段: <pre> 55: intx::uint128 *pX = (intx::uint128*)pLeft; 56: const intx::uint128 *pY = (const intx::uint128*)pRight; 57: *pX %= *pY; 58: /* __uint128_t *px = (__uint128_t*)pLeft; 59: const __uint128_t *py = (__uint128_t*)pRight; </pre>	
	污染路径: 函数'operator%='的定义点.	
357	缺陷发生位置: /source/libs/new-stream/src/streamRunner.c	行号: 517
	步骤描述: 不使用返回值的函数'ST_TASK_DLOG'应使用'void'说明.	
	代码片段: <pre> 515: code = streamBuildBlockResultNotifyContent(pTask, pBlock, &pContent, pTask->output.outCols, startRow, endRow); 516: if (code == 0) { 517: ST_TASK_DLOG("prepare notify:%s", pContent); 518: SSTriggerCalcParam* pTriggerCalcParams = taosArrayGet(pExec- >runtimeInfo.funcInfo.pStreamPesudoFuncVals, curWinIdx); 519: if (pTriggerCalcParams == NULL) { </pre>	
	污染路径: 函数'ST_TASK_DLOG'的定义点.	
358	缺陷发生位置: /tests/taosc_test/taoscTest.cpp	行号: 129
	步骤描述:	

	不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 127: void* tmpParam = pUserParam; 128: tsem_init(&query_sem, 0, 0); 129: sprintf(sql, "select * from taosc_test_db.%;s;", table_name); 130: taos_query_a(taos, sql, queryCallback, pUserParam); 131: tsem_wait(&query_sem);	
	污染路径: 无	
359	缺陷发生位置: /source/util/src/tskiplist.c	行号: 412
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 410: if (pSkipList->lock) { 411: if (taosThreadRwlockUnlock(pSkipList->lock) != 0) { 412: uError("failed to unlock skip list"); 413: } 414: }	
360	污染路径: 函数'uError'的定义点.	
	缺陷发生位置: /source/client/src/clientMain.c	行号: 186
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
361	代码片段: 184: STscObj *pObj = acquireTscObj(*(int64_t *)taos); 185: if (NULL == pObj) { 186: tscError("invalid parameter for %s", __func__); 187: return terrno; 188: }	
	污染路径: 函数'tscError'的定义点.	
	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1950
	步骤描述: 不使用返回值的函数'HUF_decodeStreamX2'应使用'void'说明.	
	代码片段: 1948: /* finish bitStreams one by one */	

	1949: HUF_decodeStreamX2(op1, &bitD1, opStart2, dt, dtLog); 1950: HUF_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 1951: HUF_decodeStreamX2(op3, &bitD3, opStart4, dt, dtLog); 1952: HUF_decodeStreamX2(op4, &bitD4, oend, dt, dtLog);	
	污染路径: 函数'HUF_decodeStreamX2'的定义点.	
362	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncIO.c	行号: 176
	步骤描述: 不使用返回值的函数'taosBlockSIGPIPE'应使用'void'说明.	
	代码片段: 174: static int32_t syncIOStartInternal(SSyncIO *io) { 175: int32_t ret = 0; 176: taosBlockSIGPIPE(); 177: 178: rpclnit();	
	污染路径: 函数'taosBlockSIGPIPE'的定义点.	
363	缺陷发生位置: /source/libs/tdb/src/db/tdbBtree.c	行号: 76
	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	
	代码片段: 74: 75: if (keyLen == 0) { 76: tdbError("tdb/btree-open: key len cannot be zero."); 77: return TSDB_CODE_INVALID_PARA; 78: }	
	污染路径: 函数'tdbError'的定义点.	
364	缺陷发生位置: /source/dnode/mnode/impl/src/mndInstance.c	行号: 278
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 276: int32_t code = tDeserializeSInstanceListReq(pReq-	

	<pre>>pCont, pReq->contLen, &req); 277: if (code != TSDB_CODE_SUCCESS) { 278: mError("instance list, failed to decode since %s", tstrerror(code)); 279: return code; 280: }</pre>	
	污染路径: 函数'mError'的定义点.	
365	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smlnt.c	行号: 179
	步骤描述: 不使用返回值的函数'dlInfo'应使用'void'说明.	
	代码片段: <pre>177: taosWUnLockLatch(&gSnode.snodeLock); 178: 179: dlInfo("snode checkpoints migrated to encrypted format and persisted encrypted flag successfully"); 180: return TSDB_CODE_SUCCESS; 181: }</pre>	
	污染路径: 函数'dlInfo'的定义点.	
366	缺陷发生位置: /utlis/test/c/tmq_td35698.c	行号: 49
	步骤描述: 不使用返回值的函数'tmq_free_json_meta'应使用'void'说明.	
	代码片段: <pre>47: char* result = tmq_get_json_meta(msg); 48: printf("meta result: %s\n", result); 49: tmq_free_json_meta(result); 50: } 51:</pre>	
	污染路径: 函数'tmq_free_json_meta'的定义点.	
367	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqMisc.c	行号: 61
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: <pre>59: struct group grp, *result; 60:</pre>	

	61: UNUSED(getgrgid_r(getgid(), &grp, buf, sizeof(buf), &result)); 62: if (result) { 63: ttq_log(NULL, TTQ_LOG_WARNING,	
	污染路径: 函数'UNUSED'的定义点.	
368	缺陷发生位置: /source/util/src/tconfig.c	行号: 338
	步骤描述: 不使用返回值的函数'TAOS_CHECK_RETURN'应使用'void'说明.	
	代码片段: 336: TAOS_RETURN(TSDB_CODE_INVALID_CFG); 337: } 338: TAOS_CHECK_RETURN(osSetTimezone(value)); 339: 340: TAOS_CHECK_RETURN(doSetConf(pltem, tsTimezoneStr, stype));	
	污染路径: 函数'TAOS_CHECK_RETURN'的定义点.	
369	缺陷发生位置: /source/libs/index/src/index.c	行号: 75
	步骤描述: 不使用返回值的函数'taosCleanUpScheduler'应使用'void'说明.	
	代码片段: 73: void indexCleanup() { 74: // refactor later 75: taosCleanUpScheduler(indexQhandle); 76: taosMemoryFreeClear(indexQhandle); 77: taosCloseRef(indexRefMgt);	
	污染路径: 函数'taosCleanUpScheduler'的定义点.	
370	缺陷发生位置: /source/client/src/clientHb.c	行号: 903
	步骤描述: 不使用返回值的函数'tscDebug'应使用'void'说明.	
	代码片段: 901: req->query = hbBasic; 902: releaseTscObj(connKey->tscRid); 903: tscDebug("no queries on connection"); 904: return TSDB_CODE_SUCCESS;	

	905: }	
	污染路径: 函数'tscDebug'的定义点.	
371	缺陷发生位置: /examples/c/schemaless.c	行号: 40
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 38: result = taos_query(taos, "drop database if exists db;"); 39: taos_free_result(result); 40: usleep(100000); 41: result = taos_query(taos, "create database db precision 'ms';"); 42: taos_free_result(result);	
	污染路径: 无	
372	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 737
	步骤描述: 不使用返回值的函数'geneDbCreateCmd'应使用'void'说明.	
	代码片段: 735: 736: // generate and execute create database sql 737: geneDbCreateCmd(database, command, remainVnodes); 738: int32_t code = queryDbExecCall(conn, command); 739: int32_t trying = g_arguments->keep_trying;	
	污染路径: 函数'geneDbCreateCmd'的定义点.	
373	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttDaemon.c	行号: 212
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: 210: if (tSerializeSRetrieveFuncReq(pReq, contLen, &retrieveReq) < 0) { 211: taosArrayDestroy(retrieveReq.pFuncNames); 212: rpcFreeCont(pReq); 213: return terno; 214: }	

	污染路径: 函数'rpcFreeCont'的定义点.	
374	缺陷发生位置: /source/client/src/clientEnv.c	行号: 284
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 282: 283: if (TSDB_CODE_SUCCESS != nodesSimReleaseAllocator(pRequest->allocatorRefId)) { 284: tscError("failed to release allocator"); 285: } 286: }	
	污染路径: 函数'tscError'的定义点.	
375	缺陷发生位置: /source/libs/qworker/src/qworker.c	行号: 823
	步骤描述: 不使用返回值的函数'QW_ERR_JRET'应使用'void'说明.	
	代码片段: 821: if (QW_EVENT_RECEIVED(ctx, QW_EVENT_DROP)) { 822: if (QW_PHASE_POST_FETCH != phase (!! QW_QUERY_RUNNING(ctx)) && qwTaskNotInExec(ctx)) { 823: QW_ERR_JRET(qwDropTask(QW_FPARAMS())); 824: QW_ERR_JRET(ctx->rspCode); 825: }	
	污染路径: 函数'QW_ERR_JRET'的定义点.	
376	缺陷发生位置: /test/new_test_framework/script/api/last-query-ws.cpp	行号: 91
	步骤描述: 不使用返回值的函数'operator<<'应使用'void'说明.	
	代码片段: 89: TAOS* taos = taos_connect("127.0.0.1", "root", "taosdata", dbName.c_str(), 0); 90: if (!taos) { 91: std::cerr << "Failed to connect to TDengine\n"; 92: return; 93: }	

	污染路径: 函数'operator<<'的定义点.	
377	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqLoop.c	行号: 56
	步骤描述: 不使用返回值的函数'ttqDbMsgStoreFree'应使用'void'说明.	
	代码片段: 54: stored->payload = ttq_malloc(stored->payloadlen + 1); 55: if (stored->payload == NULL) { 56: ttqDbMsgStoreFree(stored); 57: return TTQ_ERR_NOMEM; 58: }	
	污染路径: 函数'ttqDbMsgStoreFree'的定义点.	
378	缺陷发生位置: /source/libs/index/src/indexTfile.c	行号: 550
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 548: stmBuilderDestroy(sb); 549: taosArrayDestroy(offsets); 550: taosMemoryFree(p); 551: 552: return code;	
	污染路径: 函数'taosMemoryFree'的定义点.	
379	缺陷发生位置: /source/libs/tmqtt/tools/src/topic-producer.c	行号: 1004
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 1002: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x%x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 1003: taos_errstr(NULL)); 1004: taos_cleanup(); 1005: return -1; 1006: }	
	污染路径: 函数'taos_cleanup'的定义点.	
380	缺陷发生位置:	行号:

	/source/libs/qworker/src/qwDbg.c	377
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 375: } 376: 377: qError("invalid qw debug option:%s", option); 378: 379: return TSDB_CODE_APP_ERROR;	
	污染路径: 函数'qError'的定义点.	
381	缺陷发生位置: /source/dnode/mnode/impl/src/mndUser.c	行号: 206
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 204: taosMemoryFree(plInfo->wllp); 205: taosMemoryFree(plInfo->wlTime); 206: taosMemoryFree(plInfo); 207: } 208: plter = taosHashIterate(userCache.users, plter);	
	污染路径: 函数'taosMemoryFree'的定义点.	
382	缺陷发生位置: /source/dnode/mnode/impl/src/mndSync.c	行号: 310
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 308: if (!pMnode->deploy) { 309: if (!pMnode->restored) { 310: mInfo("vgId:1, sync restore finished, and will handle outstanding transactions"); 311: mndTransPullup(pMnode); 312: mndSetRestored(pMnode, true);	
	污染路径: 函数'mInfo'的定义点.	
383	缺陷发生位置: /source/libs/function/src/detail/tminmaxavx.c	行号: 90
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	

	代码片段: 88: return TSDB_CODE_SUCCESS; 89: #else 90: uError("unable run %s without avx2 instructions", __func__); 91: return TSDB_CODE_OPS_NOT_SUPPORT; 92: #endif	
	污染路径: 函数'uError'的定义点.	
384	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReaderWriter.c	行号: 839
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 837: tFree(pReader->aBuf[iBuf]); 838: } 839: taosMemoryFree(pReader); 840: } 841:	
	污染路径: 函数'taosMemoryFree'的定义点.	
385	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 601
	步骤描述: 不使用返回值的函数'varDataSetLen'应使用'void'说明.	
	代码片段: 599: } 600: 601: varDataSetLen(output, resLen); 602: } 603:	
	污染路径: 函数'varDataSetLen'的定义点.	
386	缺陷发生位置: /source/libs/metrics/test/metricsTest.cpp	行号: 498
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 496: int32_t vgId = 2000 + t * 100 + i; 497: char dbname[32];	

	498: snprintf(dbname, sizeof(dbname), "thread_%d_db_%d", t, i); 499: bool exists = CheckVgroupMetricsExist(vgId, 123456789 + t, t + 1, "localhost:6030", dbname); 500: ASSERT_TRUE(exists);	
	污染路径: 无	
387	缺陷发生位置: /source/dnode/mgmt/mgmt_dnode/src/dmHandle.c	行号: 97
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: 95: contLen = tSerializeRetrieveWhiteListReq(pHead, contLen, &req); 96: if (contLen < 0) { 97: rpcFreeCont(pHead); 98: dError("failed to serialize ip white list request since:%s", tstrerror(contLen)); 99: return;	
	污染路径: 函数'rpcFreeCont'的定义点.	
388	缺陷发生位置: /source/dnode/vnode/src/tq/tqMeta.c	行号: 217
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 215: if (taosHashPut(pTq->pOffset, subkey, strlen(subkey), &offset, sizeof(STqOffset)) != 0) { 216: tDeleteSTqOffset(&offset); 217: tdbFree(data); 218: return terrno; 219: }	
	污染路径: 函数'tdbFree'的定义点.	
389	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftLogDebug.c	行号: 136
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 134: printf("logStorePrint2 len:%d %s %s \n",	

	<pre> (int32_t)strlen(serialized), s, serialized); 135: fflush(NULL); 136: taosMemoryFree(serialized); 137: } 138: </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
390	缺陷发生位置: /source/libs/nodes/src/nodesUtilFuncs.c	行号: 232
	步骤描述: 不使用返回值的函数'nodesWarn'应使用'void'说明.	
	代码片段: <pre> 230: int32_t nodesInitAllocatorSet() { 231: if (g_allocatorReqRefPool >= 0) { 232: nodesWarn("nodes already initialized"); 233: return TSDB_CODE_SUCCESS; 234: } </pre>	
	污染路径: 函数'nodesWarn'的定义点.	
391	缺陷发生位置: /source/dnode/xnode/src/xnode.c	行号: 50
	步骤描述: 不使用返回值的函数'xndInfo'应使用'void'说明.	
	代码片段: <pre> 48: 49: void xndClose(SXnode *pXnode) { 50: xndInfo("xnode: dnode is closing xnode"); 51: xnodeMgmtStopXnode(); 52: } </pre>	
	污染路径: 函数'xndInfo'的定义点.	
392	缺陷发生位置: /source/os/src/osDir.c	行号: 233
	步骤描述: 不使用返回值的函数'tstrncpy'应使用'void'说明.	
	代码片段: <pre> 231: if (temp[1] == ':') pos += 3; 232: #else 233: tstrncpy(temp, dirname, sizeof(temp)); 234: #endif </pre>	

	235:	
	污染路径: 函数'tstrncpy'的定义点.	
393	缺陷发生位置: /source/dnode/mnode/impl/src/mndUser.c	行号: 265
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 263: 264: if (taosHashPut(userCache.users, user, userLen, &pInfo, sizeof(pInfo)) != 0) { 265: taosMemoryFree(pInfo); 266: return NULL; 267: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
394	缺陷发生位置: /test/new_test_framework/script/api/stmt2.c	行号: 52
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 50: taos_free_result(result); 51: 52: usleep(100000); 53: taos_select_db(taos, "test"); 54:	
	污染路径: 无	
395	缺陷发生位置: /source/libs/new-stream/src/dataSinkFile.c	行号: 285
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 283: } 284: 285: taosMemoryFree(buf); 286: return TSDB_CODE_SUCCESS; 287: }	
	污染路径:	

	函数'taosMemoryFree'的定义点.	
396	缺陷发生位置: /source/client/src/clientFirewall.c	行号: 312
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: <pre> 310: if (fp != NULL) { 311: if (taosWriteFile(fp, jsonStr, strlen(jsonStr)) != strlen(jsonStr)) { 312: tscError("sql security: failed to write learned rules to %s", ruleFile); 313: } 314: TAOS_UNUSED(taosCloseFile(&fp)); </pre>	
	污染路径: 函数'tscError'的定义点.	
397	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeSvr.c	行号: 149
	步骤描述: 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段: <pre> 147: if (tDecodeI32v(&dc, &nReqs) < 0) { 148: code = TSDB_CODE_INVALID_MSG; 149: TSDB_CHECK_CODE(code, lino, _exit); 150: } 151: for (int32_t iReq = 0; iReq < nReqs; iReq++) { </pre>	
	污染路径: 函数'TSDB_CHECK_CODE'的定义点.	
398	缺陷发生位置: /utils/tsim/src/simExec.c	行号: 267
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: <pre> 265: } 266: } else if (op2[0] == '.') { 267: snprintf(t3, sizeof(t3), "%s%s", t1, t2); 268: } 269: } else { </pre>	
	污染路径: 函数'snprintf'的定义点.	
399	缺陷发生位置:	行号:

	/contrib/test/lucene/main.cpp	4
	步骤描述: 不使用返回值的函数'operator<<'应使用'void'说明.	
	代码片段: 2: 3: int main(int argc, char const *argv[]) { 4: std::cout << "Hello, this is lucene test" << std::endl; 5: return 0; 6: }	
	污染路径: 函数'operator<<'的定义点.	
400	缺陷发生位置: /source/os/src/osLocale.c	行号: 89
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 87: if (NULL == locale) { 88: terrno = TSDB_CODE_INVALID_PARA; 89: uError("failed to set locale:%s", inLocale); 90: return terrno; 91: }	
	污染路径: 函数'uError'的定义点.	
401	缺陷发生位置: /source/libs/function/src/builtinsimpl.c	行号: 314
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 312: plter->pPrevData = taosMemoryMalloc(plter->pDataCol->info.bytes); 313: if (NULL == plter->pPrevData) { 314: qError("out of memory when function get input row."); 315: return terrno; 316: }	
	污染路径: 函数'qError'的定义点.	
402	缺陷发生位置: /docs/examples/c-ws-new/with_reqid_demo.c	行号: 73
	步骤描述:	

	不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 71: // close & clean 72: taos_close(taos); 73: taos_cleanup(); 74: return 0; 75: // ANCHOR_END: with_reqid	
	污染路径: 函数'taos_cleanup'的定义点.	
403	缺陷发生位置: /source/libs/decimal/src/detail/wideInteger.cpp	行号: 41
	步骤描述: 不使用返回值的函数'operator*='应使用'void'说明.	
	代码片段: 39: intx::uint128 *pX = (intx::uint128*)pLeft; 40: const intx::uint128 *pY = (const intx::uint128*)pRight; 41: *pX *= *pY; 42: /* __uint128_t *px = (__uint128_t*)pLeft; 43: const __uint128_t *py = (__uint128_t*)pRight;	
	污染路径: 函数'operator*='的定义点.	
404	缺陷发生位置: /source/libs/executor/src/exchangeoperator.c	行号: 499
	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: 497: 498: if (blockDataGetNumOfRows(pBlock) == 0) { 499: qDebug("rows 0 block got, continue next load"); 500: continue; 501: }	
	污染路径: 函数'qDebug'的定义点.	
405	缺陷发生位置: /source/libs/new-stream/src/dataSinkCache.c	行号: 270
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 268: stError("failed to encode data since %s, lineno:%d", tsterror(code), lino);	

	269: if (buf) { 270: taosMemoryFree(buf); 271: } 272: return code;	
	污染路径: 函数'taosMemoryFree'的定义点.	
406	缺陷发生位置: /test/new_test_framework/script/api/stopquery.c	行号: 234
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 232: sqExecSQL(taos, sql); 233: 234: sprintf(sql, "select * from %s", tbName); 235: TAOS_RES* pRes = taos_query(taos, sql); 236: code = taos_errno(pRes);	
	污染路径: 无	
407	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSnapshot.c	行号: 1219
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 1217: } 1218: 1219: taosMemoryFree(writer[0]); 1220: writer[0] = NULL; 1221:	
	污染路径: 函数'taosMemoryFree'的定义点.	
408	缺陷发生位置: /source/libs/wal/test/walMetaTest.cpp	行号: 448
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 446: ASSERT_EQ(pRead->curVersion, ver + 1); 447: char newStr[100]; 448: sprintf(newStr, "%s-%d", ranStr, ver); 449: int len = strlen(newStr); 450: ASSERT_EQ(pRead->pHead->head.bodyLen, len);	

	污染路径: 无	
409	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeSvr.c	行号: 109
	步骤描述: 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段: 107: if (tDecodeCStr(pCoder, &name) < 0) { 108: code = TSDB_CODE_INVALID_MSG; 109: TSDB_CHECK_CODE(code, lino, _exit); 110: } 111:	
	污染路径: 函数'TSDB_CHECK_CODE'的定义点.	
410	缺陷发生位置: /contrib/lemon/lemon.c	行号: 1503
	步骤描述: 不使用返回值的函数'SetAdd'应使用'void'说明.	
	代码片段: 1501: xsp = rp->rhs[i]; 1502: if(xsp->type==TERMINAL){ 1503: SetAdd(newcfp->fws,xsp->index); 1504: break; 1505: }else if(xsp->type==MULTITERMINAL){	
	污染路径: 函数'SetAdd'的定义点.	
411	缺陷发生位置: /tools/taos-tools/src/benchCommandOpt.c	行号: 470
	步骤描述: 不使用返回值的函数'prctl'应使用'void'说明.	
	代码片段: 468: } 469: #ifdef LINUX 470: prctl(PR_SET_NAME, "queryNtableAggrFunc"); 471: #endif 472: char * command = benchCalloc(1, TOOLS_MAX_ALLOWED_SQL_LEN, false);	
	污染路径: 无	

412	缺陷发生位置: /source/libs/txnode/src/txnodeMgmt.c	行号: 77
	步骤描述: 不使用返回值的函数'xndDebug'应使用'void'说明.	
	代码片段: 75: snprintf(pipeName, size, "%s%s", tsDataDir, XNODED_XNODED_PID_NAME); 76: } 77: xndDebug("xnode get xnoded pid path:%s", pipeName); 78: } 79:	
	污染路径: 函数'xndDebug'的定义点.	
413	缺陷发生位置: /source/libs/executor/src/sortoperator.c	行号: 599
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 597: taosArrayDestroy(pCalc->pSortColsArr); 598: taosMemoryFree(pCalc->keyBuf); 599: taosMemoryFree(pCalc); 600: } 601: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
414	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqDb.c	行号: 329
	步骤描述: 不使用返回值的函数'ttq_free'应使用'void'说明.	
	代码片段: 327: 328: tmqtt_property_free_all(&item->properties); 329: ttq_free(item); 330: } 331:	
	污染路径: 函数'ttq_free'的定义点.	
415	缺陷发生位置: /docs/examples/c/create_db_demo.c	行号: 63
	步骤描述:	

	不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 61: fprintf(stderr, "Failed to create stable power.meters, ErrCode: 0x%x, ErrorMessage: %s\n.", code, taos_errstr(result)); 62: taos_close(taos); 63: taos_cleanup(); 64: return -1; 65: }</pre>	
	污染路径: 函数'taos_cleanup'的定义点.	
416	缺陷发生位置: /source/util/src/talgo.c	行号: 409
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 407: } 408: 409: taosMemoryFree(buf); 410: return 0; 411: }</pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
417	缺陷发生位置: /source/os/src/osTimezone.c	行号: 1217
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: <pre> 1215: char str2[TD_TIMEZONE_LEN] = {0}; 1216: if (taosStrfTime(str2, sizeof(str2), "%z", &tm1) == 0) { 1217: uError("failed to get timezone offset"); 1218: return TSDB_CODE_TIME_ERROR; 1219: }</pre>	
	污染路径: 函数'uError'的定义点.	
418	缺陷发生位置: /source/libs/catalog/src/ctgRemote.c	行号: 821
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: <pre> 819: if (msgSize < 0) {</pre>	

	820: taosMemoryFree(*msg); 821: qError("tSerializeSBatchReq failed"); 822: CTG_ERR_RET(msgSize); 823: }	
	污染路径: 函数'qError'的定义点.	
419	缺陷发生位置: /examples/c/asyncdemo.c	行号: 112
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 110: queryDB(taos, sql); 111: 112: sprintf(sql, "create database %s", db); 113: queryDB(taos, sql); 114:	
	污染路径: 无	
420	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCacheRead.c	行号: 246
	步骤描述: 不使用返回值的函数'tsdbWarn'应使用'void'说明.	
	代码片段: 244: // all queried tables have been dropped already, return immediately. 245: if (p->pSchema == NULL) { 246: tsdbWarn("all queried tables has been dropped, try next group, %s", idstr); 247: code = TSDB_CODE_PAR_TABLE_NOT_EXIST; 248: TSDB_CHECK_CODE(code, lino, _end);	
	污染路径: 函数'tsdbWarn'的定义点.	
421	缺陷发生位置: /contrib/test/traft/join_into_vgroup/node_follower10001.c	行号: 108
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 106: // for test: submit value every second 107: while (1) { 108: sleep(1);	

	109: submitValue(); 110: }	
	污染路径: 无	
422	缺陷发生位置: /source/libs/scalar/test/filter/filterTests.cpp	行号: 1469
	步骤描述: 不使用返回值的函数'fp'应使用'void'说明.	
	代码片段: 1467: for (SignedT l = signedMin; l <= signedMax; ++l) { 1468: for (UnsignedT r = unsignedMin; r <= unsignedMax; ++r) { 1469: ASSERT_EQ(fp(&l, &r), compareSignedWithUnsigned(l, r)); 1470: } 1471: }	
	污染路径: 无	
423	缺陷发生位置: /source/dnode/mnode/impl/src/mndSubscribe.c	行号: 1230
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 1228: code = mndDoRebalance(pMnode, &rebInput, &rebOutput); 1229: if (code != 0) { 1230: mError("mq rebalance do rebalance error, msg: %s", tstrerror(code)) 1231: } 1232: }	
	污染路径: 函数'mError'的定义点.	
424	缺陷发生位置: /source/os/test/osStringTests.cpp	行号: 371
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 369: // 测试超出范围的值 370: char large_num[50]; 371: snprintf(large_num, sizeof(large_num), "%d",	

	INT8_MAX); 372: result = taosStr2int8(large_num, &val); 373: ASSERT_EQ(result, 0);	
	污染路径: 无	
425	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeAsync.c	行号: 142
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 140: task->channel->scheduled = NULL; 141: if (task->channel->state == EVA_CHANNEL_STATE_CLOSE) { 142: taosMemoryFree(task->channel); 143: } else { 144: for (int32_t i = 0; i < EVA_PRIORITY_MAX; i++) {	
	污染路径: 函数'taosMemoryFree'的定义点.	
426	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 709
	步骤描述: 不使用返回值的函数'int32ToBytes_bigEndian'应使用'void'说明.	
	代码片段: 707: 708: *out = (unsigned char*)malloc(length*sizeof(int) +treeByteSize); 709: int32ToBytes_bigEndian(buffer, nodeCount); 710: memcpy(*out, buffer, 4); 711: int32ToBytes_bigEndian(buffer, huffmanTree->stateNum/2); //real number of intervals	
	污染路径: 函数'int32ToBytes_bigEndian'的定义点.	
427	缺陷发生位置: /contrib/TSZ/zstd/compress/fse_compress.c	行号: 346
	步骤描述: 不使用返回值的函数'assert'应使用'void'说明.	
	代码片段: 344: U32 tableLog = maxTableLog; 345: U32 minBits = FSE_minTableLog(srcSize, maxSymbolValue);	

	(SSharedStorageFS*)taosMemCalloc(1, sizeof(SSharedStorageFS) + asLen); 128: if (!ss) { 129: tssError("failed to allocate memory for SSharedStorageFS"); 130: TAOS_RETURN(TSDB_CODE_OUT_OF_MEMORY); 131: }	
	污染路径: 函数'tssError'的定义点.	
431	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 441
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 439: snprintf(zhongguoUtf8, sizeof(zhongguoUtf8), "%c%c%c%c %c%c%c%c", 0xE4, 0xB8, 0xAD, 0xE5, 0x9B, 0xBD); 440: snprintf(guoUtf8, sizeof(guoUtf8), "%c%c%c%c", 0xE5, 0x9B, 0xBD); 441: snprintf(zhongguoGbk, sizeof(zhongguoGbk), "%c%c%c%c %c", 0xD6, 0xD0, 0xB9, 0xFA); 442: snprintf(zhongGbk, sizeof(zhongGbk), "%c%c", 0xD6, 0xD0); 443:	
	污染路径: 无	
432	缺陷发生位置: /source/dnode/mnode/impl/src/mndEncryptAlgr.c	行号: 324
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 322: pTrans = mndTransCreate(pMnode, TRN_POLICY_RETRY, TRN_CONFLICT_NOTHING, NULL, "create-enc-algr"); 323: if (pTrans == NULL) { 324: mError("failed to create since %s",terrstr()); 325: code = terrno; 326: goto _OVER;	
	污染路径: 函数'mError'的定义点.	
433	缺陷发生位置: /source/dnode/vnode/src/meta/metaSnapshot.c	行号: 201

	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 199: if (code) { 200: metaError("vgId:%d, %s failed at %s:%d since %s", TD_VID(pMeta->pVnode), __func__, __FILE__, lino, tstrerror(code)); 201: taosMemoryFree(pWriter); 202: *ppWriter = NULL; 203: } else {	
	污染路径: 函数'taosMemoryFree'的定义点.	
434	缺陷发生位置: /source/libs/qworker/src/qwMsg.c	行号: 421
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: 419: if (msgSize < 0) { 420: QW_SCH_TASK_ELOG("tSerializeSTaskDropReq failed, msgSize:%d", msgSize); 421: rpcFreeCont(msg); 422: QW_ERR_RET(msgSize); 423: }	
	污染路径: 函数'rpcFreeCont'的定义点.	
435	缺陷发生位置: /source/libs/executor/src/dataDispatcher.c	行号: 66
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 64: } 65: if (pInput == NULL pInput->pData == NULL pInput->pData->info.rows <= 0) { 66: qError("invalid input data"); 67: return TSDB_CODE_QRY_INVALID_INPUT; 68: }	
	污染路径: 函数'qError'的定义点.	
436	缺陷发生位置: /source/libs/new-stream/src/streamTimer.c	行号: 50

	步骤描述: 不使用返回值的函数'stTrace'应使用'void'说明.	
	代码片段: 48: } 49: 50: stTrace("start %s tmr succ", pMsg); 51: } 52:	
	污染路径: 函数'stTrace'的定义点.	
437	缺陷发生位置: /test/new_test_framework/script/api/stmt2-blob-performance.c	行号: 135
	步骤描述: 不使用返回值的函数'clock_gettime'应使用'void'说明.	
	代码片段: 133: } 134: 135: clock_gettime(CLOCK_MONOTONIC, &end); // 获取开始 时间 TAOS_STMT2_BINDV bindv; 136: bind_time += ((double)(end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9); 137:	
	污染路径: 无	
438	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmEnv.c	行号: 58
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 56: static int32_t dmInitSystem() { 57: if (taosIgnSIGPIPE() != 0) { 58: dError("failed to ignore SIGPIPE"); 59: } 60:	
	污染路径: 函数'dError'的定义点.	
439	缺陷发生位置: /source/libs/transport/test/ivrBench.c	行号: 120
	步骤描述: 不使用返回值的函数'taosBlockSIGPIPE'应使用'void'说明.	

	代码片段: 118: char dataName[20] = "server.data"; 119: 120: taosBlockSIGPIPE(); 121: 122: memset(&rpclnit, 0, sizeof(rpclnit));	
	污染路径: 函数'taosBlockSIGPIPE'的定义点.	
440	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeCommit.c	行号: 209
	步骤描述: 不使用返回值的函数'vInfo'应使用'void'说明.	
	代码片段: 207: } 208: 209: vInfo("vnode info is committed, dir:%s", dir); 210: return 0; 211: }	
	污染路径: 函数'vInfo'的定义点.	
441	缺陷发生位置: /source/dnode/vnode/src/bse/bseTableMgt.c	行号: 378
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 376: } 377: } 378: void blockFree(void *pBlock) { taosMemoryFree(pBlock); } 379: 380: int32_t tableReaderMgtInit(STableReaderMgt *pReader, SBse *pBse, int64_t timestamp) {	
	污染路径: 函数'taosMemoryFree'的定义点.	
442	缺陷发生位置: /utils/test/c/tmq_write_raw_with_blob_test.c	行号: 84
	步骤描述: 不使用返回值的函数'tmq_free_json_meta'应使用'void'说明.	
	代码片段: 82: char* result = tmq_get_json_meta(msg);	

	83: printf("meta result: %s\n", result); 84: tmq_free_json_meta(result); 85: } 86:	
	污染路径: 函数'tmq_free_json_meta'的定义点.	
443	缺陷发生位置: /source/client/wrapper/src/clientJniConnector.c	行号: 69
	步骤描述: 不使用返回值的函数'taosMsleep'应使用'void'说明.	
	代码片段: 67: case 1: 68: do { 69: taosMsleep(0); 70: } while (atomic_load_32(&__init) == 1); 71: case 2:	
	污染路径: 函数'taosMsleep'的定义点.	
444	缺陷发生位置: /docs/examples/c/query_data_demo.c	行号: 82
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 80: // close & clean 81: taos_close(taos); 82: taos_cleanup(); 83: return 0; 84: // ANCHOR_END: query_data	
	污染路径: 函数'taos_cleanup'的定义点.	
445	缺陷发生位置: /source/dnode/mnode/impl/src/mndTopic.c	行号: 937
	步骤描述: 不使用返回值的函数'STR_TO_VARSTR'应使用'void'说明.	
	代码片段: 935: size_t len = strlen(string); 936: if (string && len <= TSDB_SHOW_SCHEMA_JSON_LEN) { 937: STR_TO_VARSTR(schemaJson, string); 938: } else { 939: mError("mndRetrieveTopic build schema error json:%p,	

	json len:%zu", string, len);	
	污染路径: 函数'STR_TO_VARSTR'的定义点.	
446	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttLogging.c	行号: 272
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 270: log_line_pos = 0; 271: } 272: UNUSED(vsnprintf(&log_line[log_line_pos], sizeof(log_line) - log_line_pos, fmt, va)); 273: log_line[sizeof(log_line) - 1] = '\0'; /* Ensure string is null terminated. */ 274:	
	污染路径: 函数'UNUSED'的定义点.	
447	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqPacket.c	行号: 162
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 160: 161: if (ttq->current_out_packet) { 162: UNUSED(ttqMuxAddOut(ttq)); 163: } 164:	
	污染路径: 函数'UNUSED'的定义点.	
448	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmNodes.c	行号: 174
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 172: if (count == 10) { 173: if(osUpdate() != 0) { 174: dError("failed to update os info"); 175: } 176: count = 0;	

	污染路径: 函数'dError'的定义点.	
449	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmEnv.c	行号: 62
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 60: 61: if (taosBlockSIGPIPE() != 0) { 62: dError("failed to block SIGPIPE"); 63: } 64:	
	污染路径: 函数'dError'的定义点.	
450	缺陷发生位置: /source/util/src/tdigest.c	行号: 256
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 254: 255: if (tdigestCompress(t) != 0) { 256: uError("failed to compress t-digest"); 257: } 258: if (t->num_centroids == 0) return NAN;	
	污染路径: 函数'uError'的定义点.	
451	缺陷发生位置: /source/libs/qworker/src/qwUtil.c	行号: 231
	步骤描述: 不使用返回值的函数'QW_TASK_DLOG_E'应使用'void'说明.	
	代码片段: 229: *ctx = taosHashGet(mgmt->ctxHash, id, sizeof(id)); 230: if (NULL == (*ctx)) { 231: QW_TASK_DLOG_E("get task ctx not exist, may be dropped"); 232: QW_ERR_RET(QW_CTX_NOT_EXISTS_ERR_CODE(mgmt)); 233: }	
	污染路径: 函数'QW_TASK_DLOG_E'的定义点.	

452	缺陷发生位置: /source/libs/scalar/src/filter.c	行号: 5653
	步骤描述: 不使用返回值的函数'fltError'应使用'void'说明.	
	代码片段: 5651: 5652: if (*p == NULL) { 5653: fltError("filterExecute failed, column data is NULL"); 5654: FLT_ERR_JRET(TSDB_CODE_APP_ERROR); 5655: }	
	污染路径: 函数'fltError'的定义点.	
453	缺陷发生位置: /source/util/test/trefTest.c	行号: 61
	步骤描述: 不使用返回值的函数'taosUsleep'应使用'void'说明.	
	代码片段: 59: } 60: 61: taosUsleep(100); 62: } 63:	
	污染路径: 函数'taosUsleep'的定义点.	
454	缺陷发生位置: /source/os/src/osThread.c	行号: 110
	步骤描述: 不使用返回值的函数'OS_PARAM_CHECK'应使用'void'说明.	
	代码片段: 108: int32_t taosThreadAttrGetStackSize(const TdThreadAttr *attr, size_t *stacksize) { 109: OS_PARAM_CHECK(attr); 110: OS_PARAM_CHECK(stacksize); 111: int32_t code = pthread_attr_getstacksize(attr, stacksize); 112: if (code) {	
	污染路径: 函数'OS_PARAM_CHECK'的定义点.	
455	缺陷发生位置: /test/new_test_framework/script/api/insertBlob.c	行号: 122

	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 120: 121: // query the records 122: sprintf(qstr, "SELECT * FROM m1 order by ts desc"); 123: result = taos_query(taos, qstr); 124: if (result == NULL taos_errno(result) != 0) {	
	污染路径: 无	
456	缺陷发生位置:	行号:
	/source/dnode/vnode/src/tsdb/tsdbMigrate.c	169
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 167: if (code != TSDB_CODE_SUCCESS) { 168: tsdbError("vgld:%d, fid:%d, failed to read manifest from shared storage since %s", vid, fid, tstrerror(code)); 169: taosMemoryFree(buf); 170: return code; 171: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
	缺陷发生位置:	行号:
	/source/client/src/clientMonitor.c	183
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
457	代码片段: 181: } 182: } 183: taosMemoryFreeClear(pCont); 184: } 185:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
458	缺陷发生位置:	行号:
	/source/libs/sync/test/sync_test_lib/src/syncRaftStoreDebug.c	83
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段:	

	81: char *serialized = raftStore2Str(pObj); 82: sTrace("raftStoreLog2 len:%d %s %s", (int32_t)strlen(serialized), s, serialized); 83: taosMemoryFree(serialized); 84: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
459	缺陷发生位置: /source/libs/new-stream/src/streamRunner.c	行号: 522
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 520: ST_TASK_ELOG("%s failed to get trigger calc params for win index:%d, size:%d", __FUNCTION__, curWinIdx, 521: (int32_t)pExec->runtimeInfo.funcInfo.pStreamPesudoFuncVals->size); 522: taosMemoryFreeClear(pContent); 523: code = TSDB_CODE_MND_STREAM_INTERNAL_ERROR; 524: goto _exit;	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
460	缺陷发生位置: /source/dnode/vnode/src/meta/metaSnapshot.c	行号: 66
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 64: if (ppReader && *ppReader) { 65: tdbTbcClose((*ppReader)->pTbc); 66: taosMemoryFree(*ppReader); 67: *ppReader = NULL; 68: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
461	缺陷发生位置: /source/util/src/theap.c	行号: 339
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 337: if (code) {	

	338: destroyPriorityQueue(q->queue); 339: taosMemoryFree(q); 340: terrno = code; 341: return NULL;	
	污染路径: 函数'taosMemoryFree'的定义点.	
462	缺陷发生位置: /source/dnode/vnode/src/meta/metaCache.c	行号: 400
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 398: if (pEntry) { 399: *ppEntry = pEntry->next; 400: taosMemoryFree(pEntry); 401: pCache->sEntryCache.nEntry--; 402: if (pCache->sEntryCache.nEntry < pCache->sEntryCache.nBucket / 4 &&	
	污染路径: 函数'taosMemoryFree'的定义点.	
463	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 1008
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 1006: size_t *keys_list_sizes = taosMemoryCalloc(2, sizeof(size_t)); 1007: if (!keys_list_sizes) { 1008: taosMemoryFree(keys_list); 1009: return terrno; 1010: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
464	缺陷发生位置: /source/libs/index/src/indexTfile.c	行号: 444
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 442: tem->suid, tem->colName, tem->colVal, offset, (int)taosArrayGetSize(tr->total), cost); 443: }	

	444: taosMemoryFree(p); 445: fstSliceDestroy(&key); 446: return 0;	
	污染路径: 函数'taosMemoryFree'的定义点.	
465	缺陷发生位置: /source/util/src/tlruCache.c	行号: 205
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 203: } 204: 205: taosMemoryFree(table->list); 206: table->list = newList; 207: table->lengthBits = newLengthBits;	
	污染路径: 函数'taosMemoryFree'的定义点.	
466	缺陷发生位置: /source/dnode/mgmt/node_util/src/dmFile.c	行号: 415
	步骤描述: 不使用返回值的函数'encryptDebug'应使用'void'说明.	
	代码片段: 413: } 414: 415: encryptDebug("succeed to read checkCode file:%s", file); 416: 417: int len = ENCRYPTED_LEN(size);	
	污染路径: 函数'encryptDebug'的定义点.	
467	缺陷发生位置: /source/dnode/vnode/src/bse/bseTable.c	行号: 614
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 612: bseError("failed to close table builder file %s since %s", p->name, tsterror(errno)); 613: } 614: taosMemoryFree(p); 615: }	

	616:	
	污染路径: 函数'taosMemoryFree'的定义点.	
468	缺陷发生位置: /source/dnode/mnode/impl/src/mndInstance.c	行号: 247
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 245: 246: if (req.id[0] == 0) { 247: mError("instance register, id is empty"); 248: return TSDB_CODE_INVALID_PARA; 249: }	
	污染路径: 函数'mError'的定义点.	
469	缺陷发生位置: /source/libs/tdb/test/tdbTest.cpp	行号: 268
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 266: 267: for (int iData = 1; iData <= nData; iData++) { 268: sprintf(key, "key%d", iData); 269: sprintf(val, "value%d", iData); 270: ret = tdbTbInsert(pDb, key, strlen(key), val, strlen(val), txn);	
	污染路径: 无	
470	缺陷发生位置: /source/libs/qworker/src/qwMsg.c	行号: 539
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: 537: if (msgSize < 0) { 538: QW_SCH_ELOG("tSerializeSSchedulerHbReq hbReq failed, size:%d", msgSize); 539: rpcFreeCont(msg); 540: QW_ERR_RET(msgSize); 541: }	

	污染路径: 函数'rpcFreeCont'的定义点.	
471	缺陷发生位置: /test/new_test_framework/script/api/apitest.c	行号: 28
	步骤描述: 不使用返回值的函数'taos_select_db'应使用'void'说明.	
	代码片段: 26: taos_free_result(result); 27: usleep(100000); 28: taos_select_db(taos, "test"); 29: 30: result = taos_query(taos, "create table meters(ts timestamp, a int) tags(area int);");	
	污染路径: 函数'taos_select_db'的定义点.	
472	缺陷发生位置: /source/libs/executor/src/tsort.c	行号: 137
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 135: } 136: 137: static void destoryAllocatedTuple(void* t) { taosMemoryFree(t); } 138: 139: #define tupleOffset(tuple, colldx) ((uint32_t*) (tuple + sizeof(uint32_t) * colldx))	
	污染路径: 函数'taosMemoryFree'的定义点.	
473	缺陷发生位置: /source/libs/executor/src/projectoperator.c	行号: 75
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 73: 74: blockDataDestroy(pInfo->pFinalRes); 75: taosMemoryFreeClear(param); 76: } 77:	
	污染路径:	

	函数'taosMemoryFreeClear'的定义点.	
474	缺陷发生位置: /contrib/libmqt/ttq/src/ttqSubs.c	行号: 181
	步骤描述: 不使用返回值的函数'ttq_free'应使用'void'说明.	
	代码片段: 179: HASH_DELETE(hh, subhier->shared, shared); 180: ttq_free(shared->name); 181: ttq_free(shared); 182: } 183: ttq_free(leaf);	
	污染路径: 函数'ttq_free'的定义点.	
475	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmFile.c	行号: 233
	步骤描述: 不使用返回值的函数'dlInfo'应使用'void'说明.	
	代码片段: 231: } 232: 233: dlInfo("successfully set and persisted encrypted flag in mnode.json"); 234: return 0; 235: }	
	污染路径: 函数'dlInfo'的定义点.	
476	缺陷发生位置: /source/util/src/talgo.c	行号: 154
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 152: } 153: tqsortImpl(src, 0, (int32_t)numOfElem - 1, (int32_t)size, param, comparFn, buf); 154: taosMemoryFreeClear(buf); 155: return 0; 156: }	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
477	缺陷发生位置:	行号:

	/source/util/src/tcompression.c	608
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: <pre> 606: output[pos] = t; 607: } else { 608: uError("Invalid compress bool value:%d", output[pos]); 609: return TSDB_CODE_INVALID_PARA; 610: } </pre>	
478	污染路径: 函数'uError'的定义点.	
	缺陷发生位置: /source/common/src/tvariant.c	行号: 257
	步骤描述: 不使用返回值的函数'SET_ERRNO'应使用'void'说明.	
	代码片段: <pre> 255: 256: int32_t toUIntegerEx(const char *z, int32_t n, uint32_t type, uint64_t *value) { 257: SET_ERRNO(0); 258: char *endPtr = NULL; 259: const char *p = z; </pre>	
	污染路径: 函数'SET_ERRNO'的定义点.	
479	缺陷发生位置: /source/client/wrapper/jni/windows/win32/bridge/AccessBridgeCalls.c	行号: 77
	步骤描述: 不使用返回值的函数'PrintDebugString'应使用'void'说明.	
	代码片段: <pre> 75: 76: LOAD_FP(SetJavaShutdown, SetJavaShutdownFP, "setJavaShutdownFP"); 77: LOAD_FP(SetFocusGained, SetFocusGainedFP, "setFocusGainedFP"); 78: LOAD_FP(SetFocusLost, SetFocusLostFP, "setFocusLostFP"); 79: </pre>	
	污染路径: expanded from macro 'LOAD_FP'	

	函数'PrintDebugString'的定义点. expanded from macro 'LOAD_FP'	
480	缺陷发生位置: /source/os/test/osStringTests.cpp	行号: 314
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 312: 313: // 测试大于 INT16 范围的值 314: snprintf(large_num, sizeof(large_num), "%lld", (long long)INT16_MAX + 1); 315: result = taosStr2int16(large_num, &val); 316: ASSERT_EQ(result, TAOS_SYSTEM_ERROR(ERANGE));	
	污染路径: 无	
481	缺陷发生位置: /source/util/src/tqueue.c	行号: 110
	步骤描述: 不使用返回值的函数'uDebug'应使用'void'说明.	
	代码片段: 108: taosMemoryFree(queue); 109: 110: uDebug("queue:%p is closed", queue); 111: } 112:	
	污染路径: 函数'uDebug'的定义点.	
482	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 1349
	步骤描述: 不使用返回值的函数'buffer_skip_whitespace'应使用'void'说明.	
	代码片段: 1347: /* parse next value */ 1348: input_buffer->offset++; 1349: buffer_skip_whitespace(input_buffer); 1350: if (!parse_value(current_item, input_buffer)) 1351: {	
	污染路径: 函数'buffer_skip_whitespace'的定义点.	
483	缺陷发生位置:	行号:

	/source/libs/command/src/explain.c	191
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: <pre> 189: SArray *rows = taosArrayInit(10, sizeof(SQueryExplainRowInfo)); 190: if (NULL == rows) { 191: qError("taosArrayInit SQueryExplainRowInfo failed"); 192: QRY_ERR_JRET(terrno); 193: } </pre>	
	污染路径: 函数'qError'的定义点.	
484	缺陷发生位置: /source/libs/new-stream/src/streamHb.c	行号: 136
	步骤描述: 不使用返回值的函数'stTrace'应使用'void'说明.	
	代码片段: <pre> 134: stError("%s failed at line %d, error:%s", __FUNCTION__, lino, tstrerror(code)); 135: } else { 136: stTrace("stream hb end"); 137: } 138: } </pre>	
	污染路径: 函数'stTrace'的定义点.	
485	缺陷发生位置: /source/libs/catalog/src/ctgUtil.c	行号: 1115
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 1113: ctgFreeSMetaData(&pJob->jobRes); 1114: 1115: taosMemoryFree(job); 1116: 1117: qTrace("QID:0x%" PRIx64 " , job:0x%" PRIx64 " , catalog job freed", qid, rid); </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
486	缺陷发生位置: /test/new_test_framework/script/api/insertSameTs.c	行号: 86

	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 84: for (i = 0; i < 10; ++i) { 85: int32_t v = n * 10 + i; 86: sprintf(qstr, "insert into m1 values (%" PRId64 ", %d, %d, %d, %d, %f, %lf, '%s')", (uint64_t)1546300800000, v, v, v, v*10000000, v*1.0, v*2.0, "hello"); 87: printf("qstr: %s\n", qstr); 88:	
	污染路径: 无	
487	缺陷发生位置: /source/libs/qworker/src/qwMem.c	行号: 110
	步骤描述: 不使用返回值的函数'QW_TASK_DLOG_E'应使用'void'说明.	
	代码片段: 108: TAOS_UNUSED(taosHashRemove(gQueryMgmt.pJobInfo, id2, sizeof(id2))); 109: 110: QW_TASK_DLOG_E("the whole query job removed"); 111: // } else { 112: // QW_TASK_DLOG("job not removed, remainSessions: %d, %d", taosHashGetSize(pJobInfo->pSessions), pJobInfo->memInfo->remainSession);	
	污染路径: 函数'QW_TASK_DLOG_E'的定义点.	
488	缺陷发生位置: /source/libs/executor/src/executorInt.c	行号: 322
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 320: QUERY_CHECK_CODE(code, lino, _end); 321: } 322: taosMemoryFree(tmp); 323: } 324:	
	污染路径: 函数'taosMemoryFree'的定义点.	

489	缺陷发生位置: /utils/test/c/tmqDemo.c	行号: 137
	步骤描述: 不使用返回值的函数'tstrncpy'应使用'void'说明.	
	代码片段: 135: tstrncpy(configDir, argv[++i], PATH_MAX); 136: } else if (strcmp(argv[i], "-s") == 0) { 137: tstrncpy(g_stConflInfo.stbName, argv[++i], sizeof(g_stConflInfo.stbName)); 138: } else if (strcmp(argv[i], "-w") == 0) { 139: tstrncpy(g_stConflInfo.vnodeWalPath, argv[++i], sizeof(g_stConflInfo.vnodeWalPath));	
	污染路径: 函数'tstrncpy'的定义点.	
490	缺陷发生位置: /source/client/test/clientTests.cpp	行号: 1917
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 1915: ASSERT_NE(pRootConn, nullptr); 1916: 1917: sprintf(sql, "drop database if exists %s", database); 1918: TAOS_RES* pRes = taos_query(pRootConn, sql); 1919: ASSERT_EQ(taos_errno(pRes), TSDB_CODE_SUCCESS);	
	污染路径: 无	
491	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 2607
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 2605: 2606: char single_stable_model[32] = {0}; 2607: sprintf(cache, "SINGLE_STABLE_MODEL %d", 2608: dblInfo->single_stable_model?1:0); 2609:	
	污染路径: 无	
492	缺陷发生位置: /source/util/src/tskiplist.c	行号: 403

	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 401: if (pSkipList->lock) { 402: if (taosThreadRwlockRdlock(pSkipList->lock) != 0) { 403: uError("failed to lock skip list"); 404: } 405: }	
	污染路径: 函数'uError'的定义点.	
493	缺陷发生位置: /source/dnode/mnode/impl/src/mndArbGroup.c	行号: 533
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: 531: if (epSet.numOfEps == 0) { 532: mError("arbgroup:%d, failed to send check-sync request since no epSet found", vglId); 533: rpcFreeCont(pHead); 534: code = -1; 535: if (terrno != 0) code = terrno;	
	污染路径: 函数'rpcFreeCont'的定义点.	
494	缺陷发生位置: /contrib/test/cos/main.c	行号: 126
	步骤描述: 不使用返回值的函数'cos_warn_log'应使用'void'说明.	
	代码片段: 124: 125: char *pstr3 = apr_pstrndup(p, (char *)fmt_str->pos, cos_buf_size(fmt_str)); 126: cos_warn_log("Format string: %s", pstr3); 127: 128: // Format-String sha1hash	
	污染路径: 函数'cos_warn_log'的定义点.	
495	缺陷发生位置: /tools/shell/src/shellCommand.c	行号: 335
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	

	代码片段: 333: } 334: 335: taosMemoryFree(total); 336: return false; 337: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
496	缺陷发生位置: /source/util/src/ttimer.c	行号: 633
	步骤描述: 不使用返回值的函数'tstrncpy'应使用'void'说明.	
	代码片段: 631: } 632: 633: tstrncpy(ctrl->label, label, sizeof(ctrl->label)); 634: 635: tmrDebug("%s timer controller is initialized, number of timer controllers: %d.", label, numOfTmrCtrl);	
	污染路径: 函数'tstrncpy'的定义点.	
497	缺陷发生位置: /source/common/src/tencrypt.c	行号: 336
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 334: } 335: if (plainBuf != NULL) { 336: taosMemoryFree(plainBuf); 337: } 338: if (encryptedBuf != NULL) {	
	污染路径: 函数'taosMemoryFree'的定义点.	
498	缺陷发生位置: /docs/examples/c/schemaless.c	行号: 40
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 38: result = taos_query(taos, "drop database if exists db;"); 39: taos_free_result(result);	

	40: usleep(100000); 41: result = taos_query(taos, "create database db precision 'ms';"); 42: taos_free_result(result);	
	污染路径: 无	
499	缺陷发生位置: /source/dnode/vnode/src/bse/bseCache.c	行号: 241
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 239: 240: (void)taosThreadMutexDestroy(&pCache->mutex); 241: taosMemoryFree(pCache); 242: } 243: void lruCacheClear(SLruCache *pCache) {	
	污染路径: 函数'taosMemoryFree'的定义点.	
500	缺陷发生位置: /source/libs/wal/src/walMgmt.c	行号: 379
	步骤描述: 不使用返回值的函数'setThreadName'应使用'void'说明.	
	代码片段: 377: 378: static void *walThreadFunc(void *param) { 379: setThreadName("wal"); 380: while (1) { 381: walUpdateSeq();	
	污染路径: 函数'setThreadName'的定义点.	
501	缺陷发生位置: /docs/examples/c/asyncdemo.c	行号: 115
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 113: queryDB(taos, sql); 114: 115: sprintf(sql, "use %s", db); 116: queryDB(taos, sql); 117:	

	污染路径: 无	
502	缺陷发生位置: /utils/test/c/createTable.c	行号: 60
	步骤描述: 不使用返回值的函数'pPrint'应使用'void'说明.	
	代码片段: 58: 59: void createDbAndStb() { 60: pPrint("start to create db and stable"); 61: char qstr[64000]; 62:	
	污染路径: 函数'pPrint'的定义点.	
503	缺陷发生位置: /source/libs/parser/src/parUtil.c	行号: 745
	步骤描述: 不使用返回值的函数'tstrncpy'应使用'void'说明.	
	代码片段: 743: char buf[10] = {0}; 744: if (NULL == p) { 745: tstrncpy(buf, pStart, 10); 746: *pNext = NULL; 747: } else {	
	污染路径: 函数'tstrncpy'的定义点.	
504	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReadUtil.c	行号: 145
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 143: } 144: if (p) { 145: taosMemoryFreeClear(p); 146: } 147: return code;	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
505	缺陷发生位置:	行号:

	/source/libs/new-stream/src/streamTriggerTask.c	5287
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 5285: QUERY_CHECK_CODE(code, lino, _end); 5286: } 5287: taosMemoryFree(buf); 5288: pContext->haveReadCheckpoint = true; 5289: } else { </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
506	缺陷发生位置: /source/libs/function/src/builtinsimpl.c	行号: 321
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: <pre> 319: plter->pPrevPk = taosMemoryMalloc(plter->pPkCol- >info.bytes); 320: if (NULL == plter->pPrevPk) { 321: qError("out of memory when function get input row."); 322: taosMemoryFree(plter->pPrevData); 323: return terrno; </pre>	
	污染路径: 函数'qError'的定义点.	
507	缺陷发生位置: /source/libs/executor/src/filloperator.c	行号: 189
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: <pre> 187: SExecTaskInfo* pTaskInfo = pOperator->pTaskInfo; 188: if (pInfo == NULL pTaskInfo == NULL) { 189: qError("%s failed at line %d since pInfo or pTaskInfo is NULL.", __func__, __LINE__); 190: return NULL; 191: } </pre>	
	污染路径: 函数'qError'的定义点.	
508	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 705

	步骤描述: 不使用返回值的函数'BIT_reloadDStream'应使用'void'说明.	
	代码片段: 703: memcpy(&DTableH, dt, sizeof(DTableH)); 704: DStatePtr->state = BIT_readBits(bitD, DTableH.tableLog); 705: BIT_reloadDStream(bitD); 706: DStatePtr->table = dt + 1; 707: }	
	污染路径: 函数'BIT_reloadDStream'的定义点.	
509	缺陷发生位置: /test/new_test_framework/script/api/stmtBatchTest.c	行号: 227
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 225: taosMemoryFree(v->br); 226: taosMemoryFree(v->nr); 227: taosMemoryFree(v); 228: taosMemoryFree(lb); 229: taosMemoryFree(params);	
	污染路径: 函数'taosMemoryFree'的定义点.	
510	缺陷发生位置: /docs/examples/c/error_handle_example.c	行号: 23
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 21: taos_close(taos); 22: } 23: taos_cleanup(); 24: }	
	污染路径: 函数'taos_cleanup'的定义点.	
511	缺陷发生位置: /source/libs/executor/src/externalwindowoperator.c	行号: 419
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 417: if (pInfo->ownTGrpCtx && pInfo->pTGrpCtx) {	

	<pre> 418: extWinDestroyTGrpCtx(plInfo->pTGrpCtx); 419: taosMemoryFree(plInfo->pTGrpCtx); 420: plInfo->pTGrpCtx = NULL; 421: }</pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
512	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 706
	步骤描述: 不使用返回值的函数'varDataSetLen'应使用'void'说明.	
	代码片段: <pre> 704: int32_t resLen = orgLen - pos; 705: (void)memcpy(varDataVal(output), orgStr + pos, resLen); 706: varDataSetLen(output, resLen); 707: return TSDB_CODE_SUCCESS; 708: }</pre>	
	污染路径: 函数'varDataSetLen'的定义点.	
513	缺陷发生位置: /utlis/test/c/tmqDemo.c	行号: 237
	步骤描述: 不使用返回值的函数'pError'应使用'void'说明.	
	代码片段: <pre> 235: int code = taos_errno(pRes); 236: if (code != 0) { 237: pError("failed to reason:%s, sql: %s", tstrerror(code), command); 238: taos_free_result(pRes); 239: return -1; </pre>	
	污染路径: 函数'pError'的定义点.	
514	缺陷发生位置: /docs/examples/c-ws-new/sml_insert_demo.c	行号: 48
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 46: fprintf(stderr, "Failed to execute use power, ErrCode: 0x %x, ErrMessage: %s\n.", code, taos_errstr(result)); 47: taos_close(taos); </pre>	

	48: taos_cleanup(); 49: return -1; 50: }	
	污染路径: 函数'taos_cleanup'的定义点.	
515	缺陷发生位置: /source/libs/executor/src/dataDeleter.c	行号: 215
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 213: if (NULL == pDeleter->nextOutput.pData) { 214: if (!pDeleter->queryEnd) { 215: qError("empty res while query not end in data deleter"); 216: return TSDB_CODE_QRY_EXECUTOR_INTERNAL_ERROR; 217: }	
	污染路径: 函数'qError'的定义点.	
516	缺陷发生位置: /test/new_test_framework/script/api/demoapi.c	行号: 113
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 111: // (i%2)?"朝阳区":"黄浦区", 112: // (i%2)?"长安街":"中山路"); 113: sprintf(command, "create table t%"PRIu64" using meters " 114: "tags(%"PRIu64", '%s', '%s', '%s');", 115: i, i,	
	污染路径: 无	
517	缺陷发生位置: /source/dnode/vnode/src/meta/metaQuery.c	行号: 537
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 535: metaULock(pMeta); 536: } 537: tdbFree(pData);	

	538: return pSchema; 539:	
	污染路径: 函数'tdbFree'的定义点.	
518	缺陷发生位置: /source/libs/function/src/functionMgt.c	行号: 911
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 909: int32_t fmSetStreamPseudoFuncParamVal(int32_t funcId, SNodeList* pParamNodes, const SStreamRuntimeFuncInfo* pStreamRuntimeInfo) { 910: if (!pStreamRuntimeInfo) { 911: uError("internal error, should have pVals for stream pseudo funcs"); 912: return TSDB_CODE_INTERNAL_ERROR; 913: }	
	污染路径: 函数'uError'的定义点.	
519	缺陷发生位置: /utlis/test/c/get_db_name_test.c	行号: 37
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 35: 36: code = taos_get_current_db(taos, NULL, 0, NULL); 37: ASSERT(code != 0); 38: 39: int required = 0;	
	污染路径: 函数'ASSERT'的定义点.	
520	缺陷发生位置: /source/util/test/timerTest.cpp	行号: 275
	步骤描述: 不使用返回值的函数'adjustSystemTime'应使用'void'说明.	
	代码片段: 273: 274: // Restore time 275: adjustSystemTime(3600LL * 1000); 276: taosTmrCleanUp(ctrl);	

	277: }	
	污染路径: 函数'adjustSystemTime'的定义点.	
521	缺陷发生位置: /source/client/src/clientFirewall.c	行号: 326
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 324: tscWarn("sql security: failed to open rule file for writing: %s", ruleFile); 325: } 326: taosMemoryFree(jsonStr); 327: } 328: }</pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
522	缺陷发生位置: /source/libs/parser/src/parser.c	行号: 720
	步骤描述: 不使用返回值的函数'taosCleanupKeywordsTable'应使用'void'说明.	
	代码片段: <pre> 718: int32_t qInitKeywordsTable() { return taosInitKeywordsTable(); } 719: 720: void qCleanupKeywordsTable() { taosCleanupKeywordsTable(); } 721: 722: int32_t qStmtBindParams(SQuery* pQuery, TAOS_MULTI_BIND* pParams, int32_t colIdx, void *charsetCxt) {</pre>	
	污染路径: 函数'taosCleanupKeywordsTable'的定义点.	
523	缺陷发生位置: /source/libs/transport/src/transSvr.c	行号: 299
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 297: pWhiteList->pList = taosHashInit(1024, taosGetDefaultHashFunction(TSDB_DATA_TYPE_BINARY), 0, HASH_NO_LOCK);</pre>	

	298: if (pWhiteList->pList == NULL) { 299: taosMemoryFree(pWhiteList); 300: return NULL; 301: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
524	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 606
	步骤描述: 不使用返回值的函数'OS_PARAM_CHECK'应使用'void'说明.	
	代码片段: 604: 605: int32_t taosGetOsReleaseName(char *releaseName, char* sName, char* ver, int32_t maxLen) { 606: OS_PARAM_CHECK(releaseName); 607: #ifdef WINDOWS 608: if (!getWinVersionReleaseName(releaseName, maxLen)) {	
	污染路径: 函数'OS_PARAM_CHECK'的定义点.	
525	缺陷发生位置: /contrib/TSZ/sz/src/dataCompression.c	行号: 390
	步骤描述: 不使用返回值的函数'computeReqLength_float'应使用'void'说明.	
	代码片段: 388: 389: int reqLength; 390: computeReqLength_float(precision, radExpo, &reqLength, medianValue); 391: 392: *reqBytesLength = reqLength/8;	
	污染路径: 函数'computeReqLength_float'的定义点.	
526	缺陷发生位置: /utills/test/c/createTable.c	行号: 79
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 77: taos_free_result(pRes); 78:	

	79: sprintf(qstr, "use %s", dbName); 80: pRes = taos_query(con, qstr); 81: code = taos_errno(pRes);	
	污染路径: 函数'sprintf'的定义点.	
527	缺陷发生位置: /utls/test/c/tmq_token.c	行号: 50
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 48: printf("token: %s\n", (char *)(*row)); 49: token = strdup((char *)(*row)); 50: ASSERT(token != NULL); 51: } 52: taos_free_result(pRes);	
	污染路径: 函数'ASSERT'的定义点.	
528	缺陷发生位置: /test/new_test_framework/script/api/stmt2-performance.c	行号: 138
	步骤描述: 不使用返回值的函数'clock_gettime'应使用'void'说明.	
	代码片段: 136: bind_time += ((double)(end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9); 137: 138: clock_gettime(CLOCK_MONOTONIC, &start); // 获取开始时间 139: // exec 140: if (taos_stmt2_exec(stmt, NULL)) {	
	污染路径: 无	
529	缺陷发生位置: /contrib/test/traft/join_into_vgroup/node_follower10001.c	行号: 67
	步骤描述: 不使用返回值的函数'printRaftState'应使用'void'说明.	
	代码片段: 65: struct raft *r = getRaft(&raftEnv, 100); 66: assert(r != NULL); 67: printRaftState(r); 68:	

	69: // submit value	
	污染路径: 函数'printRaftState'的定义点.	
530	缺陷发生位置: /source/common/src/msg/xnode.c	行号: 466
	步骤描述: 不使用返回值的函数'DECODESQL'应使用'void'说明.	
	代码片段: 464: 465: TAOS_CHECK_EXIT(tStartDecode(&decoder)); 466: DECODESQL(); 467: TAOS_CHECK_EXIT(tDecodeI32(&decoder, &pReq->xnodeId)); 468: TAOS_CHECK_EXIT(xDecodeCowStr(&decoder, &pReq->name, true));	
	污染路径: 函数'DECODESQL'的定义点.	
531	缺陷发生位置: /source/libs/executor/src/executil.c	行号: 218
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 216: } 217: 218: static void freeEx(void* p) { taosMemoryFree(*(void**)p); } 219: 220: void cleanupGroupResInfo(SGroupResInfo* pGroupResInfo) {	
	污染路径: 函数'taosMemoryFree'的定义点.	
532	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSttFileRW.c	行号: 138
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 136: reader->footer->statisBlkPtr->size, 0, pEncryptData); 137: if (code) { 138: taosMemoryFree(data);	

	139: return code; 140: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
533	缺陷发生位置: /source/libs/qworker/src/qwMem.c	行号: 87
	步骤描述: 不使用返回值的函数'QW_TASK_ELOG'应使用'void'说明.	
	代码片段: 85: int32_t code = taosHashRemove(pJobInfo->pSessions, id, sizeof(id)); 86: if (TSDB_CODE_SUCCESS != code && needRemoveHashKey) { 87: QW_TASK_ELOG("fail to remove session from query session hash, error(0x%x):%s", code, strerror(code)); 88: return; 89: }	
	污染路径: 函数'QW_TASK_ELOG'的定义点.	
534	缺陷发生位置: /contrib/test/traft/cluster/node10001.c	行号: 112
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 110: // for test: submit a value every second 111: while (1) { 112: sleep(1); 113: submitValue(); 114: }	
	污染路径: 无	
535	缺陷发生位置: /source/libs/planner/src/planLogicCreator.c	行号: 1769
	步骤描述: 不使用返回值的函数'planError'应使用'void'说明.	
	代码片段: 1767: SNode* pRangeInterval = ((SRangeAroundNode*)pSelect->pRangeAround)->pInterval; 1768: if (!pRangeInterval nodeType(pRangeInterval) != QUERY_NODE_VALUE) {	

	1769: planError("%s failed at line %d since range interval is invalid", __func__, __LINE__); 1770: PLAN_ERR_JRET(TSDB_CODE_PLAN_INTERNAL_ERROR); 1771: } else {	
	污染路径: 函数'planError'的定义点.	
536	缺陷发生位置: /contrib/TSZ/sz/src/iniparser.c	行号: 336
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 334: 335: seclen = (int)strlen(s); 336: sprintf(keym, "%s:", s); 337: 338: i = 0;	
	污染路径: 函数'sprintf'的定义点.	
537	缺陷发生位置: /source/client/src/clientHb.c	行号: 659
	步骤描述: 不使用返回值的函数'tscWarn'应使用'void'说明.	
	代码片段: 657: if (pRsp->query->pQnodeList) { 658: if (TSDB_CODE_SUCCESS != updateQnodeList(pTscObj->pAppInfo, pRsp->query->pQnodeList)) { 659: tscWarn("update qnode list failed"); 660: } 661: }	
	污染路径: 函数'tscWarn'的定义点.	
538	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqHandleConnect.c	行号: 226
	步骤描述: 不使用返回值的函数'ttqCxtAddToByld'应使用'void'说明.	
	代码片段: 224: connection_check_acl(context, &context->msgs_out.queued);	

	225: 226: ttqCxtAddToByld(context); 227: 228: #ifdef WITH_PERSISTENCE	
	污染路径: 函数'ttqCxtAddToByld'的定义点.	
539	缺陷发生位置: /source/libs/tmqtt/tools/src/topic-producer.c	行号: 277
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 275: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x%x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 276: taos_errstr(NULL)); 277: taos_cleanup(); 278: return -1; 279: }	
	污染路径: 函数'taos_cleanup'的定义点.	
540	缺陷发生位置: /docs/examples/c-ws-new/stmt_insert_demo.c	行号: 75
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 73: sprintf(table_name, "d_bind_%d", i); 74: char location[20]; 75: sprintf(location, "location_%d", i); 76: 77: // set table name and tags	
	污染路径: 无	
541	缺陷发生位置: /source/libs/command/src/command.c	行号: 285
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 283: } 284: if (pBlock->info.rows <= 0) { 285: qError("no permission to view any columns"); 286: return TSDB_CODE_PAR_PERMISSION_DENIED;	

	287: }	
	污染路径: 函数'qError'的定义点.	
542	缺陷发生位置: /source/libs/new-stream/src/stream.c	行号: 41
	步骤描述: 不使用返回值的函数'stInfo'应使用'void'说明.	
	代码片段: 39: } 40: 41: stInfo("snode disabled"); 42: gStreamMgmt.snodeEnabled = false; 43: smUndeploySnodeTasks(cleanup);	
	污染路径: 函数'stInfo'的定义点.	
543	缺陷发生位置: /test/new_test_framework/script/api/demoapi.c	行号: 161
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 159: __func__, __LINE__, taos_errstr(res)); 160: } 161: sprintf(command, "insert into t%"PRId64" " 162: "values(%" PRId64 ", %f, %"PRId64", " 163: ""%c%d', '%s%c%d', '%c%d')",	
	污染路径: 无	
544	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 5750
	步骤描述: 不使用返回值的函数'tsdbDebug'应使用'void'说明.	
	代码片段: 5748: pReader = *ppReader; 5749: if (isEmptyQueryTimeWindow(&pReader->info.window) && pCond->type == TIMEWINDOW_RANGE_CONTAINED) { 5750: tsdbDebug("%p query window not overlaps with the data set, no result returned, %s", pReader, pReader->idStr); 5751: goto _end; 5752: }	

	污染路径: 函数'tsdbDebug'的定义点.	
545	缺陷发生位置: /source/libs/index/src/index.c	行号: 76
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 74: // refactor later 75: taosCleanUpScheduler(indexQhandle); 76: taosMemoryFreeClear(indexQhandle); 77: taosCloseRef(indexRefMgt); 78: }	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
546	缺陷发生位置: /docs/examples/c-ws-new/async_demo.c	行号: 102
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 100: queryDB(taos, sql); 101: 102: sprintf(sql, "use %s", db); 103: queryDB(taos, sql); 104:	
	污染路径: 无	
547	缺陷发生位置: /source/util/src/tsched.c	行号: 37
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 35: pSched = (SSchedQueue *)taosMemoryCalloc(sizeof(SSchedQueue), 1); 36: if (pSched == NULL) { 37: uError("%s: no enough memory for pSched", label); 38: return NULL; 39: }	
	污染路径: 函数'uError'的定义点.	
548	缺陷发生位置:	行号:

	/source/dnode/xnode/src/xnode.c		91
	步骤描述: 不使用返回值的函数'xndDebug'应使用'void'说明.		
	代码片段: 89: #endif 90: } 91: xndDebug("xnode get unix socket pipe path:%s", pipeName); 92: }		
	污染路径: 函数'xndDebug'的定义点.		
549	缺陷发生位置: /source/dnode/mgmt/mgmt_qnode/src/qmlnt.c		行号: 77
	步骤描述: 不使用返回值的函数'tmsgReportStartup'应使用'void'说明.		
	代码片段: 75: return code; 76: } 77: tmsgReportStartup("qnode-worker", "initialized"); 78: 79: pOutput->pMgmt = pMgmt;		
	污染路径: 函数'tmsgReportStartup'的定义点.		
550	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c		行号: 235
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.		
	代码片段: 233: int32_t code = tNameFromString(&sn, db, T_NAME_ACCT T_NAME_DB); 234: if (code != TSDB_CODE_SUCCESS) { 235: qError("%s failed at line %d since %s", __func__, __LINE__, strerror(code)); 236: return code; 237: }		
	污染路径: 函数'qError'的定义点.		
551	缺陷发生位置: /source/dnode/mgmt/mgmt_dnode/src/dmWorker.c		行号: 71
	步骤描述:		

	不使用返回值的函数'setThreadName'应使用'void'说明.	
	代码片段: <pre>69: SDnodeMgmt *pMgmt = param; 70: int64_t lastTime = taosGetTimestampMs(); 71: setThreadName("dnode-keystsync"); 72: 73: // Wait a bit before first sync attempt</pre>	
	污染路径: 函数'setThreadName'的定义点.	
552	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqNet2.c	行号: 51
	步骤描述: 不使用返回值的函数'net__cleanup'应使用'void'说明.	
	代码片段: <pre>49: spare_sock = INVALID_SOCKET; 50: } 51: net__cleanup(); 52: } 53:</pre>	
	污染路径: 函数'net__cleanup'的定义点.	
553	缺陷发生位置: /source/libs/sync/src/syncMain.c	行号: 386
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: <pre>384: code = TSDB_CODE_OUT_OF_MEMORY; 385: sError("vgId:%d, failed to serialize SVArbSetAssignedLeaderRsp", ths->vgId); 386: rpcFreeCont(pHead); 387: goto _OVER; 388: }</pre>	
	污染路径: 函数'rpcFreeCont'的定义点.	
554	缺陷发生位置: /source/libs/metrics/src/metrics.c	行号: 59
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: <pre>57: if (keys) taosMemoryFree(keys);</pre>	

	58: if (vgroup_ids) taosMemoryFree(vgroup_ids); 59: uError("failed to get vgroup ids for counter %s", counterName); 60: return; 61: }	
	污染路径: 函数'uError'的定义点.	
555	缺陷发生位置: /docs/examples/c-ws-new/tmq_demo.c	行号: 129
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 127: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x%x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 128: taos_errstr(NULL)); 129: taos_cleanup(); 130: return NULL; 131: }	
	污染路径: 函数'taos_cleanup'的定义点.	
556	缺陷发生位置: /source/libs/monitorfw/src/taos_metric_formatter_custom.c	行号: 148
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 146: taosMemoryFreeClear(arr); 147: taosMemoryFreeClear(keyvalue); 148: taosMemoryFreeClear(keyvalues); 149: 150: SJson* metric = tjsonCreateObject();	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
557	缺陷发生位置: /docs/examples/c-ws-new/stmt_insert_demo.c	行号: 182
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 180: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x%x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 181: taos_errstr(NULL));	

	182: taos_cleanup(); 183: exit(EXIT_FAILURE); 184: }	
	污染路径: 函数'taos_cleanup'的定义点.	
558	缺陷发生位置: /source/dnode/mnode/impl/src/mndVgroup.c	行号: 252
	步骤描述: 不使用返回值的函数'TAOS_RETURN'应使用'void'说明.	
	代码片段: 250: if (pVgroup) mndVgroupActionDelete(pSdb, pVgroup); 251: taosMemoryFreeClear(pRow); 252: TAOS_RETURN(code); 253: } 254:	
	污染路径: 函数'TAOS_RETURN'的定义点.	
559	缺陷发生位置: /source/dnode/vnode/src/tq/tqScan.c	行号: 77
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 75: tDeleteSchemaWrapper(pSchema); 76: if (code != TSDB_CODE_SUCCESS){ 77: taosMemoryFree(buf); 78: tqError("%s failed at line %d with msg:%s", __func__, lino, tstrerror(code)); 79: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
560	缺陷发生位置: /source/libs/command/src/command.c	行号: 536
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 534: } 535: 536: taosMemoryFree(pRetentions); 537: 538: (varDataLen(buf2)) = len;	

	污染路径: 函数'taosMemoryFree'的定义点.	
561	缺陷发生位置: /source/libs/tdb/test/tdbPageFlushTest.cpp	行号: 400
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 398: // char const *key = "key123456789"; 399: char key[32] = {0}; 400: sprintf(key, "key-%d", i); 401: ret = tdbTbInsert(pDb, key, strlen(key) + 1, val, valLen, txn); 402: GTEST_ASSERT_EQ(ret, 0);	
	污染路径: 无	
562	缺陷发生位置: /test/new_test_framework/script/api/stopquery.c	行号: 261
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 259: if (taos == NULL) sqExit("taos_connect", taos_errstr(NULL)); 260: 261: sprintf(sql, "reset query cache"); 262: sqExecSQL(taos, sql); 263:	
	污染路径: 无	
563	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRetention.c	行号: 311
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 309: } 310: 311: void tsdbRetentionCancel(void *arg) { taosMemoryFree(arg); } 312: 313: static bool tsdbFSetNeedRetention(STFileSet *fset, int32_t expLevel) {	

	污染路径: 函数'taosMemoryFree'的定义点.	
564	缺陷发生位置: /source/libs/scheduler/src/schRemote.c	行号: 663
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 661: SSchHbCallbackParam *param = taosMemoryCalloc(1, sizeof(SSchHbCallbackParam)); 662: if (NULL == param) { 663: qError("calloc SSchTaskCallbackParam failed"); 664: SCH_ERR_RET(terrno); 665: }	
	污染路径: 函数'qError'的定义点.	
565	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmMgmt.c	行号: 128
	步骤描述: 不使用返回值的函数'indexCleanup'应使用'void'说明.	
	代码片段: 126: rpcCleanup(); 127: streamCleanup(); 128: indexCleanup(); 129: taosConvDestroy(); 130:	
	污染路径: 函数'indexCleanup'的定义点.	
566	缺陷发生位置: /source/os/src/osThread.c	行号: 22
	步骤描述: 不使用返回值的函数'OS_PARAM_CHECK'应使用'void'说明.	
	代码片段: 20: int32_t taosThreadCreate(TdThread *tid, const TdThreadAttr *attr, void *(*start)(void *), void *arg) { 21: OS_PARAM_CHECK(tid); 22: OS_PARAM_CHECK(start); 23: #ifdef TD_ASTR 24: int32_t code = 0;	
	污染路径:	

	函数'OS_PARAM_CHECK'的定义点.	
567	缺陷发生位置: /source/client/src/clientSmlJson.c	行号: 145
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 143: pVal->type = TSDB_DATA_TYPE_NCHAR; 144: } else { 145: uError("SML:invalid type(%s) for JSON String", typeStr); 146: return TSDB_CODE_TSC_INVALID_JSON_TYPE; 147: }	
	污染路径: 函数'uError'的定义点.	
568	缺陷发生位置: /source/dnode/mnode/impl/src/mndSnode.c	行号: 636
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 634: 635: if ((code = mndTransAppendRedoAction(pTrans, &action)) != 0) { 636: taosMemoryFree(pReq); 637: TAOS_RETURN(code); 638: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
569	缺陷发生位置: /source/libs/tdb/src/db/tdbPCache.c	行号: 96
	步骤描述: 不使用返回值的函数'tdbPCacheClose'应使用'void'说明.	
	代码片段: 94: if (code) { 95: tdbError("%s failed at %s:%d since %s", __func__, __FILE__, __LINE__, tstrerror(code)); 96: tdbPCacheClose(pCache); 97: *ppCache = NULL; 98: } else {	
	污染路径: 函数'tdbPCacheClose'的定义点.	

570	缺陷发生位置: /source/libs/qcom/src/queryUtil.c	行号: 243
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: <pre> 241: int32_t coce = tQueryAutoQWorkerInit(&taskQueue.wrokrerPool); 242: if (TSDB_CODE_SUCCESS != coce) { 243: qError("failed to init task thread pool"); 244: return -1; 245: }</pre>	
	污染路径: 函数'qError'的定义点.	
571	缺陷发生位置: /source/libs/transport/src/transCli.c	行号: 852
	步骤描述: 不使用返回值的函数'tTrace'应使用'void'说明.	
	代码片段: <pre> 850: QUEUE_INIT(&plist->conns); 851: *ppList = plist; 852: tTrace("create connlist:%p for key:%s", plist, key); 853: } else { 854: *ppList = plist;</pre>	
	污染路径: 函数'tTrace'的定义点.	
572	缺陷发生位置: /source/dnode/mgmt/node_util/src/dmEps.c	行号: 253
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: <pre> 251: } 252: 253: dDebug("reset dnode list on startup"); 254: dmResetEps(pData, pData->dnodeEps); 255:</pre>	
	污染路径: 函数'dDebug'的定义点.	
573	缺陷发生位置: /source/libs/executor/src/dataDispatcher.c	行号: 313
	步骤描述:	

	不使用返回值的函数'taosFreeQitem'应使用'void'说明.	
	代码片段: 311: code = allocBuf(pDispatcher, pInput, pBuf); 312: if (code) { 313: taosFreeQitem(pBuf); 314: return code; 315: }	
	污染路径: 函数'taosFreeQitem'的定义点.	
574	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncIO.c	行号: 216
	步骤描述: 不使用返回值的函数'sError'应使用'void'说明.	
	代码片段: 214: void *pRpc = rpcOpen(&rpclnit); 215: if (pRpc == NULL) { 216: sError("failed to start RPC server"); 217: return terrno; 218: }	
	污染路径: 函数'sError'的定义点.	
575	缺陷发生位置: /source/client/src/clientMain.c	行号: 155
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 153: } 154: if (code != 0) { 155: tscError("failed to put timezone %s to map", val); 156: } 157:	
	污染路径: 函数'tscError'的定义点.	
576	缺陷发生位置: /source/libs/parser/src/parInsertSql.c	行号: 623
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 621: pVal->flag = CV_FLAG_NULL; 622:	

	623: taosMemoryFree(tmpTokenBuf); 624: 625: return TSDB_CODE_SUCCESS;	
	污染路径: 函数'taosMemoryFree'的定义点.	
577	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 18
	步骤描述: 不使用返回值的函数"应使用'void'说明.	
	代码片段: 16: { 17: template <unsigned N> 18: struct uint 19: { 20: static_assert((N & (N - 1)) == 0, "Number of bits must be power of 2");	
	污染路径: 函数"的定义点.	
578	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 1111
	步骤描述: 不使用返回值的函数'BITv06_reloadDStream'应使用'void'说明.	
	代码片段: 1109: const FSEv06_DTableHeader* const DTableH = (const FSEv06_DTableHeader*)ptr; 1110: DStatePtr->state = BITv06_readBits(bitD, DTableH- >tableLog); 1111: BITv06_reloadDStream(bitD); 1112: DStatePtr->table = dt + 1; 1113: }	
	污染路径: 函数'BITv06_reloadDStream'的定义点.	
579	缺陷发生位置: /source/client/src/clientSmlLine.c	行号: 538
	步骤描述: 不使用返回值的函数'JUMP_SPACE'应使用'void'说明.	
	代码片段: 536: 537: // parse cols 538: JUMP_SPACE(sql, sqlEnd)	

	539: elements->cols = sql; 540:	
	污染路径: 函数'JUMP_SPACE'的定义点.	
580	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 216
	步骤描述: 不使用返回值的函数'putc'应使用'void'说明.	
	代码片段: 214: for (i = 0; i < pad; i++) 215: { 216: putc (' ', fs->stream); 217: } 218: }	
	污染路径: 无	
581	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqHandleConnect.c	行号: 63
	步骤描述: 不使用返回值的函数'DL_FOREACH_SAFE'应使用'void'说明.	
	代码片段: 61: int access; 62: 63: DL_FOREACH_SAFE((*head), msg_tail, tmp) { 64: if (msg_tail->direction == ttq_md_out) { 65: access = TTQ_ACL_READ;	
	污染路径: 函数'DL_FOREACH_SAFE'的定义点.	
582	缺陷发生位置: /source/client/src/clientSmlJson.c	行号: 133
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 131: 132: // if reach here means type is unsupported 133: uError("SML:invalid type(%s) for JSON Number", typeStr); 134: return TSDB_CODE_TSC_INVALID_JSON_TYPE; 135: }	

	污染路径: 函数'uError'的定义点.	
583	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqDb.c	行号: 219
	步骤描述: 不使用返回值的函数'ttqDbMsgStoreClean'应使用'void'说明.	
	代码片段: 217: subhier_clean(&db.shared_subs); 218: // retain__clean(&db.retains); 219: ttqDbMsgStoreClean(); 220: 221: return TTQ_ERR_SUCCESS;	
	污染路径: 函数'ttqDbMsgStoreClean'的定义点.	
584	缺陷发生位置: /source/libs/executor/src/streamexternalwindowoperator.c	行号: 269
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 267: pInfo->stat.resBlockReused, pInfo->stat.resBlockAppend); 268: 269: taosMemoryFreeClear(pInfo); 270: } 271:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
585	缺陷发生位置: /source/dnode/vnode/src/bse/bseTableMgt.c	行号: 117
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 115: tableMetaMgtDestroy(p->pTableMetaMgt); 116: } 117: taosMemoryFree(p); 118: } 119: int32_t bseTableMgtGet(STableMgt *pMgt, int64_t seq, uint8_t **pValue, int32_t *len) {	
	污染路径:	

	函数'taosMemoryFree'的定义点.	
586	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttContext.c	行号: 132
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 130: UNUSED(net__socket_close(context)); 131: if (force_free) { 132: UNUSED(ttqSubCleanSession(context)); 133: } 134:	
	污染路径: 函数'UNUSED'的定义点.	
587	缺陷发生位置: /source/dnode/vnode/src/meta/metaTtl.c	行号: 273
	步骤描述: 不使用返回值的函数'metaInfo'应使用'void'说明.	
	代码片段: 271: SMeta *meta = pMeta; 272: 273: metaInfo("ttlMgr convert start."); 274: 275: SConvertData cvData = {.pNewTtlIdx = pNewTtlIdx, .pMeta = meta};	
	污染路径: 函数'metaInfo'的定义点.	
588	缺陷发生位置: /utils/test/c/tmq_td37436.c	行号: 37
	步骤描述: 不使用返回值的函数'tmq_free_json_meta'应使用'void'说明.	
	代码片段: 35: printf("meta result: %s\n", result); 36: ASSERT(strcmp(result, result1) == 0 strcmp(result, result2) == 0); 37: tmq_free_json_meta(result); 38: } else { 39: printf("no meta result\n");	
	污染路径: 函数'tmq_free_json_meta'的定义点.	
589	缺陷发生位置:	行号:

	/source/dnode/mnode/impl/src/mndGrant.c	49
	步骤描述: 不使用返回值的函数'STR_WITH_MAXSIZE_TO_VARSTR'应使用'void'说明.	
	代码片段: 47: COL_DATA_SET_VAL_GOTO(tmp, false, NULL, NULL, _exit); 48: 49: GRANT_ITEM_SHOW("unlimited"); 50: GRANT_ITEM_SHOW("limited"); 51: GRANT_ITEM_SHOW("false");	
	污染路径: expanded from macro 'GRANT_ITEM_SHOW' 函数'STR_WITH_MAXSIZE_TO_VARSTR'的定义点.	
590	缺陷发生位置: /test/new_test_framework/script/api/asyncQuery.c	行号: 70
	步骤描述: 不使用返回值的函数'atomic_add_fetch_64'应使用'void'说明.	
	代码片段: 68: atomic_add_fetch_64(&errTimes, 1); 69: taos_free_result(res); 70: atomic_add_fetch_64(&finQueries, 1); 71: return; 72: }	
	污染路径: 函数'atomic_add_fetch_64'的定义点.	
591	缺陷发生位置: /source/os/src/osDir.c	行号: 358
	步骤描述: 不使用返回值的函数'OS_PARAM_CHECK'应使用'void'说明.	
	代码片段: 356: 357: int32_t taosExpandDir(const char *dirname, char *outname, int32_t maxlen) { 358: OS_PARAM_CHECK(dirname); 359: OS_PARAM_CHECK(outname); 360: if (dirname[0] == 0) return 0;	
	污染路径: 函数'OS_PARAM_CHECK'的定义点.	
592	缺陷发生位置: /source/dnode/mnode/impl/src/mndRole.c	行号: 1073

	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 1071: _exit; 1072: taosMemoryFreeClear(pBuf); 1073: taosMemoryFreeClear(sql); 1074: if (code < 0) { 1075: mError("%s failed at line %d since %s", __func__, lino, strerror(code));	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
593	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/legacy/zstd_v03.c	1786
	步骤描述: 不使用返回值的函数'HUF_decodeStreamX2'应使用'void'说明.	
	代码片段: 1784: /* finish bitStreams one by one */ 1785: HUF_decodeStreamX2(op1, &bitD1, opStart2, dt, dtLog); 1786: HUF_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 1787: HUF_decodeStreamX2(op3, &bitD3, opStart4, dt, dtLog); 1788: HUF_decodeStreamX2(op4, &bitD4, oend, dt, dtLog);	
594	污染路径: 函数'HUF_decodeStreamX2'的定义点.	
	缺陷发生位置:	行号:
	/source/client/wrapper/src/clientTmqConnector.c	53
	步骤描述: 不使用返回值的函数'jniDebug'应使用'void'说明.	
595	代码片段: 51: 52: atomic_store_32(&__init_tmq, 2); 53: jniDebug("tmq method register finished"); 54: } 55:	
	污染路径: 函数'jniDebug'的定义点.	
595	缺陷发生位置:	行号:

	/source/libs/transport/test/svrBench.c	205
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 203: } 204: int ch = getchar(); 205: UNUSED(ch); 206: 207: return 0;	
	污染路径: 函数'UNUSED'的定义点.	
596	缺陷发生位置: /source/dnode/vnode/src/tq/tqUtil.c	行号: 237
	步骤描述: 不使用返回值的函数'tqError'应使用'void'说明.	
	代码片段: 235: tEncodeSize(tEncodeMqMetaRsp, &tmpMetaRsp, len, code); 236: if (TSDB_CODE_SUCCESS != code) { 237: tqError("tmq extract meta from log, tEncodeMqMetaRsp error"); 238: goto END; 239: }	
	污染路径: 函数'tqError'的定义点.	
597	缺陷发生位置: /source/libs/transport/src/trans.c	行号: 194
	步骤描述: 不使用返回值的函数'tlInfo'应使用'void'说明.	
	代码片段: 192: } 193: void rpcClose(void* arg) { 194: tlInfo("start to close rpc"); 195: if (arg == NULL) { 196: return;	
	污染路径: 函数'tlInfo'的定义点.	
598	缺陷发生位置: /source/libs/planner/src/planOptimizer.c	行号: 2736
	步骤描述:	

	不使用返回值的函数'planError'应使用'void'说明.	
	代码片段: 2734: pCxt->optimized = true; 2735: } else { 2736: planError("%s failed at line %d since %s", __func__, lino, tstrerror(code)); 2737: } 2738:	
	污染路径: 函数'planError'的定义点.	
599	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 878
	步骤描述: 不使用返回值的函数'prctl'应使用'void'说明.	
	代码片段: 876: 877: #ifdef LINUX 878: prctl(PR_SET_NAME, "createTable"); 879: #endif 880: pThreadInfo->buffer = benchCalloc(1, TOOLS_MAX_ALLOWED_SQL_LEN, false);	
	污染路径: 无	
600	缺陷发生位置: /test/new_test_framework/script/api/stmt2-memleak.c	行号: 77
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 75: for (int i = 0; i < CTB_NUMS; i++) { 76: tbs[i] = (char*)malloc(sizeof(char) * 20); 77: sprintf(tbs[i], "ctb_%d", i); 78: if (preCreateTb) { 79: createCtb(taos, tbs[i]);	
	污染路径: 无	
601	缺陷发生位置: /source/libs/planner/src/planLogicCreator.c	行号: 2476
	步骤描述: 不使用返回值的函数'planError'应使用'void'说明.	
	代码片段:	

	2474: 2475: if (NULL == pExternal->pCol) { 2476: planError("%s failed, External window can not find pk column", __func__); 2477: nodesDestroyNode((SNode*)pWindow); 2478: return TSDB_CODE_PLAN_INTERNAL_ERROR;	
	污染路径: 函数'planError'的定义点.	
602	缺陷发生位置: /source/dnode/mnode/impl/src/mndVgroup.c	行号: 502
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 500: 501: if (tSerializeSAlterVnodeConfigReq((char *)pReq + sizeof(SMsgHead), contLen, &alterReq) < 0) { 502: taosMemoryFree(pReq); 503: mError("vgId:%d, failed to serialize alter vnode config req,since %s", pVgroup->vgId, terrstr()); 504: terrno = TSDB_CODE_OUT_OF_MEMORY;	
	污染路径: 函数'taosMemoryFree'的定义点.	
603	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttSocket.c	行号: 333
	步骤描述: 不使用返回值的函数'ttqSessionExpiryRemoveAll'应使用'void'说明.	
	代码片段: 331: #endif 332: 333: ttqSessionExpiryRemoveAll(); 334: ttq_cxt_cleanup(); 335: listeners__stop();	
	污染路径: 函数'ttqSessionExpiryRemoveAll'的定义点.	
604	缺陷发生位置: /source/dnode/mgmt/mgmt_dnode/src/dmHandle.c	行号: 81
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 79: dDebug("disable ip-white-list on dnode ver: %" PRIu64	

	<pre> ", status ver: %" PRIu64, pMgmt->pData->ipWhiteVer, ver); 80: if (rpcSetIpWhite(pMgmt->msgCb.serverRpc, NULL) != 0) { 81: dError("failed to disable ip white list on dnode"); 82: } 83: }</pre>	
	污染路径: 函数'dError'的定义点.	
605	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 2209
	步骤描述: 不使用返回值的函数'HUFv06_decodeStreamX2'应使用'void'说明.	
	代码片段: 2207: 2208: /* finish bitStreams one by one */ 2209: HUFv06_decodeStreamX2(op1, &bitD1, opStart2, dt, dtLog); 2210: HUFv06_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 2211: HUFv06_decodeStreamX2(op3, &bitD3, opStart4, dt, dtLog);	
	污染路径: 函数'HUFv06_decodeStreamX2'的定义点.	
606	缺陷发生位置: /test/new_test_framework/script/api/insertSameTs.c	行号: 71
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 69: Test(taos, qstr); 70: taos_close(taos); 71: taos_cleanup(); 72: } 73: void Test(TAOS *taos, char *qstr) {	
	污染路径: 函数'taos_cleanup'的定义点.	
607	缺陷发生位置: /docs/examples/c/telnet_line_example.c	行号: 50
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段:	

	48: taos_free_result(res); 49: taos_close(taos); 50: taos_cleanup(); 51: } 52: // output:	
	污染路径: 函数'taos_cleanup'的定义点.	
608	缺陷发生位置: /source/libs/scheduler/src/schTask.c	行号: 122
	步骤描述: 不使用返回值的函数'SCH_TASK_DLOG'应使用'void'说明.	
	代码片段: 120: code = epsetToStr(&pAddr->epSet, buf, tListLen(buf)); 121: if (code == TSDB_CODE_SUCCESS) { 122: SCH_TASK_DLOG("recode the success addr:%s", buf); 123: } else { 124: SCH_TASK_ELOG("failed to print epset due to convert to string failed, code:%s, ignore and continue",	
	污染路径: 函数'SCH_TASK_DLOG'的定义点.	
609	缺陷发生位置: /source/util/src/tqueue.c	行号: 108
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 106: 107: (void)taosThreadMutexDestroy(&queue->mutex); 108: taosMemoryFree(queue); 109: 110: uDebug("queue:%p is closed", queue);	
	污染路径: 函数'taosMemoryFree'的定义点.	
610	缺陷发生位置: /utls/test/c/tmq_batch_alter_tag.c	行号: 52
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 50: int32_t ret = tmq_write_raw(pConn, raw); 51: printf("write raw data: %s\n", tmq_err2str(ret)); 52: ASSERT(ret == 0);	

	53: 54: tmq_free_raw(raw);	
	污染路径: 函数'ASSERT'的定义点.	
611	缺陷发生位置: /source/client/src/clientMain.c	行号: 135
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 133: tz = tzalloc("UTC"); 134: if (tz == NULL) { 135: tscError("%s set timezone UTC error", __func__); 136: terrno = TAOS_SYSTEM_ERROR(errno); 137: goto END;	
	污染路径: 函数'tscError'的定义点.	
612	缺陷发生位置: /source/client/test/session.cpp	行号: 450
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 448: snprintf(fenGbk, sizeof(fenGbk), "%c%c", 0xB7, 0xD2); 449: snprintf(zhongguoUtf8, sizeof(zhongguoUtf8), "%c%c%c%c", 0xE4, 0xB8, 0xAD, 0xE5, 0x9B, 0xBD); 450: snprintf(guoUtf8, sizeof(guoUtf8), "%c%c%c", 0xE5, 0x9B, 0xBD); 451: snprintf(zhongguoGbk, sizeof(zhongguoGbk), "%c%c%c%c", 0xD6, 0xD0, 0xB9, 0xFA); 452: snprintf(zhongGbk, sizeof(zhongGbk), "%c%c", 0xD6, 0xD0);	
	污染路径: 无	
613	缺陷发生位置: /source/libs/executor/src/exchangeoperator.c	行号: 533
	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: 531: T_LONG_JMP(pTaskInfo->env, code); 532: } else { 533: qDebug("empty block returned in exchange");	

	534: } 535:	
	污染路径: 函数'qDebug'的定义点.	
614	缺陷发生位置: /test/new_test_framework/script/api/last-query-ws.cpp	行号: 98
	步骤描述: 不使用返回值的函数'operator<<'应使用'void'说明.	
	代码片段: 96: } 97: 98: std::cout << "1. Success to connect to TDengine\n"; 99: } 100:	
	污染路径: 函数'operator<<'的定义点.	
615	缺陷发生位置: /test/new_test_framework/script/api/demoapi.c	行号: 150
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 148: taos_free_result(res); 149: } 150: sprintf(command, "insert into t%"PRIu64" values(%" PRIu64 " , " 151: "NULL, NULL, NULL, NULL, NULL)", 152: i, 1650000000000+j);	
	污染路径: 无	
616	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncMessageDebug.c	行号: 246
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 244: char* serialized = syncPing2Str(pMsg); 245: sTrace("syncPingLog len:%d %s", (int32_t)strlen(serialized), serialized); 246: taosMemoryFree(serialized); 247: } 248:	

	污染路径: 函数'taosMemoryFree'的定义点.	
617	缺陷发生位置: /source/libs/transport/src/transCli.c	行号: 828
	步骤描述: 不使用返回值的函数'taosHashCleanup'应使用'void'说明.	
	代码片段: 826: connList = taosHashIterate((SHashObj*)pool, connList); 827: } 828: taosHashCleanup(pool); 829: pThrd->pool = NULL; 830: return NULL;	
	污染路径: 函数'taosHashCleanup'的定义点.	
618	缺陷发生位置: /source/libs/decimal/test/decimalTest.cpp	行号: 238
	步骤描述: 不使用返回值的函数'r'应使用'void'说明.	
	代码片段: 236: } 237: 238: DEFINE_OPERATOR(+, OP_TYPE_ADD); 239: DEFINE_OPERATOR(-, OP_TYPE_SUB); 240: DEFINE_OPERATOR(*, OP_TYPE_MULTI);	
	污染路径: expanded from macro 'DEFINE_OPERATOR'	
619	缺陷发生位置: /tools/taos-tools/src/benchDataMix.c	行号: 132
	步骤描述: 不使用返回值的函数'tmpStr'应使用'void'说明.	
	代码片段: 130: case TSDB_DATA_TYPE_VARBINARY: 131: format = "%s"; 132: tmpStr(val, 0, fd, *k); 133: break; 134: case TSDB_DATA_TYPE_GEOMETRY:	
	污染路径: 函数'tmpStr'的定义点.	

620	缺陷发生位置: /source/libs/new-stream/src/dataSinkMgr.c	行号: 41
	步骤描述: 不使用返回值的函数'stError'应使用'void'说明.	
	代码片段: 39: code = initDataSinkFileDir(); 40: if (code != 0) { 41: stError("failed to create data sink file dir, err: 0x%0x", code); 42: return code; 43: }	
	污染路径: 函数'stError'的定义点.	
621	缺陷发生位置: /source/libs/parser/src/parlInsertUtil.c	行号: 1113
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 1111: *pLen = len; 1112: } else { 1113: taosMemoryFree(pBuf); 1114: } 1115: return code;	
	污染路径: 函数'taosMemoryFree'的定义点.	
622	缺陷发生位置: /test/new_test_framework/script/api/whiteListTest.c	行号: 146
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 144: char qstr[1024]; 145: for (int i = 0; i < nUser; ++i) { 146: sprintf(users[i], "user%d", i); 147: sprintf(qstr, "DROP USER %s", users[i]); 148: queryDB(taos, qstr);	
	污染路径: 无	
623	缺陷发生位置: /test/new_test_framework/script/api/whiteListTest.c	行号: 157
	步骤描述:	

	不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 155: // users 156: for (int i = 0; i < nUser; ++i) { 157: sprintf(users[i], "user%d", i); 158: sprintf(qstr, "CREATE USER %s PASS 'taosdata'", users[i]); 159: queryDB(taos, qstr); </pre>	
	污染路径: 无	
624	缺陷发生位置: /source/client/test/clientTests.cpp	行号: 1927
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 1925: taos_free_result(pRes); 1926: 1927: sprintf(sql, "create stable %s.%s (ts timestamp, v int) tags (t1 int)", database, stb); 1928: pRes = taos_query(pRootConn, sql); 1929: ASSERT_EQ(taos_errno(pRes), TSDB_CODE_SUCCESS); </pre>	
	污染路径: 无	
625	缺陷发生位置: /source/libs/parser/src/parlInsertUtil.c	行号: 1252
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: <pre> 1250: pColSchema->name, tDataTypes[pColSchema->type].name, precision, scale, tDataTypes[*fields].name, 1251: precisionData, scaleData); 1252: uError("checkSchema %s", buf); 1253: return TSDB_CODE_INVALID_PARA; 1254: } </pre>	
	污染路径: 函数'uError'的定义点.	
626	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqSend.c	行号: 124
	步骤描述:	

	不使用返回值的函数'ttq_free'应使用'void'说明.	
	代码片段: <pre> 122: rc = packet__alloc(packet); 123: if (rc) { 124: ttq_free(packet); 125: return rc; 126: }</pre>	
	污染路径: 函数'ttq_free'的定义点.	
627	缺陷发生位置: /utils/test/c/tmq_td32471.c	行号: 75
	步骤描述: 不使用返回值的函数'taosSsleep'应使用'void'说明.	
	代码片段: <pre> 73: } 74: 75: taosSsleep(5); 76: TAOS_RES* tmqmessage = tmq_consumer_poll(tmq, 1000); 77: ASSERT(tmqmessage == NULL);</pre>	
	污染路径: 函数'taosSsleep'的定义点.	
628	缺陷发生位置: /source/libs/index/src/indexFstFile.c	行号: 41
	步骤描述: 不使用返回值的函数'SERIALIZE_STR_VAR_TO_BUF'应使用'void'说明.	
	代码片段: <pre> 39: static FORCE_INLINE void idxGenLRUKey(char* buf, const char* path, int32_t blockId) { 40: char* p = buf; 41: SERIALIZE_STR_VAR_TO_BUF(p, path, strlen(path)); 42: SERIALIZE_VAR_TO_BUF(p, '_', char); 43: TAOS_UNUSED(idxInt2str(blockId, p, 0));</pre>	
	污染路径: 函数'SERIALIZE_STR_VAR_TO_BUF'的定义点.	
629	缺陷发生位置: /tools/taos-tools/src/toolstime.c	行号: 905
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段:	

	903: 904: if (precision == TSDB_TIME_PRECISION_NANO) { 905: sprintf(buf + pos, "%.09d", ms); 906: } else if (precision == TSDB_TIME_PRECISION_MICRO) { 907: sprintf(buf + pos, "%.06d", ms);	
	污染路径: 无	
630	缺陷发生位置: /source/libs/monitor/src/monMain.c	行号: 619
	步骤描述: 不使用返回值的函数'ulnfoL'应使用'void'说明.	
	代码片段: 617: char *pCont = tjsonToString(pMonitor->pjson); 618: if (tsMonitorLogProtocol) { 619: ulnfoL("report cont:\n%s", pCont); 620: } 621: if (pCont != NULL) {	
	污染路径: 函数'ulnfoL'的定义点.	
631	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 1617
	步骤描述: 不使用返回值的函数'FSE_buildDTable_raw'应使用'void'说明.	
	代码片段: 1615: case bt_raw : 1616: LLlog = LLbits; 1617: FSE_buildDTable_raw(DTableLL, LLbits); break; 1618: default : 1619: { U32 max = MaxLL;	
	污染路径: 函数'FSE_buildDTable_raw'的定义点.	
632	缺陷发生位置: /utils/test/c/tmq_ts5776.c	行号: 49
	步骤描述: 不使用返回值的函数'tmq_free_json_meta'应使用'void'说明.	
	代码片段: 47: char* result = tmq_get_json_meta(msg); 48: printf("meta result: %s\n", result); 49: tmq_free_json_meta(result); 50: }	

	51:	
	污染路径: 函数'tmq_free_json_meta'的定义点.	
633	缺陷发生位置: /source/dnode/vnode/src/meta/metaTable.c	行号: 549
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 547: tDecoderClear(&dc); 548: } 549: tdbFree(pData); 550: tdbFree(pKey); 551: tdbTbcClose(pCur);	
	污染路径: 函数'tdbFree'的定义点.	
634	缺陷发生位置: /source/os/src/osTimezone.c	行号: 1016
	步骤描述: 不使用返回值的函数'uDebug'应使用'void'说明.	
	代码片段: 1014: } 1015: int32_t code = TSDB_CODE_SUCCESS; 1016: uDebug("[tz]set timezone to %s", tz); 1017: #ifdef WINDOWS 1018: char winStr[TD_TIMEZONE_LEN * 2] = {0};	
	污染路径: 函数'uDebug'的定义点.	
635	缺陷发生位置: /docs/examples/c-ws/stmt_insert_demo.c	行号: 86
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 84: for (int i = 1; i <= num_of_sub_table; i++) { 85: char table_name[20]; 86: sprintf(table_name, "d_bind_%d", i); 87: char location[20]; 88: sprintf(location, "location_%d", i);	
	污染路径: 无	

636	缺陷发生位置: /source/util/src/thash.c	行号: 564
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 562: 563: taosHashClear(pHashObj); 564: taosMemoryFreeClear(pHashObj->hashList); 565: 566: // destroy mem block	
	污染路径: 函数'taosMemoryFreeClear'的定义点. expanded from macro 'FREE_HASH_NODE'	
637	缺陷发生位置: /source/libs/tdb/test/tdbExOVFLTest.cpp	行号: 439
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 437: 438: for (int i = 1; i <= nData; i++) { 439: sprintf(key, "key%d", i); 440: // sprintf(val, "value%d", i); 441:	
	污染路径: 无	
638	缺陷发生位置: /source/dnode/mgmt/node_util/src/dmEps.c	行号: 349
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 347: void dmUpdateEps(SDnodeData *pData, SArray *eps) { 348: if (taosThreadRwlockWrlck(&pData->lock) != 0) { 349: dError("failed to lock dnode lock"); 350: } 351:	
	污染路径: 函数'dError'的定义点.	
639	缺陷发生位置: /contrib/test/rocksdb/main.c	行号: 176
	步骤描述:	

	不使用返回值的函数'kvSerial'应使用'void'说明.	
	代码片段: <pre> 174: char buf[128] = {0}; 175: KV kv = {.k1 = 0, .k2 = 0}; 176: kvSerial(&kv, buf); 177: char *v = rocksdb_get_cf(db, rOpt, cfHandle[1], buf, sizeof(kv), &vlen, &err); 178: printf("Get value %s, and len = %d, xxxx\n", v, (int)vlen); </pre>	
	污染路径: 函数'kvSerial'的定义点.	
640	缺陷发生位置: /source/client/src/clientHb.c	行号: 744
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: <pre> 742: code = tDeserializeSClientHbBatchRsp(pMsg->pData, pMsg->len, &pRsp); 743: if (TSDB_CODE_SUCCESS != code) { 744: tscError("deserialize hb rsp failed"); 745: } 746: int32_t now = taosGetTimestampSec(); </pre>	
641	污染路径: 函数'tscError'的定义点.	
	缺陷发生位置: /source/libs/function/src/detail/tminmaxavx.c	行号: 224
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
642	代码片段: <pre> 222: return TSDB_CODE_SUCCESS; 223: #else 224: uError("unable run %s without avx2 instructions", __func__); 225: return TSDB_CODE_OPS_NOT_SUPPORT; 226: #endif </pre>	
	污染路径: 函数'uError'的定义点.	
	缺陷发生位置: /source/client/src/clientStmt2.c	行号: 1976
	步骤描述:	

	不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 1974: 1975: if (tsem_destroy(&cbParam.sem) != 0) { 1976: tscError("failed to destroy semaphore"); 1977: code = TSDB_CODE_CTG_INTERNAL_ERROR; 1978: catalogFreeMetaData(pMetaData);	
	污染路径: 函数'tscError'的定义点.	
643	缺陷发生位置: /source/dnode/mgmt/node_util/src/dmEps.c	行号: 355
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 353: dmResetEps(pData, eps); 354: if (dmWriteEps(pData) != 0) { 355: dError("failed to write dnode file"); 356: } 357:	
	污染路径: 函数'dError'的定义点.	
644	缺陷发生位置: /source/dnode/mgmt/mgmt_qnode/src/qmWorker.c	行号: 136
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: 134: } 135: 136: dDebug("qnode workers are initialized"); 137: return code; 138: }	
	污染路径: 函数'dDebug'的定义点.	
645	缺陷发生位置: /source/libs/tdb/src/db/tdbPager.c	行号: 848
	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	
	代码片段: 846: int code = tdbTbPushFreePage(pPager->pEnv->pFreeDb, pPage, pTxn);	

	847: if (code < 0) { 848: tdbError("tdb/insert-free-page: tb push failed with ret: %d.", code); 849: return code; 850: }	
	污染路径: 函数'tdbError'的定义点.	
646	缺陷发生位置: /source/libs/scalar/src/sclvector.c	行号: 1462
	步骤描述: 不使用返回值的函数'SCL_RET'应使用'void'说明.	
	代码片段: 1460: doReleaseVec(pLeftCol, leftConvert); 1461: doReleaseVec(pRightCol, rightConvert); 1462: SCL_RET(code); 1463: } 1464:	
	污染路径: 函数'SCL_RET'的定义点.	
647	缺陷发生位置: /source/dnode/mnode/impl/src/mndVgroup.c	行号: 664
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 662: if (tSerializeSAlterVnodeReplicaReq(pReq, contLen, &req) < 0) { 663: mError("vgId:%d, failed to serialize alter vnode req,since %s", req.vgId, terrstr()); 664: taosMemoryFree(pReq); 665: terrno = TSDB_CODE_OUT_OF_MEMORY; 666: return NULL;	
	污染路径: 函数'taosMemoryFree'的定义点.	
648	缺陷发生位置: /source/libs/decimal/test/decimalTest.cpp	行号: 16
	步骤描述: 不使用返回值的函数'cout'应使用'void'说明.	
	代码片段: 14: auto it = arr.rbegin(); 15: for (; it != arr.rend(); ++it) {	

	<pre> 16: cout << *it; 17: } 18: cout << endl; </pre>	
	污染路径: 无	
649	缺陷发生位置: /source/libs/wal/src/walRead.c	行号: 516
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 514: (void)memcpy(pHead->head.body, newBody, plainBodyLen); 515: 516: taosMemoryFree(newBody); 517: } 518: </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
650	缺陷发生位置: /contrib/test/rocksdb/main.c	行号: 167
	步骤描述: 不使用返回值的函数'kvSerial'应使用'void'说明.	
	代码片段: <pre> 165: char buf[128] = {0}; 166: KV kv = {.k1 = (100 - i) % 26, .k2 = i % 26}; 167: kvSerial(&kv, buf); 168: rocksdb_writebatch_put_cf(wBatch, cfHandle[1], buf, sizeof(kv), "value", strlen("value")); 169: } </pre>	
	污染路径: 函数'kvSerial'的定义点.	
651	缺陷发生位置: /source/client/src/clientImpl.c	行号: 263
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: <pre> 261: if (TSDB_CODE_SUCCESS != code) { 262: (void)taosThreadMutexUnlock(&appInfo.mutex); 263: taosMemoryFreeClear(key); 264: return code; </pre>	

	265: } else {	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
652	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1788
	步骤描述: 不使用返回值的函数'HUF_decodeStreamX2'应使用'void'说明.	
	代码片段: 1786: 1787: /* finish bitStreams one by one */ 1788: HUF_decodeStreamX2(op1, &bitD1, opStart2, dt, dtLog); 1789: HUF_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 1790: HUF_decodeStreamX2(op3, &bitD3, opStart4, dt, dtLog);	
	污染路径: 函数'HUF_decodeStreamX2'的定义点.	
653	缺陷发生位置: /utils/test/c/tmq_td33798.c	行号: 147
	步骤描述: 不使用返回值的函数'taosSsleep'应使用'void'说明.	
	代码片段: 145: taos_free_result(pRes); 146: 147: taosSsleep(1); 148: pRes = taos_query(pConn, "insert into ct1 using st1 tags(1000, \"ttt\", true) values(1626006833400, 1, 2, 'a') ct2 using st1 tags(1000, \"ttt\", true) values(1626006833400, 1, 2, 'a') ct3 using st1 tags(1000, \"ttt\", true) values(1626006833400, 1, 2, 'a')"); 149: if (taos_errno(pRes) != 0) {	
	污染路径: 函数'taosSsleep'的定义点.	
654	缺陷发生位置: /contrib/test/traft/join_into_vgroup/node_leader10002.c	行号: 132
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 130:	

	131: // wait for join over 132: sleep(2); 133: 134: r = joinRaftPeer(&raftEnv, 100, "127.0.0.1", 10001, joinRaftPeerCb);	
	污染路径: 无	
655	缺陷发生位置: /source/libs/sync/src/syncRespMgr.c	行号: 47
	步骤描述: 不使用返回值的函数'TAOS_RETURN'应使用'void'说明.	
	代码片段: 45: *ppObj = pObj; 46: 47: TAOS_RETURN(0); 48: } 49:	
	污染路径: 函数'TAOS_RETURN'的定义点.	
656	缺陷发生位置: /utlis/test/c/tmq_td33798.c	行号: 85
	步骤描述: 不使用返回值的函数'tmq_free_json_meta'应使用'void'说明.	
	代码片段: 83: } 84: 85: tmq_free_json_meta(result); 86: } 87:	
	污染路径: 函数'tmq_free_json_meta'的定义点.	
657	缺陷发生位置: /source/dnode/mgmt/node_util/src/dmEps.c	行号: 359
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 357: 358: if (taosThreadRwlockUnlock(&pData->lock) != 0) { 359: dError("failed to unlock dnode lock"); 360: }	

	361: }	
	污染路径: 函数'dError'的定义点.	
658	缺陷发生位置: /source/libs/index/src/indexFst.c	行号: 775
	步骤描述: 不使用返回值的函数'indexInfo'应使用'void'说明.	
	代码片段: 773: return true; 774: } 775: indexInfo("fst write key must be ordered"); 776: return false; 777: }	
	污染路径: 函数'indexInfo'的定义点.	
659	缺陷发生位置: /source/dnode/mnode/impl/src/mndQnode.c	行号: 504
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 502: SArray *qnodeList = taosArrayInit(5, sizeof(SQueryNodeLoad)); 503: if (NULL == qnodeList) { 504: mError("failed to alloc epSet while process qnode list req"); 505: code = terrno; 506: TAOS_RETURN(code);	
	污染路径: 函数'mError'的定义点.	
660	缺陷发生位置: /source/libs/monitorfw/src/taos_collector.c	行号: 50
	步骤描述: 不使用返回值的函数'TAOS_LOG'应使用'void'说明.	
	代码片段: 48: if (r) { 49: if (taos_collector_destroy(self) != 0) { 50: TAOS_LOG("taos_collector_destroy failed"); 51: } 52: return NULL;	

	污染路径: 函数'TAOS_LOG'的定义点.	
661	缺陷发生位置: /docs/examples/c-ws-new/stmt2_insert_demo.c	行号: 206
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 204: insertData(taos); 205: taos_close(taos); 206: taos_cleanup(); 207: }	
	污染路径: 函数'taos_cleanup'的定义点.	
662	缺陷发生位置: /source/os/src/osString.c	行号: 191
	步骤描述: 不使用返回值的函数'OS_PARAM_CHECK'应使用'void'说明.	
	代码片段: 189: 190: int32_t taosStr2int32(const char *str, int32_t *val) { 191: OS_PARAM_CHECK(str); 192: OS_PARAM_CHECK(val); 193: int64_t tmp = 0;	
	污染路径: 函数'OS_PARAM_CHECK'的定义点.	
663	缺陷发生位置: /source/os/src/osDir.c	行号: 172
	步骤描述: 不使用返回值的函数'tstrncpy'应使用'void'说明.	
	代码片段: 170: if (temp[1] == ':') pos += 3; 171: #else 172: tstrncpy(temp, dirname, sizeof(temp)); 173: #endif 174:	
	污染路径: 函数'tstrncpy'的定义点.	
664	缺陷发生位置: /source/libs/scheduler/src/schRemote.c	行号: 528
	步骤描述:	

	不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 526: // called if drop task rsp received code 527: if (pMsg->handle == NULL) { 528: qError("sch handle is NULL, may be already released and mem leak"); 529: } else { 530: (void)rpcReleaseHandle(pMsg->handle, TAOS_CONN_CLIENT, 0); // ignore error	
	污染路径: 函数'qError'的定义点.	
665	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeSvr.c	行号: 153
	步骤描述: 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段: 151: for (int32_t iReq = 0; iReq < nReqs; iReq++) { 152: code = vnodePreprocessCreateTableReq(pVnode, &dc, btime, NULL); 153: TSDB_CHECK_CODE(code, lino, _exit); 154: } 155:	
	污染路径: 函数'TSDB_CHECK_CODE'的定义点.	
666	缺陷发生位置: /source/dnode/vnode/src/meta/metaEntry2.c	行号: 115
	步骤描述: 不使用返回值的函数'tdbFreeClear'应使用'void'说明.	
	代码片段: 113: } 114: 115: tdbFreeClear(value); 116: tDecoderClear(&decoder); 117: return code;	
	污染路径: 函数'tdbFreeClear'的定义点.	
667	缺陷发生位置: /source/client/src/clientFirewall.c	行号: 315
	步骤描述: 不使用返回值的函数'tsclInfo'应使用'void'说明.	

	代码片段: 313: } 314: TAOS_UNUSED(taosCloseFile(&fp)); 315: tscInfo("sql security: saved %d learned rules to %s", added, ruleFile); 316: 317: // Force reload rules by clearing the cache	
	污染路径: 函数'tscInfo'的定义点.	
668	缺陷发生位置: /source/dnode/bnode/src/bnode.c	行号: 57
	步骤描述: 不使用返回值的函数'bndInfo'应使用'void'说明.	
	代码片段: 55: taosMemoryFree(pBnode); 56: 57: bndInfo("Bnode closed."); 58: }	
	污染路径: 函数'bndInfo'的定义点.	
669	缺陷发生位置: /source/dnode/mnode/impl/src/mndView.c	行号: 46
	步骤描述: 不使用返回值的函数'mDebug'应使用'void'说明.	
	代码片段: 44: } 45: 46: void mndCleanupView(SMnode *pMnode) { mDebug("mnd view cleanup"); } 47: 48: int32_t mndProcessCreateViewReq(SRpcMsg *pReq) {	
	污染路径: 函数'mDebug'的定义点.	
670	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdblter.c	行号: 711
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 709: void tsdblterMergerClose(SlterMerger **merger) { 710: if (merger[0]) {	

	711: taosMemoryFree(merger[0]); 712: merger[0] = NULL; 713: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
671	缺陷发生位置: /utlis/test/c/tmqOffset.c	行号: 31
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 29: } 30: if ((code = taosReadFile(pFile, memBuf, size)) != size) { 31: taosMemoryFree(memBuf); 32: printf("code:%d != size:%d\n", code, size); 33: return -1;	
	污染路径: 函数'taosMemoryFree'的定义点.	
672	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 5851
	步骤描述: 不使用返回值的函数'tsdbReaderClose2'应使用'void'说明.	
	代码片段: 5849: if (code != TSDB_CODE_SUCCESS) { 5850: tsdbError("%s failed at line %d since %s, %s", __func__, lino, tstrerror(code), idstr); 5851: tsdbReaderClose2(*ppReader); 5852: *ppReader = NULL; // reset the pointer value. 5853: }	
	污染路径: 函数'tsdbReaderClose2'的定义点.	
673	缺陷发生位置: /test/new_test_framework/script/api/stmt_function.c	行号: 92
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 90: stmt = taos_stmt_init(taos); 91: assert(stmt != NULL); 92: sprintf(stmt_sql, "insert into ? values (?,?,?,?,?,?,?,1,?,?,?));	

	93: assert(taos_stmt_prepare(stmt, stmt_sql, 0) == 0); 94: assert(taos_stmt_close(stmt) == 0);	
	污染路径: 无	
674	缺陷发生位置: /source/dnode/mnode/impl/src/mndMnode.c	行号: 374
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 372: 373: if ((code = mndTransAppendRedoAction(pTrans, &action)) != 0) { 374: taosMemoryFree(pReq); 375: TAOS_RETURN(code); 376: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
675	缺陷发生位置: /source/libs/sync/src/syncSnapshot.c	行号: 805
	步骤描述: 不使用返回值的函数'sInfo'应使用'void'说明.	
	代码片段: 803: goto _START_RECEIVER; 804: } else if (order == 0) { // order == 0 805: sInfo("prepare snapshot, received a duplicate snapshot preparation. send reply."); 806: goto _SEND_REPLY; 807: } else { // order > 0	
	污染路径: 函数'sInfo'的定义点.	
676	缺陷发生位置: /source/dnode/mnode/impl/src/mndRsma.c	行号: 301
	步骤描述: 不使用返回值的函数'mFatal'应使用'void'说明.	
	代码片段: 299: } else { 300: terrno = TSDB_CODE_APP_ERROR; 301: mFatal("rsma:%s, failed to acquire rsma since %s", name, terrstr()); 302: }	

	303: }	
	污染路径: 函数'mFatal'的定义点.	
677	缺陷发生位置: /source/util/src/tqueue.c	行号: 72
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 70: int32_t code = taosThreadMutexInit(&(*queue)->mutex, NULL); 71: if (code) { 72: taosMemoryFreeClear(*queue); 73: return (terrno = TAOS_SYSTEM_ERROR(code)); 74: }	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
678	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nchar.c	行号: 52
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 50: taos_free_result(result); 51: 52: usleep(100000); 53: taos_select_db(taos, "test"); 54:	
	污染路径: 无	
679	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmWorker.c	行号: 533
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: 531: tWWorkerCleanup(&pMgmt->fetchPool); 532: tQueryAutoQWorkerCleanup(&pMgmt->streamReaderPool); 533: dDebug("vnode workers are closed"); 534: }	
	污染路径:	

	函数'dDebug'的定义点.	
680	缺陷发生位置: /source/libs/executor/src/filloperator.c	行号: 354
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 352: cleanupExprSupp(&pInfo->fillNullExprSupp); 353: 354: taosMemoryFreeClear(pInfo->p); 355: taosArrayDestroy(pInfo->matchInfo.pList); 356: taosMemoryFreeClear(param);	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
681	缺陷发生位置: /source/dnode/mnode/impl/src/mndDnode.c	行号: 2031
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 2029: SArray *fqdns = taosArrayInit(4, sizeof(void *)); 2030: if (fqdns == NULL) { 2031: mError("failed to init fqdns array"); 2032: return NULL; 2033: }	
	污染路径: 函数'mError'的定义点.	
682	缺陷发生位置: /test/new_test_framework/script/api/stmt2-blob-performance.c	行号: 36
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 34: void createCtb(TAOS* taos, const char* tname) { 35: char* tmp = (char*)malloc(sizeof(char) * 100); 36: sprintf(tmp, "create table db.%s using db.stb tags(0, 'after')", tname); 37: do_query(taos, tmp); 38: }	
	污染路径: 无	
683	缺陷发生位置: /source/dnode/mnode/impl/src/mndCluster.c	行号: 372

	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 370: 371: if (clusterObj.id <= 0) { 372: mError("can't get cluster info while update uptime"); 373: return 0; 374: }	
	污染路径: 函数'mError'的定义点.	
684	缺陷发生位置: /contrib/test/craft/raftServer.c	行号: 162
	步骤描述: 不使用返回值的函数'splitString'应使用'void'说明.	
	代码片段: 160: 161: char arr[2][MAX_TOKEN_LEN]; 162: splitString(msg, "--", arr, 2); 163: putKV(arr[0], arr[1]); 164:	
	污染路径: 函数'splitString'的定义点.	
685	缺陷发生位置: /source/libs/new-stream/src/streamCheckpoint.c	行号: 218
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 216: end: 217: if (code != TSDB_CODE_SUCCESS) { 218: taosMemoryFreeClear(*data); 219: *dataLen = 0; 220: }	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
686	缺陷发生位置: /source/dnode/vnode/src/meta/metaQuery.c	行号: 544
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 542: metaULock(pMeta);	

	543: } 544: tdbFree(pData); 545: tDeleteSchemaWrapper(pSchema); 546: if (extSchema != NULL) {	
	污染路径: 函数'tdbFree'的定义点.	
687	缺陷发生位置: /contrib/TSZ/zstd/compress/zstd_compress.c	行号: 1031
	步骤描述: 不使用返回值的函数'assert'应使用'void'说明.	
	代码片段: 1029: size_t const tableSpace = (chainSize + hSize + h3Size) * sizeof(U32); 1030: 1031: assert(((size_t)ptr & 3) == 0); 1032: 1033: ms->hashLog3 = hashLog3;	
	污染路径: 函数'assert'的定义点.	
688	缺陷发生位置: /test/new_test_framework/script/api/stmt2-blob-exmaple.c	行号: 265
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 263: 264: taos_close(taos); 265: taos_cleanup(); 266: }	
	污染路径: 函数'taos_cleanup'的定义点.	
689	缺陷发生位置: /source/libs/tdb/test/tdbPageFlushTest.cpp	行号: 412
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 410: for (int i = 1024 * 4; i < 1024 * 8; ++i) { 411: char key[32] = {0}; 412: sprintf(key, "key-%d", i); 413: ret = tdbTbInsert(pDb, key, strlen(key) + 1, val, valLen, txn);	

	414: GTEST_ASSERT_EQ(ret, 0);	
	污染路径: 无	
690	缺陷发生位置: /examples/c/asyncdemo.c	行号: 115
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 113: queryDB(taos, sql); 114: 115: sprintf(sql, "use %s", db); 116: queryDB(taos, sql); 117:	
	污染路径: 无	
691	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftStoreDebug.c	行号: 77
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 75: char *serialized = raftStore2Str(pObj); 76: sTrace("raftStoreLog len:%d %s", (int32_t)strlen(serialized), serialized); 77: taosMemoryFree(serialized); 78: } 79:	
	污染路径: 函数'taosMemoryFree'的定义点.	
692	缺陷发生位置: /source/client/wrapper/src/clientTmqConnector.c	行号: 52
	步骤描述: 不使用返回值的函数'atomic_store_32'应使用'void'说明.	
	代码片段: 50: (*env)->DeleteLocalRef(env, offset); 51: 52: atomic_store_32(&__init_tmq, 2); 53: jniDebug("tmq method register finished"); 54: }	
	污染路径:	

	函数'atomic_store_32'的定义点.	
693	缺陷发生位置: /tools/taos-tools/src/benchTmq.c	行号: 370
	步骤描述: 不使用返回值的函数'pthread_join'应使用'void'说明.	
	代码片段: 368: 369: for (int i = 0; i < pConsumerInfo->concurrent; i++) { 370: pthread_join(pids[i], NULL); 371: } 372:	
	污染路径: 无	
694	缺陷发生位置: /tools/taos-tools/src/benchMain.c	行号: 128
	步骤描述: 不使用返回值的函数'initLog'应使用'void'说明.	
	代码片段: 126: 127: // log 128: initLog(); 129: initArgument(); 130: srand(time(NULL)%1000000);	
	污染路径: 函数'initLog'的定义点.	
695	缺陷发生位置: /docs/examples/c-ws/tmq_demo.c	行号: 479
	步骤描述: 不使用返回值的函数'pthread_join'应使用'void'说明.	
	代码片段: 477: 478: thread_stop = 1; 479: pthread_join(thread_id, NULL); 480: 481: if (drop_topic(pConn) < 0) {	
	污染路径: 无	
696	缺陷发生位置: /source/libs/wal/src/walMeta.c	行号: 49
	步骤描述:	

	不使用返回值的函数'wError'应使用'void'说明.	
	代码片段: <pre> 47: 48: if (pWal == NULL) { 49: wError("failed to set keep version, pWal is NULL"); 50: return TSDB_CODE_INVALID_PARA; 51: }</pre>	
	污染路径: 函数'wError'的定义点.	
697	缺陷发生位置: /source/os/src/osSocket.c	行号: 223
	步骤描述: 不使用返回值的函数'TAOS_SKIP_ERROR'应使用'void'说明.	
	代码片段: <pre> 221: if (-1 == bind(pSocket->fd, (struct sockaddr *)&serverAdd, sizeof(serverAdd))) { 222: terrno = TAOS_SYSTEM_ERROR(ERRNO); 223: TAOS_SKIP_ERROR(taosCloseSocket(&pSocket)); 224: return 0; 225: }</pre>	
	污染路径: 函数'TAOS_SKIP_ERROR'的定义点.	
698	缺陷发生位置: /source/libs/executor/src/dataDeleter.c	行号: 199
	步骤描述: 不使用返回值的函数'taosFreeQitem'应使用'void'说明.	
	代码片段: <pre> 197: if (pBuf != NULL) { 198: TAOS_MEMCPY(&pDeleter->nextOutput, pBuf, sizeof(SDataDeleterBuf)); 199: taosFreeQitem(pBuf); 200: } 201:</pre>	
	污染路径: 函数'taosFreeQitem'的定义点.	
699	缺陷发生位置: /source/dnode/vnode/src/tq/tqSnapshot.c	行号: 65
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段:	

	<pre> 63: if (code != 0){ 64: tqError("%s failed at %d, vnode tq snapshot reader open failed since %s", __func__, lino, tstrerror(code)); 65: taosMemoryFreeClear(pReader); 66: } 67: </pre>	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
700	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 880
	步骤描述: 不使用返回值的函数'DEBUGLOG'应使用'void'说明.	
	代码片段: <pre> 878: 879: /* check conditions */ 880: DEBUGLOG(4, "ZDICT_finalizeDictionary"); 881: if (dictBufferCapacity < dictContentSize) return ERROR(dstSize_tooSmall); 882: if (dictContentSize < ZDICT_CONTENTSIZE_MIN) return ERROR(srcSize_wrong); </pre>	
	污染路径: 函数'DEBUGLOG'的定义点.	
701	缺陷发生位置: /source/dnode/mnode/impl/src/mndStreamUtil.c	行号: 508
	步骤描述: 不使用返回值的函数'mstError'应使用'void'说明.	
	代码片段: <pre> 506: SDBVgHashInfo* dbInfo = (SDBVgHashInfo*)tSimpleHashGet(pDbVgroups, dbFName, strlen(dbFName) + 1); 507: if (NULL == dbInfo) { 508: mstError("db %s does not exist", dbFName); 509: TAOS_CHECK_EXIT(TSDB_CODE_MND_DB_NOT_EXIST); 510: } </pre>	
	污染路径: 函数'mstError'的定义点.	
702	缺陷发生位置: /source/libs/sync/src/syncSnapshot.c	行号: 802
	步骤描述: 不使用返回值的函数'sWarn'应使用'void'说明.	

	代码片段: <pre> 800: int32_t order = 0; 801: if ((order = snapshotReceiverSignatureCmp(pReceiver, pMsg)) < 0) { // order < 0 802: sWarn("failed to prepare snapshot, received a new snapshot preparation. restart receiver."); 803: goto _START_RECEIVER; 804: } else if (order == 0) { // order == 0 </pre>	
	污染路径: 函数'sWarn'的定义点.	
703	缺陷发生位置: /source/dnode/vnode/src/tq/tqRead.c	行号: 852
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 850: code = tBlobSetGet(pBlobRow2, seq, &item); 851: if (code != 0) { 852: taosMemoryFree(val); 853: terrno = code; 854: uError("tq set blob val, idx:%d, get blob item failed, seq:%" PRIu64 " , code:%d", idx, seq, code); </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
704	缺陷发生位置: /source/common/src/tvariant.c	行号: 127
	步骤描述: 不使用返回值的函数'SET_ERRNO'应使用'void'说明.	
	代码片段: <pre> 125: 126: int32_t toIntegerPure(const char *z, int32_t n, int32_t base, int64_t *value) { 127: SET_ERRNO(0); 128: 129: char *endPtr = NULL; </pre>	
	污染路径: 函数'SET_ERRNO'的定义点.	
705	缺陷发生位置: /source/dnode/mnode/impl/src/mndQuery.c	行号: 178
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	

	代码片段: <pre> 176: int32_t mndInitQuery(SMnode *pMnode) { 177: if (qWorkerInit(NODE_TYPE_MNODE, MNODE_HANDLE, (void **)&pMnode->pQuery, &pMnode->msgCb) != 0) { 178: mError("failed to init qworker in mnode since %s", terrstr()); 179: return -1; 180: } </pre>	
	污染路径: 函数'mError'的定义点.	
706	缺陷发生位置: /test/new_test_framework/script/api/last-query-ws.cpp	行号: 64
	步骤描述: 不使用返回值的函数'operator<<'应使用'void'说明.	
	代码片段: <pre> 62: } 63: 64: std::cout << "2. Threads created\n"; 65: } 66: </pre>	
	污染路径: 函数'operator<<'的定义点.	
707	缺陷发生位置: /source/libs/tdb/test/tdbPageRecycleTest.cpp	行号: 379
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 377: 378: for (int iData = 1; iData <= nData; iData++) { 379: sprintf(key, "key0"); 380: sprintf(val, "value%d", iData); 381: </pre>	
	污染路径: 无	
708	缺陷发生位置: /source/libs/function/src/builtins.c	行号: 609
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 607: } else { </pre>	

	608: (void)snprintf(errMsg, msgLen, "%s", msg3); 609: taosMemoryFree(intervals); 610: cJSON_Delete(binDesc); 611: return TSDB_CODE_FUNC_HISTOGRAM_ERROR;	
	污染路径: 函数'taosMemoryFree'的定义点.	
709	缺陷发生位置: /utils/test/c/tmqDemo.c	行号: 139
	步骤描述: 不使用返回值的函数'tstrncpy'应使用'void'说明.	
	代码片段: 137: tstrncpy(g_stConflInfo.stbName, argv[++i], sizeof(g_stConflInfo.stbName)); 138: } else if (strcmp(argv[i], "-w") == 0) { 139: tstrncpy(g_stConflInfo.vnodeWalPath, argv[++i], sizeof(g_stConflInfo.vnodeWalPath)); 140: } else if (strcmp(argv[i], "-f") == 0) { 141: tstrncpy(g_stConflInfo.resultFileName, argv[++i], sizeof(g_stConflInfo.resultFileName));	
	污染路径: 函数'tstrncpy'的定义点.	
710	缺陷发生位置: /test/new_test_framework/script/api/insertSameTs.c	行号: 111
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 109: 110: // query the records 111: sprintf(qstr, "SELECT * FROM m1 order by ts desc"); 112: result = taos_query(taos, qstr); 113: if (result == NULL taos_errno(result) != 0) {	
	污染路径: 无	
711	缺陷发生位置: /source/dnode/mnode/impl/src/mndRetention.c	行号: 68
	步骤描述: 不使用返回值的函数'mDebug'应使用'void'说明.	
	代码片段: 66: } 67:	

	68: void mndCleanupRetention(SMnode *pMnode) { mDebug("mnd retention cleanup"); } 69: 70: void tFreeRetentionObj(SRetentionObj *pObj) { tFreeCompactObj((SCompactObj *)pObj); }	
	污染路径: 函数'mDebug'的定义点.	
712	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 439
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 437: snprintf(fenUtf8, sizeof(fenUtf8), "%c%c%c", 0xE8, 0x8A, 0xAC); 438: snprintf(fenGbk, sizeof(fenGbk), "%c%c", 0xB7, 0xD2); 439: snprintf(zhongguoUtf8, sizeof(zhongguoUtf8), "%c%c%c%c", 0xE4, 0xB8, 0xAD, 0xE5, 0x9B, 0xBD); 440: snprintf(guoUtf8, sizeof(guoUtf8), "%c%c%c", 0xE5, 0x9B, 0xBD); 441: snprintf(zhongguoGbk, sizeof(zhongguoGbk), "%c%c%c%c", 0xD6, 0xD0, 0xB9, 0xFA);	
	污染路径: 无	
713	缺陷发生位置: /source/libs/new-stream/src/streamRunner.c	行号: 672
	步骤描述: 不使用返回值的函数'ST_TASK_DLOG'应使用'void'说明.	
	代码片段: 670: empty ? 0 : pBlock->info.rows - 1); 671: if (code == 0) { 672: ST_TASK_DLOG("start to send notify:%s", pContent); 673: int32_t index = pExec->runtimeInfo.funcInfo.curOutIdx; 674: SSTriggerCalcParam* pTriggerCalcParams =	
	污染路径: 函数'ST_TASK_DLOG'的定义点.	
714	缺陷发生位置: /test/new_test_framework/script/api/sameReqidTest.c	行号: 218
	步骤描述:	

	不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 216: taos_query_a_with_reqid(taos, "select *from t3", selectCallback, NULL, CONST_REQ_ID); 217: 218: sleep(1); 219: } 220:	
	污染路径: 无	
715	缺陷发生位置: /source/dnode/mnode/impl/src/mndDnode.c	行号: 446
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 444: } 445: if (taosArrayPush(pDnodeEps, &dnodeEp) == NULL) { 446: mError("failed to put ep into array, but continue at this call"); 447: } 448: }	
	污染路径: 函数'mError'的定义点.	
716	缺陷发生位置: /utils/test/c/tmq_write_raw_test.c	行号: 85
	步骤描述: 不使用返回值的函数'tmq_free_json_meta'应使用'void'说明.	
	代码片段: 83: char* result = tmq_get_json_meta(msg); 84: printf("meta result: %s\n", result); 85: tmq_free_json_meta(result); 86: } 87:	
	污染路径: 函数'tmq_free_json_meta'的定义点.	
717	缺陷发生位置: /tools/taos-tools/src/dumpUtil.c	行号: 505
	步骤描述: 不使用返回值的函数'pthread_mutex_destroy'应使用'void'说明.	
	代码片段:	

	503: } 504: } 505: pthread_mutex_destroy(&map->lock); 506: } 507:	
	污染路径: 无	
718	缺陷发生位置: /utls/tsim/src/simSystem.c	行号: 140
	步骤描述: 不使用返回值的函数'simInfo'应使用'void'说明.	
	代码片段: 138: } 139: 140: simInfo("thread is stopped"); 141: return NULL; 142: }	
	污染路径: 函数'simInfo'的定义点.	
719	缺陷发生位置: /source/libs/transport/src/transComm.c	行号: 283
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 281: r = uv_ip6_name(addr, buf, sizeof(buf)); 282: if (r != 0) { 283: uError("failed to get ip from sockaddr6, err:%s", uv_strerror(r)); 284: } 285:	
	污染路径: 函数'uError'的定义点.	
720	缺陷发生位置: /test/new_test_framework/script/api/stmtTest.c	行号: 219
	步骤描述: 不使用返回值的函数'taos_stmt_bind_param'应使用'void'说明.	
	代码片段: 217: strcpy(v.c10, "一二三四五六七八"); 218: c10len=strlen(v.c10); 219: taos_stmt_bind_param(stmt, params);	

	220: taos_stmt_add_batch(stmt); 221: }	
	污染路径: 函数'taos_stmt_bind_param'的定义点.	
721	缺陷发生位置: /source/libs/qworker/src/qwUtil.c	行号: 252
	步骤描述: 不使用返回值的函数'QW_TASK_ELOG_E'应使用'void'说明.	
	代码片段: 250: QW_RET(qwGetTaskCtx(QW_FPARAMS(), ctx)); 251: } else { 252: QW_TASK_ELOG_E("task ctx already exist"); 253: QW_ERR_RET(TSDB_CODE_QRY_TASK_ALREADY_EXIST); 254: }	
	污染路径: 函数'QW_TASK_ELOG_E'的定义点.	
722	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 660
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 658: *outputLen = taosUcs4ToMbs((TdUcs4 *)input, inputLen, *output, charsetCxt); 659: if (*outputLen < 0) { 660: taosMemoryFree(*output); 661: return TSDB_CODE_SCALAR_CONVERT_ERROR; 662: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
723	缺陷发生位置: /source/libs/function/test/runUdf.c	行号: 42
	步骤描述: 不使用返回值的函数'fnError'应使用'void'说明.	
	代码片段: 40: 41: if (doSetupUdf("udf1", &handle) != 0) { 42: fnError("setup udf failure"); 43: return -1; 44: }	

	污染路径: 函数'fnError'的定义点.	
724	缺陷发生位置: /source/libs/sync/src/syncSnapshot.c	行号: 403
	步骤描述: 不使用返回值的函数'taosMsleep'应使用'void'说明.	
	代码片段: 401: } 402: 403: taosMsleep(1); 404: 405: sInfo("vgId:%d, snapshot replication progress:1/8:leader:1/4 to dnode:%d, reason:%s", pSyncNode->vgId, DID(pDestId),	
	污染路径: 函数'taosMsleep'的定义点.	
725	缺陷发生位置: /docs/examples/c-ws-new/insert_data_demo.c	行号: 49
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 47: taos_errstr(result)); 48: taos_close(taos); 49: taos_cleanup(); 50: return -1; 51: }	
	污染路径: 函数'taos_cleanup'的定义点.	
726	缺陷发生位置: /source/libs/executor/src/scanoperator.c	行号: 2100
	步骤描述: 不使用返回值的函数'qTrace'应使用'void'说明.	
	代码片段: 2098: 2099: QRY_PARAM_CHECK(ppRes); 2100: qTrace("%s call", __FUNCTION__); 2101: 2102: code = createVTableScanInfoFromParam(pOperator);	
	污染路径:	

	函数'qTrace'的定义点.	
727	缺陷发生位置: /source/libs/index/src/indexTfile.c	行号: 435
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 433: if (ret != 0) { 434: indexError("failed to search json since %s", tstrerror(ret)); 435: taosMemoryFree(p); 436: fstSliceDestroy(&key); 437: return ret;	
	污染路径: 函数'taosMemoryFree'的定义点.	
728	缺陷发生位置: /source/libs/txnode/src/txnodeMgmt.c	行号: 92
	步骤描述: 不使用返回值的函数'xndError'应使用'void'说明.	
	代码片段: 90: atomic_load_32(&pData->isStopped)) { 91: if (uv_async_send(&pData->stopAsync) != 0) { 92: xndError("stop xnoded: failed to send stop async"); 93: } 94: char xnodedPipeSocket[PATH_MAX] = {0};	
	污染路径: 函数'xndError'的定义点.	
729	缺陷发生位置: /source/libs/scalar/test/filter/filterTests.cpp	行号: 1480
	步骤描述: 不使用返回值的函数'fp'应使用'void'说明.	
	代码片段: 1478: for (UnsignedT l = unsignedMin; l <= unsignedMax; ++l) { 1479: for (SignedT r = signedMin; r <= signedMax; ++r) { 1480: ASSERT_EQ(fp(&l, &r), compareUnsignedWithSigned(l, r)); 1481: } 1482: }	
	污染路径: 无	

730	缺陷发生位置： /contrib/TSZ/zstd/compress/fse_compress.c	行号： 104
	步骤描述： 不使用返回值的函数'assert'应使用'void'说明.	
	代码片段： 102: tableU16[-2] = (U16) tableLog; 103: tableU16[-1] = (U16) maxSymbolValue; 104: assert(tableLog < 16); /* required for the threshold strategy to work */ 105: 106: /* For explanations on how to distribute symbol values over the table :	
	污染路径： 函数'assert'的定义点.	
731	缺陷发生位置： /contrib/TSZ/sz/src/conf.c	行号： 168
	步骤描述： 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段： 166: 167: par = iniparser_getstring(ini, "ENV:sol_name", NULL); 168: snprintf(sol_name, 256, "%s", par); 169: 170: if(strcmp(sol_name, "SZ")==0)	
	污染路径： 函数'snprintf'的定义点.	
732	缺陷发生位置： /utils/test/c/replay_test.c	行号： 271
	步骤描述： 不使用返回值的函数'taosSsleep'应使用'void'说明.	
	代码片段： 269: 270: if(totalRows == len * 4){ 271: taosSsleep(1); 272: tsem_post(sem); 273: }	
	污染路径： 函数'taosSsleep'的定义点.	
733	缺陷发生位置： /source/libs/executor/src/dataDispatcher.c	行号： 327

	步骤描述: 不使用返回值的函数'taosFreeQitem'应使用'void'说明.	
	代码片段: 325: 326: taosMemoryFreeClear(pBuf->pData); 327: taosFreeQitem(pBuf); 328: return code; 329: }	
	污染路径: 函数'taosFreeQitem'的定义点.	
734	缺陷发生位置:	行号:
	/source/libs/planner/src/planLogicCreator.c	2005
	步骤描述: 不使用返回值的函数'PLAN_ERR_RET'应使用'void'说明.	
	代码片段: 2003: code = rewriteExprsForSelect(pPartKeys, pSelect, SQL_CLAUSE_EXT_WINDOW, &pOutputPartitionKeys); 2004: nodesDestroyList(pPartKeys); 2005: PLAN_ERR_RET(code); 2006: 2007: if (NULL != pOutputPartitionKeys) {	
735	污染路径: 函数'PLAN_ERR_RET'的定义点.	
	缺陷发生位置:	行号:
	/docs/examples/c-ws-new/query_data_demo.c	40
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
736	代码片段: 38: taos_errstr(result); 39: taos_close(taos); 40: taos_cleanup(); 41: return -1; 42: }	
	污染路径: 函数'taos_cleanup'的定义点.	
	缺陷发生位置:	行号:
736	/source/libs/executor/src/tlinearhash.c	329
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段:	

	327: taosMemoryFree(pHashObj->pBucket); 328: } 329: taosMemoryFree(pHashObj); 330: terrno = code; 331: return NULL;	
	污染路径: 函数'taosMemoryFree'的定义点.	
737	缺陷发生位置: /source/libs/catalog/src/ctgRemote.c	行号: 809
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 807: int32_t msgSize = tSerializeSBatchReq(NULL, 0, &batchReq); 808: if (msgSize < 0) { 809: qError("tSerializeSBatchReq failed"); 810: CTG_ERR_RET(msgSize); 811: }	
	污染路径: 函数'qError'的定义点.	
738	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmTransport.c	行号: 401
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 399: code = rpcSendRequest(pDnode->trans.clientRpc, pEpSet, pMsg, NULL); 400: if (code != 0) { 401: dError("failed to send rpc msg"); 402: return code; 403: }	
	污染路径: 函数'dError'的定义点.	
739	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/test/argp-test.c	行号: 247
	步骤描述: 不使用返回值的函数'argp_parse'应使用'void'说明.	
	代码片段: 245: params.foonly = 0;	

	246: params.foonly_default = random (); 247: argp_parse (&argp, argc, argv, 0, 0, ¶ms); 248: printf ("After parsing: foonly = %x\n", params.foonly); 249: return 0;	
	污染路径: 无	
740	缺陷发生位置: /source/libs/executor/src/operator.c	行号: 249
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 247: if (p.code == TSDB_CODE_SUCCESS) { 248: if (!(*order == TSDB_ORDER_ASC *order == TSDB_ORDER_DESC)) { 249: qError("operator failed at: %s:%d", __func__, __LINE__); 250: p.code = TSDB_CODE_INVALID_PARA; 251: }	
	污染路径: 函数'qError'的定义点.	
741	缺陷发生位置: /source/libs/index/src/indexCache.c	行号: 407
	步骤描述: 不使用返回值的函数'indexError'应使用'void'说明.	
	代码片段: 405: 406: if (taosThreadMutexInit(&cache->mtx, NULL) != 0) { 407: indexError("failed to create mutex for index cache"); 408: taosMemoryFree(cache); 409: return NULL;	
	污染路径: 函数'indexError'的定义点.	
742	缺陷发生位置: /source/libs/transport/src/transSvr.c	行号: 341
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 339: 340: int32_t len = snprintf(pBuf, bufSize, "user:%s, ver:%" PRId64 " ", ip:{%s}", user, plist->ver, tmp);	

	341: taosMemoryFree(tmp); 342: 343: if (len < 0) {	
	污染路径: 函数'taosMemoryFree'的定义点.	
743	缺陷发生位置: /source/libs/new-stream/src/dataSinkCache.c	行号: 408
	步骤描述: 不使用返回值的函数'stError'应使用'void'说明.	
	代码片段: 406: _end: 407: if (code != TSDB_CODE_SUCCESS) { 408: stError("failed to encode data since %s, lineno:%d", tstrerror(code), lino); 409: } 410: return code;	
	污染路径: 函数'stError'的定义点.	
744	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmlnt.c	行号: 136
	步骤描述: 不使用返回值的函数'tmsgReportStartup'应使用'void'说明.	
	代码片段: 134: return code; 135: } 136: tmsgReportStartup("mnode-impl", "initialized"); 137: 138: if ((code = mmStartWorker(pMgmt)) != 0) {	
	污染路径: 函数'tmsgReportStartup'的定义点.	
745	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmMonitor.c	行号: 194
	步骤描述: 不使用返回值的函数'dlInfo'应使用'void'说明.	
	代码片段: 192: 193: void dmSetVnodeSyncTimeout() { 194: dlInfo("dmSetVnodeSyncTimeout"); 195: SDnode *pDnode = dmInstance(); 196: SMgmtWrapper *pWrapper = &pDnode-	

	>wrappers[VNODE];	
	污染路径: 函数'dInfo'的定义点.	
746	缺陷发生位置: /source/libs/scheduler/src/schRemote.c	行号: 714
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 712: } 713: 714: taosMemoryFree(msg); 715: 716: SCH_RET(code);	
	污染路径: 函数'taosMemoryFree'的定义点.	
747	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeSimpleTest.cpp	行号: 219
	步骤描述: 不使用返回值的函数'sdbSetRawInt32'应使用'void'说明.	
	代码片段: 217: sdbSetRawInt32(pRaw, dataPos, obj.id); 218: dataPos += 4; 219: sdbSetRawInt32(pRaw, dataPos, obj.urlLen); 220: dataPos += 4; 221: sdbSetRawInt32(pRaw, dataPos, obj.statusLen);	
	污染路径: 函数'sdbSetRawInt32'的定义点.	
748	缺陷发生位置: /source/libs/planner/src/planOptimizer.c	行号: 1965
	步骤描述: 不使用返回值的函数'PLAN_ERR_JRET'应使用'void'说明.	
	代码片段: 1963: 1964: nodesRewriteExpr(ppRewrittenCond, pdcRewriteVtablePrimaryTsCol, &rewriteCxt); 1965: PLAN_ERR_JRET(rewriteCxt.errCode); 1966: 1967: _return:	
	污染路径:	

	函数'PLAN_ERR_JRET'的定义点.	
749	缺陷发生位置: /contrib/test/craft/raftMain.c	行号: 30
	步骤描述: 不使用返回值的函数'sscanf'应使用'void'说明.	
	代码片段: 28: token = strtok_r(NULL, separator, &context); 29: if (token) { 30: sscanf(token, "%u", port); 31: } 32:	
	污染路径: 无	
750	缺陷发生位置: /source/dnode/mnode/impl/src/mndCompactDetail.c	行号: 48
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 46: SDbObj *pDb = NULL; 47: 48: mInfo("retrieve compact detail"); 49: 50: if (strlen(pShow->db) > 0) {	
	污染路径: 函数'mInfo'的定义点.	
751	缺陷发生位置: /source/libs/monitor/src/monMain.c	行号: 232
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 230: char buf[40] = {0}; 231: if (taosFormatUtcTime(buf, sizeof(buf), pMonitor->curTime, TSDB_TIME_PRECISION_MILLI) != 0) { 232: uError("failed to format time"); 233: return; 234: }	
	污染路径: 函数'uError'的定义点.	
752	缺陷发生位置: /source/libs/monitorfw/src/taos_map.c	行号: 120

	步骤描述: 不使用返回值的函数'TAOS_LOG'应使用'void'说明.	
	代码片段: 118: TAOS_LOG(TAOS_PTHREAD_RWLOCK_INIT_ERROR); 119: if (taos_map_destroy(self) != 0) { 120: TAOS_LOG("TAOS_MAP_DESTROY_ERROR"); 121: } 122: return NULL;	
	污染路径: 函数'TAOS_LOG'的定义点.	
753	缺陷发生位置: /source/client/wrapper/src/clientTmqConnector.c	行号: 84
	步骤描述: 不使用返回值的函数'atomic_store_32'应使用'void'说明.	
	代码片段: 82: (*env)->DeleteLocalRef(env, assignment); 83: 84: atomic_store_32(&__init_assignment, 2); 85: jniDebug("tmq method assignment finished"); 86: }	
	污染路径: 函数'atomic_store_32'的定义点.	
754	缺陷发生位置: /source/libs/executor/src/dataInserter.c	行号: 71
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 69: SSubmitTbDataMsg* pVg = p; 70: taosMemoryFree(pVg->pData); 71: taosMemoryFree(pVg); 72: } 73:	
	污染路径: 函数'taosMemoryFree'的定义点.	
755	缺陷发生位置: /source/libs/transport/src/transTLS.c	行号: 222
	步骤描述: 不使用返回值的函数'tInfo'应使用'void'说明.	
	代码片段: 220: tInfo("transport instance is ready to reload tls config");	

	221: } else { 222: tInfo("transport instance is not ready to reload tls config"); 223: } 224: return ready;	
	污染路径: 函数'tInfo'的定义点.	
756	缺陷发生位置: /utls/test/c/blob_test.c	行号: 71
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 69: int code = taos_errno(pRes); 70: taos_free_result(pRes); 71: ASSERT(code == 0); 72: 73: pRes = taos_query(	
	污染路径: 函数'ASSERT'的定义点.	
757	缺陷发生位置: /source/libs/tmqtt/mgmt/src/tmqttMgmt.c	行号: 58
	步骤描述: 不使用返回值的函数'TAOS_MQTT_MGMT_CHECK_PTR_RCODE'应使用'void'说明.	
	代码片段: 56: static int32_t mqttMgmtSpawnMqttd(SMqttdData *pData) { 57: bndInfo("start to init taosmqtt"); 58: TAOS_MQTT_MGMT_CHECK_PTR_RCODE(pData); 59: 60: int32_t err = 0;	
	污染路径: 函数'TAOS_MQTT_MGMT_CHECK_PTR_RCODE'的定义点.	
758	缺陷发生位置: /source/libs/qworker/src/qwMem.c	行号: 214
	步骤描述: 不使用返回值的函数'QW_TASK_DLOG_E'应使用'void'说明.	
	代码片段: 212: } while (true); 213:	

	214: QW_TASK_DLOG_E("session initialized"); 215: 216: _return:	
	污染路径: 函数'QW_TASK_DLOG_E'的定义点.	
759	缺陷发生位置: /tools/lastqps-test/taos_query_a_simple.c	行号: 333
	步骤描述: 不使用返回值的函数'pthread_mutex_destroy'应使用'void'说明.	
	代码片段: 331: free(threads); 332: free(thread_qps); 333: pthread_mutex_destroy(&mutex); 334: return 0; 335: }	
	污染路径: 无	
760	缺陷发生位置: /source/libs/function/src/builtins.c	行号: 569
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 567: if (width->valuedouble == 0) { 568: (void)snprintf(errMsg, msgLen, "%s", msg6); 569: taosMemoryFree(intervals); 570: cJSON_Delete(binDesc); 571: return TSDB_CODE_FUNC_HISTOGRAM_ERROR;	
	污染路径: 函数'taosMemoryFree'的定义点.	
761	缺陷发生位置: /source/common/test/commonTests.cpp	行号: 446
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 444: colDataSetVal(p0, i, (const char*)&i, false); 445: 446: sprintf(buf, "the number of row:%d", i); 447: int32_t len = sprintf(buf1, buf, i); 448: STR_TO_VARSTR(buf1, buf)	

	污染路径: 无	
762	缺陷发生位置: /source/dnode/mnode/impl/src/mndSsMigrate.c	行号: 1145
	步骤描述: 不使用返回值的函数'mTrace'应使用'void'说明.	
	代码片段: 1143: 1144: static int32_t mndProcessUpdateSsMigrateProgressTimer(SRpcMsg *pReq) { 1145: mTrace("start to process update ssmigrate progress timer"); 1146: 1147: int32_t code = 0;	
	污染路径: 函数'mTrace'的定义点.	
763	缺陷发生位置: /source/dnode/mnode/impl/src/mndStreamErrorInjection.c	行号: 43
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 41: "not started***"); 42: } else { // pAction->msgType == TDMT_VND_STREAM_CHECK_POINT_SOURCE 43: mError(44: "***sleep 5s and core dump, following tasks will not recv checkpoint-source info, so the tasks will " 45: "started after restart***");	
	污染路径: 函数'mError'的定义点.	
764	缺陷发生位置: /source/os/src/osFile.c	行号: 212
	步骤描述: 不使用返回值的函数'tstrncpy'应使用'void'说明.	
	代码片段: 210: // Try to create directory recursively 211: char s[PATH_MAX]; 212: tstrncpy(s, path, sizeof(s)); 213: if (taosMkDir(taosDirName(s)) != 0) { 214: return NULL;	

	污染路径: 函数'tstrncpy'的定义点.	
765	缺陷发生位置: /source/libs/new-stream/src/streamTriggerTask.c	行号: 2308
	步骤描述: 不使用返回值的函数'ST_TASK_DLOG'应使用'void'说明.	
	代码片段: 2306: if (infoBuf) { 2307: infoBuf[bufCap - 1] = '\0'; 2308: ST_TASK_DLOG("virtual table info: %s", infoBuf); 2309: } 2310:	
	污染路径: 函数'ST_TASK_DLOG'的定义点.	
766	缺陷发生位置: /source/libs/monitor/src/monFramework.c	行号: 192
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 190: 191: if(pBasicInfo->cluster_id == 0) { 192: uError("failed to generate dnode info table since cluster_id is 0"); 193: return; 194: }	
	污染路径: 函数'uError'的定义点.	
767	缺陷发生位置: /utils/tsim/src/simExec.c	行号: 47
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 45: // hostName[strEndIndex + 1] = '\0'; 46: //else 47: snprintf(hostName, sizeof(hostName), "%s", "localhost"); 48: //endif 49: return hostName;	
	污染路径: 函数'snprintf'的定义点.	
768	缺陷发生位置:	行号:

	/source/dnode/vnode/src/meta/metaTable.c	845
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 843: 844: tDecoderClear(&dc); 845: tdbFree(pData); 846: 847: return 0;	
	污染路径: 函数'tdbFree'的定义点.	
769	缺陷发生位置: /utils/test/c/tmq_multi_thread_test.c	行号: 113
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 111: int64_t t2 = taosGetTimestampUs(); 112: printf("total cost %"PRId64" us\n", t2 - t1); 113: taosMemoryFree(paras); 114: return 0; 115: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
770	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 1632
	步骤描述: 不使用返回值的函数'FSE_buildDTable_rle'应使用'void'说明.	
	代码片段: 1630: Offlog = 0; 1631: if (ip > iend-2) return ERROR(srcSize_wrong); /* min : "raw", hence no header, but at least xxLog bits */ 1632: FSE_buildDTable_rle(DTableOffb, *ip++); break; 1633: case bt_raw : 1634: Offlog = Offbits;	
	污染路径: 函数'FSE_buildDTable_rle'的定义点.	
771	缺陷发生位置: /contrib/test/craft/simulate_vnode.c	行号: 204
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	

	代码片段: 202: snprintf(cmd, sizeof(cmd), "rm -rf ./%s", dir); 203: system(cmd); 204: snprintf(cmd, sizeof(cmd), "mkdir -p ./%s", dir); 205: system(cmd); 206:	
	污染路径: 无	
772	缺陷发生位置: /utils/test/c/write_raw_block_test.c	行号: 35
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 33: int32_t numOfRows = 0; 34: int error_code = taos_fetch_raw_block(pRes, &numOfRows, &data); 35: ASSERT(error_code == 0); 36: error_code = taos_write_raw_block(pConn, numOfRows, data, dst); 37: taos_free_result(pRes);	
	污染路径: 函数'ASSERT'的定义点.	
773	缺陷发生位置: /test/new_test_framework/script/api/stopquery.c	行号: 231
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 229: sqExecSQL(taos, sql); 230: 231: sprintf(sql, "use %s", dbName); 232: sqExecSQL(taos, sql); 233:	
	污染路径: 无	
774	缺陷发生位置: /source/libs/planner/src/planSpliter.c	行号: 643
	步骤描述: 不使用返回值的函数'planError'应使用'void'说明.	
	代码片段: 641: }	

	642: if (index < 0 && indexExt < 0) { 643: planError("%s failed since no pkts placeholder set", __FUNCTION__); 644: code = TSDB_CODE_INTERNAL_ERROR; 645: PLAN_ERR_JRET(code);	
	污染路径: 函数'planError'的定义点.	
775	缺陷发生位置: /source/libs/nodes/src/nodesUtilFuncs.c	行号: 238
	步骤描述: 不使用返回值的函数'nodesError'应使用'void'说明.	
	代码片段: 236: g_allocatorReqRefPool = taosOpenRef(1024, destroyNodeAllocator); 237: if (g_allocatorReqRefPool < 0) { 238: nodesError("init nodes failed"); 239: return TSDB_CODE_OUT_OF_MEMORY; 240: }	
	污染路径: 函数'nodesError'的定义点.	
776	缺陷发生位置: /tools/lastqps-test/taos_query_a_simple.c	行号: 167
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 165: taos_query_a_func(param->taos, param->sql, queryCallback, param); 166: } else { 167: usleep(50); 168: } 169: }	
	污染路径: 无	
777	缺陷发生位置: /source/util/test/lrucacheTest.cpp	行号: 212
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 210: for (int32_t i = 0; i < numItems; i++) { 211: char key[32];	

	212: snprintf(key, sizeof(key), "key_%d", i); 213: char value[64]; 214: memset(value, 'A' + (i % 26), itemSize);	
	污染路径: 无	
778	缺陷发生位置: /tools/taos-tools/src/benchCommandOpt.c	行号: 376
	步骤描述: 不使用返回值的函数'prctl'应使用'void'说明.	
	代码片段: 374: } 375: #ifdef LINUX 376: prctl(PR_SET_NAME, "queryStableAggrFunc"); 377: #endif 378: char *command = benchCalloc(1, TOOLS_MAX_ALLOWED_SQL_LEN, false);	
	污染路径: 无	
779	缺陷发生位置: /source/libs/new-stream/src/streamHb.c	行号: 120
	步骤描述: 不使用返回值的函数'stTrace'应使用'void'说明.	
	代码片段: 118: TAOS_CHECK_EXIT(streamHbBuildRequestMsg(&reqMsg, &skipHb)); 119: if (skipHb) { 120: stTrace("stream hb skipped"); 121: goto _exit; 122: }	
	污染路径: 函数'stTrace'的定义点.	
780	缺陷发生位置: /contrib/lemon/lemon.c	行号: 1445
	步骤描述: 不使用返回值的函数'Configtable_insert'应使用'void'说明.	
	代码片段: 1443: *currentend = cfp; 1444: currentend = &cfp->next; 1445: Configtable_insert(cfp);	

	1446: } 1447: return cfp;	
	污染路径: 函数'Configtable_insert'的定义点.	
781	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqUtil.c	行号: 80
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 78: UNUSED(net__socket_close(ttq)); 79: #else 80: UNUSED(net__socket_close(ttq)); 81: state = tmqtt__get_state(ttq); 82: if (state == ttq_cs_disconnecting) {	
	污染路径: 函数'UNUSED'的定义点.	
782	缺陷发生位置: /source/libs/planner/src/planSpliter.c	行号: 2529
	步骤描述: 不使用返回值的函数'qDebugL'应使用'void'说明.	
	代码片段: 2527: qDebugL("before split, JsonPlan: %s", pStr); 2528: } else { 2529: qDebugL("apply split %s rule, JsonPlan: %s", pRuleName, pStr); 2530: } 2531: taosMemoryFree(pStr);	
	污染路径: 函数'qDebugL'的定义点.	
783	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttContext.c	行号: 71
	步骤描述: 不使用返回值的函数'ttq_free'应使用'void'说明.	
	代码片段: 69: if (!context->address) { 70: /* getpeername and inet_ntop failed and not a bridge */ 71: ttq_free(context); 72: return NULL; 73: }	

	污染路径: 函数'ttq_free'的定义点.	
784	缺陷发生位置: /source/dnode/mnode/impl/src/mndVgroup.c	行号: 586
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 584: if (tSerializeSAlterVnodeReplicaReq(pReq, contLen, &alterReq) < 0) { 585: mError("vgld:%d, failed to serialize alter vnode req,since %s", alterReq.vgld, terrstr()); 586: taosMemoryFree(pReq); 587: terrno = TSDB_CODE_OUT_OF_MEMORY; 588: return NULL;	
	污染路径: 函数'taosMemoryFree'的定义点.	
785	缺陷发生位置: /test/new_test_framework/script/api/dbTableRoute.c	行号: 112
	步骤描述: 不使用返回值的函数'rtGetTimeOfDay'应使用'void'说明.	
	代码片段: 110: static int64_t rtGetTimestampMs() { 111: struct timeval systemTime; 112: rtGetTimeOfDay(&systemTime); 113: return (int64_t)systemTime.tv_sec * 1000LL + (int64_t)systemTime.tv_usec/1000; 114: }	
	污染路径: 函数'rtGetTimeOfDay'的定义点.	
786	缺陷发生位置: /source/util/test/arrayTest.cpp	行号: 179
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 177: char value[100]; 178: for (int32_t i = 1; i <= count; i++) { 179: sprintf(value, format, i); 180: char * v = (char *)taosArrayGetP(pa, i - 1); 181: //printf(" needfree get i=%d v=%s\n", i, v);	

	污染路径: 无	
787	缺陷发生位置: /source/client/src/clientStmt2.c	行号: 2166
	步骤描述: 不使用返回值的函数'STMT2_ELOG_E'应使用'void'说明.	
	代码片段: 2164: if (colIdx == -1) { 2165: if (pStmt->sql.bindRowFormat) { 2166: STMT2_ELOG_E("can't mix bind row format and bind column format"); 2167: STMT_ERR_RET(TSDB_CODE_TSC_STMT_API_ERROR); 2168: } 	
	污染路径: 函数'STMT2_ELOG_E'的定义点.	
788	缺陷发生位置: /source/libs/command/src/explain.c	行号: 226
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 224: int32_t code = nodesMakeList(pChildren); 225: if (NULL == *pChildren) { 226: qError("nodesMakeList failed"); 227: QRY_ERR_RET(code); 228: } 	
	污染路径: 函数'qError'的定义点.	
789	缺陷发生位置: /source/libs/transport/test/http_test.c	行号: 51
	步骤描述: 不使用返回值的函数'taosUsleep'应使用'void'说明.	
	代码片段: 49: int32_t code = taosSendHttpReport(monitor, "/crash", 6050, msg, 10, HTTP_FLAT); 50: taosMemoryFree(msg); 51: taosUsleep(10); 52: } 53: } 	
	污染路径:	

	函数'taosUsleep'的定义点.	
790	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncBatch.c	行号: 146
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 144: s = syncUtilPrintBin2((char*)(pMsg->data), pMsg->dataLen); 145: cJSON_AddStringToObject(pRoot, "data2", s); 146: taosMemoryFree(s); 147: } 148:	
	污染路径: 函数'taosMemoryFree'的定义点.	
791	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 2556
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 2554: char vgroups[32] = ""; 2555: if (0 < dbInfo->vgroups) { 2556: sprintf(vgroups, "VGROUPS %d", dbInfo->vgroups); 2557: } 2558:	
	污染路径: 无	
792	缺陷发生位置: /source/dnode/mnode/impl/src/mndStreamTransAct.c	行号: 50
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 48: if (code == -1) { 49: tEncoderClear(&encoder); 50: taosMemoryFree(pBuf); 51: return code; 52: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
793	缺陷发生位置:	行号:

	/source/libs/scalar/src/sclvector.c	186
	步骤描述: 不使用返回值的函数'sclError'应使用'void'说明.	
	代码片段: <pre> 184: int32_t getVectorBigintValue_JSON(void *src, int32_t index, int64_t *res) { 185: if (colDataIsNull_var(((SColumnInfoData *)src), index)) { 186: sclError("getVectorBigintValue_JSON get json data null with index %d", index); 187: SCL_ERR_RET(TSDB_CODE_SCALAR_CONVERT_ERROR); 188: } </pre>	
	污染路径: 函数'sclError'的定义点.	
794	缺陷发生位置: /tests/taosc_test/taoscTest.cpp	行号: 109
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 107: 108: const char* table_name = "taosc_test_table1"; 109: sprintf(sql, "create table taosc_test_db.%s (ts TIMESTAMP, val INT)", table_name); 110: TAOS_RES* res = taos_query(taos, sql); 111: if (taos_errno(res) != 0) { </pre>	
	污染路径: 无	
795	缺陷发生位置: /source/dnode/mnode/impl/src/mndStreamUtil.c	行号: 556
	步骤描述: 不使用返回值的函数'mstDebug'应使用'void'说明.	
	代码片段: <pre> 554: SCMCreateStreamReq* q = p->pCreate; 555: if (NULL == q) { 556: mstDebug("stream pCreate is NULL"); 557: return; 558: } </pre>	
	污染路径: 函数'mstDebug'的定义点.	
796	缺陷发生位置:	行号:

	/contrib/test/sqlite/sqliteTest.c	63
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 61: for (int i = 0; i < batch; i++) { 62: v++; 63: sprintf(tsql, "insert into t values (%d)", v); 64: rc = sqlite3_exec(db, tsql, 0, 0, &err_msg); 65: </pre>	
	污染路径: 无	
797	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 173
	步骤描述: 不使用返回值的函数'showHelp'应使用'void'说明.	
	代码片段: <pre> 171: if (strncasecmp(command, "help", 4) == 0) { 172: if (command[4] == ';' command[4] == ' ' command[4] == 0) { 173: showHelp(); 174: return 0; 175: } </pre>	
	污染路径: 函数'showHelp'的定义点.	
798	缺陷发生位置: /source/libs/executor/src/scanoperator.c	行号: 542
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 540: 541: STableCachedVal* pVal = param; 542: taosMemoryFree((void*)pVal->pName); 543: taosMemoryFree(pVal->pTags); 544: taosMemoryFree(pVal); </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
799	缺陷发生位置: /source/dnode/mnode/impl/src/mndStreamErrorInjection.c	行号: 32
	步骤描述: 不使用返回值的函数'taosMsleep'应使用'void'说明.	

	代码片段: 30: if (choseltem == 0) { 31: // 1. one of update-checkpoint not send, restart and send it again 32: taosMsleep(5000); 33: if (pAction->msgType == TDMT_STREAM_TASK_UPDATE_CHKPT) { 34: mError(	
	污染路径: 函数'taosMsleep'的定义点.	
800	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqPacketDataTypes.c	行号: 79
	步骤描述: 不使用返回值的函数'ttq_free'应使用'void'说明.	
	代码片段: 77: 78: if (tmqtt_validate_utf8(*str, *length)) { 79: ttq_free(*str); 80: *str = NULL; 81: *length = 0;	
	污染路径: 函数'ttq_free'的定义点.	
801	缺陷发生位置: /tools/taos-tools/src/benchCommandOpt.c	行号: 167
	步骤描述: 不使用返回值的函数'benchArrayPush'应使用'void'说明.	
	代码片段: 165: for (int i = 0; i < 3; ++i) { 166: Field *col = benchCalloc(1, sizeof(Field), true); 167: benchArrayPush(stbInfo->cols, col); 168: } 169: Field * c1 = benchArrayGet(stbInfo->cols, 0);	
	污染路径: 函数'benchArrayPush'的定义点.	
802	缺陷发生位置: /tests/taosc_test/taoscTest.cpp	行号: 152
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 150: const char* table_name = "taosc_test_table1";	

	151: tsem_init(&query_sem, 0, 0); 152: sprintf(sql, "select * from taosc_test_db.%;", table_name); 153: taos_query_a(taos, sql, queryCallback, pUserParam); 154: tsem_wait(&query_sem);	
	污染路径: 无	
803	缺陷发生位置: /source/libs/catalog/src/ctgRemote.c	行号: 820
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 818: msgSize = tSerializeSBatchReq(*msg, msgSize, &batchReq); 819: if (msgSize < 0) { 820: taosMemoryFree(*msg); 821: qError("tSerializeSBatchReq failed"); 822: CTG_ERR_RET(msgSize);	
	污染路径: 函数'taosMemoryFree'的定义点.	
804	缺陷发生位置: /utlis/test/c/tmq_get_meta_json.c	行号: 54
	步骤描述: 不使用返回值的函数'tmq_free_json_meta'应使用'void'说明.	
	代码片段: 52: } 53: idx++; 54: tmq_free_json_meta(result); 55: } 56: }	
	污染路径: 函数'tmq_free_json_meta'的定义点.	
805	缺陷发生位置: /source/libs/tdb/test/tdbPageDefragmentTest.cpp	行号: 511
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 509: for (int iData = 0; iData < nData; ++iData) { 510: sprintf(key, "key%d", iData); 511: sprintf(val, "value%d", iData);	

	512: 513: ret = tdbTbInsert(pDb, key, strlen(key), val, strlen(val), txn);	
	污染路径: 无	
806	缺陷发生位置: /source/libs/new-stream/src/dataSinkFile.c	行号: 237
	步骤描述: 不使用返回值的函数'QUERY_CHECK_CODE'应使用'void'说明.	
	代码片段: 235: if (readLen < 0 readLen != pBlockInfo->dataLen) { 236: code = terrno; 237: QUERY_CHECK_CODE(code, lino, _exit); 238: } 239:	
	污染路径: 函数'QUERY_CHECK_CODE'的定义点.	
807	缺陷发生位置: /source/dnode/mnode/impl/src/mndSubscribe.c	行号: 1237
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 1235: code = mndPersistRebResult(pMnode, pMsg, &rebOutput); 1236: if (code != 0) { 1237: mError("mq rebalance persist output error, possibly vnode splitted or dropped,msg:%s", tstrerror(code)) 1238: } 1239: }	
	污染路径: 函数'mError'的定义点.	
808	缺陷发生位置: /source/dnode/vnode/src/tq/tqSnapshot.c	行号: 106
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 104: } 105: tdbFree(pKey); 106: tdbFree(pVal); 107: return code;	

	108: }	
	污染路径: 函数'tdbFree'的定义点.	
809	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 1307
	步骤描述: 不使用返回值的函数'buffer_skip_whitespace'应使用'void'说明.	
	代码片段: 1305: 1306: input_buffer->offset++; 1307: buffer_skip_whitespace(input_buffer); 1308: if (can_access_at_index(input_buffer, 0) && (buffer_at_offset(input_buffer)[0] == 'J')) 1309: {	
	污染路径: 函数'buffer_skip_whitespace'的定义点.	
810	缺陷发生位置: /docs/examples/c-ws-new/sml_insert_demo.c	行号: 37
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 35: fprintf(stderr, "Failed to create database power, ErrCode: 0x%x, ErrorMessage: %s.\n", code, taos_errstr(result)); 36: taos_close(taos); 37: taos_cleanup(); 38: return -1; 39: }	
	污染路径: 函数'taos_cleanup'的定义点.	
811	缺陷发生位置: /source/libs/monitorfw/src/taos_metric_formatter.c	行号: 216
	步骤描述: 不使用返回值的函数'taos_free'应使用'void'说明.	
	代码片段: 214: r = taos_string_builder_clear(self->string_builder); 215: if (r) { 216: taos_free(data); 217: return NULL; 218: }	

	污染路径: 函数'taos_free'的定义点.	
812	缺陷发生位置: /source/libs/parser/src/parInsertSql.c	行号: 615
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 613: *pData = taosMemoryMalloc(outputBytes); 614: if (NULL == *pData) { 615: taosMemoryFree(tmpTokenBuf); 616: 617: return terrno;	
	污染路径: 函数'taosMemoryFree'的定义点.	
813	缺陷发生位置: /utils/test/c/tmq_ts5776.c	行号: 57
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 55: int32_t ret = tmq_write_raw(pConn, raw); 56: printf("write raw data: %s\n", tmq_err2str(ret)); 57: ASSERT(ret == 0); 58: 59: tmq_free_raw(raw);	
	污染路径: 函数'ASSERT'的定义点.	
814	缺陷发生位置: /source/dnode/mnode/impl/src/mndSnode.c	行号: 276
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 274: 275: if ((code = mndTransAppendRedoAction(pTrans, &action)) != 0) { 276: taosMemoryFree(pReq); 277: TAOS_RETURN(code); 278: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
815	缺陷发生位置:	行号:

	/source/dnode/mgmt/node_mgmt/src/dmEnv.c	49
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: <pre> 47: int32_t code = 0; 48: if (atomic_val_compare_exchange_8(&pDnode->once, DND_ENV_INIT, DND_ENV_READY) != DND_ENV_INIT) { 49: dError("env is already initialized"); 50: code = TSDB_CODE_REPEAT_INIT; 51: return code; </pre>	
	污染路径: 函数'dError'的定义点.	
816	缺陷发生位置: /source/dnode/mnode/impl/src/mndDb.c	行号: 2904
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: <pre> 2902: STrans *pTrans = mndTransCreate(pMnode, TRN_POLICY_RETRY, TRN_CONFLICT_DB, pReq, "ssmigrate- db"); 2903: if (pTrans == NULL) { 2904: mError("failed to create ssmigrate-db trans since %s", terrstr()); 2905: goto _OVER; 2906: } </pre>	
	污染路径: 函数'mError'的定义点.	
817	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 380
	步骤描述: 不使用返回值的函数'taosGetTimeOfDay'应使用'void'说明.	
	代码片段: <pre> 378: static int64_t taosGetTimestampUs() { 379: struct timeval systemTime; 380: taosGetTimeOfDay(&systemTime); 381: return (int64_t)systemTime.tv_sec * 1000000LL + (int64_t)systemTime.tv_usec; 382: } </pre>	
	污染路径: 函数'taosGetTimeOfDay'的定义点.	

818	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftEntryDebug.c	行号: 72
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 70: printf("syncEntryPrint2 len:%zu %s %s \n", strlen(serialized), s, serialized); 71: fflush(NULL); 72: taosMemoryFree(serialized); 73: } 74: </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
819	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 2211
	步骤描述: 不使用返回值的函数'HUFv06_decodeStreamX2'应使用'void'说明.	
	代码片段: <pre> 2209: HUFv06_decodeStreamX2(op1, &bitD1, opStart2, dt, dtLog); 2210: HUFv06_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 2211: HUFv06_decodeStreamX2(op3, &bitD3, opStart4, dt, dtLog); 2212: HUFv06_decodeStreamX2(op4, &bitD4, oend, dt, dtLog); 2213: </pre>	
	污染路径: 函数'HUFv06_decodeStreamX2'的定义点.	
820	缺陷发生位置: /source/libs/executor/src/dataInserter.c	行号: 166
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: <pre> 164: gStreamGrpTableHash = taosHashInit(8, taosGetDefaultHashFunction(TSDB_DATA_TYPE_BIGINT), false, HASH_ENTRY_LOCK); 165: if (NULL == gStreamGrpTableHash) { 166: qError("failed to create stream group table hash"); 167: return terno; 168: } </pre>	

	函数'taosMemoryFree'的定义点.	
824	缺陷发生位置: /source/libs/qworker/src/qwMsg.c	行号: 503
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: 501: if (msgSize < 0) { 502: QW_SCH_TASK_ELOG("tSerializeSTaskDropReq failed, msgSize:%d", msgSize); 503: rpcFreeCont(msg); 504: QW_ERR_RET(msgSize); 505: }	
	污染路径: 函数'rpcFreeCont'的定义点.	
825	缺陷发生位置: /source/dnode/vnode/src/tq/tqSnapshot.c	行号: 141
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 139: if (code != 0){ 140: tqError("%s failed at %d tq snapshot writer open failed since %s", __func__, lino, tstrerror(code)); 141: taosMemoryFreeClear(pWriter); 142: } 143: return code;	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
826	缺陷发生位置: /source/libs/executor/src/dataInserter.c	行号: 78
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 76: SSubmitTbDataSendInfo* pSendInfo = p; 77: taosMemoryFree(pSendInfo->msg.pData); 78: taosMemoryFree(pSendInfo); 79: } 80:	
	污染路径: 函数'taosMemoryFree'的定义点.	
827	缺陷发生位置:	行号:

	/source/util/src/tlog.c	229
	步骤描述: 不使用返回值的函数'TAOS_UNUSED'应使用'void'说明.	
	代码片段: 227: if (tsLogObj.slowHandle == NULL) return terrno; 228: 229: TAOS_UNUSED(taosUmaskFile(0)); 230: tsLogObj.slowHandle->pFile = taosOpenFile(name, TD_FILE_CREATE TD_FILE_READ TD_FILE_WRITE TD_FILE_APPEND); 231: if (tsLogObj.slowHandle->pFile == NULL) {	
	污染路径: 函数'TAOS_UNUSED'的定义点.	
828	缺陷发生位置: /source/dnode/vnode/src/meta/metaTable2.c	行号: 170
	步骤描述: 不使用返回值的函数'tdbFreeClear'应使用'void'说明.	
	代码片段: 168: } else { 169: int64_t uid = *(int64_t *)value; 170: tdbFreeClear(value); 171: 172: if (uid != pReq->suid) {	
	污染路径: 函数'tdbFreeClear'的定义点.	
829	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 2107
	步骤描述: 不使用返回值的函数'HUFv06_decodeStreamX2'应使用'void'说明.	
	代码片段: 2105: if (HUFv06_isError(errorCode)) return errorCode; } 2106: 2107: HUFv06_decodeStreamX2(op, &bitD, oend, dt, dtLog); 2108: 2109: /* check */	
	污染路径: 函数'HUFv06_decodeStreamX2'的定义点.	
830	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 630

	步骤描述: 不使用返回值的函数'symTransform_4bytes'应使用'void'说明.	
	代码片段: 628: while(1) 629: { 630: symTransform_4bytes(p); 631: i+=sizeof(unsigned int); 632: if(i<size)	
	污染路径: 函数'symTransform_4bytes'的定义点.	
831	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 907
	步骤描述: 不使用返回值的函数'benchArrayAddBatch'应使用'void'说明.	
	代码片段: 905: BArray * copyBArray(BArray *pArray) { 906: BArray * pNew = benchArrayInit(pArray->size, pArray->elemSize); 907: benchArrayAddBatch(pNew, pArray->pData, pArray->size, false); 908: return pNew; 909: }	
	污染路径: 函数'benchArrayAddBatch'的定义点.	
832	缺陷发生位置: /source/util/src/mpDirect.c	行号: 107
	步骤描述: 不使用返回值的函数'taosMemFree'应使用'void'说明.	
	代码片段: 105: MP_LOCK(MP_READ, &pPool->cfgLock); // tmp test 106: 107: taosMemFree(ptr); 108: 109: if (NULL != pSession) {	
	污染路径: 函数'taosMemFree'的定义点.	
833	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbDataFileRW.c	行号: 598
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	

	代码片段: 596: tsdbError("vgId:%d %s failed at %s:%d since %s", TD_VID(reader->config->tsdb->pVnode), __func__, __FILE__, lino, 597: tstrerror(code)); 598: taosMemoryFree(data); 599: } 600: return code;	
	污染路径: 函数'taosMemoryFree'的定义点.	
834	缺陷发生位置: /utils/test/c/tmqOffset.c	行号: 39
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 37: tDecoderInit(&decoder, memBuf, size); 38: if (tDecodeSTqOffset(&decoder, &offset) < 0) { 39: taosMemoryFree(memBuf); 40: tDecoderClear(&decoder); 41: printf("tDecodeSTqOffset error\n");	
	污染路径: 函数'taosMemoryFree'的定义点.	
835	缺陷发生位置: /contrib/TSZ/zstd/compress/zstd_compress.c	行号: 66
	步骤描述: 不使用返回值的函数'assert'应使用'void'说明.	
	代码片段: 64: cctx->bmi2 = ZSTD_cpuid_bmi2(ZSTD_cpuid()); 65: { size_t const err = ZSTD_CCtx_resetParameters(cctx); 66: assert(!ZSTD_isError(err)); 67: (void)err; 68: }	
	污染路径: 函数'assert'的定义点.	
836	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftEntryDebug.c	行号: 45
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 43: s = syncUtilPrintBin2((char*)(pEntry->data), pEntry-	

	<pre> >dataLen); 44: cJSON_AddStringToObject(pRoot, "data2", s); 45: taosMemoryFree(s); 46: } 47: </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
837	缺陷发生位置: /source/dnode/mgmt/mgmt_dnode/src/dmWorker.c	行号: 52
	步骤描述: 不使用返回值的函数'setThreadName'应使用'void'说明.	
	代码片段: <pre> 50: SDnodeMgmt *pMgmt = param; 51: int64_t lastTime = taosGetTimestampMs(); 52: setThreadName("dnode-config"); 53: while (1) { 54: taosMsleep(50); </pre>	
	污染路径: 函数'setThreadName'的定义点.	
838	缺陷发生位置: /source/dnode/vnode/src/tq/tqOffset.c	行号: 118
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 116: } 117: (void)taosCloseFile(&pFile); 118: taosMemoryFree(pMemBuf); 119: 120: tDeleteSTqOffset(pOffset); </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
839	缺陷发生位置: /source/client/src/clientMain.c	行号: 129
	步骤描述: 不使用返回值的函数'tscDebug'应使用'void'说明.	
	代码片段: <pre> 127: } 128: 129: tscDebug("set timezone to %s", val); 130: tz = tzalloc(val); </pre>	

	131: if (tz == NULL) {	
	污染路径: 函数'tscDebug'的定义点.	
840	缺陷发生位置: /source/libs/qcom/src/querymsg.c	行号: 113
	步骤描述: 不使用返回值的函数'QUERY_PARAM_CHECK'应使用'void'说明.	
	代码片段: 111: void (*freeFp)(void *) { 112: QUERY_PARAM_CHECK(input); 113: QUERY_PARAM_CHECK(msg); 114: QUERY_PARAM_CHECK(msgLen); 115: SBuildUseDBInput *pInput = input;	
	污染路径: 函数'QUERY_PARAM_CHECK'的定义点.	
841	缺陷发生位置: /docs/examples/c/with_reqid_demo.c	行号: 51
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 49: taos_errstr(result)); 50: taos_close(taos); 51: taos_cleanup(); 52: return -1; 53: }	
	污染路径: 函数'taos_cleanup'的定义点.	
842	缺陷发生位置: /utils/test/c/tmq_blob.c	行号: 242
	步骤描述: 不使用返回值的函数'create_topic'应使用'void'说明.	
	代码片段: 240: return -1; 241: } 242: create_topic(); 243: 244: tmq_t* tmq = build_consumer();	
	污染路径: 函数'create_topic'的定义点.	

843	缺陷发生位置: /source/client/src/clientSmlLine.c	行号: 483
	步骤描述: 不使用返回值的函数'JUMP_SPACE'应使用'void'说明.	
	代码片段: 481: int32_t smlParseInfluxString(SSmlHandle *info, char *sql, char *sqlEnd, SSmlLineInfo *elements) { 482: if (!sql) return TSDB_CODE_SML_INVALID_DATA; 483: JUMP_SPACE(sql, sqlEnd) 484: if (unlikely(*sql == COMMA)) return TSDB_CODE_SML_INVALID_DATA; 485: elements->measure = sql;	
	污染路径: 函数'JUMP_SPACE'的定义点.	
844	缺陷发生位置: /source/dnode/vnode/src/tq/tqScan.c	行号: 253
	步骤描述: 不使用返回值的函数'tqDebug'应使用'void'说明.	
	代码片段: 251: SSDataBlock* pDataBlock = NULL; 252: uint64_t ts = 0; 253: tqDebug("tmqsnap task start to execute"); 254: code = qExecTask(task, &pDataBlock, &ts); 255: TSDB_CHECK_CODE(code, lino, END);	
	污染路径: 函数'tqDebug'的定义点.	
845	缺陷发生位置: /test/new_test_framework/script/api/demoapi.c	行号: 132
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 130: int64_t affected; 131: for (; j < g_num_of_rec -2; j++) { 132: sprintf(command, "insert into t%"PRIu64" " 133: "values(%" PRIu64 " , %f, %"PRIu64" , " 134: "'%c%d', '%s%c%d', '%c%d')",	
	污染路径: 无	
846	缺陷发生位置: /source/os/test/osStringTests.cpp	行号: 308

	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 306: ASSERT_EQ(val, INT16_MAX); 307: 308: snprintf(large_num, sizeof(large_num), "%d", INT16_MIN); 309: result = taosStr2int16(large_num, &val); 310: ASSERT_EQ(result, 0);	
	污染路径: 无	
847	缺陷发生位置: /source/libs/executor/src/filloperator.c	行号: 196
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 194: SSDataBlock* pResBlock = pInfo->pFinalRes; 195: if (pResBlock == NULL) { 196: qError("%s failed at line %d since pResBlock is NULL.", __func__, __LINE__); 197: return NULL; 198: }	
	污染路径: 函数'qError'的定义点.	
848	缺陷发生位置: /source/libs/scheduler/src/schRemote.c	行号: 100
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 98: } 99: 100: taosMemoryFreeClear(msg); 101: 102: return TSDB_CODE_SUCCESS;	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
849	缺陷发生位置: /source/libs/command/src/command.c	行号: 752
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	

	代码片段: <pre> 750: *len += snprintf(buf + VARSTR_HEADER_SIZE + *len, SHOW_CREATE_TB_RESULT_FIELD2_LEN (VARSTR_HEADER_SIZE + *len), 751: ""%s"", escapedJson); 752: taosMemoryFree(escapedJson); 753: } else { 754: taosMemoryFree(pjson); </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
850	缺陷发生位置: /contrib/TSZ/sz/src/sz_float.c	行号: 127
	步骤描述: 不使用返回值的函数'int32ToBytes_bigEndian'应使用'void'说明.	
	代码片段: <pre> 125: 126: unsigned char preDiffBytes[4]; 127: int32ToBytes_bigEndian(preDiffBytes, 0); 128: 129: // calc save byte length and bit lengths with reqLength </pre>	
	污染路径: 函数'int32ToBytes_bigEndian'的定义点.	
851	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 682
	步骤描述: 不使用返回值的函数'OS_PARAM_CHECK'应使用'void'说明.	
	代码片段: <pre> 680: int32_t taosGetCpuInfo(char *cpuModel, int32_t maxLen, float *numOfCores) { 681: OS_PARAM_CHECK(cpuModel); 682: OS_PARAM_CHECK(numOfCores); 683: #ifdef WINDOWS 684: char value[100]; </pre>	
	污染路径: 函数'OS_PARAM_CHECK'的定义点.	
852	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSet2.c	行号: 653
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段:	

	<pre> 651: int32_t code = tsdbTFileSetInitRef(pTsdb, fset1, &fsr[0]- >fset); 652: if (code) { 653: taosMemoryFree(fsr[0]); 654: fsr[0] = NULL; 655: }</pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
853	缺陷发生位置: /source/os/src/osFile.c	行号: 237
	步骤描述: 不使用返回值的函数'OS_PARAM_CHECK'应使用'void'说明.	
	代码片段: <pre> 235: 236: int32_t taosRenameFile(const char *oldName, const char *newName) { 237: OS_PARAM_CHECK(oldName); 238: OS_PARAM_CHECK(newName); 239: #ifdef WINDOWS</pre>	
	污染路径: 函数'OS_PARAM_CHECK'的定义点.	
854	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttKeepalive.c	行号: 51
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: <pre> 49: 50: int ttqKeepaliveRemove(struct tmqtt *context) { 51: UNUSED(context); 52: 53: return TTQ_ERR_SUCCESS;</pre>	
	污染路径: 函数'UNUSED'的定义点.	
855	缺陷发生位置: /source/libs/tdb/src/db/tdbBtree.c	行号: 114
	步骤描述: 不使用返回值的函数'tdbOsFree'应使用'void'说明.	
	代码片段: <pre> 112: ret = tdbBegin(pEnv, &txn, tdbDefaultMalloc, tdbDefaultFree, NULL, TDB_TXN_WRITE </pre>	

	TDB_TXN_READ_UNCOMMITTED); 113: if (ret < 0) { 114: tdbOsFree(pBt); 115: return ret; 116: }	
	污染路径: 函数'tdbOsFree'的定义点.	
856	缺陷发生位置: /source/dnode/xnode/src/xnode.c	行号: 51
	步骤描述: 不使用返回值的函数'xnodeMgmtStopXnoded'应使用'void'说明.	
	代码片段: 49: void xndClose(SXnode *pXnode) { 50: xndInfo("xnode: dnode is closing xnoded"); 51: xnodeMgmtStopXnoded(); 52: } 53:	
	污染路径: 函数'xnodeMgmtStopXnoded'的定义点.	
857	缺陷发生位置: /source/dnode/mnode/impl/src/mndScan.c	行号: 992
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 990: 991: if (tSerializeSScanVnodeReq((char *)pReq + sizeof(SMsgHead), contLen, &scanReq) < 0) { 992: taosMemoryFree(pReq); 993: terrno = TSDB_CODE_OUT_OF_MEMORY; 994: return NULL;	
	污染路径: 函数'taosMemoryFree'的定义点.	
858	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmlnt.c	行号: 263
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 261: SVnodeObj *pClosedVnode = taosMemoryCalloc(1, sizeof(SVnodeObj)); 262: if (pClosedVnode == NULL) {	

	263: dError("failed to alloc vnode since %s", terrstr()); 264: return terrno; 265: }	
	污染路径: 函数'dError'的定义点.	
859	缺陷发生位置: /source/dnode/mnode/impl/src/mndSsMigrate.c	行号: 59
	步骤描述: 不使用返回值的函数'mDebug'应使用'void'说明.	
	代码片段: 57: } 58: 59: void mndCleanupSsMigrate(SMnode *pMnode) { mDebug("mnd ssmigrate cleanup"); } 60: 61: void tFreeSsMigrateObj(SSsMigrateObj *pSsMigrate) {	
	污染路径: 函数'mDebug'的定义点.	
860	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbMerge.c	行号: 524
	步骤描述: 不使用返回值的函数'taosMsleep'应使用'void'说明.	
	代码片段: 522: tsdbError("vgId:%d %s failed at %s:%d since %s", TD_VID(tsdb->pVnode), __func__, __FILE__, lino, tstrerror(code)); 523: tsdbFatal("vgId:%d, failed to merge stt files since %s. code:%d", TD_VID(tsdb->pVnode), terrstr(), code); 524: taosMsleep(100); 525: exit(EXIT_FAILURE); 526: }	
	污染路径: 函数'taosMsleep'的定义点.	
861	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCacheRead.c	行号: 311
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 309: p->rowKey.pks[0].pData = taosMemoryCalloc(1, pPkCol->bytes);	

	<pre> 310: if (p->rowKey.pks[0].pData == NULL) { 311: taosMemoryFreeClear(p); 312: code = terrno; 313: TSDB_CHECK_CODE(code, lino, _end); </pre>	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
862	缺陷发生位置: /source/os/src/osSemaphore.c	行号: 244
	步骤描述: 不使用返回值的函数'OS_PARAM_CHECK'应使用'void'说明.	
	代码片段: <pre> 242: int32_t taosGetAppName(char* name, int32_t* len) { 243: #ifndef TD_ASTR 244: OS_PARAM_CHECK(name); 245: const char* self = "/proc/self/exe"; 246: char path[PATH_MAX] = {0}; </pre>	
	污染路径: 函数'OS_PARAM_CHECK'的定义点.	
863	缺陷发生位置: /source/client/src/clientSmlTelnet.c	行号: 77
	步骤描述: 不使用返回值的函数'JUMP_SPACE'应使用'void'说明.	
	代码片段: <pre> 75: const char *sql = data; 76: while (sql < sqlEnd) { 77: JUMP_SPACE(sql, sqlEnd) 78: if (unlikely(*sql == '\0' *sql == '\n')) break; 79: </pre>	
	污染路径: 函数'JUMP_SPACE'的定义点.	
864	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1517
	步骤描述: 不使用返回值的函数'tstrncpy'应使用'void'说明.	
	代码片段: <pre> 1515: char* p = taosMemoryCalloc(strLen + 8, 1); 1516: if (p) { 1517: tstrncpy(p, str, strLen + 1); 1518: tstrncpy(p + strLen, ";", 1 + 1); 1519: lastWordBytes += 1; </pre>	

	污染路径: 函数'tstrncpy'的定义点.	
865	缺陷发生位置: /test/new_test_framework/script/api/stmt_function.c	行号: 96
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 94: assert(taos_stmt_close(stmt) == 0); 95: 96: taosMemoryFree(stmt_sql); 97: printf("finish taos_stmt_prepare test\n"); 98: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
866	缺陷发生位置: /test/new_test_framework/script/api/sameReqidTest.c	行号: 403
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 401: 402: taos_close(taos); 403: taos_cleanup(); 404: 405: return 0;	
	污染路径: 函数'taos_cleanup'的定义点.	
867	缺陷发生位置: /tools/rocks-reader/rreader.cpp	行号: 152
	步骤描述: 不使用返回值的函数'operator='应使用'void'说明.	
	代码片段: 150: // pks 151: for (int32_t i = 0; i < pLastCol->rowKey.numOfPKs; i++) { 152: pLastCol->rowKey.pks[i] = *(SValue*)(value + offset); 153: offset += sizeof(SValue); 154:	
	污染路径: 函数'operator='的定义点.	

868	缺陷发生位置: /docs/examples/c/create_db_demo.c	行号: 48
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 46: fprintf(stderr, "Failed to create database power, ErrCode: 0x%x, ErrorMessage: %s.\n", code, taos_errstr(result)); 47: taos_close(taos); 48: taos_cleanup(); 49: return -1; 50: }</pre>	
	污染路径: 函数'taos_cleanup'的定义点.	
869	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 215
	步骤描述: 不使用返回值的函数'DEBUGLOG'应使用'void'说明.	
	代码片段: <pre> 213: buffer.capacity = (start==NULL) ? 0 : bSize; 214: if (start==NULL) { 215: DEBUGLOG(5, "ZSTDMT_getBuffer: buffer allocation failure !!"); 216: } else { 217: DEBUGLOG(5, "ZSTDMT_getBuffer: created buffer of size %u", (U32)bSize);</pre>	
	污染路径: 函数'DEBUGLOG'的定义点.	
870	缺陷发生位置: /utils/test/c/tmq_token.c	行号: 150
	步骤描述: 不使用返回值的函数'taosSsleep'应使用'void'说明.	
	代码片段: <pre> 148: taosSsleep(5); 149: alter_token_enable(); 150: taosSsleep(5); 151: drop_token(); 152: return NULL;</pre>	
	污染路径: 函数'taosSsleep'的定义点.	
871	缺陷发生位置:	行号:

	/source/util/src/ttimer.c	659
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 657: 658: taosCleanUpScheduler(tmrQhandle); 659: taosMemoryFreeClear(tmrQhandle); 660: 661: taosCleanUpScheduler(tmrQhandleHigh);	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
872	缺陷发生位置: /docs/examples/c-ws-new/async_demo.c	行号: 109
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 107: tableList[i].id = i; 108: tableList[i].taos = taos; 109: sprintf(tableList[i].name, "%s%d", prefix, i); 110: sprintf(sql, "create table %s%d (ts timestamp, volume bigint)", prefix, i); 111: queryDB(taos, sql);	
	污染路径: 无	
873	缺陷发生位置: /tests/taosc_test/taoscTest.cpp	行号: 120
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 118: for (int i = 0; i < insertCounts; i++) { 119: char sql[128]; 120: sprintf(sql, "insert into taosc_test_db.%s values(now() + %ds, %d)", table_name, i, i); 121: res = taos_query(taos, sql); 122: ASSERT_TRUE(taos_errno(res) == 0);	
	污染路径: 无	
874	缺陷发生位置: /source/os/src/osSystem.c	行号: 288
	步骤描述:	

	不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: <pre> 286: } 287: if (*ptrBuf != NULL) { 288: taosMemoryFreeClear(*ptrBuf); 289: } 290: </pre>	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
875	缺陷发生位置: /contrib/TSZ/zstd/decompress/zstd_decompress.c	行号: 2467
	步骤描述: 不使用返回值的函数'assert'应使用'void'说明.	
	代码片段: <pre> 2465: sizeof(ZSTD_DDict) + (dictLoadMethod == ZSTD_dlm_byRef ? 0 : dictSize); 2466: ZSTD_DDict* const ddict = (ZSTD_DDict*)workspace; 2467: assert(workspace != NULL); 2468: assert(dict != NULL); 2469: if ((size_t)workspace & 7) return NULL; /* 8-aligned */ </pre>	
	污染路径: 函数'assert'的定义点.	
876	缺陷发生位置: /source/libs/parser/src/parUtil.c	行号: 366
	步骤描述: 不使用返回值的函数'tstrncpy'应使用'void'说明.	
	代码片段: <pre> 364: 365: char buf[64] = {0}; // only extract part of sql string 366: tstrncpy(buf, sourceStr, tListLen(buf)); 367: 368: if (additionalInfo != NULL) { </pre>	
	污染路径: 函数'tstrncpy'的定义点.	
877	缺陷发生位置: /utils/test/c/tmq_ts7402.c	行号: 40
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段:	

	<pre> 38: char* result = tmq_get_json_meta(msg); 39: printf("meta result: %s\n", result); 40: ASSERT(strcmp(result, result1[cnt]) == 0); 41: tmq_free_json_meta(result); 42: } else { </pre>	
	污染路径: 函数'ASSERT'的定义点.	
878	缺陷发生位置: /source/util/src/thash.c	行号: 632
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 630: 631: if (taosArrayPush(pHashObj->pMemBlock, &p) == NULL) { 632: taosMemoryFree(p); 633: return; 634: } </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
879	缺陷发生位置: /source/client/src/clientRawBlockWrite.c	行号: 136
	步骤描述: 不使用返回值的函数'uDebug'应使用'void'说明.	
	代码片段: <pre> 134: buildDefaultCompress = 1; 135: } 136: RAW_LOG_START 137: char* string = NULL; 138: cJSON* json = cJSON_CreateObject(); </pre>	
	污染路径: expanded from macro 'RAW_LOG_START' 函数'uDebug'的定义点. expanded from macro 'RAW_LOG_START'	
880	缺陷发生位置: /docs/examples/c/multi_bind_example.c	行号: 143
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 141: insertData(taos); </pre>	

	142: taos_close(taos); 143: taos_cleanup(); 144: } 145:	
	污染路径: 函数'taos_cleanup'的定义点.	
881	缺陷发生位置: /source/client/wrapper/src/clientTmqConnector.c	行号: 37
	步骤描述: 不使用返回值的函数'taosMsleep'应使用'void'说明.	
	代码片段: 35: case 1: 36: do { 37: taosMsleep(0); 38: } while (atomic_load_32(&__init_tmq) == 1); 39: case 2:	
	污染路径: 函数'taosMsleep'的定义点.	
882	缺陷发生位置: /utls/test/c/tmq_vtable.c	行号: 72
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 70: int32_t ret = tmq_write_raw(pConn, raw); 71: printf("write raw data: %s\n", tmq_err2str(ret)); 72: ASSERT(ret == 0); 73: 74: tmq_free_raw(raw);	
	污染路径: 函数'ASSERT'的定义点.	
883	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeSync.c	行号: 629
	步骤描述: 不使用返回值的函数'vnodeSnapReaderClose'应使用'void'说明.	
	代码片段: 627: static void vnodeSnapshotStopRead(const SSyncFSM *pFsm, void *pReader) { 628: SVnode *pVnode = pFsm->data; 629: vnodeSnapReaderClose(pReader); 630: }	

	631:	
	污染路径: 函数'vnodeSnapReaderClose'的定义点.	
884	缺陷发生位置: /docs/examples/c/insert_data_demo.c	行号: 38
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 36: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x %x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 37: taos_errstr(NULL)); 38: taos_cleanup(); 39: return -1; 40: }	
	污染路径: 函数'taos_cleanup'的定义点.	
885	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smWorker.c	行号: 263
	步骤描述: 不使用返回值的函数'streamReleaseTask'应使用'void'说明.	
	代码片段: 261: taosArrayDestroyEx(batchReq.pMsgs, tFreeSBatchReqMsg); 262: if (taskAddr != NULL) { 263: streamReleaseTask(taskAddr); 264: } 265: if (code) {	
	污染路径: 函数'streamReleaseTask'的定义点.	
886	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 3504
	步骤描述: 不使用返回值的函数'parserError'应使用'void'说明.	
	代码片段: 3502: nodesDestroyNode((SNode*)pCol); 3503: } 3504: parserError("%s failed to rewrite count(1), code:%d", __func__, code); 3505: return code; 3506: }	

	污染路径: 函数'parserError'的定义点.	
887	缺陷发生位置: /source/util/src/tmempool.c	行号: 456
	步骤描述: 不使用返回值的函数'ulInfo'应使用'void'说明.	
	代码片段: 454: } 455: 456: ulInfo("[bytes]:"); 457: switch (gMPMgmt.strategy) { 458: case E_MP_STRATEGY_DIRECT:	
	污染路径: 函数'ulInfo'的定义点.	
888	缺陷发生位置: /source/client/src/clientMsgHandler.c	行号: 115
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 113: if (rpcSetDefaultAddr(pTscObj->pAppInfo->pTransporter, srcEpSet.eps[srcEpSet.inUse].fqdn, 114: dstEpSet.eps[dstEpSet.inUse].fqdn) != 0) { 115: tscError("failed to set default addr for rpc"); 116: } 117: updateEpSet = 0;	
	污染路径: 函数'tscError'的定义点.	
889	缺陷发生位置: /source/util/test/trefTest.c	行号: 78
	步骤描述: 不使用返回值的函数'taosReleaseRef'应使用'void'说明.	
	代码片段: 76: if (p) { 77: taosUsleep(id % 5 + 1); 78: taosReleaseRef(pSpace->rsetId, (int64_t)pSpace->p[id]); 79: } 80: }	

	污染路径: 函数'taosReleaseRef'的定义点.	
890	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 2019
	步骤描述: 不使用返回值的函数'parserError'应使用'void'说明.	
	代码片段: 2017: 2018: if (code) { 2019: parserError("%s failed to find and set column info, code:%d", __func__, code); 2020: } 2021: return code;	
	污染路径: 函数'parserError'的定义点.	
891	缺陷发生位置: /source/common/src/tglobal.c	行号: 1479
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 1477: 1478: *pScope = slowScope; 1479: taosMemoryFreeClear(tmp); 1480: TAOS_RETURN(TSDB_CODE_SUCCESS); 1481: }	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
892	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 428
	步骤描述: 不使用返回值的函数'z'应使用'void'说明.	
	代码片段: 426: z[i] = x[i] + y[i]; 427: auto k1 = z[i] < x[i]; 428: z[i] += k; 429: k = (z[i] < k) k1; 430: }	
	污染路径: 无	
893	缺陷发生位置:	行号:

896	缺陷发生位置: /contrib/TSZ/zstd/compress/zstd_compress.c	行号: 995
	步骤描述: 不使用返回值的函数'DEBUGLOG'应使用'void'说明.	
	代码片段: 993: size_t const windowSize = MAX(1, (size_t)MIN(((U64)1 << params.cParams.windowLog), pledgedSrcSize)); 994: size_t const blockSize = MIN(ZSTD_BLOCKSIZE_MAX, windowSize); 995: DEBUGLOG(4, "ZSTD_continueCCTX: re-use context in place"); 996: 997: cctx->blockSize = blockSize; /* previous block size could be different even for same windowLog, due to pledgedSrcSize */	
	污染路径: 函数'DEBUGLOG'的定义点.	
897	缺陷发生位置: /source/libs/executor/src/groupoperator.c	行号: 101
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 99: cleanupGroupResInfo(&pInfo->groupResInfo); 100: cleanupAggSup(&pInfo->aggSup); 101: taosMemoryFreeClear(param); 102: } 103:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
898	缺陷发生位置: /source/libs/scheduler/src/scheduler.c	行号: 340
	步骤描述: 不使用返回值的函数'SCH_JOB_ELOG'应使用'void'说明.	
	代码片段: 338: HASH_ENTRY_LOCK); 339: if (NULL == pJob->taskList) { 340: SCH_JOB_ELOG("taosHashInit %d taskList failed", 100); 341: SCH_ERR_JRET(terrno); 342: }	
	污染路径:	

	函数'SCH_JOB_ELOG'的定义点.	
899	缺陷发生位置: /source/dnode/mnode/impl/src/mndInfoSchema.c	行号: 99
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: <pre> 97: strcmp(tbName, TSDB_INS_TABLE_XNODES) == 0 strcmp(tbName, TSDB_INS_TABLE_XNODES_FULL) == 0 98: strcmp(tbName, TSDB_INS_TABLE_LICENCES) == 0)) { 99: mError("no permission to get schema of table name: %s", tbName); 100: code = TSDB_CODE_PAR_PERMISSION_DENIED; 101: TAOS_RETURN(code); </pre>	
	污染路径: 函数'mError'的定义点.	
900	缺陷发生位置: /source/dnode/vnode/src/bse/bseTableMgt.c	行号: 207
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 205: 206: taosHashCleanup(p->pHashObj); 207: taosMemoryFree(p); 208: } 209: </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
901	缺陷发生位置: /source/client/wrapper/src/clientTmqConnector.c	行号: 85
	步骤描述: 不使用返回值的函数'jniDebug'应使用'void'说明.	
	代码片段: <pre> 83: 84: atomic_store_32(&__init_assignment, 2); 85: jniDebug("tmq method assignment finished"); 86: } 87: </pre>	
	污染路径: 函数'jniDebug'的定义点.	

902	缺陷发生位置: /source/dnode/mnode/impl/src/mndArbGroup.c	行号: 369
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: 367: 368: if (tSerializeSVArbHeartBeatReq(pReq, contLen, &req) <= 0) { 369: rpcFreeCont(pReq); 370: return NULL; 371: }	
	污染路径: 函数'rpcFreeCont'的定义点.	
903	缺陷发生位置: /source/libs/transport/test/uv.c	行号: 132
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 130: uv_close((uv_handle_t *)pConn->pClient, NULL); 131: taosMemoryFree(pConn->pClient); 132: taosMemoryFree(pConn); 133: } 134: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
904	缺陷发生位置: /source/common/src/msg/streamMsg.c	行号: 4612
	步骤描述: 不使用返回值的函数'blockDataEmpty'应使用'void'说明.	
	代码片段: 4610: decoder->pos = (uint8_t*)pEndPos - decoder->data; 4611: } else if (pBlock != NULL) { 4612: blockDataEmpty(pBlock); 4613: } 4614:	
	污染路径: 函数'blockDataEmpty'的定义点.	
905	缺陷发生位置: /source/libs/planner/src/planSpliter.c	行号: 645
	步骤描述:	

	不使用返回值的函数'PLAN_ERR_JRET'应使用'void'说明.	
	代码片段: <pre> 643: planError("%s failed since no pkts placeholder set", __FUNCTION__); 644: code = TSDB_CODE_INTERNAL_ERROR; 645: PLAN_ERR_JRET(code); 646: } 647: </pre>	
	污染路径: 函数'PLAN_ERR_JRET'的定义点.	
906	缺陷发生位置: /source/libs/executor/src/groupoperator.c	行号: 1137
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: <pre> 1135: pInfo->orderedRows = 0; 1136: } else if (pInfo->pOrderInfoArr == NULL) { 1137: qError("Exception, remainRows not zero, but pOrderInfoArr is NULL"); 1138: } 1139: </pre>	
	污染路径: 函数'qError'的定义点.	
907	缺陷发生位置: /source/dnode/xnode/src/xnode.c	行号: 44
	步骤描述: 不使用返回值的函数'xndInfo'应使用'void'说明.	
	代码片段: <pre> 42: // } 43: 44: xndInfo("xnode: opened & initialized by dnode"); 45: 46: return TSDB_CODE_SUCCESS; </pre>	
	污染路径: 函数'xndInfo'的定义点.	
908	缺陷发生位置: /source/libs/transport/test/cliBench.c	行号: 81
	步骤描述: 不使用返回值的函数'tDebug'应使用'void'说明.	
	代码片段:	

	79: } 80: 81: tDebug("thread:%d, it is over", pInfo->index); 82: tcount++; 83:	
	污染路径: 函数'tDebug'的定义点.	
909	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncMessageDebug.c	行号: 213
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 211: s = syncUtilPrintBin2((char*)(pMsg->data), pMsg->dataLen); 212: cJSON_AddStringToObject(pRoot, "data2", s); 213: taosMemoryFree(s); 214: } 215:	
	污染路径: 函数'taosMemoryFree'的定义点.	
910	缺陷发生位置: /source/dnode/snode/src/snode.c	行号: 112
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 110: stDebug("[checkpoint] streamId:%" PRIx64 " , checkpoint no need send back, ver:%d, dataLen:%" PRId64, streamId, ver, dataLen); 111: dataLen = 0; 112: taosMemoryFreeClear(data); 113: } 114:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
911	缺陷发生位置: /docs/examples/c/schemaless.c	行号: 58
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 56: for (int k = 0; k < numRowsPerChildTable; ++k) {	

	57: char* line = calloc(512, 1); 58: snprintf(line, 512, lineFormat, i, j, ts + 10 * l); 59: lines[l] = line; 60: ++l;	
	污染路径: 无	
912	缺陷发生位置: /utls/test/c/tmq_batch_alter_tag.c	行号: 106
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 104: 105: printf("\n\nmeta result:%s\n%s\n\n\n", json_get, json_result); 106: ASSERT(strcmp(json_get, json_result) == 0); 107: code = tmq_consumer_close(tmq); 108: if (code)	
	污染路径: 函数'ASSERT'的定义点.	
913	缺陷发生位置: /source/libs/monitor/src/monMain.c	行号: 172
	步骤描述: 不使用返回值的函数'monCleanupMonitorFW'应使用'void'说明.	
	代码片段: 170: (void)taosThreadMutexDestroy(&tsMonitor.lock); 171: 172: monCleanupMonitorFW(); 173: } 174:	
	污染路径: 函数'monCleanupMonitorFW'的定义点.	
914	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttCtx.c	行号: 214
	步骤描述: 不使用返回值的函数'bndError'应使用'void'说明.	
	代码片段: 212: tmq_list_t* topic_list = tmq_list_new(); 213: if (!topic_list) { 214: bndError("failed to remove topic: %s", topic_name); 215:	

	216: return NULL;	
	污染路径: 函数'bndError'的定义点.	
915	缺陷发生位置: /docs/examples/c/with_reqid_demo.c	行号: 38
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 36: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x %x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 37: taos_errstr(NULL)); 38: taos_cleanup(); 39: return -1; 40: }	
	污染路径: 函数'taos_cleanup'的定义点.	
916	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqSessionExpiry.c	行号: 123
	步骤描述: 不使用返回值的函数'DL_FOREACH_SAFE'应使用'void'说明.	
	代码片段: 121: last_check = db.now_real_s; 122: 123: DL_FOREACH_SAFE(expiry_list, item, tmp) { 124: if (item->context->session_expiry_time < db.now_real_s) { 125: context = item->context;	
	污染路径: 函数'DL_FOREACH_SAFE'的定义点.	
917	缺陷发生位置: /tools/shell/src/shellCommand.c	行号: 539
	步骤描述: 不使用返回值的函数'pressOtherKey'应使用'void'说明.	
	代码片段: 537: } 538: shellInsertChar(&cmd, utf8_array, count); 539: pressOtherKey(c); 540: } else if (c == TAB_KEY) { 541: // press TAB key	

	污染路径: 函数'pressOtherKey'的定义点.	
918	缺陷发生位置: /source/libs/executor/src/executil.c	行号: 874
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 872: // taosMemoryFree(tmp); 873: 874: taosMemoryFree(payload); 875: return TSDB_CODE_SUCCESS; 876: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
919	缺陷发生位置: /docs/examples/c/connect_example.c	行号: 38
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 36: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x%x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 37: taos_errstr(NULL)); 38: taos_cleanup(); 39: return -1; 40: }	
	污染路径: 函数'taos_cleanup'的定义点.	
920	缺陷发生位置: /tools/shell/src/shellMain.c	行号: 82
	步骤描述: 不使用返回值的函数'shellGenerateAuth'应使用'void'说明.	
	代码片段: 80: 81: if (shell.args.is_gen_auth) { 82: shellGenerateAuth(); 83: return 0; 84: }	
	污染路径: 函数'shellGenerateAuth'的定义点.	
921	缺陷发生位置:	行号:

	/utils/test/c/tmq_offset_test.c	216
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 214: } 215: 216: ASSERT(numOfAssign1 == 2); 217: ASSERT(numOfAssign1 == numOfAssign2); 218: ASSERT(numOfAssign1 == numOfAssign3);	
	污染路径: 函数'ASSERT'的定义点.	
922	缺陷发生位置: /source/libs/executor/src/executorInt.c	行号: 472
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 470: // do nothing 471: } else { 472: qError("invalid constant type for sma info"); 473: } 474:	
	污染路径: 函数'qError'的定义点.	
923	缺陷发生位置: /test/new_test_framework/script/api/stmtTest.c	行号: 185
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 183: for (int i = 0; i < 10; i++) { 184: char buf[32]; 185: sprintf(buf, "t%d", i); 186: if (i == 0) { 187: code = taos_stmt_set_tbname(stmt, buf);	
	污染路径: 无	
924	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 1010
	步骤描述: 不使用返回值的函数'close'应使用'void'说明.	
	代码片段:	

	1008: WSACleanup(); 1009: #else 1010: close(sockfd); 1011: #endif 1012: }	
	污染路径: 无	
925	缺陷发生位置: /source/dnode/mnode/impl/src/mndQnode.c	行号: 410
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 408: 409: if ((code = mndTransAppendRedoAction(pTrans, &action)) != 0) { 410: taosMemoryFree(pReq); 411: TAOS_RETURN(code); 412: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
926	缺陷发生位置: /contrib/test/craft/simulate_vnode.c	行号: 42
	步骤描述: 不使用返回值的函数'vnodeApplyWMsg'应使用'void'说明.	
	代码片段: 40: 41: SVnode* tmp_vnodes = (SVnode*)(fsm->data); 42: vnodeApplyWMsg(&tmp_vnodes[vid], value, NULL); 43: 44: return 0;	
	污染路径: 函数'vnodeApplyWMsg'的定义点.	
927	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeStream.c	行号: 802
	步骤描述: 不使用返回值的函数'STREAM_PRINT_LOG_END_WITHID'应使用'void'说明.	
	代码片段: 800: end: 801: taosArrayDestroy(uidListAdd); 802: STREAM_PRINT_LOG_END_WITHID(code, lino);	

	803: return code; 804: }	
	污染路径: 函数'STREAM_PRINT_LOG_END_WITHID'的定义点.	
928	缺陷发生位置: /source/dnode/mnode/impl/src/mndStreamUtil.c	行号: 546
	步骤描述: 不使用返回值的函数'mstDebug'应使用'void'说明.	
	代码片段: 544: 545: if (NULL == p) { 546: mstDebug("%s: stream is NULL", tips); 547: return; 548: }	
	污染路径: 函数'mstDebug'的定义点.	
929	缺陷发生位置: /source/client/src/clientSml.c	行号: 232
	步骤描述: 不使用返回值的函数'smlStrReplace'应使用'void'说明.	
	代码片段: 230: PROCESS_SLASH_IN_MEASUREMENT(measure, measureLen); 231: } 232: smlStrReplace(measure, measureLen); 233: code = smlGetMeta(info, measure, measureLen, &pTableMeta); 234: taosMemoryFree(measure);	
	污染路径: 函数'smlStrReplace'的定义点.	
930	缺陷发生位置: /source/dnode/mnode/impl/src/mndProfile.c	行号: 274
	步骤描述: 不使用返回值的函数'taosCacheDestroyIter'应使用'void'说明.	
	代码片段: 272: static void mndCancelGetNextConn(SMnode *pMnode, void *pIter) { 273: if (pIter != NULL) { 274: taosCacheDestroyIter(pIter); 275: }	

	276: }	
	污染路径: 函数'taosCacheDestroyIter'的定义点.	
931	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 1248
	步骤描述: 不使用返回值的函数'close'应使用'void'说明.	
	代码片段: 1246: WSACleanup(); 1247: #else 1248: close(pThreadInfo->sockfd); 1249: #endif 1250: } else {	
	污染路径: 无	
932	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttDaemon.c	行号: 239
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 237: code = msgInfo->code; 238: } 239: taosMemoryFree(msgInfo); 240: return code; 241: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
933	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncBatch.c	行号: 173
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 171: printf("syncClientRequestBatchPrint2 len:%d %s %s \n", (int32_t)strlen(serialized), s, serialized); 172: fflush(NULL); 173: taosMemoryFree(serialized); 174: } 175:	
	污染路径:	

	函数'taosMemoryFree'的定义点.	
934	缺陷发生位置: /source/libs/index/src/indexFstFile.c	行号: 143
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 141: blk->nread = taosPReadFile(ctx->file.pFile, blk->buf, kBlockSize, blkId * kBlockSize); 142: if (blk->nread < kBlockSize && blk->nread < len) { 143: taosMemoryFree(blk); 144: break; 145: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
935	缺陷发生位置: /source/libs/scalar/src/scVector.c	行号: 1373
	步骤描述: 不使用返回值的函数'SCL_RET'应使用'void'说明.	
	代码片段: 1371: *converted = VECTOR_DO_CONVERT; 1372: *pOutputCol = output.columnData; 1373: SCL_RET(code); 1374: } 1375:	
	污染路径: 函数'SCL_RET'的定义点.	
936	缺陷发生位置: /tools/taos-tools/src/benchCsv.c	行号: 874
	步骤描述: 不使用返回值的函数'toolsFormatTimestamp'应使用'void'说明.	
	代码片段: 872: static int csvGenRowColData(char* buf, int size, SSuperTable* stb, int64_t ts, int32_t precision, int64_t *k) { 873: char ts_fmt[128] = {0}; 874: toolsFormatTimestamp(ts_fmt, ts, precision); 875: int pos = snprintf(buf, size, "\"'%s'\",", ts_fmt); 876:	
	污染路径: 函数'toolsFormatTimestamp'的定义点.	
937	缺陷发生位置:	行号:

	/source/dnode/mnode/impl/src/mndStreamMgmt.c	719
	步骤描述: 不使用返回值的函数'mstDebug'应使用'void'说明.	
	代码片段: <pre> 717: int32_t vgNum = taosArrayGetSize(pCtx->pReq- >pVgLeaders); 718: 719: mstDebug("start to update vgroups upTs"); 720: 721: for (int32_t i = 0; i < vgNum; ++i) { </pre>	
	污染路径: 函数'mstDebug'的定义点.	
938	缺陷发生位置: /source/libs/executor/src/projectoperator.c	行号: 89
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: <pre> 87: cleanupExprSupp(&pInfo->scalarSup); 88: 89: taosMemoryFreeClear(param); 90: } 91: </pre>	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
939	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqSend.c	行号: 45
	步骤描述: 不使用返回值的函数'util__increment_receive_quota'应使用'void'说明.	
	代码片段: <pre> 43: ttq_log(NULL, TTQ_LOG_DEBUG, "Sending PUBCOMP to %s (m%d)", SAFE_PRINT(ttq->id), mid); 44: 45: util__increment_receive_quota(ttq); 46: 47: return send__command_with_mid(ttq, CMD_PUBCOMP, mid, false, 0, properties); </pre>	
	污染路径: 函数'util__increment_receive_quota'的定义点.	
940	缺陷发生位置: /source/common/src/msg/tmsg.c	行号: 996

	步骤描述: 不使用返回值的函数'ENCODESQL'应使用'void'说明.	
	代码片段: 994: } 995: TAOS_CHECK_EXIT(tEncode164(&encoder, pReq->suid)); 996: ENCODESQL(); 997: tEndEncode(&encoder); 998:	
	污染路径: 函数'ENCODESQL'的定义点.	
941	缺陷发生位置: /source/libs/transport/src/transCli.c	行号: 651
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 649: static FORCE_INLINE void cliConnClearInitUserMsg(SCliConn* conn) { 650: if (conn->pInitUserReq) { 651: taosMemoryFree(conn->pInitUserReq); 652: conn->pInitUserReq = NULL; 653: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
942	缺陷发生位置: /tools/taos-tools/src/toolstime.c	行号: 907
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 905: sprintf(buf + pos, "%.09d", ms); 906: } else if (precision == TSDB_TIME_PRECISION_MICRO) { 907: sprintf(buf + pos, "%.06d", ms); 908: } else { 909: sprintf(buf + pos, "%.03d", ms);	
	污染路径: 无	
943	缺陷发生位置: /source/dnode/mnode/impl/src/mndRole.c	行号: 853
	步骤描述: 不使用返回值的函数'TAOS_RETURN'应使用'void'说明.	
	代码片段:	

	851: static int32_t mndProcessUpgradeRoleResp(SRpcMsg *pReq) { return 0; } 852: 853: static int32_t mndProcessGetRoleAuthReq(SRpcMsg *pReq) { TAOS_RETURN(0); } 854: 855: static int32_t mndRetrieveRoles(SRpcMsg *pReq, SShowObj *pShow, SSDataBlock *pBlock, int32_t rows) {	
	污染路径: 函数'TAOS_RETURN'的定义点.	
944	缺陷发生位置: /source/util/src/mpDirect.c	行号: 47
	步骤描述: 不使用返回值的函数'taosMemFree'应使用'void'说明.	
	代码片段: 45: (void)atomic_sub_fetch_64(&pJob->job.allocMemSize, taosMemSize(ptr)); 46: } 47: taosMemFree(ptr); 48: } 49:	
	污染路径: 函数'taosMemFree'的定义点.	
945	缺陷发生位置: /source/libs/transport/test/cliBench.c	行号: 75
	步骤描述: 不使用返回值的函数'tDebug'应使用'void'说明.	
	代码片段: 73: rpcMsg.info.noResp = 0; 74: rpcMsg.msgType = 1; 75: tDebug("thread:%d, send request, contLen:%d num:%d", plInfo->index, plInfo->msgSize, plInfo->num); 76: rpcSendRequest(plInfo->pRpc, &plInfo->epSet, &rpcMsg, NULL); 77: if (plInfo->num % 20000 == 0) tInfo("thread:%d, %d requests have been sent", plInfo->index, plInfo->num);	
	污染路径: 函数'tDebug'的定义点.	
946	缺陷发生位置: /source/common/src/tencrypt.c	行号: 339

	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 337: } 338: if (encryptedBuf != NULL) { 339: taosMemoryFree(encryptedBuf); 340: } 341: if (code != 0) {	
	污染路径: 函数'taosMemoryFree'的定义点.	
947	缺陷发生位置:	行号:
	/source/libs/executor/src/executor.c	110
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 108: _end: 109: if (code != TSDB_CODE_SUCCESS) { 110: qError("%s failed at line %d since %s", __func__, lino, tstrerror(code)); 111: } 112: return code;	
948	污染路径: 函数'qError'的定义点.	
	缺陷发生位置:	行号:
	/source/dnode/vnode/src/meta/metaQuery.c	125
	步骤描述: 不使用返回值的函数'metaError'应使用'void'说明.	
949	代码片段: 123: code = tdbTbcMoveToPrev(pCur); 124: if (code != 0) { 125: metaError("%s move to prev failed", __func__); 126: goto END; 127: }	
	污染路径: 函数'metaError'的定义点.	
	缺陷发生位置:	行号:
	/source/dnode/mnode/impl/src/mndRetention.c	989
	步骤描述: 不使用返回值的函数'mTrace'应使用'void'说明.	
	代码片段:	

	987: 988: static int32_t mndProcessTrimDbTimer(SRpcMsg *pReq) { 989: mTrace("start to process trim db timer"); 990: return mndTrimDbDispatch(pReq); 991: }	
	污染路径: 函数'mTrace'的定义点.	
950	缺陷发生位置: /source/libs/parser/src/parser.c	行号: 179
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 177: t = tStrGetToken((char*)pSql, &index, false, NULL); 178: if (t.n == 0 t.z == NULL) { 179: taosMemoryFree(newSql); 180: code = generateSyntaxErrMsgExt(&pMsgBuf, TSDB_CODE_PAR_SYNTAX_ERROR, "Invalid table name"); 181: return code;	
	污染路径: 函数'taosMemoryFree'的定义点.	
951	缺陷发生位置: /source/util/src/tlog.c	行号: 334
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 332: (void)taosThreadMutexDestroy(&tsLogObj.slowHandle->buffMutex); 333: (void)taosCloseFile(&tsLogObj.slowHandle->pFile); 334: taosMemoryFreeClear(tsLogObj.slowHandle->buffer); 335: taosMemoryFreeClear(tsLogObj.slowHandle); 336: }	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
952	缺陷发生位置: /source/util/src/tdecompressavx.c	行号: 191
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 189: return nelements * word_length; 190: #else	

	191: uError("unable run %s without avx512 instructions", __func__); 192: return -1; 193: #endif	
	污染路径: 函数'uError'的定义点.	
953	缺陷发生位置: /docs/examples/c/demo.c	行号: 87
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 85: int i = 0; 86: for (i = 0; i < 10; ++i) { 87: sprintf(qstr, "insert into m1 values (%" PRIu64 " ", %d, %d, %d, %d, %f, %lf, '%s')", (uint64_t)(1546300800000 + i * 1000), i, i, i, i*10000000, i*1.0, i*2.0, "hello"); 88: printf("qstr: %s\n", qstr); 89:	
	污染路径: 无	
954	缺陷发生位置: /source/util/src/tcache.c	行号: 235
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 233: } 234: 235: taosMemoryFree(pNode); 236: } 237:	
	污染路径: 函数'taosMemoryFree'的定义点.	
955	缺陷发生位置: /test/new_test_framework/script/api/passwdTest.c	行号: 250
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 248: // users 249: for (int i = 0; i < nUser; ++i) { 250: sprintf(users[i], "user%d", i);	

	251: printf(qstr, "CREATE USER %s PASS 'AAbb1122' PASSWORD_REUSE_MAX 0 PASSWORD_REUSE_TIME 0", users[i]); 252: queryDB(taos, qstr);	
	污染路径: 无	
956	缺陷发生位置: /source/libs/scheduler/src/scheduler.c	行号: 321
	步骤描述: 不使用返回值的函数'qInfo'应使用'void'说明.	
	代码片段: 319: taosHashCleanup(schMgmt.hbConnections); 320: schMgmt.hbConnections = NULL; 321: qInfo("hbConnections cleanup"); 322: } 323: SCH_UNLOCK(SCH_WRITE, &schMgmt.hbLock);	
	污染路径: 函数'qInfo'的定义点.	
957	缺陷发生位置: /source/util/src/ttimer.c	行号: 121
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 119: static void timerDecRef(tmr_obj_t* timer) { 120: if (atomic_sub_fetch_8(&timer->refCount, 1) == 0) { 121: taosMemoryFree(timer); 122: } 123: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
958	缺陷发生位置: /test/new_test_framework/script/api/stmt2-example.c	行号: 265
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 263: 264: taos_close(taos); 265: taos_cleanup(); 266: }	

	污染路径: 函数'taos_cleanup'的定义点.	
959	缺陷发生位置: /docs/examples/c/asyncdemo.c	行号: 127
	步骤描述: 不使用返回值的函数'gettimeofday'应使用'void'说明.	
	代码片段: <pre> 125: } 126: 127: gettimeofday(&systemTime, NULL); 128: for (i = 0; i < numOfTables; ++i) 129: tableList[i].timeStamp = (time_t)(systemTime.tv_sec) * 1000 + systemTime.tv_usec / 1000; </pre>	
	污染路径: 无	
960	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 253
	步骤描述: 不使用返回值的函数'DEBUGLOG'应使用'void'说明.	
	代码片段: <pre> 251: { 252: if (buf.start == NULL) return; /* compatible with release on NULL */ 253: DEBUGLOG(5, "ZSTDMT_releaseBuffer"); 254: ZSTD_pthread_mutex_lock(&bufPool->poolMutex); 255: if (bufPool->nbBuffers < bufPool->totalBuffers) { </pre>	
	污染路径: 函数'DEBUGLOG'的定义点.	
961	缺陷发生位置: /source/util/src/tref.c	行号: 76
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 74: lockedBy = taosMemoryCalloc(sizeof(int64_t), (size_t)max); 75: if (lockedBy == NULL) { 76: taosMemoryFree(nodeList); 77: return terrno; 78: } </pre>	
	污染路径:	

	函数'taosMemoryFree'的定义点.	
962	缺陷发生位置: /source/libs/new-stream/src/streamUtil.c	行号: 170
	步骤描述: 不使用返回值的函数'stsDebug'应使用'void'说明.	
	代码片段: 168: 169: ST_TASK_DLOG("task status added to hb %s mgmtReq", pTask->pMgmtReq ? "with" : "without"); 170: stsDebug("%d trigger tasks status added to hb", 1); 171: } 172:	
	污染路径: 函数'stsDebug'的定义点.	
963	缺陷发生位置: /source/libs/index/src/indexUtil.c	行号: 77
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 75: } 76: } 77: taosMemoryFreeClear(mi); 78: return code; 79: }	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
964	缺陷发生位置: /source/libs/executor/src/dataDeleter.c	行号: 341
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 339: if (deleter != NULL) { 340: (void)destroyDataSink((SDataSinkHandle*)deleter); 341: taosMemoryFree(deleter); 342: } else { 343: taosMemoryFree(pManager);	
	污染路径: 函数'taosMemoryFree'的定义点.	
965	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 2199

	步骤描述: 不使用返回值的函数'getSampleFileNameByPattern'应使用'void'说明.	
	代码片段: 2197: for (int64_t child = 0; child < stbInfo->childTblCount; child++) { 2198: char sampleFilePath[MAX_PATH_LEN] = {0}; 2199: getSampleFileNameByPattern(sampleFilePath, stbInfo, child); 2200: if (0 != access(sampleFilePath, F_OK)) { 2201: continue;	
	污染路径: 函数'getSampleFileNameByPattern'的定义点.	
966	缺陷发生位置: /source/dnode/mnode/impl/src/mndStreamMgmt.c	行号: 2424
	步骤描述: 不使用返回值的函数'mstsInfo'应使用'void'说明.	
	代码片段: 2422: mstsWarn("stream %s already dropped by user, ignore deploy it", pAction->streamName); 2423: atomic_store_8(&pStatus->stopped, 2); 2424: mstsInfo("set stream %s stopped by user since streamId mismatch", streamName); 2425: TAOS_CHECK_EXIT(TSDB_CODE_MND_STREAM_NOT_EXIST); 2426: }	
	污染路径: 函数'mstsInfo'的定义点.	
967	缺陷发生位置: /utils/test/c/tmq_offset_test.c	行号: 218
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 216: ASSERT(numOfAssign1 == 2); 217: ASSERT(numOfAssign1 == numOfAssign2); 218: ASSERT(numOfAssign1 == numOfAssign3); 219: 220: for(int i = 0; i < numOfAssign1; i++){	
	污染路径: 函数'ASSERT'的定义点.	
968	缺陷发生位置:	行号:

	/source/libs/executor/src/aggregateoperator.c	197
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 195: 196: if(!pAgglInfo) { 197: qError("function:%s, pAgglInfo is NULL", __func__); 198: return false; 199: }	
	污染路径: 函数'qError'的定义点.	
969	缺陷发生位置:	行号:
	/test/new_test_framework/script/api/stmtBatchTest.c	224
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 222: printf("insert total %d records, used %u seconds, avg: %u useconds per record\n", totalRows, (endtime-starttime)/1000000UL, (endtime-starttime)/totalRows); 223: 224: taosMemoryFree(v->ts); 225: taosMemoryFree(v->br); 226: taosMemoryFree(v->nr);	
970	缺陷发生位置:	行号:
	/utils/test/c/tmq_vtable.c	41
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 39: 40: char useDB[64] = {0}; 41: sprintf(useDB, "use %s", topic); 42: TAOS_RES* pRes = taos_query(pConn, useDB); 43: if (taos_errno(pRes) != 0) {	
971	缺陷发生位置:	行号:
	/source/libs/new-stream/src/streamHb.c	52
	污染路径: 无	

	步骤描述: 不使用返回值的函数'stTrace'应使用'void'说明.	
	代码片段: 50: TAOS_CHECK_EXIT(tmsgSendReq(pEpset, &msg)); 51: 52: stTrace("stream hb sent"); 53: 54: return code;	
	污染路径: 函数'stTrace'的定义点.	
972	缺陷发生位置: /source/os/src/osTimezone.c	行号: 980
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 978: timezone_t ntz = tzalloc(tzname); 979: if (!ntz) { 980: uError("tzalloc(%s) failed", tzname); 981: return TSDB_CODE_TIME_ERROR; 982: }	
	污染路径: 函数'uError'的定义点.	
973	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftLogDebug.c	行号: 151
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 149: char* serialized = logStore2Str(pLogStore); 150: sLTrace("logStoreLog2 len:%d %s %s", (int32_t)strlen(serialized), s, serialized); 151: taosMemoryFree(serialized); 152: } 153: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
974	缺陷发生位置: /source/util/src/tcurl.c	行号: 112
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段:	

	<pre> 110: tNotificationConnHash = taosHashInit(4, taosGetDefaultHashFunction(TSDB_DATA_TYPE_BINARY), false, HASH_NO_LOCK); 111: if (tNotificationConnHash == NULL) { 112: uError("[curl]failed to init NotificationConnHash"); 113: return terrno; 114: } </pre>	
	污染路径: 函数'uError'的定义点.	
975	缺陷发生位置: /source/libs/parser/src/parser.c	行号: 169
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 167: t = tStrGetToken((char*)pSql, &index, false, NULL); 168: if (TK_UPDATE != t.type) { 169: taosMemoryFree(newSql); 170: code = generateSyntaxErrMsgExt(&pMsgBuf, TSDB_CODE_PAR_SYNTAX_ERROR, "Expected UPDATE keyword"); 171: return code; </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
976	缺陷发生位置: /source/dnode/vnode/src/bse/bseCache.c	行号: 441
	步骤描述: 不使用返回值的函数'T_REF_INC'应使用'void'说明.	
	代码片段: <pre> 439: void bseCacheRefltem(SCacheItem *pItem) { 440: if (pItem == NULL) return; 441: T_REF_INC(pItem); 442: } 443: </pre>	
	污染路径: 函数'T_REF_INC'的定义点.	
977	缺陷发生位置: /docs/examples/c-ws-new/sml_insert_demo.c	行号: 27
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段:	

	25: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x %x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 26: taos_errstr(NULL)); 27: taos_cleanup(); 28: return -1; 29: }	
	污染路径: 函数'taos_cleanup'的定义点.	
978	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 2544
	步骤描述: 不使用返回值的函数'dumpExtraInfo'应使用'void'说明.	
	代码片段: 2542: char sqlstr[TSDB_DEFAULT_PKT_SIZE] = {0}; 2543: 2544: dumpExtraInfo(taos_v, fp); 2545: 2546: char *pstr = sqlstr;	
	污染路径: 函数'dumpExtraInfo'的定义点.	
979	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 619
	步骤描述: 不使用返回值的函数'symTransform_2bytes'应使用'void'说明.	
	代码片段: 617: while(1) 618: { 619: symTransform_2bytes(p); 620: i+=sizeof(unsigned short); 621: if(i<size)	
	污染路径: 函数'symTransform_2bytes'的定义点.	
980	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 1788
	步骤描述: 不使用返回值的函数'HUF_decodeStreamX2'应使用'void'说明.	
	代码片段: 1786: HUF_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 1787: HUF_decodeStreamX2(op3, &bitD3, opStart4, dt,	

	60: reader[0] = NULL; 61: return 0;	
	污染路径: 函数'taosMemoryFree'的定义点.	
984	缺陷发生位置: /utls/test/c/write_raw_block_test.c	行号: 47
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 45: int32_t numOfRows = 0; 46: int error_code = taos_fetch_raw_block(pRes, &numOfRows, &data); 47: ASSERT(error_code == 0); 48: 49: int numFields = taos_num_fields(pRes);	
	污染路径: 函数'ASSERT'的定义点.	
985	缺陷发生位置: /utls/test/c/varbinary_test.c	行号: 47
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 45: int code = taos_errno(pRes); 46: taos_free_result(pRes); 47: ASSERT(code == 0); 48: 49: pRes = taos_query(taos, "create stable stb (ts timestamp, c1 nchar(32), c2 varbinary(16), c3 float) tags (t1 int, t2 binary(8), t3 varbinary(8))");	
	污染路径: 函数'ASSERT'的定义点.	
986	缺陷发生位置: /source/libs/qcom/src/queryUtil.c	行号: 287
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 285: int32_t taosAsyncWait() { 286: if (!taskQueue.wrokerPool.pCb) { 287: qError("query task thread pool callback function is null");	

	288: return -1; 289: }	
	污染路径: 函数'qError'的定义点.	
987	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmMonitor.c	行号: 182
	步骤描述: 不使用返回值的函数'dlInfo'应使用'void'说明.	
	代码片段: 180: 181: void dmSetMnodeSyncTimeout() { 182: dlInfo("dmSetMnodeSyncTimeout"); 183: SDnode *pDnode = dmInstance(); 184: SMgmtWrapper *pWrapper = &pDnode->wrappers[MNODE];	
	污染路径: 函数'dlInfo'的定义点.	
988	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqSubs.c	行号: 206
	步骤描述: 不使用返回值的函数'ttq_free'应使用'void'说明.	
	代码片段: 204: shared->name = ttq_strdup(sharename); 205: if (shared->name == NULL) { 206: ttq_free(shared); 207: return TTQ_ERR_NOMEM; 208: }	
	污染路径: 函数'ttq_free'的定义点.	
989	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeQuery.c	行号: 164
	步骤描述: 不使用返回值的函数'SET_ERRNO'应使用'void'说明.	
	代码片段: 162: metaReaderDoInit(&mer1, pVnode->pMeta, META_READER_LOCK); 163: if (reqTbUid) { 164: SET_ERRNO(0); 165: uint64_t tbUid = taosStr2UInt64(infoReq.tbName, NULL, 10);	

	166: if (ERRNO == ERANGE tbUid == 0) {	
	污染路径: 函数'SET_ERRNO'的定义点.	
990	缺陷发生位置: /source/dnode/mnode/impl/src/mndQnode.c	行号: 269
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 267: 268: if ((code = mndTransAppendUndoAction(pTrans, &action)) != 0) { 269: taosMemoryFree(pReq); 270: TAOS_RETURN(code); 271: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
991	缺陷发生位置: /test/new_test_framework/script/api/stmt2-insert-dupkeys.c	行号: 209
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 207: 208: taos_close(taos); 209: taos_cleanup(); 210: } 211:	
	污染路径: 函数'taos_cleanup'的定义点.	
992	缺陷发生位置: /source/common/src/tatablock.c	行号: 3791
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 3789: pBlock->info.rows = maxRows; 3790: if (pBitmap != NULL) { 3791: taosMemoryFree(pBitmap); 3792: } 3793:	
	污染路径:	

	函数'taosMemoryFree'的定义点.	
993	缺陷发生位置: /test/new_test_framework/script/api/dbTableRoute.c	行号: 62
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 60: taos_free_result(pSql); 61: taos_close(taos); 62: taos_cleanup(); 63: exit(EXIT_FAILURE); 64: }	
	污染路径: 函数'taos_cleanup'的定义点.	
994	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smWorker.c	行号: 314
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: 312: tSingleWorkerCleanup(&pMgmt->runnerWorker); 313: tDispatchWorkerCleanup(&pMgmt->triggerWorkerPool); 314: dDebug("snode workers are closed"); 315: } 316:	
	污染路径: 函数'dDebug'的定义点.	
995	缺陷发生位置: /source/dnode/mnode/impl/src/mndEncryptAlgr.c	行号: 567
	步骤描述: 不使用返回值的函数'TAOS_RETURN'应使用'void'说明.	
	代码片段: 565: 566: mndTransDrop(pTrans); 567: TAOS_RETURN(0); 568: } 569:	
	污染路径: 函数'TAOS_RETURN'的定义点.	
996	缺陷发生位置: /source/libs/tss/src/tss.c	行号: 160
	步骤描述:	

	不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: <pre>158: } 159: 160: taosArrayDestroy(paths); 161: return TSDB_CODE_SUCCESS; 162: }</pre>	
	污染路径: 函数'taosArrayDestroy'的定义点.	
997	缺陷发生位置: /source/libs/catalog/src/ctgRemote.c	行号: 455
	步骤描述: 不使用返回值的函数'CTG_API_JENTER'应使用'void'说明.	
	代码片段: <pre>453: SCtgJob* pJob = NULL; 454: 455: CTG_API_JENTER(); 456: 457: pJob = taosAcquireRef(gCtgMgmt.jobPool, cbParam->refId);</pre>	
	污染路径: 函数'CTG_API_JENTER'的定义点.	
998	缺陷发生位置: /contrib/test/craft/raftMain.c	行号: 380
	步骤描述: 不使用返回值的函数'taosSeedRand'应使用'void'说明.	
	代码片段: <pre>378: 379: int main(int argc, char **argv) { 380: taosSeedRand(time(NULL)); 381: int32_t ret; 382:</pre>	
	污染路径: 函数'taosSeedRand'的定义点.	
999	缺陷发生位置: /source/common/src/msg/streamMsg.c	行号: 1737
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre>1735: }</pre>	

	1736: 1737: taosMemoryFree(*(void**)param); 1738: } 1739:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1000	缺陷发生位置: /examples/c/schemaless.c	行号: 58
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 56: for (int k = 0; k < numRowsPerChildTable; ++k) { 57: char* line = calloc(512, 1); 58: snprintf(line, 512, lineFormat, i, j, ts + 10 * l); 59: lines[l] = line; 60: ++l;	
	污染路径: 无	
1001	缺陷发生位置: /source/libs/monitorfw/src/taos_metric_formatter_custom.c	行号: 51
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 49: char** keyvalue = taosMemoryMalloc(sizeof(char*) * (count + 1)); 50: if (keyvalue == NULL) { 51: taosMemoryFreeClear(keyvalues); 52: return 1; 53: }	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1002	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeSnapshot.c	行号: 250
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 248: bseSnapReaderClose(&pReader->pBseReader); 249: } 250: taosMemoryFree(pReader); 251: }	

	252:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1003	缺陷发生位置: /source/dnode/vnode/src/tq/tqUtil.c	行号: 56
	步骤描述: 不使用返回值的函数'tqDebug'应使用'void'说明.	
	代码片段: 54: int32_t code = TDB_CODE_SUCCESS; 55: int32_t lino = 0; 56: tqDebug("%s called", __FUNCTION__); 57: TSDB_CHECK_NULL(pRsp, code, lino, END, TSDB_CODE_INVALID_PARA); 58: tOffsetCopy(&pRsp->reqOffset, &pOffset);	
	污染路径: 函数'tqDebug'的定义点.	
1004	缺陷发生位置: /source/libs/transport/test/cliBench.c	行号: 168
	步骤描述: 不使用返回值的函数'tlInfo'应使用'void'说明.	
	代码片段: 166: } 167: 168: tlInfo("client is initialized"); 169: tlInfo("threads:%d msgSize:%d requests:%d", appThreads, msgSize, numOfReqs); 170:	
	污染路径: 函数'tlInfo'的定义点.	
1005	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 1027
	步骤描述: 不使用返回值的函数'prepareColData'应使用'void'说明.	
	代码片段: 1025: for (int b = 0; b < gCurCase->tblNum; b++) { 1026: for (int c = 0; c < gCurCase->bindTagNum; ++c) { 1027: prepareColData(false, BP_BIND_TAG, data, b * gCurCase->bindTagNum + c, b, c); 1028: } 1029: }	

	污染路径: 函数'prepareColData'的定义点.	
1006	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbUtil.c	行号: 626
	步骤描述: 不使用返回值的函数'tsdbError'应使用'void'说明.	
	代码片段: 624: if (pColData) { 625: if (tColDataGetValue(pColData, pRow->iRow, pColVal) != 0) { 626: tsdbError("failed to tColDataGetValue"); 627: } 628: } else {	
	污染路径: 函数'tsdbError'的定义点.	
1007	缺陷发生位置: /source/libs/executor/src/executor.c	行号: 72
	步骤描述: 不使用返回值的函数'doDestroyExchangeOperatorInfo'应使用'void'说明.	
	代码片段: 70: 71: if (*(bool*)param) { 72: doDestroyExchangeOperatorInfo(param); 73: } else { 74: destroySubJobCtx((STaskSubJobCtx *)param);	
	污染路径: 函数'doDestroyExchangeOperatorInfo'的定义点.	
1008	缺陷发生位置: /source/dnode/mnode/impl/src/mndStb.c	行号: 1284
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 1282: 1283: if (pRsp == NULL) { 1284: mError("audit record rsp, null response message"); 1285: return -1; 1286: }	
	污染路径: 函数'mError'的定义点.	

1009	缺陷发生位置： /source/libs/sync/test/sync_test_lib/src/syncRaftEntryDebug.c	行号： 77
	步骤描述： 不使用返回值的函数'sTrace'应使用'void'说明.	
	代码片段： <pre> 75: void syncEntryLog(const SSyncRaftEntry* pObj) { 76: char* serialized = syncEntry2Str(pObj); 77: sTrace("syncEntryLog len:%zu %s", strlen(serialized), serialized); 78: taosMemoryFree(serialized); 79: }</pre>	
	污染路径： 函数'sTrace'的定义点.	
1010	缺陷发生位置： /source/os/src/osString.c	行号： 239
	步骤描述： 不使用返回值的函数'SET_ERRNO'应使用'void'说明.	
	代码片段： <pre> 237: } 238: char *endptr = NULL; 239: SET_ERRNO(0); 240: uint64_t ret = strtoull(str, &endptr, 10); 241:</pre>	
	污染路径： 函数'SET_ERRNO'的定义点.	
1011	缺陷发生位置： /source/common/src/msg/xnode.c	行号： 441
	步骤描述： 不使用返回值的函数'ENCODESQL'应使用'void'说明.	
	代码片段： <pre> 439: 440: TAOS_CHECK_EXIT(tStartEncode(&encoder)); 441: ENCODESQL(); 442: TAOS_CHECK_EXIT(tEncodeI32(&encoder, pReq- >xnodeId)); 443: TAOS_CHECK_EXIT(xEncodeCowStr(&encoder, &pReq- >name));</pre>	
	污染路径： 函数'ENCODESQL'的定义点.	
1012	缺陷发生位置：	行号：

	/source/libs/qworker/src/qwMsg.c	235
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 233: int32_t msgSize = tSerializeSQueryTableRsp(NULL, 0, &rsp); 234: if (msgSize < 0) { 235: qError("tSerializeSQueryTableRsp failed"); 236: QW_RET(msgSize); 237: }	
	污染路径: 函数'qError'的定义点.	
1013	缺陷发生位置: /source/dnode/vnode/src/meta/metaOpen.c	行号: 478
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 476: tDecoderClear(&dc); 477: } 478: tdbFree(value); 479: } 480:	
	污染路径: 函数'tdbFree'的定义点.	
1014	缺陷发生位置: /docs/examples/c-ws-new/connect_example.c	行号: 26
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 24: 25: taos_close(taos); 26: taos_cleanup(); 27: }	
	污染路径: 函数'taos_cleanup'的定义点.	
1015	缺陷发生位置: /source/dnode/mnode/impl/src/mndProfile.c	行号: 111
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段:	

	<pre> 109: if (pMgmt->appCache == NULL) { 110: code = TSDB_CODE_OUT_OF_MEMORY; 111: mError("failed to alloc profile cache since %s", terrstr()); 112: TAOS_RETURN(code); 113: }</pre>	
	污染路径: 函数'mError'的定义点.	
1016	缺陷发生位置: /source/libs/executor/src/sortoperator.c	行号: 559
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: <pre> 557: taosArrayDestroy(pInfo->matchInfo.pList); 558: destroySortOpGroupIdCalc(pInfo->pGroupIdCalc); 559: taosMemoryFreeClear(param); 560: } 561:</pre>	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1017	缺陷发生位置: /source/dnode/mnode/impl/src/mndInstance.c	行号: 147
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: <pre> 145: code = sdbWrite(pMnode->pSdb, pRaw); 146: if (code != TSDB_CODE_SUCCESS) { 147: mError("instance:%s, failed to remove from sdb since %s", id, tstrerror(code)); 148: } 149: return code;</pre>	
	污染路径: 函数'mError'的定义点.	
1018	缺陷发生位置: /source/dnode/mnode/impl/src/mndGrant.c	行号: 50
	步骤描述: 不使用返回值的函数'STR_WITH_MAXSIZE_TO_VARSTR'应使用'void'说明.	
	代码片段: <pre> 48: 49: GRANT_ITEM_SHOW("unlimited");</pre>	

	50: GRANT_ITEM_SHOW("limited"); 51: GRANT_ITEM_SHOW("false"); 52: GRANT_ITEM_SHOW("ungranted");	
	污染路径: expanded from macro 'GRANT_ITEM_SHOW' 函数'STR_WITH_MAXSIZE_TO_VARSTR'的定义点.	
1019	缺陷发生位置: /source/libs/sync/src/syncMain.c	行号: 454
	步骤描述: 不使用返回值的函数'sError'应使用'void'说明.	
	代码片段: 452: code = TSDB_CODE_SYN_RETURN_VALUE_NULL; 453: if (terrno != 0) code = terrno; 454: sError("sync begin snapshot error"); 455: TAOS_RETURN(code); 456: }	
	污染路径: 函数'sError'的定义点.	
1020	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmEnv.c	行号: 182
	步骤描述: 不使用返回值的函数'dlInfo'应使用'void'说明.	
	代码片段: 180: 181: int32_t dmlnit() { 182: dlInfo("start to init dnode env"); 183: int32_t code = 0; 184:	
	污染路径: 函数'dlInfo'的定义点.	
1021	缺陷发生位置: /utls/test/c/tmq_offset_test.c	行号: 138
	步骤描述: 不使用返回值的函数'taosSleep'应使用'void'说明.	
	代码片段: 136: break; 137: } 138: taosSleep(3); 139: } 140: tmq_topic_assignment* pAssign = NULL;	

	污染路径: 函数'taosSsleep'的定义点.	
1022	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncBatch.c	行号: 143
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 141: s = syncUtilPrintBin((char*)(pMsg->data), pMsg->dataLen); 142: cJSON_AddStringToObject(pRoot, "data", s); 143: taosMemoryFree(s); 144: s = syncUtilPrintBin2((char*)(pMsg->data), pMsg->dataLen); 145: cJSON_AddStringToObject(pRoot, "data2", s);	
	污染路径: 函数'taosMemoryFree'的定义点.	
1023	缺陷发生位置: /source/client/src/clientMsgHandler.c	行号: 56
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 54: } else { 55: if (tsem_post(&pRequest->body.rspSem) != 0) { 56: tscError("failed to post semaphore"); 57: } 58: }	
	污染路径: 函数'tscError'的定义点.	
1024	缺陷发生位置: /source/dnode/mnode/impl/src/mndScan.c	行号: 1019
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 1017: 1018: if ((code = mndTransAppendRedoAction(pTrans, &action)) != 0) { 1019: taosMemoryFree(pReq); 1020: TAOS_RETURN(code); 1021: }	

	污染路径: 函数'taosMemoryFree'的定义点.	
1025	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 1005
	步骤描述: 不使用返回值的函数'BITv05_reloadDStream'应使用'void'说明.	
	代码片段: <pre> 1003: const FSEv05_DTableHeader* const DTableH = (const FSEv05_DTableHeader*)ptr; 1004: DStatePtr->state = BITv05_readBits(bitD, DTableH- >tableLog); 1005: BITv05_reloadDStream(bitD); 1006: DStatePtr->table = dt + 1; 1007: }</pre>	
	污染路径: 函数'BITv05_reloadDStream'的定义点.	
1026	缺陷发生位置: /source/libs/transport/src/thttp.c	行号: 954
	步骤描述: 不使用返回值的函数'atomic_store_8'应使用'void'说明.	
	代码片段: <pre> 952: } 953: 954: atomic_store_8(&load->quit, 1); 955: ret = httpSendQuit(load, channelId); 956: if (ret != 0) {</pre>	
	污染路径: 函数'atomic_store_8'的定义点.	
1027	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 4455
	步骤描述: 不使用返回值的函数'tsdbDebug'应使用'void'说明.	
	代码片段: <pre> 4453: *pReturnType = TSDB_READ_RETURN; 4454: 4455: tsdbDebug("seq load data blocks from stt files %s", pReader->idStr); 4456: 4457: while (1) {</pre>	
	污染路径:	

	函数'tsdbDebug'的定义点.	
1028	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmlnt.c	行号: 162
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: 160: static int32_t mmStart(SMnodeMgmt *pMgmt) { 161: int32_t code = 0; 162: dDebug("mnode-mgmt start to run"); 163: SMnodeOpt option = {0}; 164: if ((code = mmReadFile(pMgmt->path, &option)) != 0) {	
	污染路径: 函数'dDebug'的定义点.	
1029	缺陷发生位置: /source/libs/function/src/builtinsimpl.c	行号: 257
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 255: plter->pPrevPk = taosMemoryMalloc(plter->pPkCol->info.bytes); 256: if (NULL == plter->pPrevPk) { 257: qError("out of memory when function get input row."); 258: taosMemoryFree(plter->pPrevData); 259: return terrno;	
	污染路径: 函数'qError'的定义点.	
1030	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmNodes.c	行号: 123
	步骤描述: 不使用返回值的函数'dmReportStartup'应使用'void'说明.	
	代码片段: 121: 122: dInfo("The daemon initialized successfully"); 123: dmReportStartup("The daemon", "initialized successfully"); 124: return 0; 125: }	
	污染路径:	

	函数'dmReportStartup'的定义点.	
1031	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nchar.c	行号: 41
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 39: TAOS_RES* result = taos_query(taos, "drop database if exists test;"); 40: taos_free_result(result); 41: usleep(100000); 42: result = taos_query(taos, "create database test;"); 43:	
	污染路径: 无	
1032	缺陷发生位置: /source/libs/executor/src/eventwindowoperator.c	行号: 207
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 205: cleanupAggSup(&pInfo->aggSup); 206: cleanupExprSup(&pInfo->scalarSup); 207: taosMemoryFreeClear(param); 208: } 209:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1033	缺陷发生位置: /source/libs/new-stream/src/streamMgmt.c	行号: 310
	步骤描述: 不使用返回值的函数'streamReleaseTask'应使用'void'说明.	
	代码片段: 308: } 309: 310: streamReleaseTask(taskAddr); 311: 312: return code;	
	污染路径: 函数'streamReleaseTask'的定义点.	
1034	缺陷发生位置: /contrib/TSZ/sz/src/dataCompression.c	行号: 528

	步骤描述: 不使用返回值的函数'computeReqLength_double'应使用'void'说明.	
	代码片段: 526: 527: int reqLength; 528: computeReqLength_double(precision, radExpo, &reqLength, medianValue); 529: 530: *reqBytesLength = reqLength/8;	
	污染路径: 函数'computeReqLength_double'的定义点.	
1035	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 1087
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 1085: taosMemoryFree(keys_list); 1086: taosMemoryFree(keys_list_sizes); 1087: taosMemoryFree(values_list); 1088: taosMemoryFree(values_list_sizes); 1089:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1036	缺陷发生位置: /contrib/test/traft/join_into_vgroup/node_follower10000.c	行号: 108
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 106: // for test: submit value every second 107: while (1) { 108: sleep(1); 109: submitValue(); 110: }	
	污染路径: 无	
1037	缺陷发生位置: /source/libs/function/src/builtins.c	行号: 671
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段:	

	669: 670: cJSON_Delete(binDesc); 671: taosMemoryFree(intervals); 672: return TSDB_CODE_SUCCESS; 673: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1038	缺陷发生位置: /test/new_test_framework/script/api/api_with_reqid_test.c	行号: 191
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 189: result = taos_query_with_reqid(taos, "drop database if exists test;", genReqid()); 190: taos_free_result(result); 191: usleep(100000); 192: result = taos_query_with_reqid(taos, "create database test precision 'us' update 1;", genReqid()); 193: taos_free_result(result);	
	污染路径: 无	
1039	缺陷发生位置: /source/client/test/clientTests.cpp	行号: 1943
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 1941: 1942: char* tempUser = "control_user"; 1943: sprintf(buf, "create user if not exists %s pass 'AAbb1122'", tempUser); 1944: pRes = taos_query(pRootConn, buf); 1945: ASSERT_EQ(taos_errno(pRes), TSDB_CODE_SUCCESS);	
	污染路径: 无	
1040	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmEnv.c	行号: 65
	步骤描述: 不使用返回值的函数'taosResolveCRC'应使用'void'说明.	
	代码片段: 63: }	

	64: 65: taosResolveCRC(); 66: return 0; 67: }	
	污染路径: 函数'taosResolveCRC'的定义点.	
1041	缺陷发生位置: /source/libs/executor/src/sortoperator.c	行号: 762
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 760: if (ps == NULL param == NULL) { 761: taosMemoryFree(ps); 762: taosMemoryFree(param); 763: return termno; 764: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1042	缺陷发生位置: /utils/test/c/tmq_poll_test.c	行号: 107
	步骤描述: 不使用返回值的函数'taosSsleep'应使用'void'说明.	
	代码片段: 105: taos_free_result(tmqmessage); 106: 107: taosSsleep(15); 108: 109: tmqmessage = tmq_consumer_poll(tmq, 500);	
	污染路径: 函数'taosSsleep'的定义点.	
1043	缺陷发生位置: /utils/test/c/tmq_token.c	行号: 136
	步骤描述: 不使用返回值的函数'taosMsleep'应使用'void'说明.	
	代码片段: 134: } 135: taos_free_result(pRes); 136: taosMsleep(200); 137: } 138: tmq_consumer_close(tmq);	

	污染路径: 函数'taosMsleep'的定义点.	
1044	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/tst-argp1.c	行号: 104
	步骤描述: 不使用返回值的函数'argp_parse'应使用'void'说明.	
	代码片段: 102: 103: /* Parse and process arguments. */ 104: argp_parse (&argp, argc, argv, 0, &remaining, NULL); 105: 106: return 0;	
	污染路径: 无	
1045	缺陷发生位置: /source/libs/executor/src/timesliceoperator.c	行号: 1596
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 1594: } 1595: nodesDestroyNode(plInfo->pWin); 1596: taosMemoryFreeClear(param); 1597: } 1598:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1046	缺陷发生位置: /source/dnode/mnode/impl/src/mndStb.c	行号: 1324
	步骤描述: 不使用返回值的函数'mDebug'应使用'void'说明.	
	代码片段: 1322: int32_t contLen = reqLen + sizeof(SMsgHead); 1323: 1324: mDebug("start to process ttl timer"); 1325: 1326: while (1) {	
	污染路径: 函数'mDebug'的定义点.	

1047	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbMigrate.c	行号: 258
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 256: snprintf(path, sizeof(path), "vnode%d/f%d/manifests.json", vnode, fset->fid); 257: code = tssUploadToDefault(path, buf, strlen(buf)); 258: taosMemoryFree(buf); 259: if (code != TSDB_CODE_SUCCESS) { 260: tsdbError("vgId:%d, fid:%d, failed to upload manifest since %s", vnode, fset->fid, tstrerror(code));	
	污染路径: 函数'taosMemoryFree'的定义点.	
1048	缺陷发生位置: /source/libs/qworker/src/qwUtil.c	行号: 147
	步骤描述: 不使用返回值的函数'QW_TASK_ELOG_E'应使用'void'说明.	
	代码片段: 145: if (NULL == (*task)) { 146: QW_UNLOCK(rwType, &sch->tasksLock); 147: QW_TASK_ELOG_E("task status not exists"); 148: QW_ERR_RET(TSDB_CODE_QRY_TASK_NOT_EXIST); 149: }	
	污染路径: 函数'QW_TASK_ELOG_E'的定义点.	
1049	缺陷发生位置: /source/util/src/tmempool.c	行号: 435
	步骤描述: 不使用返回值的函数'uInfo'应使用'void'说明.	
	代码片段: 433: uInfo("Max Used Memory Size: %" PRId64, maxAllocSize); 434: 435: uInfo("[times]:"); 436: switch (gMPMgmt.strategy) { 437: case E_MP_STRATEGY_DIRECT:	
	污染路径: 函数'uInfo'的定义点.	
1050	缺陷发生位置:	行号:

	/test/new_test_framework/script/api/apitest.c	27
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 25: result = taos_query(taos, "create database test precision 'us';"); 26: taos_free_result(result); 27: usleep(100000); 28: taos_select_db(taos, "test"); 29:	
	污染路径: 无	
1051	缺陷发生位置: /source/libs/command/src/explain.c	行号: 281
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 279: if (NULL == pNode) { 280: *pResNode = NULL; 281: qError("physical node is NULL"); 282: return TSDB_CODE_APP_ERROR; 283: }	
	污染路径: 函数'qError'的定义点.	
1052	缺陷发生位置: /source/client/src/clientImpl.c	行号: 148
	步骤描述: 不使用返回值的函数'tscInfo'应使用'void'说明.	
	代码片段: 146: taosHashCleanup(appInfo.pInstMap); 147: taosHashCleanup(appInfo.pInstMapByClusterId); 148: tscInfo("cluster instance map cleaned"); 149: } 150:	
	污染路径: 函数'tscInfo'的定义点.	
1053	缺陷发生位置: /source/libs/tdb/src/db/tdbTable.c	行号: 287
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	

	代码片段: 285: } 286: tdbFree(pKey); 287: tdbFree(pValue); 288: tdbTbcClose(pCur); 289:	
	污染路径: 函数'tdbFree'的定义点.	
1054	缺陷发生位置: /source/libs/tmqtt/mgmt/src/tmqttMgmt.c	行号: 128
	步骤描述: 不使用返回值的函数'bndError'应使用'void'说明.	
	代码片段: 126: bndInfo("[MQTTD]Success to set TAOS_FQDN:%s", taosFqdn); 127: } else { 128: bndError("[MQTTD]Failed to allocate memory for TAOS_FQDN"); 129: return terrno; 130: }	
	污染路径: 函数'bndError'的定义点.	
1055	缺陷发生位置: /source/util/src/tdecompressavx.c	行号: 339
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 337: return (int32_t)(out - output); 338: #else 339: uError("unable run %s without avx2 instructions", __func__); 340: return -1; 341: #endif	
	污染路径: 函数'uError'的定义点.	
1056	缺陷发生位置: /test/new_test_framework/script/api/asyncQuery.c	行号: 55
	步骤描述: 不使用返回值的函数'atomic_add_fetch_64'应使用'void'说明.	
	代码片段:	

	53: if (nrows < 0) { 54: taos_free_result(res); 55: atomic_add_fetch_64(&finQueries, 1); 56: atomic_add_fetch_64(&errTimes, 1); 57: } else if (nrows == 0) {	
	污染路径: 函数'atomic_add_fetch_64'的定义点.	
1057	缺陷发生位置: /contrib/test/rocksdb/main.c	行号: 126
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 124: size_t vallen = 0; 125: char *val = rocksdb_get(db, readoptions, "key", 3, &vallen, &err); 126: snprintf(buf, vallen + 5, "val:%s", val); 127: printf("%ld %ld %s\n", strlen(val), vallen, buf); 128:	
	污染路径: 无	
1058	缺陷发生位置: /contrib/test/craft/simulate_vnode.c	行号: 202
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 200: 201: char cmd[128]; 202: snprintf(cmd, sizeof(cmd), "rm -rf ./%s", dir); 203: system(cmd); 204: snprintf(cmd, sizeof(cmd), "mkdir -p ./%s", dir);	
	污染路径: 无	
1059	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSnapshotRAW.c	行号: 481
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 479: tsdbFSDestroyCopySnapshot(&writer[0]->fsetArr); 480: 481: taosMemoryFree(writer[0]);	

	482: writer[0] = NULL; 483:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1060	缺陷发生位置: /source/client/src/clientStmt2.c	行号: 2185
	步骤描述: 不使用返回值的函数'STMT2_ELOG_E'应使用'void'说明.	
	代码片段: 2183: } else { 2184: if (pStmt->sql.stbInterlaceMode) { 2185: STMT2_ELOG_E("bind single column not allowed in stb insert mode"); 2186: STMT_ERR_RET(TSDB_CODE_TSC_STMT_API_ERROR); 2187: } 	
	污染路径: 函数'STMT2_ELOG_E'的定义点.	
1061	缺陷发生位置: /docs/examples/c-ws-new/query_data_demo.c	行号: 72
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 70: // close & clean 71: taos_close(taos); 72: taos_cleanup(); 73: return 0; 74: // ANCHOR_END: query_data	
	污染路径: 函数'taos_cleanup'的定义点.	
1062	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqSend.c	行号: 37
	步骤描述: 不使用返回值的函数'util__increment_receive_quota'应使用'void'说明.	
	代码片段: 35: ttq_log(NULL, TTQ_LOG_DEBUG, "Sending PUBACK to %s (m%d, rc%d)", SAFE_PRINT(ttq->id), mid, reason_code); 36: 37: util__increment_receive_quota(ttq); 38:	

	39: return send__command_with_mid(ttq, CMD_PUBACK, mid, false, reason_code, properties);	
	污染路径: 函数'util__increment_receive_quota'的定义点.	
1063	缺陷发生位置: /source/libs/tss/src/tss.c	行号: 147
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 145: int32_t code = tssListFile(ss, prefix, paths); 146: if (code != TSDB_CODE_SUCCESS) { 147: taosArrayDestroy(paths); 148: return code; 149: }	
	污染路径: 函数'taosArrayDestroy'的定义点.	
1064	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmTransport.c	行号: 639
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: 637: rpcClose(pTrans->clientRpc); 638: pTrans->clientRpc = NULL; 639: dDebug("dnode rpc client is closed"); 640: } 641: }	
	污染路径: 函数'dDebug'的定义点.	
1065	缺陷发生位置: /source/dnode/mnode/impl/src/mndBnode.c	行号: 193
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 191: 192: if ((code = mndTransAppendRedoAction(pTrans, &action)) != 0) { 193: taosMemoryFree(pReq); 194: TAOS_RETURN(code); 195: }	

	污染路径: 函数'taosMemoryFree'的定义点.	
1066	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 2210
	步骤描述: 不使用返回值的函数'HUFv06_decodeStreamX2'应使用'void'说明.	
	代码片段: <pre> 2208: /* finish bitStreams one by one */ 2209: HUFv06_decodeStreamX2(op1, &bitD1, opStart2, dt, dtLog); 2210: HUFv06_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 2211: HUFv06_decodeStreamX2(op3, &bitD3, opStart4, dt, dtLog); 2212: HUFv06_decodeStreamX2(op4, &bitD4, oend, dt, dtLog); </pre>	
	污染路径: 函数'HUFv06_decodeStreamX2'的定义点.	
1067	缺陷发生位置: /test/new_test_framework/script/api/whiteListTest.c	行号: 147
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 145: for (int i = 0; i < nUser; ++i) { 146: sprintf(users[i], "user%d", i); 147: sprintf(qstr, "DROP USER %s", users[i]); 148: queryDB(taos, qstr); 149: taos_close(taosu[i]); </pre>	
	污染路径: 无	
1068	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqSendPublish.c	行号: 179
	步骤描述: 不使用返回值的函数'packet__write_uint16'应使用'void'说明.	
	代码片段: <pre> 177: packet__write_string(packet, topic, (uint16_t)strlen(topic)); 178: } else { 179: packet__write_uint16(packet, 0); 180: } 181: if (qos > 0) { </pre>	

	污染路径: 函数'packet__write_uint16'的定义点.	
1069	缺陷发生位置: /source/libs/tss/src/tss.c	行号: 95
	步骤描述: 不使用返回值的函数'tssError'应使用'void'说明.	
	代码片段: 93: } 94: 95: tssError("shared storage type not found, access string is: %s\n", as); 96: return TSDB_CODE_NOT_FOUND; 97: }	
	污染路径: 函数'tssError'的定义点.	
1070	缺陷发生位置: /source/os/src/osFile.c	行号: 238
	步骤描述: 不使用返回值的函数'OS_PARAM_CHECK'应使用'void'说明.	
	代码片段: 236: int32_t taosRenameFile(const char *oldName, const char *newName) { 237: OS_PARAM_CHECK(oldName); 238: OS_PARAM_CHECK(newName); 239: #ifdef WINDOWS 240: bool finished = false;	
	污染路径: 函数'OS_PARAM_CHECK'的定义点.	
1071	缺陷发生位置: /source/util/src/tconfig.c	行号: 347
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 345: static int32_t cfgSetCharset(SConfigItem *pltem, const char *value, ECfgSrcType stype) { 346: if (stype == CFG_STYPE_ALTER_SERVER_CMD stype == CFG_STYPE_ALTER_CLIENT_CMD) { 347: uError("failed to config charset, not support"); 348: TAOS_RETURN(TSDB_CODE_INVALID_CFG); 349: }	

	污染路径: 函数'uError'的定义点.	
1072	缺陷发生位置: /test/new_test_framework/script/api/stmt2-blob-performance.c	行号: 122
	步骤描述: 不使用返回值的函数'clock_gettime'应使用'void'说明.	
	代码片段: 120: // bind 121: struct timespec start, end; 122: clock_gettime(CLOCK_MONOTONIC, &start); // 获取开始时间 123: TAOS_STMT2_BINDV bindv; 124: if (hasTag) {	
	污染路径: 无	
1073	缺陷发生位置: /source/libs/tss/test/tssTest.cpp	行号: 297
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 295: for(int i = 0; i < 10; i++) { 296: char path[64]; 297: snprintf(path, sizeof(path), "test/test%d.txt", i); 298: code = tssUploadFileToDefault(path, "/tmp/ss_test.txt", 0, 0); 299: GTEST_ASSERT_EQ(code, TSDB_CODE_SUCCESS);	
	污染路径: 无	
1074	缺陷发生位置: /docs/examples/c/asyncdemo.c	行号: 112
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 110: queryDB(taos, sql); 111: 112: sprintf(sql, "create database %s", db); 113: queryDB(taos, sql); 114:	
	污染路径:	

	无	
1075	缺陷发生位置: /source/dnode/mnode/impl/src/mndGrant.c	行号: 51
	步骤描述: 不使用返回值的函数'STR_WITH_MAXSIZE_TO_VARSTR'应使用'void'说明.	
	代码片段: 49: GRANT_ITEM_SHOW("unlimited"); 50: GRANT_ITEM_SHOW("limited"); 51: GRANT_ITEM_SHOW("false"); 52: GRANT_ITEM_SHOW("ungranted"); 53:	
	污染路径: expanded from macro 'GRANT_ITEM_SHOW' 函数'STR_WITH_MAXSIZE_TO_VARSTR'的定义点.	
1076	缺陷发生位置: /tools/taos-tools/src/benchQuery.c	行号: 135
	步骤描述: 不使用返回值的函数'prctl'应使用'void'说明.	
	代码片段: 133: qThreadInfo *pThreadInfo = (qThreadInfo*)sarg; 134: #ifdef LINUX 135: prctl(PR_SET_NAME, "specQueryMixThread"); 136: #endif 137: // use db	
	污染路径: 无	
1077	缺陷发生位置: /source/libs/new-stream/src/streamMgmt.c	行号: 224
	步骤描述: 不使用返回值的函数'streamReleaseTask'应使用'void'说明.	
	代码片段: 222: ST_TASK_ELOG("trigger task fail to deploy, error:%s", tstrerror(code)); 223: TAOS_UNUSED(taosHashRemove(gStreamMgmt.taskMap, &pSrc->streamId, sizeof(pSrc->streamId) + sizeof(pSrc->taskId)); 224: streamReleaseTask(taskAddr); 225: taosMemoryFree(tdListPopHead(pStream->triggerList)); 226: TAOS_CHECK_EXIT(code);	

	污染路径: 函数'streamReleaseTask'的定义点.	
1078	缺陷发生位置: /utils/test/c/tmq_blob.c	行号: 57
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 55: int32_t ret = tmq_write_raw(pConn, raw); 56: printf("write raw data: %s\n", tmq_err2str(ret)); 57: ASSERT(ret == 0); 58: 59: tmq_free_raw(raw);	
	污染路径: 函数'ASSERT'的定义点.	
1079	缺陷发生位置: /contrib/TSZ/sz/src/dataCompression.c	行号: 524
	步骤描述: 不使用返回值的函数'computeRangeSize_double'应使用'void'说明.	
	代码片段: 522: double valueRangeSize; 523: 524: computeRangeSize_double(oriData, nbEle, &valueRangeSize, medianValue); 525: short radExpo = getExponent_double(valueRangeSize/2); 526:	
	污染路径: 函数'computeRangeSize_double'的定义点.	
1080	缺陷发生位置: /docs/examples/c/demo.c	行号: 73
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 71: } 72: taos_close(taos); 73: taos_cleanup(); 74: } 75: void Test(TAOS *taos, char *qstr, int index) {	
	污染路径:	

	函数'taos_cleanup'的定义点.	
1081	缺陷发生位置: /docs/examples/c/asyncdemo.c	行号: 72
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 70: taos_free_result(pSql); 71: taos_close(taos); 72: taos_cleanup(); 73: exit(EXIT_FAILURE); 74: }	
	污染路径: 函数'taos_cleanup'的定义点.	
1082	缺陷发生位置: /source/util/src/tcompression.c	行号: 224
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 222: return compressedSize - 1; 223: } else if (input[1] == 2) { 224: uError("Invalid decompress string indicator:%d", input[0]); 225: return TSDB_CODE_THIRDPARTY_ERROR; 226: }	
	污染路径: 函数'uError'的定义点.	
1083	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttContext.c	行号: 130
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 128: context->password = NULL; 129: 130: UNUSED(net__socket_close(context)); 131: if (force_free) { 132: UNUSED(ttqSubCleanSession(context));	
	污染路径: 函数'UNUSED'的定义点.	
1084	缺陷发生位置: /source/dnode/mnode/impl/src/mndEncryptAlgr.c	行号: 651

	步骤描述: 不使用返回值的函数'tstrncpy'应使用'void'说明.	
	代码片段: <pre> 649: cols = 0; 650: SColumnInfoData *pCollInfo = taosArrayGet(pBlock->pDataBlock, cols++); 651: tstrncpy(varDataVal(tmpBuf), "config", 32); 652: varDataSetLen(tmpBuf, strlen(varDataVal(tmpBuf))); 653: TAOS_CHECK_GOTO(colDataSetVal(pCollInfo, numOfRows, (const char *)tmpBuf, false), &lino, _OVER); </pre>	
	污染路径: 函数'tstrncpy'的定义点.	
1085	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftLogDebug.c	行号: 143
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 141: char* serialized = logStore2Str(pLogStore); 142: sLTrace("logStoreLog len:%d %s", (int32_t)strlen(serialized), serialized); 143: taosMemoryFree(serialized); 144: } 145: } </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
1086	缺陷发生位置: /source/util/test/utilTests.cpp	行号: 501
	步骤描述: 不使用返回值的函数'checkNull'应使用'void'说明.	
	代码片段: <pre> 499: checkNull(1, false); 500: checkNull(2, false); 501: checkNull(totalRows - 2, false); 502: checkNull(totalRows - 1, false); 503: </pre>	
	污染路径: 函数'checkNull'的定义点.	
1087	缺陷发生位置: /source/libs/scheduler/src/scheduler.c	行号: 49
	步骤描述:	

	不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: <pre> 47: schMgmt.hbConnections = taosHashInit(100, taosGetDefaultHashFunction(TSDB_DATA_TYPE_BINARY), false, HASH_ENTRY_LOCK); 48: if (NULL == schMgmt.hbConnections) { 49: qError("taosHashInit hb connections failed"); 50: SCH_ERR_RET(terrno); 51: }</pre>	
	污染路径: 函数'qError'的定义点.	
1088	缺陷发生位置:	行号:
	/source/libs/catalog/src/ctgUtil.c	2430
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 2428: _return: 2429: taosMemoryFree(pMeta); 2430: taosMemoryFree(stbName); 2431: 2432: CTG_RET(code);</pre>	
1089	污染路径: 函数'taosMemoryFree'的定义点.	
	缺陷发生位置:	行号:
	/source/dnode/mgmt/node_util/src/dmFile.c	337
	步骤描述: 不使用返回值的函数'encryptDebug'应使用'void'说明.	
1090	代码片段: <pre> 335: TAOS_CHECK_GOTO(taosRenameFile(file, realfile), NULL, _OVER); 336: 337: encryptDebug("succeed to write checkCode file:%s", realfile); 338: 339: code = 0;</pre>	
	污染路径: 函数'encryptDebug'的定义点.	
	缺陷发生位置:	行号:
1090	/source/util/test/decompressTest.cpp	539
	步骤描述:	

	不使用返回值的函数'origData'应使用'void'说明.	
	代码片段: 537: ONE_STAGE_COMP, nullptr, 0); 538: ASSERT_EQ(cnt, compData.size() - 1); 539: EXPECT_EQ(origData, decompData); 540: } 541: }	
	污染路径: 无	
1091	缺陷发生位置: /tools/taos-tools/src/toolstime.c	行号: 827
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 825: res = localtime(timep); 826: if (res == NULL && buf != NULL) { 827: sprintf(buf, "NaN"); 828: } 829: return res;	
	污染路径: 无	
1092	缺陷发生位置: /source/util/src/tlockfree.c	行号: 64
	步骤描述: 不使用返回值的函数'taosMsleep'应使用'void'说明.	
	代码片段: 62: oLatch = atomic_load_32(pLatch); 63: if (oLatch & TD_RWLATCH_WRITE_FLAG) { 64: taosMsleep(1); 65: continue; 66: }	
	污染路径: 函数'taosMsleep'的定义点.	
1093	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 1969
	步骤描述: 不使用返回值的函数'HUFv05_decodeStreamX2'应使用'void'说明.	
	代码片段: 1967: if (HUFv05_isError(errorCode)) return errorCode; } 1968:	

	1969: HUFv05_decodeStreamX2(op, &bitD, oend, dt, dtLog); 1970: 1971: /* check */	
	污染路径: 函数'HUFv05_decodeStreamX2'的定义点.	
1094	缺陷发生位置: /contrib/test/traft/single_node/singleNode.c	行号: 106
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 104: // for test: submit a value every second 105: while (1) { 106: sleep(1); 107: submitValue(); 108: }	
	污染路径: 无	
1095	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqNet2.c	行号: 98
	步骤描述: 不使用返回值的函数'G_SOCKET_CONNECTIONS_INC'应使用'void'说明.	
	代码片段: 96: } 97: 98: G_SOCKET_CONNECTIONS_INC(); 99: 100: if (net__socket_nonblock(&new_sock)) {	
	污染路径: 函数'G_SOCKET_CONNECTIONS_INC'的定义点.	
1096	缺陷发生位置: /source/libs/tdb/test/tdbPageDefragmentTest.cpp	行号: 377
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 375: 376: for (int iData = 1; iData <= nData; iData++) { 377: sprintf(key, "key0"); 378: sprintf(val, "value%d", iData); 379:	

	污染路径: 无	
1097	缺陷发生位置: /source/dnode/mnode/impl/src/mndMain.c	行号: 132
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 130: // TODO check return value 131: if (tmsgPutToQueue(&pMnode->msgCb, WRITE_QUEUE, &rpcMsg) < 0) { 132: mError("failed to put into write-queue since %s, line: %d", terrstr(), __LINE__); 133: } 134: }	
	污染路径: 函数'mError'的定义点.	
1098	缺陷发生位置: /source/libs/executor/src/operator.c	行号: 955
	步骤描述: 不使用返回值的函数'recordOpExecBeforeDownstream'应使用'void'说明.	
	代码片段: 953: 954: SSDataBlock* getNextBlockFromDownstream(struct SOperatorInfo* pOperator, int32_t idx) { 955: recordOpExecBeforeDownstream(pOperator); 956: SSDataBlock* p = NULL; 957: int32_t code = getNextBlockFromDownstreamImpl(pOperator, idx, true, &p);	
	污染路径: 函数'recordOpExecBeforeDownstream'的定义点.	
1099	缺陷发生位置: /source/libs/scalar/test/filter/filterTests.cpp	行号: 1489
	步骤描述: 不使用返回值的函数'fp'应使用'void'说明.	
	代码片段: 1487: switch (rType) { 1488: case TSDB_DATA_TYPE_UTINYINT: 1489: doCompareWithValueRange_SignedWithUnsigned<LType, uint8_t>(fp);	

	1490: break; 1491: case TSDB_DATA_TYPE_USMALLINT:	
	污染路径: 无	
1100	缺陷发生位置: /source/util/src/talgo.c	行号: 485
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 483: } 484: 485: taosMemoryFreeClear(tmp); 486: } 487: return 0;	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1101	缺陷发生位置: /source/util/test/utilTests.cpp	行号: 500
	步骤描述: 不使用返回值的函数'checkNull'应使用'void'说明.	
	代码片段: 498: checkNull(0, false); 499: checkNull(1, false); 500: checkNull(2, false); 501: checkNull(totalRows - 2, false); 502: checkNull(totalRows - 1, false);	
	污染路径: 函数'checkNull'的定义点.	
1102	缺陷发生位置: /source/libs/executor/src/executor.c	行号: 59
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 57: int32_t getCurrentMnodeEpset(SEpSet* pEpSet) { 58: if (gExecInfo.dnode == NULL gExecInfo.getMnode == NULL) { 59: qError("gExecInfo is not initialized"); 60: return TSDB_CODE_APP_ERROR; 61: }	

	污染路径: 函数'qError'的定义点.	
1103	缺陷发生位置: /source/dnode/vnode/src/meta/metaTable.c	行号: 655
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 653: rc = tdbTbGet(pMeta->pTbDb, &(STbDbKey){.version = version, .uid = uid}, sizeof(STbDbKey), &pData, &nData); 654: if (rc < 0) { 655: tdbFree(pData); 656: return rc; 657: }	
	污染路径: 函数'tdbFree'的定义点.	
1104	缺陷发生位置: /source/dnode/mnode/impl/src/mndArbGroup.c	行号: 594
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: 592: if (epSet.numOfEps == 0) { 593: mError("arbgrouop:%d, failed to send arb-set-assigned request to dnode:%d since no epSet found", vglId, dnodeId); 594: rpcFreeCont(pHead); 595: code = -1; 596: if (terrno != 0) code = terrno;	
	污染路径: 函数'rpcFreeCont'的定义点.	
1105	缺陷发生位置: /utils/test/c/tmqSim.c	行号: 194
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 192: 193: if (0 != strlen(g_stConflInfo.topic)) { 194: sprintf(filename, "/tmp/tmqlog-%d-%s.txt", process_id, getCurrentTimeString(tmpString)); 195: } else { 196: sprintf(filename, "%s/./log/tmqlog-%d-%s.txt", configDir, process_id, getCurrentTimeString(tmpString));	

	污染路径: 无	
1106	缺陷发生位置: /source/dnode/mnode/impl/src/mndRole.c	行号: 721
	步骤描述: 不使用返回值的函数'TAOS_RETURN'应使用'void'说明.	
	代码片段: 719: } 720: mndTransDrop(pTrans); 721: TAOS_RETURN(0); 722: } 723:	
	污染路径: 函数'TAOS_RETURN'的定义点.	
1107	缺陷发生位置: /source/libs/tdb/test/tdbPageFlushTest.cpp	行号: 345
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 343: for (int i = 1024 * 4; i < 1024 * 8; ++i) { 344: char key[32] = {0}; 345: sprintf(key, "key-%d", i); 346: ret = tdbTbInsert(pDb, key, strlen(key) + 1, val, valLen, txn); 347: GTEST_ASSERT_EQ(ret, 0);	
	污染路径: 无	
1108	缺陷发生位置: /source/util/src/tworker.c	行号: 715
	步骤描述: 不使用返回值的函数'closeThreadNotificationConn'应使用'void'说明.	
	代码片段: 713: 714: DestoryThreadLocalRegComp(); 715: closeThreadNotificationConn(); 716: 717: return NULL;	
	污染路径: 函数'closeThreadNotificationConn'的定义点.	
1109	缺陷发生位置:	行号:

	/source/dnode/vnode/src/meta/metaCommit.c	100
	步骤描述: 不使用返回值的函数'TAOS_RETURN'应使用'void'说明.	
	代码片段: 98: } 99: 100: TAOS_RETURN(code); 101: } 102:	
	污染路径: 函数'TAOS_RETURN'的定义点.	
1110	缺陷发生位置: /source/libs/scalar/src/scalar.c	行号: 45
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 43: return code; 44: } 45: taosMemoryFree(timeStr); 46: valueNode->datum.i = value; 47: valueNode->typeData = valueNode->datum.i;	
	污染路径: 函数'taosMemoryFree'的定义点.	
1111	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/gnu-test-skeleton.c	行号: 328
	步骤描述: 不使用返回值的函数'signal'应使用'void'说明.	
	代码片段: 326: # define TIMEOUT 2 327: #endif 328: signal (SIGALRM, timeout_handler); 329: alarm (TIMEOUT * timeoutfactor); 330:	
	污染路径: 无	
1112	缺陷发生位置: /source/util/src/tsched.c	行号: 56
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	

	代码片段: 54: pSched->qthread = taosMemoryCalloc(sizeof(TdThread), numOfThreads); 55: if (pSched->qthread == NULL) { 56: uError("%s: no enough memory for qthread", label); 57: taosCleanUpScheduler(pSched); 58: if (schedMallocated) {	
	污染路径: 函数'uError'的定义点.	
1113	缺陷发生位置: /source/dnode/vnode/src/tq/tqScan.c	行号: 39
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 37: END: 38: if (code != TSDB_CODE_SUCCESS) { 39: taosMemoryFree(buf); 40: tqError("%s failed at %d, failed to add block data to response:%s", __FUNCTION__, lino, strerror(code)); 41: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1114	缺陷发生位置: /contrib/libmqtt/tq/src/ttqNet.c	行号: 953
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 951: } 952: if (net__socket_nonblock(&sv[0])) { 953: UNUSED(COMPAT_CLOSE(sv[1])); 954: return TTQ_ERR_ERRNO; 955: }	
	污染路径: 函数'UNUSED'的定义点.	
1115	缺陷发生位置: /source/dnode/mnode/impl/src/mndStreamUtil.c	行号: 550
	步骤描述: 不使用返回值的函数'mstDebug'应使用'void'说明.	
	代码片段: 548: }	

	549: 550: mstDebug("%s: stream obj", tips); 551: mstDebug("name:%s mainSnodeId:%d userDropped:%d userStopped:%d createTime:%" PRIu64 " updateTime:%" PRIu64, 552: p->name, p->mainSnodeId, p->userDropped, p-> userStopped, p->createTime, p->updateTime);	
	污染路径: 函数'mstDebug'的定义点.	
1116	缺陷发生位置: /source/dnode/mnode/impl/src/mndDump.c	行号: 733
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 731: 732: int32_t mndDumpSdb() { 733: mInfo("start to dump sdb info to sdb.json"); 734: 735: char path[PATH_MAX * 2] = {0};	
	污染路径: 函数'mInfo'的定义点.	
1117	缺陷发生位置: /source/dnode/mnode/impl/src/mndPrivilege.c	行号: 102
	步骤描述: 不使用返回值的函数'TAOS_RETURN'应使用'void'说明.	
	代码片段: 100: pWhiteListRsp->ver = 0; 101: pWhiteListRsp->numWhiteLists = 0; 102: TAOS_RETURN(0); 103: } 104:	
	污染路径: 函数'TAOS_RETURN'的定义点.	
1118	缺陷发生位置: /source/libs/catalog/src/ctgCache.c	行号: 247
	步骤描述: 不使用返回值的函数'ctgTrace'应使用'void'说明.	
	代码片段: 245: CTG_CACHE_HIT_INC(CTG_CI_DB_VGROUP, 1); 246:	

	247: ctgTrace("db:%s, get db vglInfo from cache", dbFName); 248: 249: return TSDB_CODE_SUCCESS;	
	污染路径: 函数'ctgTrace'的定义点.	
1119	缺陷发生位置: /examples/c/asyncdemo.c	行号: 109
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 107: printf("success to connect to server\n"); 108: 109: sprintf(sql, "drop database if exists %s", db); 110: queryDB(taos, sql); 111:	
	污染路径: 无	
1120	缺陷发生位置: /source/libs/tdb/src/db/tdbPCache.c	行号: 67
	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	
	代码片段: 65: static void tdbPCacheUnlock(SPCache *pCache) { 66: if (tdbMutexUnlock(&(pCache->mutex)) != 0) { 67: tdbError("tdb/pcache: mutex unlock failed."); 68: } 69: }	
	污染路径: 函数'tdbError'的定义点.	
1121	缺陷发生位置: /utls/test/c/tmq_ts7402.c	行号: 41
	步骤描述: 不使用返回值的函数'tmq_free_json_meta'应使用'void'说明.	
	代码片段: 39: printf("meta result: %s\n", result); 40: ASSERT(strcmp(result, result1[cnt]) == 0); 41: tmq_free_json_meta(result); 42: } else { 43: printf("no meta result\n");	

	污染路径: 函数'tmq_free_json_meta'的定义点.	
1122	缺陷发生位置: /source/util/src/tlruCache.c	行号: 702
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 700: cache->shards = taosMemoryCalloc(numShards, sizeof(SLRUCacheShard)); 701: if (!cache->shards) { 702: taosMemoryFree(cache); 703: return NULL; 704: } </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
1123	缺陷发生位置: /docs/examples/c/insert_data_demo.c	行号: 58
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 56: fprintf(stderr, "Failed to insert data to power.meters, sql: %s, ErrCode: 0x%x, ErrorMessage: %s\n.", sql, code, taos_errstr(result)); 57: taos_close(taos); 58: taos_cleanup(); 59: return -1; 60: } </pre>	
	污染路径: 函数'taos_cleanup'的定义点.	
1124	缺陷发生位置: /source/dnode/mnode/impl/src/mndQnode.c	行号: 560
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: <pre> 558: code = tSerializeSQnodeListRsp(pRsp, rspLen, &qlistRsp); 559: if (code < 0) { 560: mError("failed to serialize qnode list response since %s", terrstr()); 561: } 562: </pre>	

	污染路径: 函数'mError'的定义点.	
1125	缺陷发生位置: /source/libs/index/src/indexUtil.c	行号: 133
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 131: } 132: } 133: taosMemoryFreeClear(mi); 134: return 0; 135: }	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1126	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeStream.c	行号: 846
	步骤描述: 不使用返回值的函数'ST_TASK_ELOG'应使用'void'说明.	
	代码片段: 844: end: 845: if (code != 0) { 846: ST_TASK_ELOG("%s failed,code:%d", __func__, code); 847: taosArrayDestroy(uidList); 848: uidList = NULL;	
	污染路径: 函数'ST_TASK_ELOG'的定义点.	
1127	缺陷发生位置: /source/libs/index/test/fstTest.cc	行号: 208
	步骤描述: 不使用返回值的函数'Performance_fstWriteRecords'应使用'void'说明.	
	代码片段: 206: void checkMillonWriteAndReadOfFst() { 207: FstWriter* fw = new FstWriter; 208: Performance_fstWriteRecords(fw); 209: delete fw; 210: FstReadMemory* fr = new FstReadMemory(1024 * 8 * 1024);	
	污染路径: 函数'Performance_fstWriteRecords'的定义点.	

1128	缺陷发生位置: /source/libs/tdb/test/tdbPageDefragmentTest.cpp	行号: 378
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 376: for (int iData = 1; iData <= nData; iData++) { 377: sprintf(key, "key0"); 378: sprintf(val, "value%d", iData); 379: 380: // ret = tdbTbInsert(pDb, key, strlen(key), val, strlen(val), &txn);	
	污染路径: 无	
1129	缺陷发生位置: /tools/taos-tools/src/benchCsv.c	行号: 223
	步骤描述: 不使用返回值的函数'setlocale'应使用'void'说明.	
	代码片段: 221: strftime(time_buf, buf_size, ts_format, &tm_result); 222: if (old_locale) { 223: setlocale(LC_TIME, old_locale); 224: } 225: return;	
	污染路径: 无	
1130	缺陷发生位置: /source/libs/executor/src/scanoperator.c	行号: 1192
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 1190: int32_t num = taosArrayGetSize(pParam->pUidList); 1191: if (num <= 0) { 1192: qError("empty table scan uid list"); 1193: return TSDB_CODE_INVALID_PARA; 1194: }	
	污染路径: 函数'qError'的定义点.	
1131	缺陷发生位置: /source/libs/catalog/src/ctgAsync.c	行号: 3601

	52: 53: if (reason_code >= 0x80 && ttq->protocol == ttq_p_mqtt5) { 54: util__increment_receive_quota(ttq); 55: } 56:	
	污染路径: 函数'util__increment_receive_quota'的定义点.	
1135	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 2000
	步骤描述: 不使用返回值的函数'HUFv07_decodeStreamX2'应使用'void'说明.	
	代码片段: 1998: HUFv07_decodeStreamX2(op1, &bitD1, opStart2, dt, dtLog); 1999: HUFv07_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 2000: HUFv07_decodeStreamX2(op3, &bitD3, opStart4, dt, dtLog); 2001: HUFv07_decodeStreamX2(op4, &bitD4, oend, dt, dtLog); 2002:	
	污染路径: 函数'HUFv07_decodeStreamX2'的定义点.	
1136	缺陷发生位置: /source/libs/planner/src/planPhysiCreator.c	行号: 51
	步骤描述: 不使用返回值的函数'TAOS_STRNCAT'应使用'void'说明.	
	代码片段: 49: if (slotKeyType == SLOT_KEY_TYPE_ALL) { 50: TAOS_STRNCAT(*ppKey, pPreName, TSDB_TABLE_NAME_LEN); 51: TAOS_STRNCAT(*ppKey, ".", 2); 52: TAOS_STRNCAT(*ppKey, name, TSDB_COL_NAME_LEN); 53: *pLen = taosHashBinary(*ppKey, strlen(*ppKey), callocLen);	
	污染路径: 函数'TAOS_STRNCAT'的定义点.	
1137	缺陷发生位置: /source/util/src/tqueue.c	行号: 76

	步骤描述: 不使用返回值的函数'uDebug'应使用'void'说明.	
	代码片段: 74: } 75: 76: uDebug("queue:%p is opened", queue); 77: return 0; 78: }	
	污染路径: 函数'uDebug'的定义点.	
1138	缺陷发生位置:	行号:
	/utils/test/c/varbinary_test.c	239
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 237: pRes = taos_query(taos, "select * from stb where c2 in ('\\x7F829000','\\x00000000')"); 238: GET_ROW_NUM 239: ASSERT(numRows == 2); 240: taos_free_result(pRes); 241:	
1139	污染路径: 函数'ASSERT'的定义点.	
	缺陷发生位置:	行号:
	/utils/test/c/timezone_test.c	58
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
1140	代码片段: 56: int code = taos_errno(pRes); 57: taos_free_result(pRes); 58: ASSERT(code == 0); 59: 60: pRes = taos_query(taos, "create stable stb (ts timestamp, c1 int) tags (t1 timestamp, t2 binary(32))");	
	污染路径: 函数'ASSERT'的定义点.	
	缺陷发生位置:	行号:
	/source/libs/parser/test/parInitialCTest.cpp	482
	步骤描述: 不使用返回值的函数'operator<<'应使用'void'说明.	

	代码片段: 480: udfFile(const std::string& filename) : path_(filename) { 481: std::ofstream file(filename, std::ios::binary); 482: file << 123 << "abc" << '\n'; 483: file.close(); 484: }	
	污染路径: 函数'operator<<'的定义点.	
1141	缺陷发生位置: /source/common/src/tvariant.c	行号: 108
	步骤描述: 不使用返回值的函数'SET_ERRNO'应使用'void'说明.	
	代码片段: 106: } 107: 108: SET_ERRNO(0); 109: char *endPtr = NULL; 110: *value = taosStr2Double(z, &endPtr); // 0x already converted here	
	污染路径: 函数'SET_ERRNO'的定义点.	
1142	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFS2.c	行号: 71
	步骤描述: 不使用返回值的函数'tsdbError'应使用'void'说明.	
	代码片段: 69: TARRAY2_DESTROY(fs[0]->fSetArrTmp, NULL); 70: if (tsem_destroy(&fs[0]->canEdit) != 0) { 71: tsdbError("failed to destroy semaphore"); 72: } 73: taosMemoryFree(fs[0]);	
	污染路径: 函数'tsdbError'的定义点.	
1143	缺陷发生位置: /source/dnode/mnode/impl/src/mndUser.c	行号: 228
	步骤描述: 不使用返回值的函数'mDebug'应使用'void'说明.	
	代码片段: 226: } 227: if (taosHashRemove(userCache.users, user, userLen) !	

	<pre> = 0) { 228: mDebug("failed to remove user %s from user cache", user); 229: } 230: userCache.verIp++; </pre>	
	污染路径: 函数'mDebug'的定义点.	
1144	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 204
	步骤描述: 不使用返回值的函数'DEBUGLOG'应使用'void'说明.	
	代码片段: <pre> 202: } 203: /* size conditions not respected : scratch this buffer, create new one */ 204: DEBUGLOG(5, "ZSTDMT_getBuffer: existing buffer does not meet size conditions => freeing"); 205: ZSTD_free(buf.start, bufPool->cMem); 206: } </pre>	
	污染路径: 函数'DEBUGLOG'的定义点.	
1145	缺陷发生位置: /docs/examples/c/sml_insert_demo.c	行号: 47
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 45: fprintf(stderr, "Failed to create database power, ErrCode: 0x%x, ErrorMessage: %s.\n", code, taos_errstr(result)); 46: taos_close(taos); 47: taos_cleanup(); 48: return -1; 49: } </pre>	
	污染路径: 函数'taos_cleanup'的定义点.	
1146	缺陷发生位置: /source/libs/transport/src/thttp.c	行号: 881
	步骤描述: 不使用返回值的函数'taosDestroyHttpChan'应使用'void'说明.	
	代码片段: <pre> 879: void transHttpEnvDestroy() { </pre>	

	880: // remove default chanId 881: taosDestroyHttpChan(httpDefaultChanId); 882: httpDefaultChanId = -1; 883: taosCloseRef(httpRecvRefMgt);	
	污染路径: 函数'taosDestroyHttpChan'的定义点.	
1147	缺陷发生位置: /contrib/TSZ/zstd/decompress/zstd_decompress.c	行号: 957
	步骤描述: 不使用返回值的函数'DEBUGLOG'应使用'void'说明.	
	代码片段: 955: const BYTE* const iend = istart + srcSize; 956: const BYTE* ip = istart; 957: DEBUGLOG(5, "ZSTD_decodeSeqHeaders"); 958: 959: /* check */	
	污染路径: 函数'DEBUGLOG'的定义点.	
1148	缺陷发生位置: /utils/test/c/createTable.c	行号: 149
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 147: } 148: 149: snprintf(qstr, 128, "use %s", dbName); 150: TAOS_RES *pRes = taos_query(con, qstr); 151: int code = taos_errno(pRes);	
	污染路径: 函数'snprintf'的定义点.	
1149	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbDataFileRW.c	行号: 209
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 207: tsdbError("vgId:%d %s failed at %s:%d since %s", TD_VID(reader->config->tsdb->pVnode), __func__, __FILE__, lino, 208: tsterror(code)); 209: taosMemoryFree(data);	

	210: } 211: return code;	
	污染路径: 函数'taosMemoryFree'的定义点.	
1150	缺陷发生位置: /source/libs/catalog/src/ctgUtil.c	行号: 2152
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 2150: 2151: taosMemoryFree(pMeta); 2152: taosMemoryFree(stbName); 2153: 2154: CTG_RET(code);	
	污染路径: 函数'taosMemoryFree'的定义点.	
1151	缺陷发生位置: /source/libs/tmqtt/mgmt/src/tmqttMgmt.c	行号: 57
	步骤描述: 不使用返回值的函数'bndInfo'应使用'void'说明.	
	代码片段: 55: 56: static int32_t mqttMgmtSpawnMqtttd(SMqtttdData *pData) { 57: bndInfo("start to init taosmqtt"); 58: TAOS_MQTT_MGMT_CHECK_PTR_RCODE(pData); 59:	
	污染路径: 函数'bndInfo'的定义点.	
1152	缺陷发生位置: /source/libs/new-stream/src/dataSinkCache.c	行号: 367
	步骤描述: 不使用返回值的函数'stError'应使用'void'说明.	
	代码片段: 365: QUERY_CHECK_CODE(code, lino, _end); 366: if (len < 0) { 367: stError("failed to encode data since %s, lineno:%d", tstrerror(len), lino); 368: return TSDB_CODE_STREAM_INTERNAL_ERROR; 369: }	

	污染路径: 函数'stError'的定义点.	
1153	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 209
	步骤描述: 不使用返回值的函数'DEBUGLOG'应使用'void'说明.	
	代码片段: 207: ZSTD_pthread_mutex_unlock(&bufPool->poolMutex); 208: /* create new buffer */ 209: DEBUGLOG(5, "ZSTDMT_getBuffer: create a new buffer"); 210: { buffer_t buffer; 211: void* const start = ZSTD_malloc(bSize, bufPool->cMem);	
	污染路径: 函数'DEBUGLOG'的定义点.	
1154	缺陷发生位置: /source/dnode/mgmt/node_util/src/dmFile.c	行号: 443
	步骤描述: 不使用返回值的函数'encryptDebug'应使用'void'说明.	
	代码片段: 441: } 442: 443: encryptDebug("succeed to compare checkCode file: %s", file); 444: code = 0; 445: _OVER:	
	污染路径: 函数'encryptDebug'的定义点.	
1155	缺陷发生位置: /source/dnode/mgmt/exe/dmMain.c	行号: 183
	步骤描述: 不使用返回值的函数'dmStop'应使用'void'说明.	
	代码片段: 181: #endif 182: 183: dmStop(); 184: } 185:	

	污染路径: 函数'qWorkerDestroy'的定义点.	
1159	缺陷发生位置: /source/os/src/osString.c	行号: 220
	步骤描述: 不使用返回值的函数'OS_PARAM_CHECK'应使用'void'说明.	
	代码片段: 218: 219: int32_t taosStr2int8(const char *str, int8_t *val) { 220: OS_PARAM_CHECK(str); 221: OS_PARAM_CHECK(val); 222: int64_t tmp = 0;	
	污染路径: 函数'OS_PARAM_CHECK'的定义点.	
1160	缺陷发生位置: /source/util/test/utilTests.cpp	行号: 502
	步骤描述: 不使用返回值的函数'checkNull'应使用'void'说明.	
	代码片段: 500: checkNull(2, false); 501: checkNull(totalRows - 2, false); 502: checkNull(totalRows - 1, false); 503: 504: if (IS_VAR_DATA_TYPE(type)) {	
	污染路径: 函数'checkNull'的定义点.	
1161	缺陷发生位置: /source/dnode/mnode/impl/src/mndConsumer.c	行号: 913
	步骤描述: 不使用返回值的函数'MND_TMQ_NULL_CHECK'应使用'void'说明.	
	代码片段: 911: } else if (pNewConsumer->updateType == CONSUMER_ADD_REB) { 912: void *tmp = taosArrayGetP(pNewConsumer- >rebNewTopics, 0); 913: MND_TMQ_NULL_CHECK(tmp); 914: char *pNewTopic = taosStrdup(tmp); 915: MND_TMQ_NULL_CHECK(pNewTopic);	
	污染路径:	

	函数'MND_TMQ_NULL_CHECK'的定义点.	
1162	缺陷发生位置: /source/common/src/msg/streamMsg.c	行号: 2367
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: <pre> 2365: code = tDeserializeSCMCreateStreamReqImpl(&decoder, pReq); 2366: if (TSDB_CODE_MND_STREAM_INVALID_JSON == code) { 2367: uError("invalid json for stream create request, try old deserialization"); 2368: // try old deserialization for backward compatibility 2369: tDecoderClear(&decoder); </pre>	
	污染路径: 函数'uError'的定义点.	
1163	缺陷发生位置: /contrib/TSZ/zstd/decompress/huf_decompress.c	行号: 967
	步骤描述: 不使用返回值的函数'assert'应使用'void'说明.	
	代码片段: <pre> 965: U32 HUF_selectDecoder (size_t dstSize, size_t cSrcSize) 966: { 967: assert(dstSize > 0); 968: assert(dstSize <= 128*1024); 969: /* decoder timing evaluation */ </pre>	
	污染路径: 函数'assert'的定义点.	
1164	缺陷发生位置: /source/util/src/tlog.c	行号: 343
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: <pre> 341: (void)taosThreadMutexDestroy(&tsLogObj.logHandle- >buffMutex); 342: (void)taosCloseFile(&tsLogObj.logHandle->pFile); 343: taosMemoryFreeClear(tsLogObj.logHandle->buffer); 344: (void)taosThreadMutexDestroy(&tsLogObj.logMutex); 345: taosMemoryFreeClear(tsLogObj.logHandle); </pre>	
	污染路径:	

	函数'taosMemoryFreeClear'的定义点.	
1165	缺陷发生位置: /source/libs/monitorfw/src/taos_collector.c	行号: 58
	步骤描述: 不使用返回值的函数'TAOS_LOG'应使用'void'说明.	
	代码片段: 56: if (self->string_builder == NULL) { 57: if (taos_collector_destroy(self) != 0) { 58: TAOS_LOG("taos_collector_destroy failed"); 59: } 60: return NULL;	
	污染路径: 函数'TAOS_LOG'的定义点.	
1166	缺陷发生位置: /source/libs/transport/test/http_test.c	行号: 50
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 48: memset(msg, 1, len); 49: int32_t code = taosSendHttpRequest(monitor, "/crash", 6050, msg, 10, HTTP_FLAT); 50: taosMemoryFree(msg); 51: taosUsleep(10); 52: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1167	缺陷发生位置: /source/dnode/vnode/src/meta/metaTable2.c	行号: 473
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 471: // for auto create table, we return the uid of the existing table 472: pReq->uid = *(tb_uid_t *)value; 473: tdbFree(value); 474: return TSDB_CODE_TDB_TABLE_ALREADY_EXIST; 475: }	
	污染路径: 函数'tdbFree'的定义点.	
1168	缺陷发生位置:	行号:

	/tools/taos-tools/src/benchData.c	2538
	步骤描述: 不使用返回值的函数'tmpGeometry'应使用'void'说明.	
	代码片段: 2536: case TSDB_DATA_TYPE_GEOMETRY: { 2537: char *buf = (char *)benchCalloc(col->length + 1, 1, false); 2538: tmpGeometry(buf, stbInfo->iface, col, 0); 2539: value_buf = benchCalloc(len_key + col->length + 3, 1, true); 2540: (void)snprintf(value_buf, len_key + col->length + 3, 	
	污染路径: 函数'tmpGeometry'的定义点.	
1169	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSnapshotRAW.c	行号: 88
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 86: TARRAY2_DESTROY(reader[0]->dataReaderArr, tsdbDataFileRAWReaderClose); 87: tsdbFSDestroyRefSnapshot(&reader[0]->fsetArr); 88: taosMemoryFree(reader[0]); 89: reader[0] = NULL; 90: return;	
	污染路径: 函数'taosMemoryFree'的定义点.	
1170	缺陷发生位置: /source/dnode/vnode/src/meta/metaCache.c	行号: 153
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 151: taosArrayDestroy((*p)->pColIds); 152: taosHashCleanup((*p)->set); 153: taosMemoryFreeClear(*p); 154: } 155:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1171	缺陷发生位置:	行号:

	/source/libs/executor/src/tlinearhash.c	327
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 325: taosMemoryFree(pHashObj->pBucket[i]); 326: } 327: taosMemoryFree(pHashObj->pBucket); 328: } 329: taosMemoryFree(pHashObj);	
	污染路径: 函数'taosMemoryFree'的定义点.	
1172	缺陷发生位置: /source/client/src/clientSmlJson.c	行号: 265
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 263: size_t keyLen = strlen(tag->string); 264: if (unlikely(IS_INVALID_COL_LEN(keyLen))) { 265: uError("SML:Tag key length is 0 or too large than 64"); 266: return TSDB_CODE_TSC_INVALID_COLUMN_LENGTH; 267: }	
	污染路径: 函数'uError'的定义点.	
1173	缺陷发生位置: /source/libs/planner/src/planLogicCreator.c	行号: 1486
	步骤描述: 不使用返回值的函数'planError'应使用'void'说明.	
	代码片段: 1484: break; 1485: } 1486: planError("%s failed at line %d since %s", __func__, __LINE__, tstrerror(code)); 1487: return code; 1488: }	
	污染路径: 函数'planError'的定义点.	
1174	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmHandle.c	行号: 47
	步骤描述:	

	不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: <pre> 45: if (!pVnode->failed) { 46: if (vnodeGetLoad(pVnode->pImpl, &vload) != 0) { 47: dError("failed to get vnode load"); 48: } 49: if (isReset) vnodeResetLoad(pVnode->pImpl, &vload); </pre>	
	污染路径: 函数'dError'的定义点.	
1175	缺陷发生位置: /source/dnode/mnode/impl/src/mndScanDetail.c	行号: 53
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: <pre> 51: SDbObj *pDb = NULL; 52: 53: mInfo("retrieve scan detail"); 54: 55: if (strlen(pShow->db) > 0) { </pre>	
	污染路径: 函数'mInfo'的定义点.	
1176	缺陷发生位置: /docs/examples/c/schemaless.c	行号: 43
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: <pre> 41: result = taos_query(taos, "create database db precision 'ms');"); 42: taos_free_result(result); 43: usleep(100000); 44: 45: (void)taos_select_db(taos, "db"); </pre>	
	污染路径: 无	
1177	缺陷发生位置: /source/libs/new-stream/src/dataSinkFile.c	行号: 39
	步骤描述: 不使用返回值的函数'stError'应使用'void'说明.	
	代码片段: <pre> 37: int ret = sprintf(gDataSinkFilePath, </pre>	

	<pre>sizeof(gDataSinkFilePath), "%s/tdengine_stream_data/", tsTempDir); 38: if (ret < 0) { 39: stError("failed to get stream data sink path ret:%d", ret); 40: return TSDB_CODE_TSC_INTERNAL_ERROR; 41: }</pre>	
	污染路径: 函数'stError'的定义点.	
1178	缺陷发生位置: /source/libs/monitorfw/src/taos_collector.c	行号: 117
	步骤描述: 不使用返回值的函数'TAOS_LOG'应使用'void'说明.	
	代码片段: <pre>115: if (self == NULL) return 1; 116: if (taos_map_get(self->metrics, metric->name) != NULL) { 117: TAOS_LOG("metric already found in collector"); 118: return 1; 119: }</pre>	
1179	污染路径: 函数'TAOS_LOG'的定义点.	
	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSttFileRW.c	行号: 928
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
1180	代码片段: <pre>926: tsdbSttFWriterDoClose(writer[0]); 927: } 928: taosMemoryFree(writer[0]); 929: writer[0] = NULL; 930:</pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
1180	缺陷发生位置: /tools/taos-tools/src/benchTmq.c	行号: 366
	步骤描述: 不使用返回值的函数'pthread_create'应使用'void'说明.	
	代码片段: <pre>364: }</pre>	

	<pre> 365: } 366: pthread_create(pids + i, NULL, tmqConsume, pthreadInfo); 367: } 368: </pre>	
	污染路径: 无	
1181	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 998
	步骤描述: 不使用返回值的函数'close'应使用'void'说明.	
	代码片段: <pre> 996: WSACleanup(); 997: #else 998: close(sockfd); 999: #endif 1000: return -1; </pre>	
	污染路径: 无	
1182	缺陷发生位置: /docs/examples/c/sml_insert_demo.c	行号: 58
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 56: fprintf(stderr, "Failed to execute use power, ErrCode: 0x %x, ErrorMessage: %s\n.", code, taos_errstr(result)); 57: taos_close(taos); 58: taos_cleanup(); 59: return -1; 60: } </pre>	
	污染路径: 函数'taos_cleanup'的定义点.	
1183	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 1999
	步骤描述: 不使用返回值的函数'HUFv07_decodeStreamX2'应使用'void'说明.	
	代码片段: <pre> 1997: /* finish bitStreams one by one */ 1998: HUFv07_decodeStreamX2(op1, &bitD1, opStart2, dt, dtLog); </pre>	

	1999: HUFv07_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 2000: HUFv07_decodeStreamX2(op3, &bitD3, opStart4, dt, dtLog); 2001: HUFv07_decodeStreamX2(op4, &bitD4, oend, dt, dtLog);	
	污染路径: 函数'HUFv07_decodeStreamX2'的定义点.	
1184	缺陷发生位置: /source/client/test/session.cpp	行号: 447
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 445: char guoUtf8[32] = {0}; 446: char zhongguoGbk[32] = {0}; 447: snprintf(fenUtf8, sizeof(fenUtf8), "%c%c%c", 0xE8, 0x8A, 0xAC); 448: snprintf(fenGbk, sizeof(fenGbk), "%c%c", 0xB7, 0xD2); 449: snprintf(zhongguoUtf8, sizeof(zhongguoUtf8), "%c%c%c%c", 0xE4, 0xB8, 0xAD, 0xE5, 0x9B, 0xBD);	
	污染路径: 无	
1185	缺陷发生位置: /utls/test/c/createTable.c	行号: 140
	步骤描述: 不使用返回值的函数'pPrint'应使用'void'说明.	
	代码片段: 138: 139: void showTables() { 140: pPrint("start to show tables"); 141: char qstr[128]; 142:	
	污染路径: 函数'pPrint'的定义点.	
1186	缺陷发生位置: /test/new_test_framework/script/api/last-query-ws.cpp	行号: 84
	步骤描述: 不使用返回值的函数'operator<<'应使用'void'说明.	
	代码片段: 82: const char* errstr = ws_errstr(wtaos);	

	<pre> 83: if (code) { 84: std::cout << "Connection failed on select db[" << code << "]: " << strerror << "\n"; 85: return; 86: }</pre>	
	污染路径: 函数'operator<<'的定义点.	
1187	缺陷发生位置: /utls/test/c/varbinary_test.c	行号: 229
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: <pre> 227: pRes = taos_query(taos, "select * from stb where c2 > 'varc1'"); 228: GET_ROW_NUM 229: ASSERT(numRows == 2); 230: taos_free_result(pRes); 231:</pre>	
	污染路径: 函数'ASSERT'的定义点.	
1188	缺陷发生位置: /docs/examples/c/taos_connect_with_example.c	行号: 22
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 20: if (taos == NULL) { 21: fprintf(stderr, "Failed to connect to TDengine server: %s\n", taos_errstr(NULL)); 22: taos_cleanup(); 23: return -1; 24: }</pre>	
	污染路径: 函数'taos_cleanup'的定义点.	
1189	缺陷发生位置: /docs/examples/c/connect_example.c	行号: 32
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: <pre> 30: } 31:</pre>	

	32: sleep(5); 33: { 34: TAOS_RES *res = taos_query(taos, "show connections");	
	污染路径: 无	
1190	缺陷发生位置: /source/libs/catalog/src/ctgDbg.c	行号: 370
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 368: } 369: 370: qError("invalid stat option:%s", option); 371: 372: return TSDB_CODE_CTG_INTERNAL_ERROR;	
	污染路径: 函数'qError'的定义点.	
1191	缺陷发生位置: /test/new_test_framework/script/api/asyncQuery.c	行号: 59
	步骤描述: 不使用返回值的函数'atomic_add_fetch_64'应使用'void'说明.	
	代码片段: 57: } else if (nrows == 0) { 58: taos_free_result(res); 59: atomic_add_fetch_64(&finQueries, 1); 60: } else { 61: printResult(res, nrows);	
	污染路径: 函数'atomic_add_fetch_64'的定义点.	
1192	缺陷发生位置: /source/libs/tdb/test/tdbPageRecycleTest.cpp	行号: 420
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 418: 419: for (int i = 1; i <= nData; i++) { 420: sprintf(key, "key%d", i); 421: // sprintf(val, "value%d", i); 422:	

	污染路径: 无	
1193	缺陷发生位置: /source/libs/transport/src/transSvr.c	行号: 269
	步骤描述: 不使用返回值的函数'tDebug'应使用'void'说明.	
	代码片段: 267: static void uvHandleActivityTimeout(uv_timer_t* handle) { 268: SSvrConn* conn = handle->data; 269: tDebug("%p timeout since no activity", conn); 270: } 271:	
	污染路径: 函数'tDebug'的定义点.	
1194	缺陷发生位置: /source/client/src/clientMonitor.c	行号: 87
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 85: static void monitorFreeSlowLogDataEx(void* paras) { 86: monitorFreeSlowLogData(paras); 87: taosMemoryFree(paras); 88: } 89:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1195	缺陷发生位置: /source/dnode/mnode/sdb/src/sdbFile.c	行号: 78
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 76: static int32_t sdbAfterRestoredData(SSdb *pSdb) { 77: int32_t code = 0; 78: mInfo("start to prepare sdb"); 79: 80: for (int32_t i = SDB_MAX - 1; i >= 0; --i) {	
	污染路径: 函数'mInfo'的定义点.	
1196	缺陷发生位置:	行号:

	/source/libs/tmqtt/mqtt/src/tmqttLogging.c	319
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 317: int rc; 318: 319: UNUSED(ttq); 320: 321: va_start(va, fmt);	
	污染路径: 函数'UNUSED'的定义点.	
1197	缺陷发生位置: /source/dnode/vnode/src/bse/bseCache.c	行号: 436
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 434: taosMemoryFree(pltem->pNode); 435: } 436: taosMemoryFree(pltem); 437: } 438:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1198	缺陷发生位置: /source/dnode/vnode/src/meta/metaOpen.c	行号: 944
	步骤描述: 不使用返回值的函数'metaError'应使用'void'说明.	
	代码片段: 942: 943: if (pTagIdxKey1->type != pTagIdxKey2->type) { 944: metaError("meta/open: incorrect tag idx type."); 945: return TSDB_CODE_FAILED; 946: }	
	污染路径: 函数'metaError'的定义点.	
1199	缺陷发生位置: /source/libs/new-stream/src/streamTriggerTask.c	行号: 5279
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段:	

	<pre> 5277: code = streamReadCheckPoint(pTask- >task.streamId, &buf, &len); 5278: if (code != TSDB_CODE_SUCCESS) { 5279: taosMemoryFree(buf); 5280: QUERY_CHECK_CODE(code, lino, _end); 5281: } </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
1200	缺陷发生位置: /contrib/lemon/lemon.c	行号: 1473
	步骤描述: 不使用返回值的函数'Configtable_insert'应使用'void'说明.	
	代码片段: <pre> 1471: *basisend = cfp; 1472: basisend = &cfp->bp; 1473: Configtable_insert(cfp); 1474: } 1475: return cfp; </pre>	
	污染路径: 函数'Configtable_insert'的定义点.	
1201	缺陷发生位置: /source/dnode/mnode/impl/src/mndQuery.c	行号: 96
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: <pre> 94: if ((code = tDeserializeSBatchReq(pMsg->pCont, pMsg- >contLen, &batchReq)) != 0) { 95: code = TSDB_CODE_OUT_OF_MEMORY; 96: mError("tDeserializeSBatchReq failed"); 97: goto _exit; 98: } </pre>	
	污染路径: 函数'mError'的定义点.	
1202	缺陷发生位置: /contrib/TSZ/zstd/decompress/zstd_decompress.c	行号: 1636
	步骤描述: 不使用返回值的函数'DEBUGLOG'应使用'void'说明.	
	代码片段: <pre> 1634: const ZSTD_longOffset_e isLongOffset) </pre>	

	1635: { 1636: DEBUGLOG(5, "ZSTD_decompressSequencesLong"); 1637: #if DYNAMIC_BMI2 1638: if (dctx->bmi2) {	
	污染路径: 函数'DEBUGLOG'的定义点.	
1203	缺陷发生位置: /source/dnode/mgmt/mgmt_dnode/src/dmHandle.c	行号: 186
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: 184: contLen = tSerializeRetrieveAnalyticAlgoReq(pHead, contLen, &req); 185: if (contLen < 0) { 186: rpcFreeCont(pHead); 187: dError("failed to serialize analysis function ver request since %s", tsterror(contLen)); 188: return;	
	污染路径: 函数'rpcFreeCont'的定义点.	
1204	缺陷发生位置: /source/libs/monitor/src/monFramework.c	行号: 320
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 318: 319: if(taos_collector_registry_deregister_metric(STATUS) != 0){ 320: uError("failed to delete metric "STATUS); 321: } 322:	
	污染路径: 函数'uError'的定义点.	
1205	缺陷发生位置: /test/new_test_framework/script/api/passwdTest.c	行号: 243
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 241: 242: taos_close(taos);	

	243: taos_cleanup(); 244: exit(EXIT_SUCCESS); 245: }	
	污染路径: 函数'taos_cleanup'的定义点.	
1206	缺陷发生位置: /source/util/test/lrucacheTest.cpp	行号: 102
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 100: for (int32_t i = 0; i < numItems; i++) { 101: char key[64]; 102: snprintf(key, sizeof(key), "key_%d", i); 103: int32_t value = i; 104:	
	污染路径: 无	
1207	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttDaemon.c	行号: 141
	步骤描述: 不使用返回值的函数'bndError'应使用'void'说明.	
	代码片段: 139: SConnectRsp connectRsp = {0}; 140: if (tDeserializeSConnectRsp(pMsg->pCont, pMsg->contLen, &connectRsp) < 0) { 141: bndError("mqtttd deserialize connect response failed"); 142: goto _return; 143: }	
	污染路径: 函数'bndError'的定义点.	
1208	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncIO.c	行号: 54
	步骤描述: 不使用返回值的函数'taosSeedRand'应使用'void'说明.	
	代码片段: 52: ASSERT(gSyncIO != NULL); 53: 54: taosSeedRand(taosGetTimestampSec()); 55: ret = syncIOStartInternal(gSyncIO);	

	56: ASSERT(ret == 0);	
	污染路径: 函数'taosSeedRand'的定义点.	
1209	缺陷发生位置: /contrib/TSZ/zstd/compress/fse_compress.c	行号: 337
	步骤描述: 不使用返回值的函数'assert'应使用'void'说明.	
	代码片段: <pre> 335: U32 minBitsSymbols = BIT_highbit32(maxSymbolValue) + 2; 336: U32 minBits = minBitsSrc < minBitsSymbols ? minBitsSrc : minBitsSymbols; 337: assert(srcSize > 1); /* Not supported, RLE should be used instead */ 338: return minBits; 339: }</pre>	
	污染路径: 函数'assert'的定义点.	
1210	缺陷发生位置: /source/dnode/mnode/impl/src/mndMain.c	行号: 119
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: <pre> 117: // TODO check return value 118: if (tmsgPutToQueue(&pMnode->msgCb, WRITE_QUEUE, &rpcMsg) < 0) { 119: mError("failed to put into write-queue since %s, line: %d", terrstr(), __LINE__); 120: } 121: }</pre>	
	污染路径: 函数'mError'的定义点.	
1211	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 921
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 919: } 920: 921: taosMemoryFree(value);</pre>	

	922: } 923:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1212	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 1354
	步骤描述: 不使用返回值的函数'buffer_skip_whitespace'应使用'void'说明.	
	代码片段: 1352: goto fail; /* failed to parse value */ 1353: } 1354: buffer_skip_whitespace(input_buffer); 1355: } 1356: while (can_access_at_index(input_buffer, 0) && (buffer_at_offset(input_buffer)[0] == ','));	
	污染路径: 函数'buffer_skip_whitespace'的定义点.	
1213	缺陷发生位置: /source/dnode/mnode/impl/src/mndQnode.c	行号: 238
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 236: 237: if ((code = mndTransAppendRedoAction(pTrans, &action)) != 0) { 238: taosMemoryFree(pReq); 239: TAOS_RETURN(code); 240: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1214	缺陷发生位置: /source/libs/executor/src/exchangeoperator.c	行号: 800
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 798: void freeSourceDataInfo(void* p) { 799: SSourceDataInfo* plInfo = (SSourceDataInfo*)p; 800: taosMemoryFreeClear(plInfo->decompBuf); 801: taosMemoryFreeClear(plInfo->pRsp); 802:	

	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1215	缺陷发生位置: /source/client/src/clientSmlLine.c	行号: 290
	步骤描述: 不使用返回值的函数'MOVE_FORWARD_ONE'应使用'void'说明.	
	代码片段: 288: } 289: (void)memcpy(tmp, value, valueLen); 290: PROCESS_SLASH_IN_TAG_FIELD_KEY(tmp, valueLen); 291: value = tmp; 292: }	
	污染路径: expanded from macro 'PROCESS_SLASH_IN_TAG_FIELD_KEY' 函数'MOVE_FORWARD_ONE'的定义点. expanded from macro 'PROCESS_SLASH_IN_TAG_FIELD_KEY'	
1216	缺陷发生位置: /source/libs/decimal/src/detail/intx/int128.hpp	行号: 183
	步骤描述: 不使用返回值的函数'operator++'应使用'void'说明.	
	代码片段: 181: { 182: auto ret = x; 183: ++x; 184: return ret; 185: }	
	污染路径: 函数'operator++'的定义点.	
1217	缺陷发生位置: /source/dnode/mnode/impl/src/mndMain.c	行号: 125
	步骤描述: 不使用返回值的函数'mTrace'应使用'void'说明.	
	代码片段: 123: 124: static void mndPullupCompacts(SMnode *pMnode) { 125: mTrace("pullup compact timer msg"); 126: int32_t contLen = 0; 127: void *pReq = mndBuildTimerMsg(&contLen);	
	污染路径:	

	函数'mTrace'的定义点.	
1218	缺陷发生位置: /source/dnode/mnode/impl/src/mndStreamHb.c	行号: 39
	步骤描述: 不使用返回值的函数'mstDebug'应使用'void'说明.	
	代码片段: 37: SRpcMsg rspMsg = {0}; 38: 39: mstDebug("start to process stream hb req msg"); 40: 41: rsp.streamGId = req.streamGId;	
	污染路径: 函数'mstDebug'的定义点.	
1219	缺陷发生位置: /source/util/src/tworker.c	行号: 714
	步骤描述: 不使用返回值的函数'DestoryThreadLocalRegComp'应使用'void'说明.	
	代码片段: 712: } 713: 714: DestoryThreadLocalRegComp(); 715: closeThreadNotificationConn(); 716:	
	污染路径: 函数'DestoryThreadLocalRegComp'的定义点.	
1220	缺陷发生位置: /source/libs/nodes/src/nodesCodeFuncs.c	行号: 9937
	步骤描述: 不使用返回值的函数'nodesError'应使用'void'说明.	
	代码片段: 9935: char** split = strsplit(privSet, ",", &split_num); 9936: if (!split) { 9937: nodesError("failed at line %d to parse privileges string: %s", __LINE__, privSet); 9938: return TSDB_CODE_INVALID_PARA; 9939: }	
	污染路径: 函数'nodesError'的定义点.	
1221	缺陷发生位置: /source/dnode/vnode/src/meta/metaQuery.c	行号: 143

	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 141: END; 142: tdbFree(pKey); 143: tdbFree(pVal); 144: tdbTbcClose(pCur); 145: return code;	
	污染路径: 函数'tdbFree'的定义点.	
1222	缺陷发生位置: /source/common/src/ttime.c	行号: 626
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 624: int len = taosUcs4ToMbs((TdUcs4*)dataVal, charLen, newColData, charsetCxt); 625: if (len < 0) { 626: taosMemoryFree(newColData); 627: TAOS_RETURN(TSDB_CODE_FAILED); 628: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1223	缺陷发生位置: /source/libs/decimal/src/detail/intx/int128.hpp	行号: 644
	步骤描述: 不使用返回值的函数'operator+='应使用'void'说明.	
	代码片段: 642: { 643: --q.hi; 644: r += d; 645: } 646:	
	污染路径: 函数'operator+='的定义点.	
1224	缺陷发生位置: /source/libs/executor/src/cachescanoperator.c	行号: 424
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段:	

	422: 423: cleanupExprSupp(&pInfo->pseudoExprSup); 424: taosMemoryFreeClear(param); 425: } 426:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1225	缺陷发生位置: /source/dnode/vnode/src/bse/bseCache.c	行号: 285
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 283: 284: lruCacheFree((SLruCache *)p->pCache); 285: taosMemoryFree(p); 286: } 287: int32_t tableCacheClear(STableCache *p) {	
	污染路径: 函数'taosMemoryFree'的定义点.	
1226	缺陷发生位置: /docs/examples/c-ws/tmq_demo.c	行号: 81
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 79: } 80: ws_free_result(pRes); 81: sleep(1); 82: } 83: fprintf(stdout, "Prepare data thread exit\n");	
	污染路径: 无	
1227	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmWorker.c	行号: 564
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: 562: pMgmt->pStreamReaderQ = tWWorkerAllocQueue(&pMgmt->streamReaderPool, pMgmt, mmProcessStreamReaderQueue); 563:	

	564: dDebug("mnode workers are initialized"); 565: return code; 566: }	
	污染路径: 函数'dDebug'的定义点.	
1228	缺陷发生位置: /test/new_test_framework/script/api/stmt2-insert-dupkeys.c	行号: 181
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 179: 180: for (int i = 0; i < CTB_NUMS; i++) { 181: sprintf(tbs[i], "ctb_%d", i % 2); 182: } 183:	
	污染路径: 无	
1229	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 1614
	步骤描述: 不使用返回值的函数'FSE_buildDTable_rle'应使用'void'说明.	
	代码片段: 1612: case bt_rle : 1613: LLlog = 0; 1614: FSE_buildDTable_rle(DTableLL, *ip++); break; 1615: case bt_raw : 1616: LLlog = LLbits;	
	污染路径: 函数'FSE_buildDTable_rle'的定义点.	
1230	缺陷发生位置: /utls/test/c/tmqOffset.c	行号: 57
	步骤描述: 不使用返回值的函数'tqOffsetRestoreFromFile'应使用'void'说明.	
	代码片段: 55: 56: int main(int argc, char *argv[]) { 57: tqOffsetRestoreFromFile("offset-ver0"); 58: return 0; 59: }	

	污染路径: 函数'tqOffsetRestoreFromFile'的定义点.	
1231	缺陷发生位置: /source/dnode/mnode/impl/src/mndTopic.c	行号: 940
	步骤描述: 不使用返回值的函数'STR_TO_VARSTR'应使用'void'说明.	
	代码片段: <pre> 938: } else { 939: mError("mndRetrieveTopic build schema error json:%p, json len:%zu", string, len); 940: STR_TO_VARSTR(schemaJson, "NULL"); 941: } 942: taosMemoryFree(string); </pre>	
	污染路径: 函数'STR_TO_VARSTR'的定义点.	
1232	缺陷发生位置: /source/libs/parser/src/parser.c	行号: 188
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 186: int written = snprintf(p, rem, "INSERT INTO %.*s (", (int)t.n, t.z); 187: if (written < 0 (size_t)written >= rem) { 188: taosMemoryFree(newSql); 189: code = generateSyntaxErrMsgExt(&pMsgBuf, TSDB_CODE_PAR_SYNTAX_ERROR, "sql too long"); 190: return code; </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
1233	缺陷发生位置: /docs/examples/c-ws-new/create_db_demo.c	行号: 38
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 36: fprintf(stderr, "Failed to create database power, ErrCode: 0x%x, ErrorMessage: %s.\n", code, taos_errstr(result)); 37: taos_close(taos); 38: taos_cleanup(); 39: return -1; 40: } </pre>	

	污染路径: 函数'taos_cleanup'的定义点.	
1234	缺陷发生位置: /test/new_test_framework/script/api/api_with_reqid_test.c	行号: 184
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 182: prepare_data(taos); 183: taos_query_a_with_reqid(taos, "select * from meters", select_callback, NULL, genReqid()); 184: usleep(1000000); 185: } 186:	
	污染路径: 无	
1235	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSet2.c	行号: 705
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 703: 704: tsdbTFileSetClear(&fsr[0]->fset); 705: taosMemoryFree(fsr[0]); 706: fsr[0] = NULL; 707: return;	
	污染路径: 函数'taosMemoryFree'的定义点.	
1236	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSetRAW.c	行号: 150
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 148: // free 149: TARRAY2_DESTROY(writer[0]->ctx->fopArr, NULL); 150: taosMemoryFree(writer[0]); 151: writer[0] = NULL; 152:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1237	缺陷发生位置:	行号:

	/source/libs/executor/src/filloperator.c		76
	步骤描述: 不使用返回值的函数'taosResetFillInfo'应使用'void'说明.		
	代码片段: <pre> 74: int32_t scanFlag = MAIN_SCAN; 75: // getTableScanInfo(pOperator, &order, &scanFlag, false); 76: taosResetFillInfo(pInfo->pFillInfo, getFillInfoStart(pInfo->pFillInfo)); 77: 78: blockDataCleanup(pInfo->pRes); </pre>		
	污染路径: 函数'taosResetFillInfo'的定义点.		
1238	缺陷发生位置: /tools/taos-tools/src/benchCsv.c		行号: 1018
	步骤描述: 不使用返回值的函数'gzclose'应使用'void'说明.		
	代码片段: <pre> 1016: } else { 1017: if (fhdl->handle.gf) { 1018: gzclose(fhdl->handle.gf); 1019: fhdl->handle.gf = NULL; 1020: } </pre>		
	污染路径: 无		
1239	缺陷发生位置: /source/libs/transport/test/strBench.c		行号: 176
	步骤描述: 不使用返回值的函数'tInfo'应使用'void'说明.		
	代码片段: <pre> 174: // taosSleep(5); 175: 176: tInfo("RPC server is running, ctrl-c to exit"); 177: 178: if (commit) { </pre>		
	污染路径: 函数'tInfo'的定义点.		
1240	缺陷发生位置: /source/libs/new-stream/src/dataSinkMgr.c		行号: 270
	步骤描述:		

	不使用返回值的函数'destroyStreamDataCache'应使用'void'说明.	
	代码片段: <pre> 268: code = taosHashPut(g_pDataSinkManager.dsStreamTaskList, key, strlen(key), ppCache, sizeof(void*)); 269: if (code != 0) { 270: destroyStreamDataCache(*ppCache); 271: stError("failed to put stream task data sink manager, err: 0x%0x", code); 272: return code; </pre>	
	污染路径: 函数'destroyStreamDataCache'的定义点.	
1241	缺陷发生位置: /source/util/src/thash.c	行号: 574
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 572: 573: taosArrayDestroy(pHashObj->pMemBlock); 574: taosMemoryFree(pHashObj); 575: } 576: </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
1242	缺陷发生位置: /test/new_test_framework/script/api/insertBlob.c	行号: 96
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 94: for (i = 0; i < 10; ++i) { 95: int32_t v = n * 10 + i; 96: sprintf(qstr, "insert into m1 values (%" PRIu64 ",'%s')", start, buf); 97: // printf("qstr: %s\n", qstr); 98: </pre>	
	污染路径: 无	
1243	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 2328
	步骤描述:	

	不使用返回值的函数'tstrncpy'应使用'void'说明.	
	代码片段: <pre> 2326: // name 2327: if (0 == strcmp(key, NAME_KEY)) { 2328: tstrncpy(recordSchema->name, json_string_value(value), RECORD_NAME_LEN - 1); 2329: // fields 2330: } else if (0 == strcmp(key, FIELDS_KEY)) { </pre>	
	污染路径: 函数'tstrncpy'的定义点.	
1244	缺陷发生位置: /contrib/TSZ/zstd/compress/zstd_compress.c	行号: 578
	步骤描述: 不使用返回值的函数'DEBUGLOG'应使用'void'说明.	
	代码片段: <pre> 576: ZSTD_CCtx* cctx, const ZSTD_CCtx_params* params) 577: { 578: DEBUGLOG(4, "ZSTD_CCtx_setParametersUsingCCtxParams"); 579: if (cctx->streamStage != zcss_init) return ERROR(stage_wrong); 580: if (cctx->cdict) return ERROR(stage_wrong); </pre>	
	污染路径: 函数'DEBUGLOG'的定义点.	
1245	缺陷发生位置: /source/os/src/osString.c	行号: 205
	步骤描述: 不使用返回值的函数'OS_PARAM_CHECK'应使用'void'说明.	
	代码片段: <pre> 203: } 204: int32_t taosStr2int16(const char *str, int16_t *val) { 205: OS_PARAM_CHECK(str); 206: OS_PARAM_CHECK(val); 207: int64_t tmp = 0; </pre>	
	污染路径: 函数'OS_PARAM_CHECK'的定义点.	
1246	缺陷发生位置: /source/dnode/mnode/impl/src/mndRetention.c	行号: 420
	步骤描述:	

	不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 418: 419: if ((code = mndTransAppendRedoAction(pTrans, &action)) != 0) { 420: taosMemoryFree(pReq); 421: TAOS_RETURN(code); 422: }</pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
1247	缺陷发生位置: /source/libs/tmqtt/mgmt/src/tmqttMgmt.c	行号: 184
	步骤描述: 不使用返回值的函数'bndInfo'应使用'void'说明.	
	代码片段: <pre> 182: bndError("can not spawn taosmqtt. path: %s, error: %s", path, uv_strerror(err)); 183: } else { 184: bndInfo("taosmqtt is initialized"); 185: } 186:</pre>	
	污染路径: 函数'bndInfo'的定义点.	
1248	缺陷发生位置: /source/libs/new-stream/src/dataSinkMgr.c	行号: 271
	步骤描述: 不使用返回值的函数'stError'应使用'void'说明.	
	代码片段: <pre> 269: if (code != 0) { 270: destroyStreamDataCache(*ppCache); 271: stError("failed to put stream task data sink manager, err: 0x%0x", code); 272: return code; 273: }</pre>	
	污染路径: 函数'stError'的定义点.	
1249	缺陷发生位置: /test/new_test_framework/script/api/stmt2-memleak.c	行号: 219
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	

	代码片段: 217: printf("all test finish\n"); 218: taos_close(taos); 219: taos_cleanup(); 220: }	
	污染路径: 函数'taos_cleanup'的定义点.	
1250	缺陷发生位置: /docs/examples/c/schemaless.c	行号: 17
	步骤描述: 不使用返回值的函数'gettimeofday'应使用'void'说明.	
	代码片段: 15: static int64_t getTimeInUs() { 16: struct timeval systemTime; 17: gettimeofday(&systemTime, NULL); 18: return (int64_t)systemTime.tv_sec * 1000000L + (int64_t)systemTime.tv_usec; 19: }	
	污染路径: 无	
1251	缺陷发生位置: /utils/test/c/tmq_token.c	行号: 146
	步骤描述: 不使用返回值的函数'taosSsleep'应使用'void'说明.	
	代码片段: 144: alter_token_enable(); 145: ASSERT(tsem2_post(&sem) == 0); 146: taosSsleep(5); 147: alter_token_disable(); 148: taosSsleep(5);	
	污染路径: 函数'taosSsleep'的定义点.	
1252	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 374
	步骤描述: 不使用返回值的函数'taosGetTimeOfDay'应使用'void'说明.	
	代码片段: 372: static int64_t taosGetTimestampMs() { 373: struct timeval systemTime; 374: taosGetTimeOfDay(&systemTime);	

	375: return (int64_t)systemTime.tv_sec * 1000LL + (int64_t)systemTime.tv_usec/1000; 376: }	
	污染路径: 函数'taosGetTimeOfDay'的定义点.	
1253	缺陷发生位置: /test/new_test_framework/script/api/stmtBatchTest.c	行号: 228
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 226: taosMemoryFree(v->nr); 227: taosMemoryFree(v); 228: taosMemoryFree(lb); 229: taosMemoryFree(params); 230: taosMemoryFree(is_null);	
	污染路径: 函数'taosMemoryFree'的定义点.	
1254	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmWorker.c	行号: 525
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: 523: 524: if ((code = tSingleWorkerInit(&pMgmt->mgmtMultiWorker, &multiMgmtCfg)) != 0) return code; 525: dDebug("vnode workers are initialized"); 526: return 0; 527: }	
	污染路径: 函数'dDebug'的定义点.	
1255	缺陷发生位置: /source/libs/transport/src/transComm.c	行号: 205
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 203: 204: if (decompLen != oriLen) { 205: taosMemoryFree(buf); 206: return TSDB_CODE_INVALID_MSG; 207: }	

	污染路径: 函数'taosMemoryFree'的定义点.	
1256	缺陷发生位置: /source/dnode/mnode/impl/src/mndDump.c	行号: 782
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 780: TdFilePtr pFile = taosOpenFile(file, TD_FILE_CREATE TD_FILE_WRITE TD_FILE_TRUNC TD_FILE_WRITE_THROUGH); 781: if (pFile == NULL) { 782: mError("failed to write %s since %s", file, terrstr()); 783: return terrno; 784: }	
	污染路径: 函数'mError'的定义点.	
1257	缺陷发生位置: /source/libs/monitorfw/src/taos_collector.c	行号: 102
	步骤描述: 不使用返回值的函数'TAOS_LOG'应使用'void'说明.	
	代码片段: 100: taos_collector_t *self = (taos_collector_t *)gen; 101: if (taos_collector_destroy(self) != 0) { 102: TAOS_LOG("taos_collector_destroy failed"); 103: } 104: }	
	污染路径: 函数'TAOS_LOG'的定义点.	
1258	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqProperty.c	行号: 1070
	步骤描述: 不使用返回值的函数'ttq_free'应使用'void'说明.	
	代码片段: 1068: if (!(*value)) { 1069: if (name) { 1070: ttq_free(*name); 1071: *name = NULL; 1072: }	
	污染路径: 函数'ttq_free'的定义点.	

1259	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmWorker.c	行号: 211
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 209: SRpcMsg rsp = {.info = pMsg->info, .code = terrno}; 210: if (rpcSendResponse(&rsp) != 0) { 211: dError("failed to send response since %s", terrstr()); 212: } 213: }	
	污染路径: 函数'dError'的定义点.	
1260	缺陷发生位置: /source/libs/parser/src/parser.c	行号: 200
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 198: t = tStrGetToken((char*)pSql, &index, false, NULL); 199: if (TK_SET != t.type) { 200: taosMemoryFree(newSql); 201: code = generateSyntaxErrMsgExt(&pMsgBuf, TSDB_CODE_PAR_SYNTAX_ERROR, "Expected SET keyword"); 202: return code;	
	污染路径: 函数'taosMemoryFree'的定义点.	
1261	缺陷发生位置: /source/libs/executor/src/countwindowoperator.c	行号: 65
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 63: cleanupExprSupp(&pInfo->scalarSup); 64: taosArrayDestroy(pInfo->countSup.pWinStates); 65: taosMemoryFreeClear(param); 66: } 67:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1262	缺陷发生位置: /source/libs/executor/src/streamexternalwindowoperator.c	行号: 284
	步骤描述:	

	不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 282: SMergeAlignedExternalWindowOperator* pMIExtInfo = (SMergeAlignedExternalWindowOperator*)pOperator; 283: destroyExternalWindowOperatorInfo(pMIExtInfo->pExtW); 284: taosMemoryFreeClear(pMIExtInfo); 285: } 286:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1263	缺陷发生位置: /source/util/src/tqueue.c	行号: 98
	步骤描述: 不使用返回值的函数'taosRemoveFromQset'应使用'void'说明.	
	代码片段: 96: 97: if (queue->qset) { 98: taosRemoveFromQset(qset, queue); 99: } 100:	
	污染路径: 函数'taosRemoveFromQset'的定义点.	
1264	缺陷发生位置: /utils/test/c/tmq_ts7662.c	行号: 33
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 31: char* result = tmq_get_json_meta(msg); 32: printf("meta result: %s\n", result); 33: ASSERT(strstr(result, "t2") == NULL); 34: tmq_free_json_meta(result); 35: }	
	污染路径: 函数'ASSERT'的定义点.	
1265	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmMgmt.c	行号: 126
	步骤描述: 不使用返回值的函数'rpcCleanup'应使用'void'说明.	
	代码片段:	

	124: 125: dmClearVars(pDnode); 126: rpcCleanup(); 127: streamCleanup(); 128: indexCleanup();	
	污染路径: 函数'rpcCleanup'的定义点.	
1266	缺陷发生位置: /source/libs/new-stream/src/streamCheckpoint.c	行号: 201
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 199: STREAM_CHECK_NULL_GOTO(*data, terrno); 200: memcpy(*data, plainContent, originalLen); 201: taosMemoryFree(plainContent); 202: 203: *dataLen = originalLen;	
	污染路径: 函数'taosMemoryFree'的定义点.	
1267	缺陷发生位置: /contrib/test/traft/cluster/node10002_restart.c	行号: 108
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 106: // for test: submit a value every second 107: while (1) { 108: sleep(1); 109: submitValue(); 110: }	
	污染路径: 无	
1268	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqHandleConnect.c	行号: 280
	步骤描述: 不使用返回值的函数'ttqKeepaliveAdd'应使用'void'说明.	
	代码片段: 278: auth_data_out = NULL; 279: 280: ttqKeepaliveAdd(context); 281:	

	282: tmqtt__set_state(context, ttq_cs_active);	
	污染路径: 函数'ttqKeepaliveAdd'的定义点.	
1269	缺陷发生位置: /contrib/test/traft/cluster/node10002.c	行号: 112
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 110: // for test: submit a value every second 111: while (1) { 112: sleep(1); 113: submitValue(); 114: }	
	污染路径: 无	
1270	缺陷发生位置: /source/libs/new-stream/src/streamMgmt.c	行号: 232
	步骤描述: 不使用返回值的函数'streamReleaseTask'应使用'void'说明.	
	代码片段: 230: 231: (void)atomic_add_fetch_32(&pStream->taskNum, 1); 232: streamReleaseTask(taskAddr); 233: } 234:	
	污染路径: 函数'streamReleaseTask'的定义点.	
1271	缺陷发生位置: /test/new_test_framework/script/api/apitest.c	行号: 79
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 77: taos_free_result(result); 78: // super tables subscription 79: usleep(1000000); 80: } 81:	
	污染路径: 无	

1272	缺陷发生位置： /source/dnode/mnode/sdb/src/sdb.c	行号： 122
	步骤描述： 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段： 120: (void)taosThreadMutexDestroy(&pSdb->filelock); 121: taosMemoryFree(pSdb); 122: mInfo("sdb is cleaned up"); 123: } 124:	
	污染路径： 函数'mInfo'的定义点.	
1273	缺陷发生位置： /utils/test/c/tmq_td38404.c	行号： 53
	步骤描述： 不使用返回值的函数'tmq_free_json_meta'应使用'void'说明.	
	代码片段： 51: printf("meta result: %s\n", result); 52: ASSERT(strcmp(result, result1) == 0 strcmp(result, result2) == 0); 53: tmq_free_json_meta(result); 54: } 55:	
	污染路径： 函数'tmq_free_json_meta'的定义点.	
1274	缺陷发生位置： /contrib/libmqtt/ttq/src/ttqNet2.c	行号： 445
	步骤描述： 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段： 443: } 444: #else 445: UNUSED(listener); 446: #endif 447: return TTQ_ERR_SUCCESS;	
	污染路径： 函数'UNUSED'的定义点.	
1275	缺陷发生位置： /source/dnode/mnode/impl/src/mndXnode.c	行号： 602
	步骤描述：	

	不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 600: } 601: 602: mError("xnode:%s, not found", url); 603: terrno = TSDB_CODE_MND_XNODE_NOT_EXIST; 604: return NULL;	
	污染路径: 函数'mError'的定义点.	
1276	缺陷发生位置: /source/dnode/vnode/src/bse/bseSnapshot.c	行号: 299
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 297: SBseSnapReader *pReader = *p; 298: bsalterDestroy(pReader->plter); 299: taosMemoryFree(pReader); 300: 301: *p = NULL;	
	污染路径: 函数'taosMemoryFree'的定义点.	
1277	缺陷发生位置: /test/new_test_framework/script/api/stmt_function.c	行号: 74
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 72: // stmt = taos_stmt_init(taos); 73: assert(stmt != NULL); 74: sprintf(stmt_sql, "select from ?"); 75: assert(taos_stmt_prepare(stmt, stmt_sql, 0) != 0); 76: assert(taos_stmt_close(stmt) == 0);	
	污染路径: 无	
1278	缺陷发生位置: /source/libs/sync/src/syncSnapshot.c	行号: 809
	步骤描述: 不使用返回值的函数'sError'应使用'void'说明.	
	代码片段: 807: } else { // order > 0 808: // ignore	

	809: sError("failed to prepare snapshot, received a stale snapshot preparation. ignore."); 810: code = TSDB_CODE_SYN_MISMATCHED_SIGNATURE; 811: goto _SEND_REPLY;	
	污染路径: 函数'sError'的定义点.	
1279	缺陷发生位置: /utils/test/c/get_db_name_test.c	行号: 34
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 32: int code = taos_errno(pRes); 33: taos_free_result(pRes); 34: ASSERT(code == 0); 35: 36: code = taos_get_current_db(taos, NULL, 0, NULL);	
	污染路径: 函数'ASSERT'的定义点.	
1280	缺陷发生位置: /source/dnode/mnode/impl/src/mndPerfSchema.c	行号: 109
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 107: STableMetaRsp *pMeta = taosHashGet(pMnode->perfsMeta, tbName, strlen(tbName)); 108: if (NULL == pMeta) { 109: mError("invalid performance schema table name:%s", tbName); 110: code = TSDB_CODE_PAR_TABLE_NOT_EXIST; 111: TAOS_RETURN(code);	
	污染路径: 函数'mError'的定义点.	
1281	缺陷发生位置: /source/dnode/vnode/src/tq/tqRead.c	行号: 1136
	步骤描述: 不使用返回值的函数'TQ_NULL_GO_TO_END'应使用'void'说明.	
	代码片段: 1134: SSchemaWrapper* pSchemaWrapper = pReader->pSchemaWrapper; 1135: char* assigned = taosMemoryCalloc(1,	

	pSchemaWrapper->nCols); 1136: TQ_NULL_GO_TO_END(assigned); 1137: 1138: SArray* pCols = pSubmitTbData->aCol;	
	污染路径: 函数'TQ_NULL_GO_TO_END'的定义点.	
1282	缺陷发生位置: /tools/taos-tools/src/benchCsv.c	行号: 219
	步骤描述: 不使用返回值的函数'localtime_r'应使用'void'说明.	
	代码片段: 217: localtime_s(&tm_result, &time_value); 218: #else 219: localtime_r(&time_value, &tm_result); 220: #endif 221: strftime(time_buf, buf_size, ts_format, &tm_result);	
	污染路径: 无	
1283	缺陷发生位置: /docs/examples/c-ws-new/create_db_demo.c	行号: 61
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 59: 60: taos_close(taos); 61: taos_cleanup(); 62: return 0; 63: // ANCHOR_END: create_db_and_table	
	污染路径: 函数'taos_cleanup'的定义点.	
1284	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSetRW.c	行号: 229
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 227: tDestroyTSchema(writer[0]->skmRow->pTSchema); 228: tDestroyTSchema(writer[0]->skmTb->pTSchema); 229: taosMemoryFree(writer[0]); 230: writer[0] = NULL; 231:	

	污染路径: 函数'taosMemoryFree'的定义点.	
1285	缺陷发生位置: /source/libs/transport/test/svrBench.c	行号: 74
	步骤描述: 不使用返回值的函数'tDebug'应使用'void'说明.	
	代码片段: 72: while (1) { 73: int numOfMsgs = taosReadAllQitemsFromQset(multiQ->qset[idx], qall, &qinfo); 74: tDebug("%d shell msgs are received", numOfMsgs); 75: if (numOfMsgs <= 0) break; 76:	
	污染路径: 函数'tDebug'的定义点.	
1286	缺陷发生位置: /source/util/src/tcache.c	行号: 222
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 220: static FORCE_INLINE void taosCacheReleaseNode(SCacheObj *pCacheObj, SCacheNode *pNode) { 221: if (pNode->signature != pNode) { 222: uError("key:%s, %p data is invalid, or has been released", pNode->key, pNode); 223: return; 224: }	
	污染路径: 函数'uError'的定义点.	
1287	缺陷发生位置: /docs/examples/c-ws-new/create_db_demo.c	行号: 28
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 26: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x %x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 27: taos_errstr(NULL)); 28: taos_cleanup(); 29: return -1;	

	30: }	
	污染路径: 函数'taos_cleanup'的定义点.	
1288	缺陷发生位置: /contrib/test/rocksdb/main.c	行号: 220
	步骤描述: 不使用返回值的函数'kvDeserial'应使用'void'说明.	
	代码片段: 218: const char *value = rocksdb_iter_value(iter, &vlen); 219: KV kv; 220: kvDeserial(&kv, (char *)key); 221: printf("kv1: %d\t kv2: %d, len:%d, value = %s\n", (int)(kv.k1), (int)(kv.k2), (int)(klen), value); 222: }	
	污染路径: 函数'kvDeserial'的定义点.	
1289	缺陷发生位置: /source/dnode/vnode/src/meta/metaSnapshot.c	行号: 266
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 264: static void destroySTableInfoForChildTable(void* data) { 265: STableInfoForChildTable* pData = (STableInfoForChildTable*)data; 266: taosMemoryFree(pData->tableName); 267: tDeleteSchemaWrapper(pData->schemaRow); 268: tDeleteSchemaWrapper(pData->tagRow);	
	污染路径: 函数'taosMemoryFree'的定义点.	
1290	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smlnt.c	行号: 88
	步骤描述: 不使用返回值的函数'dlInfo'应使用'void'说明.	
	代码片段: 86: } 87: 88: dlInfo("snode checkpoints not encrypted but tsMetaKey available, starting migration"); 89:	

	90: // List all checkpoint files and re-encrypt them	
	污染路径: 函数'dInfo'的定义点.	
1291	缺陷发生位置: /source/libs/scalar/src/filter.c	行号: 4320
	步骤描述: 不使用返回值的函数'FLT_RET'应使用'void'说明.	
	代码片段: 4318: } 4319: } 4320: FLT_RET(0); 4321: } 4322:	
	污染路径: 函数'FLT_RET'的定义点.	
1292	缺陷发生位置: /source/libs/executor/src/executor.c	行号: 80
	步骤描述: 不使用返回值的函数'taosCloseRef'应使用'void'说明.	
	代码片段: 78: static void cleanupRefPool() { 79: int32_t ref = atomic_val_compare_exchange_32(&fetchObjRefPool, fetchObjRefPool, 0); 80: taosCloseRef(ref); 81: } 82:	
	污染路径: 函数'taosCloseRef'的定义点.	
1293	缺陷发生位置: /utils/test/c/tmq_ts6115.c	行号: 152
	步骤描述: 不使用返回值的函数'create_topic'应使用'void'说明.	
	代码片段: 150: } 151: 152: create_topic(); 153: 154: for (int i = 0; i < runTimes; i++){	

	污染路径: 函数'create_topic'的定义点.	
1294	缺陷发生位置: /source/common/src/tdatablock.c	行号: 3155
	步骤描述: 不使用返回值的函数'QRY_PARAM_CHECK'应使用'void'说明.	
	代码片段: 3153: 3154: int32_t buildCtbNameByGroupId(const char* stbFullName, uint64_t groupId, char** pName) { 3155: QRY_PARAM_CHECK(pName); 3156: 3157: char* pBuf = taosMemoryCalloc(1, TSDB_TABLE_NAME_LEN + 1);	
	污染路径: 函数'QRY_PARAM_CHECK'的定义点.	
1295	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFS2.c	行号: 134
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 132: json[0] = NULL; 133: } 134: taosMemoryFree(data); 135: return code; 136: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1296	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmNodes.c	行号: 165
	步骤描述: 不使用返回值的函数'dlInfo'应使用'void'说明.	
	代码片段: 163: while (1) { 164: if (pDnode->stop) { 165: dlInfo("The daemon is about to stop"); 166: dmSetStatus(pDnode, DND_STAT_STOPPED); 167: dmStopNodes(pDnode);	
	污染路径: 函数'dlInfo'的定义点.	

1297	缺陷发生位置: /source/util/src/ttimer.c	行号: 658
	步骤描述: 不使用返回值的函数'taosCleanUpScheduler'应使用'void'说明.	
	代码片段: 656: taosUninitTimer(); 657: 658: taosCleanUpScheduler(tmrQhandle); 659: taosMemoryFreeClear(tmrQhandle); 660:	
	污染路径: 函数'taosCleanUpScheduler'的定义点.	
1298	缺陷发生位置: /source/client/test/smlTest.cpp	行号: 278
	步骤描述: 不使用返回值的函数'operator='应使用'void'说明.	
	代码片段: 276: info->dataFormat = false; 277: SSmlLineInfo elements = {0}; 278: info->msgBuf = msgBuf; 279: ASSERT_EQ(smlInitHandle(NULL), TSDB_CODE_INVALID_PARA); 280:	
	污染路径: 函数'operator='的定义点.	
1299	缺陷发生位置: /source/libs/parser/src/parUtil.c	行号: 684
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 682: 683: if (needFree) { 684: taosMemoryFree((char*)valueStr); 685: } 686:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1300	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 4030
	步骤描述:	

	不使用返回值的函数'parserError'应使用'void'说明.	
	代码片段: 4028: _return: 4029: pCxt->errCode = code; 4030: parserError("translatePlaceHolderFunc failed with code %d", code); 4031: nodesDestroyNode(extraValue); 4032: return DEAL_RES_ERROR;	
	污染路径: 函数'parserError'的定义点.	
1301	缺陷发生位置: /test/new_test_framework/script/api/stmt2-performance.c	行号: 122
	步骤描述: 不使用返回值的函数'clock_gettime'应使用'void'说明.	
	代码片段: 120: // bind 121: struct timespec start, end; 122: clock_gettime(CLOCK_MONOTONIC, &start); // 获取开始时间 123: TAOS_STMT2_BINDV bindv; 124: if (hasTag) {	
	污染路径: 无	
1302	缺陷发生位置: /tools/taos-tools/src/benchMain.c	行号: 251
	步骤描述: 不使用返回值的函数'pthread_join'应使用'void'说明.	
	代码片段: 249: #ifdef LINUX 250: pthread_cancel(spId); 251: pthread_join(spId, NULL); 252: #endif 253:	
	污染路径: 无	
1303	缺陷发生位置: /test/new_test_framework/script/api/stopquery.c	行号: 228
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段:	

	226: if (taos == NULL) sqExit("taos_connect", taos_errstr(NULL)); 227: 228: sprintf(sql, "reset query cache"); 229: sqExecSQL(taos, sql); 230:	
	污染路径: 无	
1304	缺陷发生位置: /source/client/src/clientRawBlockWrite.c	行号: 264
	步骤描述: 不使用返回值的函数'uDebug'应使用'void'说明.	
	代码片段: 262: int32_t code = 0; 263: int32_t lino = 0; 264: RAW_LOG_START 265: RAW_RETURN_CHECK(tDeserializeSMAAlterStbReq(alterData, alterDataLen, &req)); 266: json = cJSON_CreateObject();	
	污染路径: expanded from macro 'RAW_LOG_START' 函数'uDebug'的定义点. expanded from macro 'RAW_LOG_START'	
1305	缺陷发生位置: /source/common/src/cos_cp.c	行号: 195
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 193: } 194: if (cp_body) { 195: taosMemoryFree(cp_body); 196: } 197: TAOS_RETURN(code);	
	污染路径: 函数'taosMemoryFree'的定义点.	
1306	缺陷发生位置: /source/libs/sync/src/syncEnv.c	行号: 49
	步骤描述: 不使用返回值的函数'syncCleanUp'应使用'void'说明.	

	代码片段: <pre> 47: if (gHbDataRefId < 0) { 48: sError("failed to init hbdata rset"); 49: syncCleanUp(); 50: return TSDB_CODE_SYN_WRONG_REF; 51: } </pre>	
	污染路径: 函数'syncCleanUp'的定义点.	
1307	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmTransport.c	行号: 630
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: <pre> 628: } 629: 630: dDebug("dnode rpc sync client is initialized"); 631: return 0; 632: } </pre>	
	污染路径: 函数'dDebug'的定义点.	
1308	缺陷发生位置: /source/common/src/cos_cp.c	行号: 227
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 225: uError("%s failed at line %d since %s", __func__, lino, tsterror(code)); 226: } 227: taosMemoryFree(data); 228: TAOS_RETURN(code); 229: } </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
1309	缺陷发生位置: /source/client/test/clientTests.cpp	行号: 1935
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 1933: for (int32_t i = 0; i < 10; i++) { 1934: char tname[24] = {0}; </pre>	

	220: dataPos += 4; 221: sdbSetRawInt32(pRaw, dataPos, obj.statusLen); 222: dataPos += 4; 223: sdbSetRawInt64(pRaw, dataPos, obj.createTime);	
	污染路径: 函数'sdbSetRawInt32'的定义点.	
1313	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncBatch.c	行号: 179
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 177: char* serialized = syncClientRequestBatch2Str(pMsg); 178: sTrace("syncClientRequestBatchLog len:%d %s", (int32_t)strlen(serialized), serialized); 179: taosMemoryFree(serialized); 180: } 181:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1314	缺陷发生位置: /utlis/tsim/src/simSystem.c	行号: 93
	步骤描述: 不使用返回值的函数'simInfo'应使用'void'说明.	
	代码片段: 91: 92: if (simScriptPos == -1 && simExecSuccess) { 93: simInfo("-----"); 94: simInfo("Simulation Test Done, " SUCCESS_PREFIX "%d" SUCCESS_POSTFIX " Passed:\n", simScriptSucceed); 95: return NULL;	
	污染路径: 函数'simInfo'的定义点.	
1315	缺陷发生位置: /source/libs/tss/src/tss.c	行号: 180
	步骤描述: 不使用返回值的函数'tssInfo'应使用'void'说明.	
	代码片段: 178: 179: if (strlen(tsSsAccessString) == 0) {	

	180: tssInfo("access string is empty, default shared storage is disabled\n"); 181: return 0; 182: }	
	污染路径: 函数'tssInfo'的定义点.	
1316	缺陷发生位置: /utlis/test/c/tmq_batch_alter_tag.c	行号: 39
	步骤描述: 不使用返回值的函数'tmq_free_json_meta'应使用'void'说明.	
	代码片段: 37: printf("meta result: %s\n", result); 38: strncat(json_get, result, sizeof(json_get) - strlen(json_get) - 1); 39: tmq_free_json_meta(result); 40: } 41:	
	污染路径: 函数'tmq_free_json_meta'的定义点.	
1317	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 1112
	步骤描述: 不使用返回值的函数'pthread_create'应使用'void'说明.	
	代码片段: 1110: debugPrint("div table by thread. i=%d from= %"PRId64" to=%"PRId64" ntable=%"PRId64"\n", i, pThreadInfo->start_table_from, 1111: pThreadInfo->end_table_to, pThreadInfo->ntables); 1112: pthread_create(pids + i, NULL, createTable, pThreadInfo); 1113: threadCnt ++; 1114: }	
	污染路径: 无	
1318	缺陷发生位置: /docs/examples/c-ws-new/create_db_demo.c	行号: 54
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段:	

	52: taos_errstr(result)); 53: taos_close(taos); 54: taos_cleanup(); 55: return -1; 56: }	
	污染路径: 函数'taos_cleanup'的定义点.	
1319	缺陷发生位置: /source/os/src/osDir.c	行号: 128
	步骤描述: 不使用返回值的函数'TAOS_UNUSED'应使用'void'说明.	
	代码片段: 126: 127: TAOS_UNUSED(taosCloseDir(&pDir)); 128: TAOS_UNUSED(rmdir(dirname)); 129: 130: // printf("dir:%s is removed\n", dirname);	
	污染路径: 函数'TAOS_UNUSED'的定义点.	
1320	缺陷发生位置: /source/libs/executor/src/executorInt.c	行号: 649
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 647: code = trimDataBlock(pBlock, pBlock->info.rows, (bool*)pIndicator); 648: } else { 649: qError("unknown filter result type: %d", status); 650: } 651: return code;	
	污染路径: 函数'qError'的定义点.	
1321	缺陷发生位置: /docs/examples/c/stmt_insert_demo.c	行号: 88
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 86: sprintf(table_name, "d_bind_%d", i); 87: char location[20]; 88: sprintf(location, "location_%d", i);	

	89: 90: // set table name and tags	
	污染路径: 无	
1322	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeCommit.c	行号: 242
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 240: tstrerror(code), fname); 241: } 242: taosMemoryFree(pData); 243: return code; 244: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1323	缺陷发生位置: /source/client/src/clientMonitorSql.c	行号: 64
	步骤描述: 不使用返回值的函数'tscLog'应使用'void'说明.	
	代码片段: 62: if (pTscObj != NULL) { 63: if (pTscObj->pAppInfo == NULL) { 64: tscLog("sqlReqLog, not found pAppInfo"); 65: } else { 66: clientSQLReqLog(pTscObj->pAppInfo->clusterId, pTscObj->user, result, type);	
	污染路径: 函数'tscLog'的定义点.	
1324	缺陷发生位置: /utls/test/c/tmq_td33798.c	行号: 300
	步骤描述: 不使用返回值的函数'create_topic'应使用'void'说明.	
	代码片段: 298: return -1; 299: } 300: create_topic(); 301: 302: tmq_list_t* topic_list = build_topic_list();	

	污染路径: 函数'create_topic'的定义点.	
1325	缺陷发生位置: /source/libs/catalog/src/catalog.c	行号: 810
	步骤描述: 不使用返回值的函数'qTrace'应使用'void'说明.	
	代码片段: <pre> 808: } 809: 810: qTrace("reset catalog timer"); 811: if (taosTmrReset(ctgProcessTimerEvent, CTG_DEFAULT_CACHE_MON_MSEC, NULL, gCtgMgmt.timer, &gCtgMgmt.cacheTimer)) { 812: qError("reset catalog cache monitor timer error, timer stopped"); </pre>	
	污染路径: 函数'qTrace'的定义点.	
1326	缺陷发生位置: /source/libs/executor/src/dataDispatcher.c	行号: 125
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: <pre> 123: } 124: if (pInput == NULL pInput->pData == NULL pInput- >pData->info.rows <= 0) { 125: qError("invalid input data"); 126: return TSDB_CODE_QRY_INVALID_INPUT; 127: } </pre>	
	污染路径: 函数'qError'的定义点.	
1327	缺陷发生位置: /source/dnode/mnode/impl/src/mndSma.c	行号: 494
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 492: 493: if ((code = mndTransAppendRedoAction(pTrans, &action)) != 0) { 494: taosMemoryFree(pReq); 495: TAOS_RETURN(code); 496: } </pre>	

	污染路径: 函数'taosMemoryFree'的定义点.	
1328	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCacheRead.c	行号: 400
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 398: } 399: 400: taosMemoryFree((void*)p->idstr); 401: (void)taosThreadMutexDestroy(&p->readerMutex); 402:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1329	缺陷发生位置: /source/dnode/mnode/impl/src/mndTopic.c	行号: 942
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 940: STR_TO_VARSTR(schemaJson, "NULL"); 941: } 942: taosMemoryFree(string); 943: } 944:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1330	缺陷发生位置: /utils/test/c/tmq_taosx_ci.c	行号: 95
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 93: int32_t ret = tmq_write_raw(pConn, raw); 94: printf("write raw data: %s\n", tmq_err2str(ret)); 95: ASSERT(ret == 0); 96: 97: tmq_free_raw(raw);	
	污染路径: 函数'ASSERT'的定义点.	
1331	缺陷发生位置:	行号:

	/source/client/src/clientMonitorSlow.c	68
	步骤描述: 不使用返回值的函数'tscLog'应使用'void'说明.	
	代码片段: <pre> 66: if (pTscObj != NULL) { 67: if(pTscObj->pApplInfo == NULL) { 68: tscLog("slowQueryLog, not found pApplInfo"); 69: } else { 70: clientSlowQueryLog(pTscObj->pApplInfo->clusterId, pTscObj->user, result, cost); </pre>	
	污染路径: 函数'tscLog'的定义点.	
1332	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1949
	步骤描述: 不使用返回值的函数'HUF_decodeStreamX2'应使用'void'说明.	
	代码片段: <pre> 1947: 1948: /* finish bitStreams one by one */ 1949: HUF_decodeStreamX2(op1, &bitD1, opStart2, dt, dtLog); 1950: HUF_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 1951: HUF_decodeStreamX2(op3, &bitD3, opStart4, dt, dtLog); </pre>	
	污染路径: 函数'HUF_decodeStreamX2'的定义点.	
1333	缺陷发生位置: /source/client/src/clientMonitor.c	行号: 39
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: <pre> 37: int ret = tsnprintf(tmpPath, size, "%s %stdengine_slow_log%s", tsTempDir, hasDelim ? "" : TD_DIRSEP, TD_DIRSEP); 38: if (ret < 0) { 39: tscError("failed to get tmp path ret:%d", ret); 40: return TSDB_CODE_TSC_INTERNAL_ERROR; 41: } </pre>	
	污染路径:	

	函数'tscError'的定义点.	
1334	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttSocket.c	行号: 155
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 153: struct timeval tv; 154: 155: UNUSED(gettimeofday(&tv, NULL)); 156: srand((unsigned int)(tv.tv_sec + tv.tv_usec)); 157: }	
	污染路径: 函数'UNUSED'的定义点.	
1335	缺陷发生位置: /docs/examples/c/insert_example.c	行号: 44
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 42: "d1004 USING meters TAGS('California.LosAngeles', 3) VALUES ('2018-10-03 14:38:05.000', 10.80000, 223, 0.29000) ('2018-10-03 14:38:06.500', 11.50000, 221, 0.35000)"); 43: taos_close(taos); 44: taos_cleanup(); 45: } 46:	
	污染路径: 函数'taos_cleanup'的定义点.	
1336	缺陷发生位置: /source/dnode/vnode/src/tq/tqScan.c	行号: 311
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 309: tqError("%s failed at %d, vgId:%d, task exec error since %s", __FUNCTION__ , lino, pTq->pVnode->config.vgId, tstrerror(code)); 310: } 311: taosMemoryFree(tbName); 312: return code; 313: }	

	污染路径: 函数'taosMemoryFree'的定义点.	
1337	缺陷发生位置: /source/dnode/mnode/impl/src/mndConsumer.c	行号: 431
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: <pre> 429: void *abuf = POINTER_SHIFT(buf, sizeof(SMqRspHead)); 430: if (tEncodeSMqAskEpRsp(&abuf, rsp) < 0) { 431: rpcFreeCont(buf); 432: return TSDB_CODE_TSC_INTERNAL_ERROR; 433: }</pre>	
	污染路径: 函数'rpcFreeCont'的定义点.	
1338	缺陷发生位置: /docs/examples/c-ws-new/tmq_demo.c	行号: 50
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 48: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x 49: taos_errstr(NULL)); 50: taos_cleanup(); 51: return NULL; 52: }</pre>	
	污染路径: 函数'taos_cleanup'的定义点.	
1339	缺陷发生位置: /contrib/test/tdev/src/main.c	行号: 106
	步骤描述: 不使用返回值的函数'gettimeofday'应使用'void'说明.	
	代码片段: <pre> 104: static uint64_t now() { 105: struct timeval tv; 106: gettimeofday(&tv, NULL); 107: 108: return tv.tv_sec * 1000000 + tv.tv_usec;</pre>	
	污染路径: 无	
1340	缺陷发生位置:	行号:

	/source/libs/executor/src/groupoperator.c	1243
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 1241: 1242: taosArrayDestroy(pInfo->pGroupColVals); 1243: taosMemoryFree(pInfo->keyBuf); 1244: 1245: int32_t size = taosArrayGetSize(pInfo->sortedGroupArray);	
1341	污染路径: 函数'taosMemoryFree'的定义点.	
	缺陷发生位置: /contrib/TSZ/sz/src/sz.c	行号: 55
	步骤描述: 不使用返回值的函数'SZ_Init'应使用'void'说明.	
	代码片段: 53: int SZ_Init_Params(sz_params *params) 54: { 55: SZ_Init(NULL); 56: 57: if(params->losslessCompressor!=GZIP_COMPRESSOR && params->losslessCompressor!=ZSTD_COMPRESSOR)	
1342	污染路径: 函数'SZ_Init'的定义点.	
	缺陷发生位置: /source/libs/index/src/indexCache.c	行号: 250
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 248: 249: taosMemoryFree(pCt); 250: taosMemoryFree(exBuf); 251: TAOS_UNUSED(tSkipListDestroyIter(iter)); 252: return code;	
1343	污染路径: 函数'taosMemoryFree'的定义点.	
	缺陷发生位置: /source/libs/tss/test/tssTest.cpp	行号: 120
步骤描述:		

	不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: <pre> 118: for(int i = 0; i < 10; i++) { 119: char path[64]; 120: snprintf(path, sizeof(path), "test/test%d.txt", i); 121: code = tssUploadFile(ss, path, "/tmp/ss_test.txt", 0, 0); 122: GTEST_ASSERT_EQ(code, TSDB_CODE_SUCCESS); </pre>	
	污染路径: 无	
1344	缺陷发生位置: /source/common/src/msg/xnode.c	行号: 189
	步骤描述: 不使用返回值的函数'DECODESQL'应使用'void'说明.	
	代码片段: <pre> 187: 188: TAOS_CHECK_EXIT(tStartDecode(&decoder)); 189: DECODESQL(); 190: 191: TAOS_CHECK_EXIT(tDecodeI32(&decoder, &pReq- >urlLen)); </pre>	
	污染路径: 函数'DECODESQL'的定义点.	
1345	缺陷发生位置: /docs/examples/c/sml_insert_demo.c	行号: 37
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 35: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x %x, ErrorMessage: %s.\n", host, port, taos_erno(NULL), 36: taos_errstr(NULL)); 37: taos_cleanup(); 38: return -1; 39: } </pre>	
	污染路径: 函数'taos_cleanup'的定义点.	
1346	缺陷发生位置: /source/dnode/mnode/impl/src/mndConfig.c	行号: 290
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	

	代码片段: 288: contLen = tSerializeSConfigRsp(pHead, contLen, &configRsp); 289: if (contLen < 0) { 290: rpcFreeCont(pHead); 291: code = contLen; 292: goto _OVER;	
	污染路径: 函数'rpcFreeCont'的定义点.	
1347	缺陷发生位置: /source/common/src/tvariant.c	行号: 162
	步骤描述: 不使用返回值的函数'SET_ERRNO'应使用'void'说明.	
	代码片段: 160: 161: int32_t toIntegerEx(const char *z, int32_t n, uint32_t type, int64_t *value) { 162: SET_ERRNO(0); 163: char *endPtr = NULL; 164: switch (type) {	
	污染路径: 函数'SET_ERRNO'的定义点.	
1348	缺陷发生位置: /source/dnode/vnode/src/tq/tqUtil.c	行号: 154
	步骤描述: 不使用返回值的函数'tqDebug'应使用'void'说明.	
	代码片段: 152: int32_t code = TDB_CODE_SUCCESS; 153: int32_t lino = 0; 154: tqDebug("%s called", __FUNCTION__); 155: uint64_t consumerId = pRequest->consumerId; 156: int32_t vgId = TD_VID(pTq->pVnode);	
	污染路径: 函数'tqDebug'的定义点.	
1349	缺陷发生位置: /source/libs/qworker/src/qworker.c	行号: 436
	步骤描述: 不使用返回值的函数'QW_SINK_DISABLE_MEMPOOL'应使用'void'说明.	
	代码片段: 434: QW_SINK_ENABLE_MEMPOOL(ctx);	

	435: code = dsGetDataBlock(ctx->sinkHandle, &output); 436: QW_SINK_DISABLE_MEMPOOL(); 437: 438: if (code) {	
	污染路径: 函数'QW_SINK_DISABLE_MEMPOOL'的定义点.	
1350	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 929
	步骤描述: 不使用返回值的函数'sigaction'应使用'void'说明.	
	代码片段: 927: act.sa_flags = SA_SIGINFO SA_RESTART; 928: act.sa_sigaction = (void (*)(int, siginfo_t *, void *)) sigfp; 929: sigaction(signum, &act, NULL); 930: } 931: #endif	
	污染路径: 无	
1351	缺陷发生位置: /source/libs/qworker/src/qwMsg.c	行号: 52
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: 50: 51: if (NULL == ctx !ctx->localExec) { 52: rpcFreeCont(msg); 53: } 54: }	
	污染路径: 函数'rpcFreeCont'的定义点.	
1352	缺陷发生位置: /source/libs/nodes/src/nodesUtilFuncs.c	行号: 188
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 186: char* ptr = (char*)p - NODE_ALLOCATOR_HEAD_LEN; 187: if (0 == *ptr) { 188: taosMemoryFree(ptr); 189: }	

	190: return;	
	污染路径: 函数'taosMemoryFree'的定义点.	
1353	缺陷发生位置: /source/libs/decimal/src/detail/wideInteger.cpp	行号: 31
	步骤描述: 不使用返回值的函数'operator+='应使用'void'说明.	
	代码片段: 29: intx::uint128 *pX = (intx::uint128*)pLeft; 30: const intx::uint128 *pY = (const intx::uint128*)pRight; 31: *pX += *pY; 32: } 33: void uInt128Subtract(UInt128* pLeft, const UInt128* pRight) {	
	污染路径: 函数'operator+='的定义点.	
1354	缺陷发生位置: /source/libs/new-stream/src/dataSinkFile.c	行号: 223
	步骤描述: 不使用返回值的函数'QUERY_CHECK_CODE'应使用'void'说明.	
	代码片段: 221: if (buf == NULL) { 222: code = terrno; 223: QUERY_CHECK_CODE(code, lino, _exit); 224: } 225:	
	污染路径: 函数'QUERY_CHECK_CODE'的定义点.	
1355	缺陷发生位置: /source/dnode/mnode/impl/src/mndSubscribe.c	行号: 638
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 636: *remainderVgCnt = totalVgNum % numOfFinal; 637: } else { 638: mInfo("tmq rebalance sub:%s no consumer subscribe this topic", pSubKey); 639: } 640: mInfo(	

	污染路径: 函数'mInfo'的定义点.	
1356	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 630
	步骤描述: 不使用返回值的函数'geneDbCreateCmd'应使用'void'说明.	
	代码片段: 628: // create database 629: int remainVnodes = INT_MAX; 630: geneDbCreateCmd(database, command, remainVnodes); 631: code = postProcessSql(command, 632: database->dbName,	
	污染路径: 函数'geneDbCreateCmd'的定义点.	
1357	缺陷发生位置: /source/client/src/clientSml.c	行号: 192
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 190: END: 191: taosMemoryFree(tag); 192: uError("%s failed code:%d line:%d", __FUNCTION__, code, lino); 193: return code; 194: }	
	污染路径: 函数'uError'的定义点.	
1358	缺陷发生位置: /tools/taos-tools/src/benchMain.c	行号: 215
	步骤描述: 不使用返回值的函数'pthread_create'应使用'void'说明.	
	代码片段: 213: } 214: pthread_t spid = {0}; 215: pthread_create(&spid, NULL, benchCancelHandler, NULL); 216: benchSetSignal(SIGINT, benchQueryInterruptHandler); 217: #endif	
	污染路径:	

	无	
1359	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 437
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 435: char guoUtf8[32] = {0}; 436: char zhongguoGbk[32] = {0}; 437: snprintf(fenUtf8, sizeof(fenUtf8), "%c%c%c", 0xE8, 0x8A, 0xAC); 438: snprintf(fenGbk, sizeof(fenGbk), "%c%c", 0xB7, 0xD2); 439: snprintf(zhongguoUtf8, sizeof(zhongguoUtf8), "%c%c%c%c", 0xE4, 0xB8, 0xAD, 0xE5, 0x9B, 0xBD);	
	污染路径: 无	
1360	缺陷发生位置: /source/libs/qworker/src/qwDbg.c	行号: 316
	步骤描述: 不使用返回值的函数'taosSsleep'应使用'void'说明.	
	代码片段: 314: *rsped = true; 315: 316: taoSsleep(3); 317: return; 318: }	
	污染路径: 函数'taosSsleep'的定义点.	
1361	缺陷发生位置: /source/common/src/tdatablock.c	行号: 3164
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 3162: int32_t code = buildCtbNameByGroupIdImpl(stbFullName, groupId, pBuf); 3163: if (code != TSDB_CODE_SUCCESS) { 3164: taosMemoryFree(pBuf); 3165: } else { 3166: *pName = pBuf;	
	污染路径: 函数'taosMemoryFree'的定义点.	

1362	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReaderWriter.c	行号: 823
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 821: *ppReader = NULL; 822: TSDB_ERROR_LOG(TD_VID(pTsdb->pVnode), lino, code); 823: taosMemoryFree(pDelFReader); 824: } else { 825: *ppReader = pDelFReader;	
	污染路径: 函数'taosMemoryFree'的定义点.	
1363	缺陷发生位置: /source/dnode/qnode/src/qnode.c	行号: 26
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 24: *pQnode = taosMemoryCalloc(1, sizeof(SQnode)); 25: if (NULL == *pQnode) { 26: qError("calloc SQnode failed"); 27: return terrno; 28: }	
	污染路径: 函数'qError'的定义点.	
1364	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftEntryDebug.c	行号: 65
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 63: printf("syncEntryPrint len:%zu %s \n", strlen(serialized), serialized); 64: fflush(NULL); 65: taosMemoryFree(serialized); 66: } 67:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1365	缺陷发生位置: /source/common/src/tvariant.c	行号: 67

	步骤描述: 不使用返回值的函数'SET_ERRNO'应使用'void'说明.	
	代码片段: 65: } 66: 67: SET_ERRNO(0); 68: char *endPtr = NULL; 69: if (z[0] == '0' && n > 2) {	
	污染路径: 函数'SET_ERRNO'的定义点.	
1366	缺陷发生位置: /test/new_test_framework/script/api/last-query-ws.cpp	行号: 78
	步骤描述: 不使用返回值的函数'operator<<'应使用'void'说明.	
	代码片段: 76: code = ws_errno(NULL); 77: const char* errstr = ws_errstr(NULL); 78: std::cout << "Connection failed[" << code << "]: " << errstr << "\n"; 79: return; 80: }	
	污染路径: 函数'operator<<'的定义点.	
1367	缺陷发生位置: /source/libs/tss/src/fs.c	行号: 72
	步骤描述: 不使用返回值的函数'tssError'应使用'void'说明.	
	代码片段: 70: char* p = strchr(ss->buf, ':'); 71: if (p == NULL) { 72: tssError("invalid access string: %s", as); 73: return false; 74: }	
	污染路径: 函数'tssError'的定义点.	
1368	缺陷发生位置: /source/dnode/mgmt/exe/dmMain.c	行号: 160
	步骤描述: 不使用返回值的函数'dWarn'应使用'void'说明.	
	代码片段:	

	158: #ifndef TD_ASTR 159: if (taosIgnSignal(SIGTERM) != 0) { 160: dWarn("failed to ignore signal SIGTERM"); 161: } 162: if (taosIgnSignal(SIGHUP) != 0) {	
	污染路径: 函数'dWarn'的定义点.	
1369	缺陷发生位置: /source/libs/index/src/indexFst.c	行号: 316
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 314: } 315: TAOS_UNUSED(idxFileWrite(w, (char*)index, 256)); 316: taosMemoryFree(index); 317: } 318: TAOS_UNUSED(idxFileWrite(w, (char*)&packSizes, 1));	
	污染路径: 函数'taosMemoryFree'的定义点.	
1370	缺陷发生位置: /source/dnode/mgmt/mgmt_dnode/src/dmWorker.c	行号: 31
	步骤描述: 不使用返回值的函数'setThreadName'应使用'void'说明.	
	代码片段: 29: SDnodeMgmt *pMgmt = param; 30: int64_t lastTime = taosGetTimestampMs(); 31: setThreadName("dnode-status"); 32: 33: while (1) {	
	污染路径: 函数'setThreadName'的定义点.	
1371	缺陷发生位置: /source/libs/executor/src/filloperator.c	行号: 356
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 354: taosMemoryFreeClear(pInfo->p); 355: taosArrayDestroy(pInfo->matchInfo.pList); 356: taosMemoryFreeClear(param); 357: }	

	358:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1372	缺陷发生位置: /docs/examples/c-ws-new/stmt2_insert_demo.c	行号: 195
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 193: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x%x, ErrMessage: %s.\n", host, port, taos_errno(NULL), 194: taos_errstr(NULL)); 195: taos_cleanup(); 196: exit(EXIT_FAILURE); 197: }	
	污染路径: 函数'taos_cleanup'的定义点.	
1373	缺陷发生位置: /docs/examples/c-ws-new/query_data_demo.c	行号: 28
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 26: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x %x, ErrMessage: %s.\n", host, port, taos_errno(NULL), 27: taos_errstr(NULL)); 28: taos_cleanup(); 29: return -1; 30: }	
	污染路径: 函数'taos_cleanup'的定义点.	
1374	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSnapshotRAW.c	行号: 69
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 67: lino, tstrerror(code), ever, type); 68: tsdbFSDestroyRefSnapshot(&reader[0]->fsetArr); 69: taosMemoryFree(reader[0]); 70: reader[0] = NULL; 71: } else {	

	污染路径: 函数'taosMemoryFree'的定义点.	
1375	缺陷发生位置: /source/client/src/clientRawBlockWrite.c	行号: 217
	步骤描述: 不使用返回值的函数'uDebug'应使用'void'说明.	
	代码片段: 215: end; 216: *pJson = json; 217: RAW_LOG_END 218: return code; 219: }	
	污染路径: expanded from macro 'RAW_LOG_END' 函数'uDebug'的定义点. expanded from macro 'RAW_LOG_START'	
1376	缺陷发生位置: /source/dnode/snode/src/snode.c	行号: 348
	步骤描述: 不使用返回值的函数'streamReleaseTask'应使用'void'说明.	
	代码片段: 346: tmsgSendRsp(&rsp); 347: } 348: streamReleaseTask(taskAddr); 349: 350: return code;	
	污染路径: 函数'streamReleaseTask'的定义点.	
1377	缺陷发生位置: /utils/test/c/tmqSim.c	行号: 321
	步骤描述: 不使用返回值的函数'pPrint'应使用'void'说明.	
	代码片段: 319: pPrint("%s configDir:%s %s", GREEN, configDir, NC); 320: pPrint("%s dbName:%s %s", GREEN, g_stConflInfo.dbName, NC); 321: pPrint("%s cdbName:%s %s", GREEN, g_stConflInfo.cdbName, NC); 322: pPrint("%s consumeDelay:%d %s", GREEN, g_stConflInfo.consumeDelay, NC); 323: pPrint("%s showMsgFlag:%d %s", GREEN,	

	g_stConflInfo.showMsgFlag, NC);	
	污染路径: 函数'pPrint'的定义点.	
1378	缺陷发生位置: /contrib/TSZ/zstd/compress/zstd_compress.c	行号: 73
	步骤描述: 不使用返回值的函数'ZSTD_STATIC_ASSERT'应使用'void'说明.	
	代码片段: 71: ZSTD_CCtx* ZSTD_createCCtx_advanced(ZSTD_customMem customMem) 72: { 73: ZSTD_STATIC_ASSERT(zcss_init==0); 74: ZSTD_STATIC_ASSERT(ZSTD_CONTENTSIZE_UNKNOWN==(0ULL - 1)); 75: if (!customMem.customAlloc ^ ! customMem.customFree) return NULL;	
	污染路径: 函数'ZSTD_STATIC_ASSERT'的定义点.	
1379	缺陷发生位置: /source/dnode/mnode/impl/src/mndStream.c	行号: 1095
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 1093: int64_t tss = taosGetTimestampMs(); 1094: if (tDeserializeSMDropStreamReq(pReq->pCont, pReq->contLen, &dropReq) < 0) { 1095: mError("invalid drop stream msg recv, discarded"); 1096: code = TSDB_CODE_INVALID_MSG; 1097: TAOS_RETURN(code);	
	污染路径: 函数'mError'的定义点.	
1380	缺陷发生位置: /source/os/src/osTimer.c	行号: 104
	步骤描述: 不使用返回值的函数'setThreadName'应使用'void'说明.	
	代码片段: 102: struct sigevent sevent = {{0}}; 103: 104: setThreadName("tmr");	

	105: 106: #ifdef _ALPINE	
	污染路径: 函数'setThreadName'的定义点.	
1381	缺陷发生位置: /source/dnode/vnode/src/meta/metaQuery.c	行号: 208
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 206: if (tdbTbGet(pMeta->pNameldx, name, strlen(name) + 1, &pData, &nData) == 0) { 207: uid = *(tb_uid_t *)pData; 208: tdbFree(pData); 209: } 210:	
	污染路径: 函数'tdbFree'的定义点.	
1382	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFS2.c	行号: 53
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 51: int32_t code = tsem_init(&fs[0]->canEdit, 0, 1); 52: if (code) { 53: taosMemoryFree(fs[0]); 54: return code; 55: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1383	缺陷发生位置: /source/libs/sync/src/syncMain.c	行号: 578
	步骤描述: 不使用返回值的函数'sError'应使用'void'说明.	
	代码片段: 576: SSyncNode* pSyncNode = syncNodeAcquire(rid); 577: if (pSyncNode == NULL) { 578: sError("sync ready for read error"); 579: return false; 580: }	

	污染路径: 函数'sError'的定义点.	
1384	缺陷发生位置: /source/libs/function/src/detail/tminmaxavx.c	行号: 148
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 146: return TSDB_CODE_SUCCESS; 147: #else 148: uError("unable run %s without avx2 instructions", __func__); 149: return TSDB_CODE_OPS_NOT_SUPPORT; 150: #endif	
	污染路径: 函数'uError'的定义点.	
1385	缺陷发生位置: /contrib/test/traft/cluster/node10001_restart.c	行号: 108
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 106: // for test: submit a value every second 107: while (1) { 108: sleep(1); 109: submitValue(); 110: }	
	污染路径: 无	
1386	缺陷发生位置: /source/libs/transport/src/thttp.c	行号: 231
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 229: code = gzipStream.total_out; 230: } 231: taosMemoryFree(pDest); 232: return code; 233: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1387	缺陷发生位置:	行号:

	/tools/shell/src/shellCommand.c	331
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 329: "\\s*clear\\s*\$"); 330: if (shellRegexMatch(total, reg_str, REG_EXTENDED REG_ICASE)) { 331: taosMemoryFree(total); 332: return true; 333: } </pre>	
1388	污染路径: 函数'taosMemoryFree'的定义点.	
	缺陷发生位置: /source/libs/transport/test/cliBench.c	行号: 66
	步骤描述: 不使用返回值的函数'tDebug'应使用'void'说明.	
1389	代码片段: <pre> 64: SRpcMsg rpcMsg = {0}; 65: 66: tDebug("thread:%d, start to send request", pInfo->index); 67: 68: while (pInfo->numOfReqs == 0 pInfo->num < pInfo->numOfReqs) { </pre>	
	污染路径: 函数'tDebug'的定义点.	
	缺陷发生位置: /utils/test/c/tmq_offset_test.c	行号: 143
1390	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: <pre> 141: int32_t numOfAssign = 0; 142: int32_t code = tmq_get_topic_assignment(tmq, "t_5679", &pAssign, &numOfAssign); 143: ASSERT (code == 0); 144: 145: for(int i = 0; i < numOfAssign; i++){ </pre>	
	污染路径: 函数'ASSERT'的定义点.	
1390	缺陷发生位置:	行号:

	/source/client/src/clientRawBlockWrite.c	228
	步骤描述: 不使用返回值的函数'uDebug'应使用'void'说明.	
	代码片段: 226: int32_t code = 0; 227: int32_t lino = 0; 228: RAW_LOG_START 229: if (encode != 0) { 230: const char* encodeStr = columnEncodeStr(encode);	
	污染路径: expanded from macro 'RAW_LOG_START' 函数'uDebug'的定义点. expanded from macro 'RAW_LOG_START'	
1391	缺陷发生位置: /source/dnode/vnode/src/meta/metaTable.c	行号: 550
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 548: } 549: tdbFree(pData); 550: tdbFree(pKey); 551: tdbTbcClose(pCur); 552:	
	污染路径: 函数'tdbFree'的定义点.	
1392	缺陷发生位置: /source/libs/tmq/tmq/src/tmqCtx.c	行号: 172
	步骤描述: 不使用返回值的函数'bndError'应使用'void'说明.	
	代码片段: 170: tmq_list_t* topic_list = tmq_list_new(); 171: if (!topic_list) { 172: bndError("failed to update topic list: %s %s", config->client_id, config->group_id); 173: 174: return NULL;	
	污染路径: 函数'bndError'的定义点.	
1393	缺陷发生位置: /source/util/src/tconfig.c	行号: 282

	步骤描述: 不使用返回值的函数'TAOS_CHECK_RETURN'应使用'void'说明.	
	代码片段: <pre> 280: static int32_t cfgSetFloat(SConfigItem *pltem, const char *value, ECfgSrcType stype) { 281: float dval = 0; 282: TAOS_CHECK_RETURN(parseCfgReal(value, &dval)); 283: if (dval < pltem->fmin dval > pltem->fmax) { 284: uError("cfg:%s, type:%s src:%s value:%g out of range[%g, %g]", pltem->name, cfgDtypeStr(pltem->dtype), </pre>	
	污染路径: 函数'TAOS_CHECK_RETURN'的定义点.	
1394	缺陷发生位置: /source/util/src/ttimer.c	行号: 656
	步骤描述: 不使用返回值的函数'taosUninitTimer'应使用'void'说明.	
	代码片段: <pre> 654: tmrDebug("time controller's tmr ctrl size: %d", numOfTmrCtrl); 655: if (numOfTmrCtrl <= 0) { 656: taosUninitTimer(); 657: 658: taosCleanUpScheduler(tmrQhandle); </pre>	
	污染路径: 函数'taosUninitTimer'的定义点.	
1395	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReadUtil.c	行号: 108
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: <pre> 106: TSDB_CHECK_NULL(px, code, lino, _end, TSDB_CODE_INVALID_PARA); 107: p = *(STableBlockScanInfo**)px; 108: taosMemoryFreeClear(p); 109: pBuf->numOfTables -= remainder; 110: } </pre>	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1396	缺陷发生位置: /source/dnode/mnode/impl/src/mndConfig.c	行号: 311

	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 309: size_t sz = 0; 310: 311: mInfo("init write cfg to sdb"); 312: STrans *pTrans = mndTransCreate(pMnode, TRN_POLICY_RETRY, TRN_CONFLICT_NOHING, NULL, "init-write-config"); 313: if (pTrans == NULL) {	
	污染路径: 函数'mInfo'的定义点.	
1397	缺陷发生位置: /source/libs/transport/src/transComm.c	行号: 274
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 272: r = uv_ip4_name(addr, (char*)buf, sizeof(buf)); 273: if (r != 0) { 274: uError("failed to get ip from sockaddr, err:%s", uv_strerror(r)); 275: } 276:	
	污染路径: 函数'uError'的定义点.	
1398	缺陷发生位置: /test/new_test_framework/script/api/stopquery.c	行号: 72
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 70: taos_free_result(pSql); 71: taos_close(taos); 72: taos_cleanup(); 73: exit(EXIT_FAILURE); 74: }	
	污染路径: 函数'taos_cleanup'的定义点.	
1399	缺陷发生位置: /tools/shell/src/shellUtil.c	行号: 45
	步骤描述:	

	不使用返回值的函数'shellExit'应使用'void'说明.	
	代码片段: 43: fprintf(stderr, "Regex match failed: %s\r\n", msgbuf); 44: regfree(®ex); 45: shellExit(); 46: } 47:	
	污染路径: 函数'shellExit'的定义点.	
1400	缺陷发生位置: /source/util/src/theap.c	行号: 359
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 357: if (!q) return; 358: destroyPriorityQueue(q->queue); 359: taosMemoryFree(q); 360: } 361:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1401	缺陷发生位置: /source/client/wrapper/jni/windows/win32/bridge/AccessBridgeCalls.c	行号: 80
	步骤描述: 不使用返回值的函数'PrintDebugString'应使用'void'说明.	
	代码片段: 78: LOAD_FP(SetFocusLost, SetFocusLostFP, "setFocusLostFP"); 79: 80: LOAD_FP(SetCaretUpdate, SetCaretUpdateFP, "setCaretUpdateFP"); 81: 82: LOAD_FP(SetMouseClicked, SetMouseClickedFP, "setMouseClickedFP");	
	污染路径: expanded from macro 'LOAD_FP' 函数'PrintDebugString'的定义点. expanded from macro 'LOAD_FP'	
1402	缺陷发生位置: /source/dnode/mnode/impl/src/mndEncryptAlgr.c	行号: 367

	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 365: 366: static int32_t mndProcessBuiltinRsp(SRpcMsg *pRsp) { 367: mInfo("builtin rsp"); 368: return 0; 369: }	
	污染路径: 函数'mInfo'的定义点.	
1403	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqLoop.c	行号: 148
	步骤描述: 不使用返回值的函数'ttqSessionExpiryCheck'应使用'void'说明.	
	代码片段: 146: if (rc) return rc; 147: 148: ttqSessionExpiryCheck(); 149: // will_delay__check(); 150: #ifdef WITH_PERSISTENCE	
	污染路径: 函数'ttqSessionExpiryCheck'的定义点.	
1404	缺陷发生位置: /source/util/test/decompressTest.cpp	行号: 495
	步骤描述: 不使用返回值的函数'func'应使用'void'说明.	
	代码片段: 493: auto start = std::chrono::high_resolution_clock::now(); 494: for (int32_t i = 0; i < nround; ++i) { 495: func(); 496: } 497: auto end = std::chrono::high_resolution_clock::now();	
	污染路径: 函数'func'的定义点.	
1405	缺陷发生位置: /source/libs/new-stream/src/streamUtil.c	行号: 241
	步骤描述: 不使用返回值的函数'stDebug'应使用'void'说明.	
	代码片段: 239: }	

	240: 241: stDebug("start to destroy stream info"); 242: 243: SStreamInfo* p = (SStreamInfo*)param;	
	污染路径: 函数'stDebug'的定义点.	
1406	缺陷发生位置: /source/client/src/clientHb.c	行号: 1482
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 1480: 1481: if (TSDB_CODE_SUCCESS != taosThreadMutexLock(&clientHbMgr.lock)) { 1482: tscError("taosThreadMutexLock failed"); 1483: return NULL; 1484: }	
	污染路径: 函数'tscError'的定义点.	
1407	缺陷发生位置: /test/new_test_framework/script/api/stmtTest.c	行号: 220
	步骤描述: 不使用返回值的函数'taos_stmt_add_batch'应使用'void'说明.	
	代码片段: 218: c10len=strlen(v.c10); 219: taos_stmt_bind_param(stmt, params); 220: taos_stmt_add_batch(stmt); 221: } 222:	
	污染路径: 函数'taos_stmt_add_batch'的定义点.	
1408	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeModule.c	行号: 38
	步骤描述: 不使用返回值的函数'vnodeAsyncClose'应使用'void'说明.	
	代码片段: 36: void vnodeCleanup() { 37: if (atomic_val_compare_exchange_32(&VINIT, 1, 0) == 0) return; 38: vnodeAsyncClose();	

	39: walCleanUp(); 40: }	
	污染路径: 函数'vnodeAsyncClose'的定义点.	
1409	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeModule.c	行号: 28
	步骤描述: 不使用返回值的函数'TAOS_CHECK_RETURN'应使用'void'说明.	
	代码片段: 26: } 27: 28: TAOS_CHECK_RETURN(vnodeAsyncOpen()); 29: TAOS_CHECK_RETURN(wallnit(stopDnodeFp)); 30:	
	污染路径: 函数'TAOS_CHECK_RETURN'的定义点.	
1410	缺陷发生位置: /docs/examples/c/async_query_example.c	行号: 163
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 161: taos_free_result(res); 162: taos_close(_taos); 163: taos_cleanup(); 164: exit(EXIT_FAILURE); 165: }	
	污染路径: 函数'taos_cleanup'的定义点.	
1411	缺陷发生位置: /source/libs/nodes/src/nodesMsgFuncs.c	行号: 5545
	步骤描述: 不使用返回值的函数'nodesError'应使用'void'说明.	
	代码片段: 5543: } 5544: if (TSDB_CODE_SUCCESS != code) { 5545: nodesError("specificNodeToMsg error node = %s", nodesNodeName(nodeType(pObj))); 5546: } 5547: // nodesWarn("specificNodeToMsg node = %s, after tlv count = %d", nodesNodeName(nodeType(pObj)), pEncoder-	

	>tlvCount);	
	污染路径: 函数'nodesError'的定义点.	
1412	缺陷发生位置: /docs/examples/c/tmq_demo.c	行号: 136
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 134: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x%x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 135: taos_errstr(NULL)); 136: taos_cleanup(); 137: return NULL; 138: }	
	污染路径: 函数'taos_cleanup'的定义点.	
1413	缺陷发生位置: /source/libs/executor/src/dataInserter.c	行号: 70
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 68: if (p == NULL) return; 69: SSubmitTbDataMsg* pVg = p; 70: taosMemoryFree(pVg->pData); 71: taosMemoryFree(pVg); 72: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1414	缺陷发生位置: /source/dnode/mnode/impl/src/mndPerfSchema.c	行号: 82
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 80: STableMetaRsp *meta = (STableMetaRsp *)taosHashGet(pMnode->perfsMeta, tbName, strlen(tbName)); 81: if (NULL == meta) { 82: mError("invalid performance schema table name:%s", tbName); 83: code = TSDB_CODE_PAR_TABLE_NOT_EXIST; 84: TAOS_RETURN(code);	

	污染路径: 函数'mError'的定义点.	
1415	缺陷发生位置: /docs/examples/c-ws-new/insert_data_demo.c	行号: 28
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 26: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x %x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 27: taos_errstr(NULL)); 28: taos_cleanup(); 29: return -1; 30: } 污染路径: 函数'taos_cleanup'的定义点.	
1416	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFile2.c	行号: 50
	步骤描述: 不使用返回值的函数'tsdbInfo'应使用'void'说明.	
	代码片段: 48: tsdbError("failed to remove file:%s, code:%d, error:%s", fname, code, tsterror(code)); 49: } else { 50: tsdbInfo("file:%s is removed", fname); 51: } 52: } 污染路径: 函数'tsdbInfo'的定义点.	
1417	缺陷发生位置: /docs/examples/c-ws-new/with_reqid_demo.c	行号: 41
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 39: taos_errstr(result)); 40: taos_close(taos); 41: taos_cleanup(); 42: return -1; 43: } 污染路径:	

	污染路径: 无	
1421	缺陷发生位置: /source/os/src/osEnv.c	行号: 54
	步骤描述: 不使用返回值的函数'taosGetSystemInfo'应使用'void'说明.	
	代码片段: 52: (void)taosGetSystemTimezone(tsTimezoneStr); 53: 54: taosGetSystemInfo(); 55: 56: // deadlock in query	
	污染路径: 函数'taosGetSystemInfo'的定义点.	
1422	缺陷发生位置: /source/libs/index/test/indexBench.cc	行号: 123
	步骤描述: 不使用返回值的函数'idx'应使用'void'说明.	
	代码片段: 121: for (int i = 0; i < limit; i += batchSize) { 122: WriteBatch wb; 123: idx->Write(&wb, i); 124: } 125: return 0;	
	污染路径: 函数'idx'的定义点.	
1423	缺陷发生位置: /source/libs/transport/src/trans.c	行号: 235
	步骤描述: 不使用返回值的函数'transFreeMsg'应使用'void'说明.	
	代码片段: 233: } 234: 235: void rpcFreeCont(void* cont) { transFreeMsg(cont); } 236: 237: void* rpcReallocCont(void* ptr, int64_t contLen) {	
	污染路径: 函数'transFreeMsg'的定义点.	
1424	缺陷发生位置: /source/libs/scheduler/src/schTask.c	行号: 1111

	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 1109: if (TSDB_CODE_SUCCESS == traceCode) { 1110: SCH_TASK_DLOGL("physical plan len:%d, %s", msgLen, msg); 1111: taosMemoryFree(msg); 1112: } else { 1113: SCH_TASK_WLOG("plan trace failed, code:%s", tstrerror(traceCode));	
	污染路径: 函数'taosMemoryFree'的定义点.	
1425	缺陷发生位置: /source/libs/executor/src/executil.c	行号: 799
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 797: (nodeType(pTagCond) != QUERY_NODE_OPERATOR && 798: nodeType(pTagCond) != QUERY_NODE_LOGIC_CONDITION)) { 799: qError("invalid parameter to extract tag filter symbol"); 800: return TSDB_CODE_STREAM_INTERNAL_ERROR; 801: }	
	污染路径: 函数'qError'的定义点.	
1426	缺陷发生位置: /test/new_test_framework/script/api/sameReqidTest.c	行号: 44
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 42: CHECK_RES(res, "create database"); 43: taos_free_result(res); 44: usleep(100000); 45: 46: code = taos_select_db(taos, TEST_DB);	
	污染路径: 无	
1427	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 438

	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 436: char zhongguoGbk[32] = {0}; 437: snprintf(fenUtf8, sizeof(fenUtf8), "%c%c%c", 0xE8, 0x8A, 0xAC); 438: snprintf(fenGbk, sizeof(fenGbk), "%c%c", 0xB7, 0xD2); 439: snprintf(zhongguoUtf8, sizeof(zhongguoUtf8), "%c%c%c%c", 0xE4, 0xB8, 0xAD, 0xE5, 0x9B, 0xBD); 440: snprintf(guoUtf8, sizeof(guoUtf8), "%c%c%c", 0xE5, 0x9B, 0xBD);	
	污染路径: 无	
1428	缺陷发生位置: /source/libs/transport/src/trans.c	行号: 244
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 242: char* nst = taosMemoryRealloc(st, sz); 243: if (nst == NULL) { 244: taosMemoryFree(st); 245: terrno = TSDB_CODE_OUT_OF_MEMORY; 246: return NULL;	
	污染路径: 函数'taosMemoryFree'的定义点.	
1429	缺陷发生位置: /test/new_test_framework/script/api/stmt2.c	行号: 327
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 325: printf("done\n"); 326: taos_close(taos); 327: taos_cleanup(); 328: }	
	污染路径: 函数'taos_cleanup'的定义点.	
1430	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeQuery.c	行号: 664
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	

	代码片段: 662: rspSize = tSerializeSBatchRsp(NULL, 0, &batchRsp); 663: if (rspSize < 0) { 664: qError("tSerializeSBatchRsp failed"); 665: code = terrno; 666: goto _exit;	
	污染路径: 函数'qError'的定义点.	
1431	缺陷发生位置: /source/libs/parser/src/parInsertStmt.c	行号: 36
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 34: int32_t qCloneCurrentTbData(STableDataCxt* pDataBlock, SSubmitTbData** pData) { 35: if (pDataBlock == NULL pDataBlock->pData == NULL) { 36: qError("stmt qCloneCurrentTbData input is null, maybe not bind data before execute"); 37: return TSDB_CODE_TSC_STMT_API_ERROR; 38: }	
	污染路径: 函数'qError'的定义点.	
1432	缺陷发生位置: /source/libs/transport/src/trans.c	行号: 167
	步骤描述: 不使用返回值的函数'tError'应使用'void'说明.	
	代码片段: 165: 166: if (pRpc->tcphandle == NULL) { 167: tError("failed to init rpc handle"); 168: TAOS_CHECK_GOTO(terrno, NULL, _end); 169: }	
	污染路径: 函数'tError'的定义点.	
1433	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/test/argp-test.c	行号: 224
	步骤描述: 不使用返回值的函数'asprintf'应使用'void'说明.	

	代码片段: 222: { 223: time_t now = time (0); 224: asprintf (&new_text, text, ctime (&now)); 225: } 226: else if (key == 'f')	
	污染路径: 函数'asprintf'的定义点.	
1434	缺陷发生位置: /contrib/test/craft/raftMain.c	行号: 407
	步骤描述: 不使用返回值的函数'pthread_create'应使用'void'说明.	
	代码片段: 405: 406: pthread_t tidRaftServer; 407: pthread_create(&tidRaftServer, NULL, startServerFunc, &raftServer); 408: 409: pthread_t tidConsole;	
	污染路径: 无	
1435	缺陷发生位置: /source/dnode/mnode/impl/src/mndArbGroup.c	行号: 388
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: 386: if (epSet.numOfEps == 0) { 387: mError("arbgroup:0, dnodeId:%d, failed to send arb-hb request to dnode since no epSet found", pDnode->id); 388: rpcFreeCont(pHead); 389: return -1; 390: }	
	污染路径: 函数'rpcFreeCont'的定义点.	
1436	缺陷发生位置: /source/os/src/osTimezone.c	行号: 1222
	步骤描述: 不使用返回值的函数'uDebug'应使用'void'说明.	
	代码片段: 1220:	

	1221: (void)snprintf(outTimezoneStr, TD_TIMEZONE_LEN, "%s (%s, %s)", tz, str1, str2); 1222: uDebug("[tz] system timezone:%s", outTimezoneStr); 1223: return 0; 1224: }	
	污染路径: 函数'uDebug'的定义点.	
1437	缺陷发生位置: /test/new_test_framework/script/api/stmt2-insert-dupkeys.c	行号: 154
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 152: TAOS_STMT2_BIND **paramv, **tags; 153: 154: INIT(tbs, ts, ts_len, b, b_len, tags, paramv); 155: 156: // insert duplicate rows	
	污染路径: expanded from macro 'INIT'	
1438	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 756
	步骤描述: 不使用返回值的函数'assert'应使用'void'说明.	
	代码片段: 754: ZDICT_flatLit(countLit); /* replace distribution by a fake "mostly flat but still compressible" distribution, that HUF_writeCTable() can encode */ 755: maxNbBits = HUF_buildCTable (hufTable, countLit, 255, huffLog); 756: assert(maxNbBits==9); 757: } 758: huffLog = (U32)maxNbBits;	
	污染路径: 函数'assert'的定义点.	
1439	缺陷发生位置: /source/dnode/mnode/impl/src/mndCompact.c	行号: 440
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 438:	

	439: if ((code = mndTransAppendRedoAction(pTrans, &action)) != 0) { 440: taosMemoryFree(pReq); 441: TAOS_RETURN(code); 442: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1440	缺陷发生位置: /source/libs/new-stream/src/streamTimer.c	行号: 26
	步骤描述: 不使用返回值的函数'stInfo'应使用'void'说明.	
	代码片段: 24: } 25: 26: stInfo("init stream timer, %p", &ppTimer); 27: return 0; 28: }	
	污染路径: 函数'stInfo'的定义点.	
1441	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smlnt.c	行号: 208
	步骤描述: 不使用返回值的函数'tmsgReportStartup'应使用'void'说明.	
	代码片段: 206: } 207: 208: tmsgReportStartup("snode-impl", "initialized"); 209: 210: // Check and migrate checkpoint files to encrypted format if needed	
	污染路径: 函数'tmsgReportStartup'的定义点.	
1442	缺陷发生位置: /source/dnode/vnode/src/tq/tqUtil.c	行号: 26
	步骤描述: 不使用返回值的函数'tqDebug'应使用'void'说明.	
	代码片段: 24: int32_t code = TDB_CODE_SUCCESS; 25: int32_t lino = 0; 26: tqDebug("%s called", __FUNCTION__);	

	27: TSDB_CHECK_NULL(pRsp, code, lino, END, TSDB_CODE_INVALID_PARA); 28:	
	污染路径: 函数'tqDebug'的定义点.	
1443	缺陷发生位置: /source/dnode/mnode/impl/src/mndStreamMgmt.c	行号: 2415
	步骤描述: 不使用返回值的函数'mstsWarn'应使用'void'说明.	
	代码片段: 2413: code = mndAcquireStream(pCtx->pMnode, streamName, &pStream); 2414: if (TSDB_CODE_MND_STREAM_NOT_EXIST == code) { 2415: mstsWarn("stream %s no longer exists, ignore deploy", streamName); 2416: return TSDB_CODE_SUCCESS; 2417: }	
	污染路径: 函数'mstsWarn'的定义点.	
1444	缺陷发生位置: /source/client/src/clientSmlLine.c	行号: 281
	步骤描述: 不使用返回值的函数'MOVE_FORWARD_ONE'应使用'void'说明.	
	代码片段: 279: } 280: (void)memcpy(tmp, key, keyLen); 281: PROCESS_SLASH_IN_TAG_FIELD_KEY(tmp, keyLen); 282: key = tmp; 283: }	
	污染路径: expanded from macro 'PROCESS_SLASH_IN_TAG_FIELD_KEY' 函数'MOVE_FORWARD_ONE'的定义点. expanded from macro 'PROCESS_SLASH_IN_TAG_FIELD_KEY'	
1445	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqNet.c	行号: 813
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 811: #else 812: UNUSED(ttq);	

	813: UNUSED(host); 814: #endif 815: return TTQ_ERR_SUCCESS;	
	污染路径: 函数'UNUSED'的定义点.	
1446	缺陷发生位置: /source/libs/new-stream/src/streamUtil.c	行号: 671
	步骤描述: 不使用返回值的函数'cJSON_free'应使用'void'说明.	
	代码片段: 669: _end: 670: if (temp != NULL) { 671: cJSON_free(temp); 672: } 673: if (obj != NULL) {	
	污染路径: 函数'cJSON_free'的定义点.	
1447	缺陷发生位置: /utls/tsim/src/simExec.c	行号: 459
	步骤描述: 不使用返回值的函数'simInfo'应使用'void'说明.	
	代码片段: 457: script->linePos++; 458: if (replaced && strstr(buf, "start") != NULL) { 459: simInfo("====> startup is slow in valgrind mode, so sleep 5 seconds after exec.sh -s start"); 460: taosMsleep(5000); 461: }	
	污染路径: 函数'simInfo'的定义点.	
1448	缺陷发生位置: /source/dnode/mnode/impl/src/mndScanDetail.c	行号: 43
	步骤描述: 不使用返回值的函数'mDebug'应使用'void'说明.	
	代码片段: 41: } 42: 43: void mndCleanupScanDetail(SMnode *pMnode) { mDebug("mnd scan detail cleanup"); } 44:	

	45: static int32_t mndRetrieveScanDetail(SRpcMsg *pReq, SShowObj *pShow, SSDataBlock *pBlock, int32_t rows) {	
	污染路径: 函数'mDebug'的定义点.	
1449	缺陷发生位置: /source/libs/executor/src/dynqueryctrloperator.c	行号: 258
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 256: } 257: 258: taosMemoryFreeClear(param); 259: } 260:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1450	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 580
	步骤描述: 不使用返回值的函数'symTransform_4bytes'应使用'void'说明.	
	代码片段: 578: while(1) 579: { 580: symTransform_4bytes(p); 581: i+=sizeof(unsigned int); 582: if(i<size)	
	污染路径: 函数'symTransform_4bytes'的定义点.	
1451	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smWorker.c	行号: 307
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: 305: } 306: 307: dDebug("snode workers are initialized"); 308: return code; 309: }	
	污染路径:	

	函数'dDebug'的定义点.	
1452	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 894
	步骤描述: 不使用返回值的函数'BIT_reloadDStream'应使用'void'说明.	
	代码片段: 892: memcpy(&DTableH, dt, sizeof(DTableH)); 893: DStatePtr->state = BIT_readBits(bitD, DTableH.tableLog); 894: BIT_reloadDStream(bitD); 895: DStatePtr->table = dt + 1; 896: }	
	污染路径: 函数'BIT_reloadDStream'的定义点.	
1453	缺陷发生位置: /contrib/test/traft/join_into_vgroup/node_follower10000.c	行号: 67
	步骤描述: 不使用返回值的函数'printRaftState'应使用'void'说明.	
	代码片段: 65: struct raft *r = getRaft(&raftEnv, 100); 66: assert(r != NULL); 67: printRaftState(r); 68: 69: // submit value	
	污染路径: 函数'printRaftState'的定义点.	
1454	缺陷发生位置: /source/dnode/mnode/impl/src/mndDump.c	行号: 792
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 790: taosMemoryFree(pCont); 791: 792: mInfo("dump sdb info success"); 793: return 0; 794: }	
	污染路径: 函数'mInfo'的定义点.	
1455	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 772

	步骤描述: 不使用返回值的函数'FSE_reloadDStream'应使用'void'说明.	
	代码片段: 770: const FSE_DTableHeader* const DTableH = (const FSE_DTableHeader*)ptr; 771: DStatePtr->state = FSE_readBits(bitD, DTableH->tableLog); 772: FSE_reloadDStream(bitD); 773: DStatePtr->table = dt + 1; 774: }	
	污染路径: 函数'FSE_reloadDStream'的定义点.	
1456	缺陷发生位置: /source/dnode/vnode/src/meta/metaSnapshot.c	行号: 54
	步骤描述: 不使用返回值的函数'metaSnapReaderClose'应使用'void'说明.	
	代码片段: 52: if (code) { 53: metaError("vgld:%d, %s failed at %s:%d since %s", TD_VID(pMeta->pVnode), __func__, __FILE__, lino, tstrerror(code)); 54: metaSnapReaderClose(&pReader); 55: *ppReader = NULL; 56: } else {	
	污染路径: 函数'metaSnapReaderClose'的定义点.	
1457	缺陷发生位置: /source/libs/tdb/src/db/tdbPCache.c	行号: 61
	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	
	代码片段: 59: static void tdbPCacheLock(SPCache *pCache) { 60: if (tdbMutexLock(&(pCache->mutex)) != 0) { 61: tdbError("tdb/pcache: mutex lock failed."); 62: } 63: }	
	污染路径: 函数'tdbError'的定义点.	
1458	缺陷发生位置: /source/common/src/msg/tmsg.c	行号: 978

	步骤描述: 不使用返回值的函数'FREESQL'应使用'void'说明.	
	代码片段: 976: taosMemoryFreeClear(pReq->pAst1); 977: taosMemoryFreeClear(pReq->pAst2); 978: FREESQL(); 979: } 980:	
	污染路径: 函数'FREESQL'的定义点.	
1459	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttCtx.c	行号: 251
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 249: UNUSED(snprintf(consumer_client_id, sizeof(consumer_client_id), "_cid-%s-%d", cid, global.dnode_id)); 250: // snprintf(group_id, sizeof(group_id), "_xnd-gid-%d", global.dnode_id); 251: UNUSED(snprintf(group_id, sizeof(group_id), "_bnd-gid-%s", sn ? sn : consumer_client_id)); 252: UNUSED(snprintf(port_str, sizeof(port_str), "%d", global.mgmtEp.epSet.eps[0].port)); 253:	
	污染路径: 函数'UNUSED'的定义点.	
1460	缺陷发生位置: /source/dnode/vnode/src/tq/tqSnapshot.c	行号: 76
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 74: } 75: tdbTbcClose((*ppReader)->pCur); 76: taosMemoryFreeClear(*ppReader); 77: } 78:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1461	缺陷发生位置:	行号:

	/source/dnode/mnode/sdb/src/sdb.c	74
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 72: 73: void sdbCleanup(SSdb *pSdb) { 74: mInfo("start to cleanup sdb"); 75: 76: int32_t code = 0;	
	污染路径: 函数'mInfo'的定义点.	
1462	缺陷发生位置: /source/dnode/vnode/src/bse/bseTable.c	行号: 971
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 969: 970: int8_t blockGetType(SBlock *p) { return p->type; } 971: void blockDestroy(SBlock *pBlock) { taosMemoryFree(pBlock); } 972: 973: int32_t metaBlockAddIndex(SBlock *p, SBlkHandle *pInfo) {	
	污染路径: 函数'taosMemoryFree'的定义点.	
1463	缺陷发生位置: /source/dnode/mgmt/mgmt_bnode/src/bmlnt.c	行号: 193
	步骤描述: 不使用返回值的函数'tmsgReportStartup'应使用'void'说明.	
	代码片段: 191: return code; 192: } 193: tmsgReportStartup("bnode-impl", "initialized"); 194: 195: /*	
	污染路径: 函数'tmsgReportStartup'的定义点.	
1464	缺陷发生位置: /source/dnode/vnode/src/bse/bseCache.c	行号: 410
	步骤描述:	

	不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 408: 409: lruCacheFree((SLruCache *)p->pCache); 410: taosMemoryFree(p); 411: } 412:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1465	缺陷发生位置: /docs/examples/c-ws/stmt_insert_demo.c	行号: 138
	步骤描述: 不使用返回值的函数'gettimeofday'应使用'void'说明.	
	代码片段: 136: for (int j = 0; j < num_of_row; j++) { 137: struct timeval tv; 138: gettimeofday(&tv, NULL); 139: long long milliseconds = tv.tv_sec * 1000LL + tv.tv_usec / 1000; // current timestamp in milliseconds 140: int64_t ts = milliseconds + j;	
	污染路径: 无	
1466	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 710
	步骤描述: 不使用返回值的函数'DEBUGLOG'应使用'void'说明.	
	代码片段: 708: 709: /* init */ 710: DEBUGLOG(4, "ZDICT_analyzeEntropy"); 711: esr.ref = ZSTD_createCctx(); 712: esr.zc = ZSTD_createCctx();	
	污染路径: 函数'DEBUGLOG'的定义点.	
1467	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmlnt.c	行号: 143
	步骤描述: 不使用返回值的函数'tmsgReportStartup'应使用'void'说明.	
	代码片段: 141: return code;	

	142: } 143: tmsgReportStartup("mnode-worker", "initialized"); 144: 145: if (option.numOfTotalReplicas > 0) {	
	污染路径: 函数'tmsgReportStartup'的定义点.	
1468	缺陷发生位置: /source/dnode/mnode/impl/src/mndXnode.c	行号: 1215
	步骤描述: 不使用返回值的函数'mDebug'应使用'void'说明.	
	代码片段: 1213: } 1214: 1215: mDebug("xnode task:%s, not found", name); 1216: terrno = TSDB_CODE_MND_XNODE_TASK_NOT_EXIST; 1217: return NULL;	
	污染路径: 函数'mDebug'的定义点.	
1469	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 707
	步骤描述: 不使用返回值的函数'BIT_reloadDStream'应使用'void'说明.	
	代码片段: 705: memcpy(&DTableH, dt, sizeof(DTableH)); 706: DStatePtr->state = BIT_readBits(bitD, DTableH.tableLog); 707: BIT_reloadDStream(bitD); 708: DStatePtr->table = dt + 1; 709: }	
	污染路径: 函数'BIT_reloadDStream'的定义点.	
1470	缺陷发生位置: /source/libs/wal/src/walMgmt.c	行号: 62
	步骤描述: 不使用返回值的函数'wWarn'应使用'void'说明.	
	代码片段: 60: 61: if (stopDnode == NULL) { 62: wWarn("failed to set stop dnode call back"); 63: }	

	64: tsWal.stopDnode = stopDnode;	
	污染路径: 函数'wWarn'的定义点.	
1471	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 733
	步骤描述: 不使用返回值的函数'operator='应使用'void'说明.	
	代码片段: 731: { 732: na.numerator_ex = 0; 733: na.numerator = numerator; 734: na.divisor = denominator; 735: }	
	污染路径: 函数'operator='的定义点.	
1472	缺陷发生位置: /contrib/TSZ/zstd/compress/zstd_ldm.c	行号: 26
	步骤描述: 不使用返回值的函数'DEBUGLOG'应使用'void'说明.	
	代码片段: 24: params->windowLog = cParams->windowLog; 25: ZSTD_STATIC_ASSERT(LDM_BUCKET_SIZE_LOG <= ZSTD_LDM_BUCKETSIZelog_MAX); 26: DEBUGLOG(4, "ZSTD_ldm_adjustParameters"); 27: if (!params->bucketSizeLog) params->bucketSizeLog = LDM_BUCKET_SIZE_LOG; 28: if (!params->minMatchLength) params-> >minMatchLength = LDM_MIN_MATCH_LENGTH;	
	污染路径: 函数'DEBUGLOG'的定义点.	
1473	缺陷发生位置: /utils/tsim/src/simEntry.c	行号: 48
	步骤描述: 不使用返回值的函数'simInfo'应使用'void'说明.	
	代码片段: 46: taos_options(TSDB_OPTION_DRIVER, "native"); 47: 48: simInfo("simulator is running ..."); 49: 50: simSystemInit();	

	污染路径: 函数'simInfo'的定义点.	
1474	缺陷发生位置: /utils/tsim/src/simSystem.c	行号: 40
	步骤描述: 不使用返回值的函数'simInitsimCmdList'应使用'void'说明.	
	代码片段: 38: void simSystemInit() { 39: simInitCfg(); 40: simInitsimCmdList(); 41: memset(simScriptList, 0, sizeof(SScript *) * MAX_MAIN_SCRIPT_NUM); 42: }	
	污染路径: 函数'simInitsimCmdList'的定义点.	
1475	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smWorker.c	行号: 191
	步骤描述: 不使用返回值的函数'streamReleaseTask'应使用'void'说明.	
	代码片段: 189: _exit: 190: if (taskAddr != NULL) { 191: streamReleaseTask(taskAddr); 192: } 193: if (ahandle != NULL) {	
	污染路径: 函数'streamReleaseTask'的定义点.	
1476	缺陷发生位置: /source/dnode/mnode/impl/src/mndProfile.c	行号: 465
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: 463: contLen = tSerializeSConnectRsp(pRsp, contLen, &connectRsp); 464: if (contLen < 0) { 465: rpcFreeCont(pRsp); 466: TAOS_CHECK_GOTO(contLen, &lino, _OVER); 467: }	
	污染路径:	

	函数'rpcFreeCont'的定义点.	
1477	缺陷发生位置: /utils/tsim/src/simEntry.c	行号: 24
	步骤描述: 不使用返回值的函数'simSystemCleanUp'应使用'void'说明.	
	代码片段: 22: 23: void simHandleSignal(int32_t signo, void *sigInfo, void *context) { 24: simSystemCleanUp(); 25: abortExecution = true; 26: }	
	污染路径: 函数'simSystemCleanUp'的定义点.	
1478	缺陷发生位置: /source/dnode/mnode/impl/src/mndMnode.c	行号: 406
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 404: 405: if ((code = mndTransAppendRedoAction(pTrans, &action)) != 0) { 406: taosMemoryFree(pReq); 407: TAOS_RETURN(code); 408: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1479	缺陷发生位置: /source/client/src/clientSmlJson.c	行号: 255
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 253: int32_t tagNum = cJSON_GetArraySize(tags); 254: if (unlikely(tagNum == 0)) { 255: uError("SML:Tag should not be empty"); 256: return TSDB_CODE_TSC_INVALID_JSON; 257: }	
	污染路径: 函数'uError'的定义点.	
1480	缺陷发生位置:	行号:

	/source/dnode/vnode/src/bse/bseMgt.c	600
	步骤描述: 不使用返回值的函数'bseBatchDestroy'应使用'void'说明.	
	代码片段: 598: 599: if (taosArrayPush(pBatchMgt->pBatchList, &b) == NULL) { 600: bseBatchDestroy(b); 601: TSDB_CHECK_CODE(code = terrno, lino, _error); 602: }	
1481	污染路径: 函数'bseBatchDestroy'的定义点.	
	缺陷发生位置: /test/new_test_framework/script/api/stmt_function.c	行号: 86
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
1482	代码片段: 84: stmt = taos_stmt_init(taos); 85: assert(stmt != NULL); 86: sprintf(stmt_sql, "insert into super values (?,?,?,?,?,?,?,?,?)"); 87: assert(taos_stmt_prepare(stmt, stmt_sql, 0) != 0); 88: assert(taos_stmt_close(stmt) == 0);	
	污染路径: 无	
	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 771
1483	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 769: pAPI->metaReaderFn.clearReader(&smrSuperTable); 770: } 771: taosMemoryFreeClear(pVtableRefInfo); 772: if (code != TSDB_CODE_SUCCESS) { 773: qError("%s failed at line %d since %s", __func__, lino, strerror(code));	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1483	缺陷发生位置: /utils/test/c/tmq_write_raw_with_blob_test.c	行号: 92

	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: <pre> 90: int32_t ret = tmq_write_raw(pConn, raw); 91: printf("write raw data: %s\n", tmq_err2str(ret)); 92: ASSERT(ret == 0); 93: 94: tmq_free_raw(raw); </pre>	
	污染路径: 函数'ASSERT'的定义点.	
1484	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1166
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 1164: void* varObtainThread(void* param) { 1165: int type = *(int*)param; 1166: taosMemoryFree(param); 1167: 1168: if (varCon == NULL type > WT_FROM_DB_MAX) { </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
1485	缺陷发生位置: /source/dnode/mnode/impl/src/mndMnode.c	行号: 422
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 420: code = tSerializeSDCreateMnodeReq(pReq, contLen, pAlterReq); 421: if (code < 0) { 422: taosMemoryFree(pReq); 423: TAOS_RETURN(code); 424: } </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
1486	缺陷发生位置: /source/libs/nodes/src/nodesCodeFuncs.c	行号: 9952
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段:	

	<pre> 9950: } 9951: } 9952: taosMemoryFree(split); 9953: } 9954: } else if (pNode->optrType == TSDB_ALTER_ROLE_ROLE) { </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
1487	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmWorker.c	行号: 358
	步骤描述: 不使用返回值的函数'ST_TASK_DLOG'应使用'void'说明.	
	代码片段: <pre> 356: 357: STREAM_CHECK_RET_GOTO(streamBuildFetchRsp(pResList, hasNext, &buf, &size, TSDB_TIME_PRECISION_MILLI)); 358: ST_TASK_DLOG("%s end:", __func__); 359: 360: end: </pre>	
	污染路径: 函数'ST_TASK_DLOG'的定义点.	
1488	缺陷发生位置: /source/libs/new-stream/src/stream.c	行号: 67
	步骤描述: 不使用返回值的函数'smUndeployAllTasks'应使用'void'说明.	
	代码片段: <pre> 65: stTriggerTaskEnvCleanup(); 66: streamTimerCleanUp(); 67: smUndeployAllTasks(); 68: destroyDataSinkMgr(); 69: streamMgmtCleanup(); </pre>	
	污染路径: 函数'smUndeployAllTasks'的定义点.	
1489	缺陷发生位置: /source/libs/tfs/src/tfsTier.c	行号: 37
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 35: if (p) { </pre>	

	36: SDiskCursor *pCursor = *(SDiskCursor **)p; 37: taosMemoryFree(pCursor); 38: } 39: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1490	缺陷发生位置: /source/common/src/tdatablock.c	行号: 87
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 85: dataLen = ((STag*)(data))->len; 86: } else { 87: uError("Invalid data type:%d in Json", *data); 88: } 89: return dataLen;	
	污染路径: 函数'uError'的定义点.	
1491	缺陷发生位置: /test/new_test_framework/script/api/dbTableRoute.c	行号: 98
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 96: 97: for (int32_t i = 0; i < rtTables; ++i) { 98: sprintf(sql, "create table tb%d (ts timestamp, f1 int)", i); 99: rtExecSQL(taos, sql); 100: }	
	污染路径: 无	
1492	缺陷发生位置: /source/dnode/mnode/impl/src/mndDb.c	行号: 3146
	步骤描述: 不使用返回值的函数'STR_WITH_MAXSIZE_TO_VARSTR'应使用'void'说明.	
	代码片段: 3144: } 3145: char precVstr[10] = {0}; 3146: STR_WITH_MAXSIZE_TO_VARSTR(precVstr, precStr, 10); 3147:	

	3148: char *statusStr = "ready";	
	污染路径: 函数'STR_WITH_MAXSIZE_TO_VARSTR'的定义点.	
1493	缺陷发生位置: /source/client/src/clientMsgHandler.c	行号: 218
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 216: } 217: if (tsem_post(&pRequest->body.rspSem) != 0) { 218: tscError("failed to post semaphore"); 219: } 220:	
	污染路径: 函数'tscError'的定义点.	
1494	缺陷发生位置: /source/dnode/mnode/impl/src/mndSsMigrate.c	行号: 909
	步骤描述: 不使用返回值的函数'TAOS_RETURN'应使用'void'说明.	
	代码片段: 907: 908: static int32_t mndProcessFollowerSsMigrateRsp(SRpcMsg *pReq) { 909: TAOS_RETURN(0); 910: } 911:	
	污染路径: 函数'TAOS_RETURN'的定义点.	
1495	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqMux.c	行号: 85
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 83: int ttqMuxAddOut(struct tmqtt *context) { 84: #ifndef WITH_EPOLL 85: UNUSED(context); 86: return -1; 87: #else	
	污染路径:	

	函数'UNUSED'的定义点.	
1496	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqSubs.c	行号: 183
	步骤描述: 不使用返回值的函数'ttq_free'应使用'void'说明.	
	代码片段: 181: ttq_free(shared); 182: } 183: ttq_free(leaf); 184: } 185:	
	污染路径: 函数'ttq_free'的定义点.	
1497	缺陷发生位置: /source/libs/tfs/src/tfs.c	行号: 182
	步骤描述: 不使用返回值的函数'TAOS_RETURN'应使用'void'说明.	
	代码片段: 180: } 181: 182: TAOS_RETURN(0); 183: } 184:	
	污染路径: 函数'TAOS_RETURN'的定义点.	
1498	缺陷发生位置: /contrib/test/cos/main.c	行号: 132
	步骤描述: 不使用返回值的函数'cos_warn_log'应使用'void'说明.	
	代码片段: 130: 131: char *pstr2 = apr_pstrndup(p, (char *)fmt_str_hex, sizeof(fmt_str_hex)); 132: cos_warn_log("Format string sha1hash: %s", pstr2); 133: 134: cos_pool_destroy(p);	
	污染路径: 函数'cos_warn_log'的定义点.	
1499	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFile2.c	行号: 354

	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 352: tsdbTFileObjRemoveLC(fobj); 353: (void)taosThreadMutexDestroy(&fobj->mutex); 354: taosMemoryFree(fobj); 355: } 356: return 0;	
	污染路径: 函数'taosMemoryFree'的定义点.	
1500	缺陷发生位置: /source/libs/qworker/src/qwDbg.c	行号: 343
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 341: if (0 == strcasecmp(option, "status")) { 342: gQWDebug.statusEnable = true; 343: qError("qw status debug enabled"); 344: return TSDB_CODE_SUCCESS; 345: }	
	污染路径: 函数'qError'的定义点.	
1501	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 647
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 645: bool ret = taosMbsToUcs4(input, inputLen, (TdUcs4 *)*output, inputLen * TSDB_NCHAR_SIZE, outputLen, charsetCxt); 646: if (!ret) { 647: taosMemoryFreeClear(*output); 648: return TSDB_CODE_SCALAR_CONVERT_ERROR; 649: }	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1502	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 877
	步骤描述: 不使用返回值的函数'BITv07_reloadDStream'应使用'void'说明.	

	代码片段: 875: const FSEv07_DTableHeader* const DTableH = (const FSEv07_DTableHeader*)ptr; 876: DStatePtr->state = BITv07_readBits(bitD, DTableH->tableLog); 877: BITv07_reloadDStream(bitD); 878: DStatePtr->table = dt + 1; 879: }	
	污染路径: 函数'BITv07_reloadDStream'的定义点.	
1503	缺陷发生位置: /source/dnode/vnode/src/meta/metaCache.c	行号: 357
	步骤描述: 不使用返回值的函数'metaError'应使用'void'说明.	
	代码片段: 355: if (*ppEntry) { // update 356: if (pInfo->suid != (*ppEntry)->info.suid) { 357: metaError("meta/cache: suid should be same as the one in cache."); 358: return TSDB_CODE_INVALID_PARA; 359: }	
	污染路径: 函数'metaError'的定义点.	
1504	缺陷发生位置: /source/util/src/talgo.c	行号: 384
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 382: 383: if (buf == NULL) { 384: taosMemoryFree(tmp); 385: } 386:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1505	缺陷发生位置: /tools/shell/src/shellCommand.c	行号: 544
	步骤描述: 不使用返回值的函数'pressOtherKey'应使用'void'说明.	
	代码片段:	

	542: pressTabKey(&cmd); 543: } else if (c < '\033') { 544: pressOtherKey(c); 545: // Ctrl keys. TODO: Implement ctrl combinations 546: switch (c) {	
	污染路径: 函数'pressOtherKey'的定义点.	
1506	缺陷发生位置: /source/libs/tmqt/mqtt/src/tmqtCtx.c	行号: 140
	步骤描述: 不使用返回值的函数'bndError'应使用'void'说明.	
	代码片段: 138: context->tmq = tmq_ctx_build_consumer(config); 139: if (!context->tmq) { 140: bndError("failed to build consumer: %s %s", config->client_id, config->group_id); 141: 142: return false;	
	污染路径: 函数'bndError'的定义点.	
1507	缺陷发生位置: /source/util/src/thash.c	行号: 204
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 202: pNewNode->next = pNode->next; 203: 204: FREE_HASH_NODE(pHashObj->freeFp, pNode); 205: } else { 206: pNewNode->next = pNode;	
	污染路径: expanded from macro 'FREE_HASH_NODE' 函数'taosMemoryFreeClear'的定义点. expanded from macro 'FREE_HASH_NODE'	
1508	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqSubs.c	行号: 169
	步骤描述: 不使用返回值的函数'DL_APPEND'应使用'void'说明.	
	代码片段: 167: leaf->retain_as_published = ((options &	

	MQTT_SUB_OPT_RETAIN_AS_PUBLISHED) != 0); 168: 169: DL_APPEND(*head, leaf); 170: *newleaf = leaf; 171:	
	污染路径: 函数'DL_APPEND'的定义点.	
1509	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 648
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 646: pAPI->metaReaderFn.clearReader(&smrTable); 647: pInfo->loadInfo.totalRows = 0; 648: qError("%s failed at line %d since %s", __func__, __LINE__, tstrerror(code)); 649: pTaskInfo->code = code; 650: T_LONG_JMP(pTaskInfo->env, code);	
	污染路径: 函数'qError'的定义点.	
1510	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 1181
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 1179: } 1180: 1181: taosMemoryFreeClear(line); 1182: taosCloseFile(&pFile); 1183: int64_t file_size;	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1511	缺陷发生位置: /source/dnode/mnode/impl/src/mndTopic.c	行号: 267
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 265: TOPIC_DECODE_OVER: 266: taosMemoryFreeClear(buf); 267: taosMemoryFreeClear(ast);	

	268: 269: if (terrno != TSDB_CODE_SUCCESS) {	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1512	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 1084
	步骤描述: 不使用返回值的函数'prepareColData'应使用'void'说明.	
	代码片段: 1082: for (int b = 0; b < bindNum; b++) { 1083: for (int c = 0; c < gCurCase->bindColNum; ++c) { 1084: prepareColData(true, BP_BIND_COL, data, b * gCurCase->bindColNum + c, b * gCurCase->bindRowNum, c); 1085: } 1086: }	
	污染路径: 函数'prepareColData'的定义点.	
1513	缺陷发生位置: /source/libs/tdb/test/tdbPageRecycleTest.cpp	行号: 380
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 378: for (int iData = 1; iData <= nData; iData++) { 379: sprintf(key, "key0"); 380: sprintf(val, "value%d", iData); 381: 382: // ret = tdbTbInsert(pDb, key, strlen(key), val, strlen(val), &txn);	
	污染路径: 无	
1514	缺陷发生位置: /utls/tsim/src/simExec.c	行号: 270
	步骤描述: 不使用返回值的函数'tstrncpy'应使用'void'说明.	
	代码片段: 268: } 269: } else { 270: tstrncpy(t3, t1, sizeof(t3)); 271: } 272:	

1518	缺陷发生位置: /source/dnode/mnode/impl/src/mndConfig.c	行号: 255
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 253: SConfigObj *vObj = sdbAcquire(pMnode->pSdb, SDB_CFG, "tsmmConfigVersion"); 254: if (vObj == NULL) { 255: mInfo("failed to acquire mnd config version, since %s", terrstr()); 256: goto _OVER; 257: }	
	污染路径: 函数'mInfo'的定义点.	
1519	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqMux.c	行号: 33
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 31: #ifndef WITH_EPOLL 32: UNUSED(listensock); 33: UNUSED(listensock_count); 34: return -1; 35: #else	
	污染路径: 函数'UNUSED'的定义点.	
1520	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmlnt.c	行号: 57
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 55: SVnodeObj *pCreatingVnode = taosMemoryCalloc(1, sizeof(SVnodeObj)); 56: if (pCreatingVnode == NULL) { 57: dError("failed to alloc vnode since %s", terrstr()); 58: return terrno; 59: }	
	污染路径: 函数'dError'的定义点.	
1521	缺陷发生位置:	行号:

	/tools/taos-tools/src/benchQuery.c	213
	步骤描述: 不使用返回值的函数'prctl'应使用'void'说明.	
	代码片段: 211: qThreadInfo *pThreadInfo = (qThreadInfo *)sarg; 212: #ifdef LINUX 213: prctl(PR_SET_NAME, "specQueryThread"); 214: #endif 215: uint64_t st = 0;	
	污染路径: 无	
1522	缺陷发生位置: /contrib/test/traft/join_into_vgroup/node_leader10002.c	行号: 87
	步骤描述: 不使用返回值的函数'raft_free'应使用'void'说明.	
	代码片段: 85: fprintf(stderr, "joinRaftPeerCb ok \n"); 86: } 87: raft_free(req); 88: } 89:	
	污染路径: 函数'raft_free'的定义点.	
1523	缺陷发生位置: /source/dnode/vnode/src/tq/tqMeta.c	行号: 150
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 148: END: 149: tEncoderClear(&encoder); 150: taosMemoryFree(buf); 151: return code; 152: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1524	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeSimpleTest.cpp	行号: 217
	步骤描述: 不使用返回值的函数'sdbSetRawInt32'应使用'void'说明.	
	代码片段:	

	215: 216: int32_t dataPos = 0; 217: sdbSetRawInt32(pRaw, dataPos, obj.id); 218: dataPos += 4; 219: sdbSetRawInt32(pRaw, dataPos, obj.urlLen);	
	污染路径: 函数'sdbSetRawInt32'的定义点.	
1525	缺陷发生位置: /source/libs/transport/src/transSvr.c	行号: 312
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 310: SWhiteUserList* pUserList = *(SWhiteUserList**)plter; 311: taosMemoryFree(pUserList->pList); 312: taosMemoryFree(pUserList); 313: 314: plter = taosHashIterate(pWhiteList, plter);	
	污染路径: 函数'taosMemoryFree'的定义点.	
1526	缺陷发生位置: /source/libs/executor/src/executil.c	行号: 2056
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 2054: (nodeType(pTagCond) != QUERY_NODE_OPERATOR && 2055: nodeType(pTagCond) != QUERY_NODE_LOGIC_CONDITION)) { 2056: qError("invalid parameter to extract tag filter symbol"); 2057: return TSDB_CODE_INTERNAL_ERROR; 2058: }	
	污染路径: 函数'qError'的定义点.	
1527	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmNodes.c	行号: 181
	步骤描述: 不使用返回值的函数'taosMsleep'应使用'void'说明.	
	代码片段: 179: }	

	180: 181: taosMsleep(100); 182: } 183: }	
	污染路径: 函数'taosMsleep'的定义点.	
1528	缺陷发生位置: /source/libs/new-stream/src/dataSinkMgr.c	行号: 108
	步骤描述: 不使用返回值的函数'stError'应使用'void'说明.	
	代码片段: 106: static void destroySStreamDSTaskMgr(void* pData) { 107: if (pData == NULL *(void**)pData == NULL) { 108: stError("invalid data sink manager"); 109: return; 110: } 	
	污染路径: 函数'stError'的定义点.	
1529	缺陷发生位置: /utls/test/c/tmq_get_meta_json.c	行号: 160
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 158: code = taos_errno(pRes); 159: printf("%s result0:%s\n", __FUNCTION__, taos_errstr(pRes)); 160: ASSERT(code == 0); 161: taos_free_result(pRes); 162:	
	污染路径: 函数'ASSERT'的定义点.	
1530	缺陷发生位置: /source/dnode/vnode/src/bse/bseMgt.c	行号: 370
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 368: bseError("vgId:%d, failed to recover since %s", BSE_VGID(pBse), tsterror(code)); 369: } 370: taosMemoryFree(pCurrent);	

	371: return code; 372: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1531	缺陷发生位置: /source/libs/executor/src/exchangeoperator.c	行号: 507
	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: 505: int32_t status = handleLimitOffset(pOperator, pLimitInfo, pBlock, false); 506: if (status == PROJECT_RETRIEVE_CONTINUE) { 507: qDebug("limit retrieve continue"); 508: continue; 509: } else if (status == PROJECT_RETRIEVE_DONE) {	
	污染路径: 函数'qDebug'的定义点.	
1532	缺陷发生位置: /source/client/src/clientTmq.c	行号: 334
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 332: if (conf) { 333: if (conf->ip) { 334: taosMemoryFree(conf->ip); 335: } 336: if (conf->user) {	
	污染路径: 函数'taosMemoryFree'的定义点.	
1533	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/gnu-test-skeleton.c	行号: 329
	步骤描述: 不使用返回值的函数'alarm'应使用'void'说明.	
	代码片段: 327: #endif 328: signal (SIGALRM, timeout_handler); 329: alarm (TIMEOUT * timeoutfactor); 330: 331: /* Wait for the regular termination. */	

	污染路径: 无	
1534	缺陷发生位置: /source/dnode/mnode/impl/src/mndUser.c	行号: 225
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 223: taosMemoryFree((*ppInfo)->wllp); 224: taosMemoryFree((*ppInfo)->wlTime); 225: taosMemoryFree(*ppInfo); 226: } 227: if (taosHashRemove(userCache.users, user, userLen) != 0) {	
	污染路径: 函数'taosMemoryFree'的定义点.	
1535	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 5816
	步骤描述: 不使用返回值的函数'tsdbError'应使用'void'说明.	
	代码片段: 5814: pReader->pSchemaMap = tSimpleHashInit(8, taosFastHash); 5815: if (pReader->pSchemaMap == NULL) { 5816: tsdbError("failed init schema hash for reader %s", pReader->idStr); 5817: TSDB_CHECK_NULL(pReader->pSchemaMap, code, lino, _end, terrno); 5818: }	
	污染路径: 函数'tsdbError'的定义点.	
1536	缺陷发生位置: /tools/taos-tools/src/benchCommandOpt.c	行号: 581
	步骤描述: 不使用返回值的函数'pthread_create'应使用'void'说明.	
	代码片段: 579: } 580: if (stbInfo->use_metric) { 581: pthread_create(&read_id, NULL, queryStableAggrFunc, pThreadInfo); 582: } else {	

	583: pthread_create(&read_id, NULL, queryNtableAggrFunc, pThreadInfo);	
	污染路径: 无	
1537	缺陷发生位置: /test/new_test_framework/script/api/asyncQuery.c	行号: 56
	步骤描述: 不使用返回值的函数'atomic_add_fetch_64'应使用'void'说明.	
	代码片段: 54: taos_free_result(res); 55: atomic_add_fetch_64(&finQueries, 1); 56: atomic_add_fetch_64(&errTimes, 1); 57: } else if (nrows == 0) { 58: taos_free_result(res);	
	污染路径: 函数'atomic_add_fetch_64'的定义点.	
1538	缺陷发生位置: /source/util/src/tjson.c	行号: 218
	步骤描述: 不使用返回值的函数'TAOS_STRCPY'应使用'void'说明.	
	代码片段: 216: return TSDB_CODE_OUT_OF_MEMORY; 217: } 218: TAOS_STRCPY(pVal, p); 219: return TSDB_CODE_SUCCESS; 220: }	
	污染路径: 函数'TAOS_STRCPY'的定义点.	
1539	缺陷发生位置: /source/libs/tdb/src/db/tdbBtree.c	行号: 130
	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	
	代码片段: 128: ret = tdbPagerWrite(pPager, pPage); 129: if (ret < 0) { 130: tdbError("failed to write page since %s", terrstr()); 131: tdbAbort(pEnv, txn); 132: tdbOsFree(pBt);	
	污染路径:	

	函数'tdbError'的定义点.	
1540	缺陷发生位置: /source/os/src/osSemaphore.c	行号: 274
	步骤描述: 不使用返回值的函数'OS_PARAM_CHECK'应使用'void'说明.	
	代码片段: 272: int32_t taosGetPIdByName(const char* name, int32_t* pPId) { 273: #ifndef TD_ASTR 274: OS_PARAM_CHECK(name); 275: OS_PARAM_CHECK(pPId); 276: DIR* dir = NULL;	
	污染路径: 函数'OS_PARAM_CHECK'的定义点.	
1541	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqLoop.c	行号: 188
	步骤描述: 不使用返回值的函数'tmq_ctx_poll_msgs'应使用'void'说明.	
	代码片段: 186: } 187: 188: tmq_ctx_poll_msgs(); 189: 190: if (ttq_ppid_changed()) {	
	污染路径: 函数'tmq_ctx_poll_msgs'的定义点.	
1542	缺陷发生位置: /tools/taos-tools/src/benchDataMix.c	行号: 135
	步骤描述: 不使用返回值的函数'tmpGeometry'应使用'void'说明.	
	代码片段: 133: break; 134: case TSDB_DATA_TYPE_GEOMETRY: 135: tmpGeometry(val, 0, fd, 0); 136: break; 137: default:	
	污染路径: 函数'tmpGeometry'的定义点.	
1543	缺陷发生位置: /source/libs/executor/src/externalwindowoperator.c	行号: 458

	不使用返回值的函数'prepareColData'应使用'void'说明.	
	代码片段: <pre> 1007: for (int b = 0; b < gCurCase->tblNum; b++) { 1008: for (int c = 0; c < gCurCase->bindTagNum; ++c) { 1009: prepareColData(false, BP_BIND_TAG, data, b*gCurCase->bindTagNum+c, b, c); 1010: } 1011: }</pre>	
	污染路径: 函数'prepareColData'的定义点.	
1547	缺陷发生位置: /source/libs/executor/src/virtualtablescanoperator.c	行号: 339
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 337: } 338: if (pCtx) { 339: taosMemoryFree(pCtx); 340: } 341: NODES_DESTORY_LIST(pMergeKeys);</pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
1548	缺陷发生位置: /test/new_test_framework/script/api/stmt2-get-fields.c	行号: 253
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 251: do_stmt(taos); 252: taos_close(taos); 253: taos_cleanup(); 254: }</pre>	
	污染路径: 函数'taos_cleanup'的定义点.	
1549	缺陷发生位置: /source/client/test/clientTests.cpp	行号: 1922
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 1920: taos_free_result(pRes); 1921:</pre>	

	1922: sprintf(sql, "create database %s vgroups 4", database); 1923: pRes = taos_query(pRootConn, sql); 1924: ASSERT_EQ(taos_errno(pRes), TSDB_CODE_SUCCESS);	
	污染路径: 无	
1550	缺陷发生位置: /source/libs/catalog/src/ctgCache.c	行号: 239
	步骤描述: 不使用返回值的函数'ctgDebug'应使用'void'说明.	
	代码片段: 237: CTG_ERR_JRET(ctgRLockVgInfo(pCtg, dbCache, &inCache)); 238: if (!inCache) { 239: ctgDebug("db:%s, vgInfo not in cache", dbFName); 240: goto _return; 241: }	
	污染路径: 函数'ctgDebug'的定义点.	
1551	缺陷发生位置: /tools/shell/src/shellCommand.c	行号: 447
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 445: } 446: 447: taosMemoryFree(total_string); 448: // Position the cursor 449: int32_t cursor_pos = cmd->screenOffset + PSIZE;	
	污染路径: 函数'taosMemoryFree'的定义点.	
1552	缺陷发生位置: /test/new_test_framework/script/api/api_with_reqid_test.c	行号: 22
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 20: result = taos_query_with_reqid(taos, "create database test precision 'us';", genReqid()); 21: taos_free_result(result); 22: usleep(100000);	

	23: taos_select_db(taos, "test"); 24:	
	污染路径: 无	
1553	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmHandle.c	行号: 151
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 149: 150: if (tfsGetMonitorInfo(pMgmt->pTfs, &pInfo->tfs) != 0) { 151: dError("failed to get tfs monitor info"); 152: } 153: taosArrayDestroy(pVloads);	
	污染路径: 函数'dError'的定义点.	
1554	缺陷发生位置: /source/client/src/clientTmq.c	行号: 345
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 343: taosMemoryFree(conf->token); 344: } 345: taosMemoryFree(conf); 346: } 347: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1555	缺陷发生位置: /source/util/src/tcompare.c	行号: 317
	步骤描述: 不使用返回值的函数'DEFAULT_DOUBLE_COMP'应使用'void'说明.	
	代码片段: 315: DEFAULT_COMP(GET_INT8_VAL(realDataLeft), GET_INT8_VAL(realDataRight)); 316: } else if (leftType == TSDB_DATA_TYPE_DOUBLE) { 317: DEFAULT_DOUBLE_COMP(GET_DOUBLE_VAL(realDataLeft), GET_DOUBLE_VAL(realDataRight)); 318: } else if (leftType == TSDB_DATA_TYPE_NCHAR) {	

	319: return compareLenPrefixedWStr(realDataLeft, realDataRight);	
	污染路径: 函数'DEFAULT_DOUBLE_COMP'的定义点.	
1556	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbScan.c	行号: 297
	步骤描述: 不使用返回值的函数'scanArgDestroy'应使用'void'说明.	
	代码片段: 295: tsdbTFileSetClear(&ctx.fset); 296: tsdbRemoveScanMonitorTask(tsdb, &scanArg->taskid); 297: scanArgDestroy(scanArg); 298: return code; 299: }	
	污染路径: 函数'scanArgDestroy'的定义点.	
1557	缺陷发生位置: /source/dnode/mnode/impl/src/mndStreamMgmt.c	行号: 93
	步骤描述: 不使用返回值的函数'mstInfo'应使用'void'说明.	
	代码片段: 91: memset(mStreamMgmt.lastTs, 0, sizeof(mStreamMgmt.lastTs)); 92: 93: mstInfo("mnode stream mgmt destroyed"); 94: } 95:	
	污染路径: 函数'mstInfo'的定义点.	
1558	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCacheRead.c	行号: 53
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 51: } 52: if (buf != NULL) { 53: taosMemoryFreeClear(buf); 54: } 55: return code;	

	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1559	缺陷发生位置: /source/os/src/osFile.c	行号: 227
	步骤描述: 不使用返回值的函数'OS_PARAM_CHECK'应使用'void'说明.	
	代码片段: 225: 226: int32_t taosRemoveFile(const char *path) { 227: OS_PARAM_CHECK(path); 228: int32_t code = remove(path); 229: if (-1 == code) {	
	污染路径: 函数'OS_PARAM_CHECK'的定义点.	
1560	缺陷发生位置: /source/libs/executor/src/tsort.c	行号: 274
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 272: if (pDesc != NULL && pDesc->type == AllocatedTupleType) { 273: destoryAllocatedTuple(pDesc->data); 274: taosMemoryFree(pDesc); 275: } 276: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1561	缺陷发生位置: /test/new_test_framework/script/api/insertBlob.c	行号: 73
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 71: Test(taos, qstr); 72: taos_close(taos); 73: taos_cleanup(); 74: free(qstr); 75: }	
	污染路径: 函数'taos_cleanup'的定义点.	
1562	缺陷发生位置:	行号:

	/source/util/src/tcompression.c	150
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: <pre> 148: return compressedSize - 1; 149: } else if (input[1] == 2) { 150: uError("Invalid decompress string indicator:%d", input[0]); 151: return TSDB_CODE_THIRDPARTY_ERROR; 152: }</pre>	
	污染路径: 函数'uError'的定义点.	
1563	缺陷发生位置: /source/util/src/tdecompressavx.c	行号: 284
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: <pre> 282: return (int32_t)(out - output); 283: #else 284: uError("unable run %s without avx2 instructions", __func__); 285: return -1; 286: #endif</pre>	
	污染路径: 函数'uError'的定义点.	
1564	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFS2.c	行号: 73
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 71: tsdbError("failed to destroy semaphore"); 72: } 73: taosMemoryFree(fs[0]); 74: fs[0] = NULL; 75: }</pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
1565	缺陷发生位置: /source/dnode/snode/src/snode.c	行号: 132
	步骤描述:	

	不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 130: memcpy(rsp.pCont, data, dataLen); 131: rsp.contLen = dataLen; 132: taosMemoryFreeClear(data); 133: } 134: }	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1566	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 724
	步骤描述: 不使用返回值的函数'geosFreeBuffer'应使用'void'说明.	
	代码片段: 722: shellPrintString(outputWKT, width); 723: 724: geosFreeBuffer(outputWKT); 725: } 726:	
	污染路径: 函数'geosFreeBuffer'的定义点.	
1567	缺陷发生位置: /contrib/libmqt/qtt/src/ttqMux.c	行号: 32
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 30: int ttqMuxInit(struct tmqtt__listener_sock *listensock, int listensock_count) { 31: #ifdef WITH_EPOLL 32: UNUSED(listensock); 33: UNUSED(listensock_count); 34: return -1;	
	污染路径: 函数'UNUSED'的定义点.	
1568	缺陷发生位置: /source/dnode/vnode/src/tq/tqRead.c	行号: 1212
	步骤描述: 不使用返回值的函数'TQ_NULL_GO_TO_END'应使用'void'说明.	
	代码片段: 1210: SSchemaWrapper* pSchemaWrapper = pReader-	

	>pSchemaWrapper; 1211: char* assigned = taosMemoryCalloc(1, pSchemaWrapper->nCols); 1212: TQ_NULL_GO_TO_END(assigned); 1213: 1214: int32_t curRow = 0;	
	污染路径: 函数'TQ_NULL_GO_TO_END'的定义点.	
1569	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbScan.c	行号: 290
	步骤描述: 不使用返回值的函数'tsdbDoScanFileSet'应使用'void'说明.	
	代码片段: 288: 289: // Do scan the file set 290: tsdbDoScanFileSet(&ctx); 291: 292: _exit:	
	污染路径: 函数'tsdbDoScanFileSet'的定义点.	
1570	缺陷发生位置: /source/util/src/tmempool.c	行号: 431
	步骤描述: 不使用返回值的函数'uInfo'应使用'void'说明.	
	代码片段: 429: } 430: 431: uInfo("MemPool [%s] stat detail:", detailName); 432: 433: uInfo("Max Used Memory Size: %" PRIu64, maxAllocSize);	
	污染路径: 函数'uInfo'的定义点.	
1571	缺陷发生位置: /source/client/src/clientSml.c	行号: 234
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 232: smlStrReplace(measure, measureLen); 233: code = smlGetMeta(info, measure, measureLen,	

	&pTableMeta); 234: taosMemoryFree(measure); 235: if (code != TSDB_CODE_SUCCESS) { 236: info->dataFormat = false;	
	污染路径: 函数'taosMemoryFree'的定义点.	
1572	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeSnapshot.c	行号: 713
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 711: } else { 712: vInfo("vgId:%d, vnode snapshot writer closed, rollback:%d", TD_VID(pVnode), rollback); 713: taosMemoryFree(pWriter); 714: } 715: if (vnodeEnableBgTask(pVnode) != 0) {	
	污染路径: 函数'taosMemoryFree'的定义点.	
1573	缺陷发生位置: /source/dnode/mnode/impl/src/mndTrans.c	行号: 856
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 854: if (pTrans == NULL) { 855: terrno = TSDB_CODE_OUT_OF_MEMORY; 856: mError("failed to create transaction since %s", terrstr()); 857: return NULL; 858: }	
	污染路径: 函数'mError'的定义点.	
1574	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdblter.c	行号: 594
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 592: return TSDB_CODE_INVALID_PARA; 593: } 594: taosMemoryFree(iter[0]);	

	595: iter[0] = NULL; 596: return 0;	
	污染路径: 函数'taosMemoryFree'的定义点.	
1575	缺陷发生位置: /source/libs/executor/src/projectoperator.c	行号: 254
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 252: // here check for a new group data, we need to handle the data of the previous group. 253: if (!(pLimitInfo->remainGroupOffset == 0 pLimitInfo->remainGroupOffset == -1)) { 254: qError("project failed at: %s:%d", __func__, __LINE__); 255: return TSDB_CODE_INVALID_PARA; 256: }	
	污染路径: 函数'qError'的定义点.	
1576	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdblter.c	行号: 556
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 554: 555: if (code) { 556: taosMemoryFree(iter[0]); 557: iter[0] = NULL; 558: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1577	缺陷发生位置: /source/libs/tdb/test/tdbExOVFLTest.cpp	行号: 396
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 394: 395: for (int iData = 1; iData <= nData; iData++) { 396: sprintf(key, "key0"); 397: sprintf(val, "value%d", iData); 398:	

	污染路径: 无	
1578	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSet2.c	行号: 735
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 733: (void)taosThreadCondDestroy(&(*fset)->conds[i].cond); 734: } 735: taosMemoryFreeClear(*fset); 736: } 737: }	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1579	缺陷发生位置: /docs/examples/c/query_data_demo.c	行号: 38
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 36: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x %x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 37: taos_errstr(NULL)); 38: taos_cleanup(); 39: return -1; 40: }	
	污染路径: 函数'taos_cleanup'的定义点.	
1580	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeAsync.c	行号: 184
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 182: if (task->numWait == 0) { 183: (void)taosThreadCondDestroy(&task->waitCond); 184: taosMemoryFree(task); 185: } else if (task->numWait == 1) { 186: (void)taosThreadCondSignal(&task->waitCond);	
	污染路径:	

	函数'taosMemoryFree'的定义点.	
1581	缺陷发生位置: /source/libs/qcom/src/querymsg.c	行号: 78
	步骤描述: 不使用返回值的函数'QUERY_PARAM_CHECK'应使用'void'说明.	
	代码片段: 76: int32_t queryBuildTableMetaReqMsg(void *input, char **msg, int32_t msgSize, int32_t *msgLen, 77: void *(*mallocFp)(int64_t), void (*freeFp)(void *)) { 78: QUERY_PARAM_CHECK(input); 79: QUERY_PARAM_CHECK(msg); 80: QUERY_PARAM_CHECK(msgLen);	
	污染路径: 函数'QUERY_PARAM_CHECK'的定义点.	
1582	缺陷发生位置: /source/libs/wal/src/walMgmt.c	行号: 79
	步骤描述: 不使用返回值的函数'wlInfo'应使用'void'说明.	
	代码片段: 77: walStopThread(); 78: taosCloseRef(tsWal.refSetId); 79: wlInfo("wal module is cleaned up"); 80: atomic_store_8(&tsWal.inited, 0); 81: }	
	污染路径: 函数'wlInfo'的定义点.	
1583	缺陷发生位置: /docs/examples/c/demo.c	行号: 107
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 105: 106: // query the records 107: sprintf(qstr, "SELECT * FROM m1"); 108: result = taos_query(taos, qstr); 109: if (result == NULL taos_errno(result) != 0) {	
	污染路径: 无	
1584	缺陷发生位置:	行号:

	/contrib/libmqtt/ttq/src/ttqPacket.c	112
	步骤描述: 不使用返回值的函数'G_MSGS_DROPPED_INC'应使用'void'说明.	
	代码片段: 110: ttq_log(NULL, TTQ_LOG_NOTICE, "Outgoing messages are being dropped for client %s.", ttq->id); 111: } 112: G_MSGS_DROPPED_INC(); 113: return TTQ_ERR_SUCCESS; 114: }	
1585	污染路径: 函数'G_MSGS_DROPPED_INC'的定义点.	
	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 306
	步骤描述: 不使用返回值的函数'callbackAutoTab'应使用'void'说明.	
1586	代码片段: 304: 305: // call auto tab 306: callbackAutoTab(command, NULL, false); 307: } 308:	
	污染路径: 函数'callbackAutoTab'的定义点.	
	缺陷发生位置: /source/util/src/tconfig.c	行号: 203
1587	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 201: (void)taosThreadMutexDestroy(&pCfg->lock); 202: 203: taosMemoryFree(pCfg); 204: } 205:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1587	缺陷发生位置: /source/dnode/mnode/impl/src/mndProfile.c	行号: 285
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	

	代码片段: 283: SCacheIter *plter = taosCacheCreateIter(pMgmt->connCache); 284: if (plter == NULL) { 285: mError("failed to create conn cache iterator"); 286: return -1; 287: }	
	污染路径: 函数'mError'的定义点.	
1588	缺陷发生位置: /source/libs/monitorfw/src/taos_metric_formatter_custom.c	行号: 55
	步骤描述: 不使用返回值的函数'taos_monitor_split_str'应使用'void'说明.	
	代码片段: 53: } 54: memset(keyvalue, 0, sizeof(char*) * (count + 1)); 55: taos_monitor_split_str(keyvalue, keyvalues, ","); 56: 57: char** arr = taosMemoryMalloc(sizeof(char*) * (count + 1) * 2);	
	污染路径: 函数'taos_monitor_split_str'的定义点.	
1589	缺陷发生位置: /utils/tsim/src/simSystem.c	行号: 124
	步骤描述: 不使用返回值的函数'simDebug'应使用'void'说明.	
	代码片段: 122: script = simProcessCallOver(script); 123: if (script == NULL) { 124: simDebug("sim test abort now!"); 125: break; 126: }	
	污染路径: 函数'simDebug'的定义点.	
1590	缺陷发生位置: /source/dnode/mgmt/node_util/src/dmFile.c	行号: 439
	步骤描述: 不使用返回值的函数'encryptError'应使用'void'说明.	
	代码片段: 437: if (strcmp(opts.result, DM_KEY_INDICATOR) != 0) {	

	438: code = TSDB_CODE_DNODE_ENCRYPTKEY_CHANGED; 439: encryptError("failed to compare decrypted result"); 440: goto _OVER; 441: }	
	污染路径: 函数'encryptError'的定义点.	
1591	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1789
	步骤描述: 不使用返回值的函数'HUF_decodeStreamX2'应使用'void'说明.	
	代码片段: 1787: /* finish bitStreams one by one */ 1788: HUF_decodeStreamX2(op1, &bitD1, opStart2, dt, dtLog); 1789: HUF_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 1790: HUF_decodeStreamX2(op3, &bitD3, opStart4, dt, dtLog); 1791: HUF_decodeStreamX2(op4, &bitD4, oend, dt, dtLog);	
	污染路径: 函数'HUF_decodeStreamX2'的定义点.	
1592	缺陷发生位置: /source/dnode/mnode/impl/src/mndSync.c	行号: 292
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 290: 291: int32_t mndSyncGetSnapshot(const SSyncFSM *pFsm, SSnapshot *pSnapshot, void *pReaderParam, void **ppReader) { 292: mInfo("start to read snapshot from sdb in atomic way"); 293: SMnode *pMnode = pFsm->data; 294: return sdbStartRead(pMnode->pSdb, (SSdbIter **)ppReader, &pSnapshot->lastApplyIndex, &pSnapshot-> lastApplyTerm,	
	污染路径: 函数'mInfo'的定义点.	
1593	缺陷发生位置: /source/util/src/theap.c	行号: 222

	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 220: else 221: taosArrayDestroyP(pq->container, NULL); 222: taosMemoryFree(pq); 223: } 224:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1594	缺陷发生位置:	行号:
	/utils/test/c/blob_test.c	341
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 339: // Allocate and assign table name 340: (*table_name)[i] = (char *)taosMemMalloc(20 * sizeof(char)); 341: sprintf((*table_name)[i], "d_bind_%d", i); 342: 343: // Allocate memory for tags data	
1595	污染路径: 无	
	缺陷发生位置:	行号:
	/tools/taos-tools/src/benchData.c	2659
	步骤描述: 不使用返回值的函数'tmpGeometry'应使用'void'说明.	
	代码片段: 2657: case TSDB_DATA_TYPE_GEOMETRY: { 2658: char *buf = (char *)benchCalloc(col->length + 1, 1, false); 2659: tmpGeometry(buf, stbInfo->iface, col, 0); 2660: tools_cJSON_AddStringToObject(value, "value", buf); 2661: tools_cJSON_AddStringToObject(value, "type", "geometry");	
1596	污染路径: 函数'tmpGeometry'的定义点.	
	缺陷发生位置:	行号:
	/source/libs/tdb/src/db/tdbPCache.c	55

	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	
	代码片段: 53: static void tdbPCacheDestroyLock(SPCache *pCache) { 54: if (tdbMutexDestroy(&(pCache->mutex)) != 0) { 55: tdbError("tdb/pcache: mutex destroy failed."); 56: } 57: }	
	污染路径: 函数'tdbError'的定义点.	
1597	缺陷发生位置: /source/libs/tss/src/fs.c	行号: 147
	步骤描述: 不使用返回值的函数'taosMemFree'应使用'void'说明.	
	代码片段: 145: static int32_t closeInstance(SSharedStorage* pss) { 146: SSharedStorageFS* ss = (SSharedStorageFS*)pss; 147: taosMemFree(ss); 148: return TSDB_CODE_SUCCESS; 149: }	
	污染路径: 函数'taosMemFree'的定义点.	
1598	缺陷发生位置: /test/new_test_framework/script/api/dbTableRoute.c	行号: 91
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 89: rtExecSQL(taos, sql); 90: 91: sprintf(sql, "create database db1 vgroups 10 table_prefix %d table_suffix %d", prefix, suffix); 92: rtExecSQL(taos, sql); 93:	
	污染路径: 无	
1599	缺陷发生位置: /docs/examples/c/tmq_demo.c	行号: 64
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段:	

	62: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x %x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 63: taos_errstr(NULL)); 64: taos_cleanup(); 65: return NULL; 66: }	
	污染路径: 函数'taos_cleanup'的定义点.	
1600	缺陷发生位置: /source/os/test/osStringTests.cpp	行号: 318
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 316: ASSERT_EQ(result, TAOS_SYSTEM_ERROR(ERANGE)); 317: 318: snprintf(large_num, sizeof(large_num), "%lld", (long long)INT16_MIN - 1); 319: result = taosStr2int16(large_num, &val); 320: ASSERT_EQ(result, TAOS_SYSTEM_ERROR(ERANGE));	
	污染路径: 无	
1601	缺陷发生位置: /source/client/wrapper/jni/windows/win32/bridge/AccessBridgeCalls.c	行号: 76
	步骤描述: 不使用返回值的函数'PrintDebugString'应使用'void'说明.	
	代码片段: 74: LOAD_FP(Windows_run, Windows_runFP, "Windows_run"); 75: 76: LOAD_FP(SetJavaShutdown, SetJavaShutdownFP, "setJavaShutdownFP"); 77: LOAD_FP(SetFocusGained, SetFocusGainedFP, "setFocusGainedFP"); 78: LOAD_FP(SetFocusLost, SetFocusLostFP, "setFocusLostFP");	
	污染路径: expanded from macro 'LOAD_FP' 函数'PrintDebugString'的定义点. expanded from macro 'LOAD_FP'	
1602	缺陷发生位置:	行号:

	/tools/shell/src/shellMain.c	146
	步骤描述: 不使用返回值的函数'shellAutoInit'应使用'void'说明.	
	代码片段: 144: #endif 145: // support port feature 146: shellAutoInit(); 147: int32_t ret = shellExecute(argc, argv); 148: shellAutoExit();	
	污染路径: 函数'shellAutoInit'的定义点.	
1603	缺陷发生位置: /tools/taos-tools/src/benchQuery.c	行号: 186
	步骤描述: 不使用返回值的函数'autoSleep'应使用'void'说明.	
	代码片段: 184: // sleep 185: if (interval > 0) { 186: autoSleep(interval, et - st); 187: } 188:	
	污染路径: 函数'autoSleep'的定义点.	
1604	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttContext.c	行号: 143
	步骤描述: 不使用返回值的函数'ttqCxtRemoveFromById'应使用'void'说明.	
	代码片段: 141: 142: if (context->id) { 143: ttqCxtRemoveFromById(context); 144: ttq_free(context->id); 145: context->id = NULL;	
	污染路径: 函数'ttqCxtRemoveFromById'的定义点.	
1605	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqNet.c	行号: 812
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段:	

	810: } 811: #else 812: UNUSED(ttq); 813: UNUSED(host); 814: #endif	
	污染路径: 函数'UNUSED'的定义点.	
1606	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1791
	步骤描述: 不使用返回值的函数'HUF_decodeStreamX2'应使用'void'说明.	
	代码片段: 1789: HUF_decodeStreamX2(op2, &bitD2, opStart3, dt, dtLog); 1790: HUF_decodeStreamX2(op3, &bitD3, opStart4, dt, dtLog); 1791: HUF_decodeStreamX2(op4, &bitD4, oend, dt, dtLog); 1792: 1793: /* check */	
	污染路径: 函数'HUF_decodeStreamX2'的定义点.	
1607	缺陷发生位置: /utils/test/c/tmq_ts7662.c	行号: 34
	步骤描述: 不使用返回值的函数'tmq_free_json_meta'应使用'void'说明.	
	代码片段: 32: printf("meta result: %s\n", result); 33: ASSERT(strstr(result, "t2") == NULL); 34: tmq_free_json_meta(result); 35: } 36: }	
	污染路径: 函数'tmq_free_json_meta'的定义点.	
1608	缺陷发生位置: /source/dnode/mnode/impl/src/mndDb.c	行号: 3159
	步骤描述: 不使用返回值的函数'STR_WITH_MAXSIZE_TO_VARSTR'应使用'void'说明.	
	代码片段: 3157: }	

	3158: char statusVstr[24] = {0}; 3159: STR_WITH_MAXSIZE_TO_VARSTR(statusVstr, statusStr, 24); 3160: 3161: if (sysDb !sysinfo) {	
	污染路径: 函数'STR_WITH_MAXSIZE_TO_VARSTR'的定义点.	
1609	缺陷发生位置: /source/libs/executor/src/tsort.c	行号: 289
	步骤描述: 不使用返回值的函数'QRY_PARAM_CHECK'应使用'void'说明.	
	代码片段: 287: int32_t lino = 0; 288: 289: QRY_PARAM_CHECK(pHandle); 290: SSortHandle* pSortHandle = taosMemoryCalloc(1, sizeof(SSortHandle)); 291: QUERY_CHECK_NULL(pSortHandle, code, lino, _err, terrno);	
	污染路径: 函数'QRY_PARAM_CHECK'的定义点.	
1610	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqDb.c	行号: 335
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 333: struct tmqtt_client_msg *msg; 334: 335: UNUSED(context); 336: 337: msg = msg_data->queued;	
	污染路径: 函数'UNUSED'的定义点.	
1611	缺陷发生位置: /source/client/src/clientSmlTelnet.c	行号: 210
	步骤描述: 不使用返回值的函数'JUMP_SPACE'应使用'void'说明.	
	代码片段: 208: } 209:	

	210: JUMP_SPACE(sql, sqlEnd) 211: 212: elements->tags = sql;	
	污染路径: 函数'JUMP_SPACE'的定义点.	
1612	缺陷发生位置: /source/libs/transport/test/http_test.c	行号: 64
	步骤描述: 不使用返回值的函数'taosMsleep'应使用'void'说明.	
	代码片段: 62: } 63: while (1) { 64: taosMsleep(5000); 65: } 66:	
	污染路径: 函数'taosMsleep'的定义点.	
1613	缺陷发生位置: /source/util/src/tcompare.c	行号: 323
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 321: return 0; 322: } else { 323: uError("data type unexpected leftType:%d rightType:%d", leftType, rightType); 324: return 0; 325: }	
	污染路径: 函数'uError'的定义点.	
1614	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 683
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 681: SVirtualTableRefInfo* pVtableRefInfo = taosMemoryCalloc(1, sizeof(SVirtualTableRefInfo)); 682: if (pVtableRefInfo == NULL) { 683: qError("%s failed at line %d since %s", __func__, __LINE__, terrstr());	

	684: pTaskInfo->code = terrno; 685: T_LONG_JMP(pTaskInfo->env, terrno);	
	污染路径: 函数'qError'的定义点.	
1615	缺陷发生位置: /source/util/src/tcurl.c	行号: 163
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: <pre> 161: SCURL* pTmp = (SCURL*)taosHashGet(tNotificationConnHash, (void*)pConn- >url, strlen(pConn->url)); 162: if (pTmp == NULL) { 163: uError("[curl]failed to get connection when reset"); 164: code = TSDB_CODE_FAILED; 165: } else { </pre>	
	污染路径: 函数'uError'的定义点.	
1616	缺陷发生位置: /source/libs/tdb/test/tdbPageRecycleTest.cpp	行号: 507
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 505: 506: for (int iData = 0; iData < nData; ++iData) { 507: sprintf(key, "key%03d", iData); 508: sprintf(val, "value%03d", iData); 509: </pre>	
	污染路径: 无	
1617	缺陷发生位置: /examples/c/demo.c	行号: 87
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 85: int i = 0; 86: for (i = 0; i < 10; ++i) { 87: sprintf(qstr, "insert into m1 values (%" PRIu64 " ", %d, %d, %d, %d, %f, %lf, '%s')", 88: (uint64_t)(1546300800000 + i * 1000), i, i, i, i * </pre>	

	10000000, i * 1.0, i * 2.0, "abc"); 89: printf("qstr: %s\n", qstr);	
	污染路径: 无	
1618	缺陷发生位置: /source/os/src/osString.c	行号: 176
	步骤描述: 不使用返回值的函数'SET_ERRNO'应使用'void'说明.	
	代码片段: 174: return TSDB_CODE_INVALID_PARA; 175: } 176: SET_ERRNO(0); 177: char *endptr = NULL; 178: int64_t ret = strtoll(str, &endptr, 10);	
	污染路径: 函数'SET_ERRNO'的定义点.	
1619	缺陷发生位置: /source/dnode/vnode/src/bse/bseSnapshot.c	行号: 229
	步骤描述: 不使用返回值的函数'bseSnapReaderClose'应使用'void'说明.	
	代码片段: 227: bseError("vgId:%d failed to open table pReader at line %d since %s", BSE_VGID((SBse *)pBse), lino, 228: tsterror(code)); 229: bseSnapReaderClose(&p); 230: } 231: *ppReader = NULL;	
	污染路径: 函数'bseSnapReaderClose'的定义点.	
1620	缺陷发生位置: /source/libs/scalar/test/filter/filterTests.cpp	行号: 1492
	步骤描述: 不使用返回值的函数'fp'应使用'void'说明.	
	代码片段: 1490: break; 1491: case TSDB_DATA_TYPE_USMALLINT: 1492: doCompareWithValueRange_SignedWithUnsigned<LType, uint16_t>(fp); 1493: break;	

	1494: case TSDB_DATA_TYPE_UINT:	
	污染路径: 无	
1621	缺陷发生位置: /source/dnode/mgmt/exe/dmMain.c	行号: 166
	步骤描述: 不使用返回值的函数'dWarn'应使用'void'说明.	
	代码片段: 164: } 165: if (taosIgnSignal(SIGINT) != 0) { 166: dWarn("failed to ignore signal SIGINT"); 167: } 168: if (taosIgnSignal(SIGABRT) != 0) {	
	污染路径: 函数'dWarn'的定义点.	
1622	缺陷发生位置: /source/dnode/vnode/src/meta/metaSnapshot.c	行号: 226
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 224: if (code) goto _err; 225: } 226: taosMemoryFree(pWriter); 227: *ppWriter = NULL; 228:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1623	缺陷发生位置: /source/libs/executor/src/dataSinkMgt.c	行号: 57
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 55: } 56: 57: taosMemoryFree(pSinkManager); 58: qError("invalid input node type:%d, %s", nodeType(*ppDataSink), id); 59:	
	污染路径:	

	函数'taosMemoryFree'的定义点.	
1624	缺陷发生位置: /source/libs/index/src/indexCache.c	行号: 428
	步骤描述: 不使用返回值的函数'indexError'应使用'void'说明.	
	代码片段: 426: 427: if ((taosThreadMutexLock(&cache->mtx)) != 0) { 428: indexError("failed to lock cache mutex"); 429: } 430:	
	污染路径: 函数'indexError'的定义点.	
1625	缺陷发生位置: /source/libs/function/src/builtinsimpl.c	行号: 249
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 247: plter->pPrevData = taosMemoryMalloc(plter->pDataCol->info.bytes); 248: if (NULL == plter->pPrevData) { 249: qError("out of memory when function get input row."); 250: return terrno; 251: }	
	污染路径: 函数'qError'的定义点.	
1626	缺陷发生位置: /source/common/src/ttime.c	行号: 635
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 633: TAOS_RETURN(ret); 634: } 635: taosMemoryFree(newColData); 636: } else { 637: TAOS_RETURN(TSDB_CODE_FAILED);	
	污染路径: 函数'taosMemoryFree'的定义点.	
1627	缺陷发生位置:	行号:

	/source/os/src/osThread.c	80
	步骤描述: 不使用返回值的函数'OS_PARAM_CHECK'应使用'void'说明.	
	代码片段: <pre> 78: int32_t taosThreadAttrGetSchedParam(const TdThreadAttr *attr, struct sched_param *param) { 79: OS_PARAM_CHECK(attr); 80: OS_PARAM_CHECK(param); 81: int32_t code = pthread_attr_getschedparam(attr, param); 82: if (code) { </pre>	
	污染路径: 函数'OS_PARAM_CHECK'的定义点.	
1628	缺陷发生位置: /source/libs/function/test/runUdf.c	行号: 93
	步骤描述: 不使用返回值的函数'fnError'应使用'void'说明.	
	代码片段: <pre> 91: ret = doTeardownUdf(handle); 92: if (ret != 0) { 93: fnError("failed to teardown udf"); 94: return -1; 95: } </pre>	
	污染路径: 函数'fnError'的定义点.	
1629	缺陷发生位置: /source/util/src/tworker.c	行号: 114
	步骤描述: 不使用返回值的函数'DestoryThreadLocalRegComp'应使用'void'说明.	
	代码片段: <pre> 112: } 113: 114: DestoryThreadLocalRegComp(); 115: 116: return NULL; </pre>	
	污染路径: 函数'DestoryThreadLocalRegComp'的定义点.	
1630	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbDataFileRW.c	行号: 171
	步骤描述:	

	不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 169: } 170: 171: taosMemoryFree(reader[0]); 172: reader[0] = NULL; 173: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1631	缺陷发生位置: /utils/test/c/tmq_vtable.c	行号: 59
	步骤描述: 不使用返回值的函数'tmq_free_json_meta'应使用'void'说明.	
	代码片段: 57: printf("meta result: %s\n", result); 58: strncat(json_get, result, sizeof(json_get) - strlen(json_get) - 1); 59: tmq_free_json_meta(result); 60: } 61:	
	污染路径: 函数'tmq_free_json_meta'的定义点.	
1632	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmMonitor.c	行号: 139
	步骤描述: 不使用返回值的函数'dTrace'应使用'void'说明.	
	代码片段: 137: void dmMonitorCleanExpiredSamples() { 138: if (!tsEnableMonitor tsMonitorFqdn[0] == 0 tsMonitorPort == 0) return; 139: dTrace("clean monitor expired samples"); 140: 141: SDnode *pDnode = dmInstance();	
	污染路径: 函数'dTrace'的定义点.	
1633	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSet2.c	行号: 121
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段:	

	119: static void tsdbSttLvlRemove(SSSttLvl **lvl) { 120: TARRAY2_DESTROY(lvl[0]->fobjArr, tsdbSttLvlRemoveFObj); 121: taosMemoryFree(lvl[0]); 122: lvl[0] = NULL; 123: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1634	缺陷发生位置: /source/libs/monitorfw/src/taos_collector_registry.c	行号: 126
	步骤描述: 不使用返回值的函数'TAOS_TEST_PARA'应使用'void'说明.	
	代码片段: 124: 125: int taos_collector_registry_deregister_metric(const char *key) { 126: TAOS_TEST_PARA(key != NULL); 127: 128: taos_collector_t *default_collector =	
	污染路径: 函数'TAOS_TEST_PARA'的定义点.	
1635	缺陷发生位置: /source/libs/index/src/indexFstFile.c	行号: 33
	步骤描述: 不使用返回值的函数'TAOS_UNUSED'应使用'void'说明.	
	代码片段: 31: 32: static void deleteDataBlockFromLRU(const void* key, size_t keyLen, void* value, void* ud) { 33: TAOS_UNUSED(ud); 34: TAOS_UNUSED(key); 35: TAOS_UNUSED(keyLen);	
	污染路径: 函数'TAOS_UNUSED'的定义点.	
1636	缺陷发生位置: /source/dnode/mnode/impl/src/mndSnode.c	行号: 320
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 318:	

	<pre> 319: if ((code = mndTransAppendRedoAction(pTrans, &action)) != 0) { 320: taosMemoryFree(pReq); 321: TAOS_RETURN(code); 322: }</pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
1637	缺陷发生位置: /source/libs/txnnode/src/txnnodeMgmt.c	行号: 104
	步骤描述: 不使用返回值的函数'xndInfo'应使用'void'说明.	
	代码片段: <pre> 102: getXnodedPidPath(pidPath, PATH_MAX); 103: (void)taosRemoveFile(pidPath); 104: xndInfo("xnoded process exit success"); 105: } else { 106: xndInfo("xnoded process restart, exit status %" PRId64 ", signal %d", exitStatus, termSignal);</pre>	
	污染路径: 函数'xndInfo'的定义点.	
1638	缺陷发生位置: /source/libs/scalar/src/scalar.c	行号: 252
	步骤描述: 不使用返回值的函数'sclError'应使用'void'说明.	
	代码片段: <pre> 250: SHashObj *pObj = taosHashInit(256, taosGetDefaultHashFunction(type), true, false); 251: if (NULL == pObj) { 252: sclError("taosHashInit failed, size:%d", 256); 253: SCL_ERR_RET(terrno); 254: }</pre>	
	污染路径: 函数'sclError'的定义点.	
1639	缺陷发生位置: /source/libs/tdb/test/tdbPageFlushTest.cpp	行号: 270
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 268: // char const *key = "key123456789"; 269: char key[32] = {0};</pre>	

	270: sprintf(key, "key-%d", i); 271: ret = tdbTbInsert(pDb, key, strlen(key) + 1, val, valLen, txn); 272: GTEST_ASSERT_EQ(ret, 0);	
	污染路径: 无	
1640	缺陷发生位置: /source/dnode/mnode/impl/src/mndStreamErrorInjection.c	行号: 34
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 32: taosMsleep(5000); 33: if (pAction->msgType == TDMT_STREAM_TASK_UPDATE_CHKPT) { 34: mError(35: "***sleep 5s and core dump, following tasks will not recv update-checkpoint info, so the checkpoint will " 36: "rollback***");	
	污染路径: 函数'mError'的定义点.	
1641	缺陷发生位置: /source/common/src/ttime.c	行号: 615
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 613: int32_t ret = taosParseTime(newColData, timeVal, charLen, (int32_t)timePrec, tz); 614: if (ret != TSDB_CODE_SUCCESS) { 615: taosMemoryFree(newColData); 616: TAOS_RETURN(TSDB_CODE_INVALID_TIMESTAMP); 617: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1642	缺陷发生位置: /source/libs/parser/src/parInsertSml.c	行号: 244
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 242: uInfo("SML data(name:%s type:%s) is not same like the db data(name:%s type:%s)", tmp, tDataTypes[kv-	

	<pre> >type].name, 243: pColSchema->name, tDataTypes[pColSchema- >type].name); 244: taosMemoryFree(tmp); 245: goto end; 246: }</pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
1643	缺陷发生位置: /source/libs/executor/src/dataDeleter.c	行号: 303
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 301: code = terrno; 302: taosArrayDestroy(pDeleterParam->pUidList); 303: taosMemoryFree(pParam); 304: goto _end; 305: }</pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
1644	缺陷发生位置: /source/libs/executor/src/dataDispatcher.c	行号: 71
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: <pre> 69: SDataBlockDescNode* pSchema = pHandle->pSchema; 70: if (pSchema == NULL pSchema->totalRowSize != pInput->pData->info.rowSize) { 71: qError("invalid schema"); 72: return TSDB_CODE_QRY_INVALID_INPUT; 73: }</pre>	
	污染路径: 函数'qError'的定义点.	
1645	缺陷发生位置: /source/common/src/tencrypt.c	行号: 512
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 510: } 511: if (fileContent != NULL) {</pre>	

	512: taosMemoryFree(fileContent); 513: } 514: if (plainContent != NULL) {	
	污染路径: 函数'taosMemoryFree'的定义点.	
1646	缺陷发生位置: /utls/test/c/tmqSim.c	行号: 170
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 168: struct tm tm; 169: taosLocalTime(&tTime, &tm, NULL, 0, NULL); 170: sprintf(timeString, "%d-%02d-%02d %02d:%02d: %02d", tm.tm_year + 1900, tm.tm_mon + 1, tm.tm_mday, tm.tm_hour, 171: tm.tm_min, tm.tm_sec); 172:	
	污染路径: 无	
1647	缺陷发生位置: /source/libs/catalog/src/ctgAsync.c	行号: 3633
	步骤描述: 不使用返回值的函数'ctgError'应使用'void'说明.	
	代码片段: 3631: SCtgMsgCtx* pMsgCtx = CTG_GET_TASK_MSGCTX(pTask, -1); 3632: if (NULL == pMsgCtx) { 3633: ctgError("fail to get the %dth pMsgCtx", -1); 3634: CTG_ERR_RET(TSDB_CODE_CTG_INTERNAL_ERROR); 3635: }	
	污染路径: 函数'ctgError'的定义点.	
1648	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 264
	步骤描述: 不使用返回值的函数'DEBUGLOG'应使用'void'说明.	
	代码片段: 262: ZSTD_pthread_mutex_unlock(&bufPool->poolMutex); 263: /* Reached bufferPool capacity (should not happen) */ 264: DEBUGLOG(5, "ZSTDMT_releaseBuffer: pool capacity	

	reached => freeing "); 265: ZSTD_free(buf.start, bufPool->cMem); 266: }	
	污染路径: 函数'DEBUGLOG'的定义点.	
1649	缺陷发生位置: /source/client/wrapper/src/wrapperDriver.c	行号: 299
	步骤描述: 不使用返回值的函数'taosCloseDll'应使用'void'说明.	
	代码片段: 297: void taosDriverCleanup() { 298: if (tsDriver != NULL) { 299: taosCloseDll(tsDriver); 300: tsDriver = NULL; 301: }	
	污染路径: 函数'taosCloseDll'的定义点.	
1650	缺陷发生位置: /source/dnode/mgmt/node_util/src/dmEps.c	行号: 127
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 125: if (pData->dnodeEps == NULL) { 126: code = terrno; 127: dError("failed to calloc dnodeEp array since %s", terrstr()); 128: goto _OVER; 129: }	
	污染路径: 函数'dError'的定义点.	
1651	缺陷发生位置: /source/os/src/osSocket.c	行号: 116
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 114: code = taosCloseSocketNoCheck1((*ppSocket)->fd); 115: (*ppSocket)->fd = -1; 116: taosMemoryFree(*ppSocket); 117: 118: return code;	

	污染路径: 函数'taosMemoryFree'的定义点.	
1652	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmHandle.c	行号: 52
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 50: } 51: if (taosArrayPush(pInfo->pVloads, &vload) == NULL) { 52: dError("failed to push vnode load"); 53: } 54: plter = taosHashIterate(pMgmt->runngingHash, plter);	
	污染路径: 函数'dError'的定义点.	
1653	缺陷发生位置: /source/dnode/mnode/impl/src/mndStream.c	行号: 69
	步骤描述: 不使用返回值的函数'mDebug'应使用'void'说明.	
	代码片段: 67: msmHandleBecomeNotLeader(pMnode); 68: 69: mDebug("mnd stream runtime info cleanup"); 70: } 71:	
	污染路径: 函数'mDebug'的定义点.	
1654	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmMetrics.c	行号: 74
	步骤描述: 不使用返回值的函数'dTrace'应使用'void'说明.	
	代码片段: 72: return; 73: } 74: dTrace("clean metrics expired samples"); 75: 76: SDnode *pDnode = dmInstance();	
	污染路径: 函数'dTrace'的定义点.	
1655	缺陷发生位置:	行号:

	/source/libs/tmqt/mqtt/src/tmqttdDaemon.c	164
	步骤描述: 不使用返回值的函数'bndError'应使用'void'说明.	
	代码片段: <pre> 162: SRetrieveFuncRsp retrieveRsp = {0}; 163: if (tDeserializeSRetrieveFuncRsp(pMsg->pCont, pMsg->contLen, &retrieveRsp) < 0) { 164: bndError("mqttdd deserialize retrieve func response failed"); 165: goto _return; 166: } </pre>	
	污染路径: 函数'bndError'的定义点.	
1656	缺陷发生位置: /utils/test/c/tmq_vtable.c	行号: 127
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: <pre> 125: 126: printf("\n\nmeta result:%s\n%s\n\n", json_get, json_result); 127: ASSERT(strcmp(json_get, json_result) == 0); 128: code = tmq_consumer_close(tmq); 129: if (code) </pre>	
	污染路径: 函数'ASSERT'的定义点.	
1657	缺陷发生位置: /source/libs/scheduler/src/schTask.c	行号: 1270
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 1268: (void)schReleaseJob(pParent->refId); 1269: 1270: taosMemoryFree(param); 1271: 1272: SCH_RET(code); </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
1658	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/	行号: 937

	argp/argp-help.c	
	步骤描述: 不使用返回值的函数'mempcpy'应使用'void'说明.	
	代码片段: 935: more->num_entries * sizeof (struct hol_entry)); 936: 937: mempcpy (short_options, hol->short_options, hol_so_len); 938: 939: /* Fix up the short options pointers from HOL. */	
	污染路径: 函数'mempcpy'的定义点.	
1659	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 524
	步骤描述: 不使用返回值的函数'OS_PARAM_CHECK'应使用'void'说明.	
	代码片段: 522: 523: int32_t taosGetEmail(char *email, int32_t maxLen) { 524: OS_PARAM_CHECK(email); 525: #ifdef WINDOWS 526: return 0;	
	污染路径: 函数'OS_PARAM_CHECK'的定义点.	
1660	缺陷发生位置: /source/dnode/mnode/impl/src/mndRole.c	行号: 647
	步骤描述: 不使用返回值的函数'TAOS_RETURN'应使用'void'说明.	
	代码片段: 645: } 646: mndTransDrop(pTrans); 647: TAOS_RETURN(0); 648: } 649:	
	污染路径: 函数'TAOS_RETURN'的定义点.	
1661	缺陷发生位置: /source/libs/scalar/src/scalar.c	行号: 1792
	步骤描述: 不使用返回值的函数'sclError'应使用'void'说明.	

	代码片段: 1790: 1791: if (NULL == node->pWhenThenList node->pWhenThenList->length <= 0) { 1792: sclError("invalid whenThen list"); 1793: SCL_ERR_RET(TSDB_CODE_INVALID_PARA); 1794: }	
	污染路径: 函数'sclError'的定义点.	
1662	缺陷发生位置: /source/dnode/vnode/src/tq/tqOffset.c	行号: 38
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 36: tqError("%s failed at %d since %s", __func__, lino, strerror(code)); 37: } 38: taosMemoryFree(fname); 39: return code; 40: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
1663	缺陷发生位置: /source/libs/qworker/src/qwMem.c	行号: 99
	步骤描述: 不使用返回值的函数'QW_TASK_DLOG'应使用'void'说明.	
	代码片段: 97: } 98: 99: QW_TASK_DLOG("task session destoryed, remainSessions:%d", remainSessions); 100: 101: if (0 == remainSessions) {	
	污染路径: 函数'QW_TASK_DLOG'的定义点.	
1664	缺陷发生位置: /utils/test/c/tmq_get_meta_json.c	行号: 149
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段:	

	147: code = taos_errno(pRes); 148: printf("%s result0:%s\n", __FUNCTION__, taos_errstr(pRes)); 149: ASSERT(code == 0); 150: taos_free_result(pRes); 151:	
	污染路径: 函数'ASSERT'的定义点.	
1665	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttSocket.c	行号: 145
	步骤描述: 不使用返回值的函数'ttq_free'应使用'void'说明.	
	代码片段: 143: } 144: 145: ttq_free(listensock); 146: 147: ttq_free(db.config->listeners[db.config->listener_count - 1].host);	
	污染路径: 函数'ttq_free'的定义点.	
1666	缺陷发生位置: /tools/lastqps-test/taos_query_a_simple.c	行号: 220
	步骤描述: 不使用返回值的函数'generate_sql_statements'应使用'void'说明.	
	代码片段: 218: qps_rate = 100; 219: } else if (query_mode == 2) { 220: generate_sql_statements(); 221: thread_count = 16; 222: queries_per_thread = 100;	
	污染路径: 函数'generate_sql_statements'的定义点.	
1667	缺陷发生位置: /docs/examples/c-ws-new/taos_connect_with_example.c	行号: 30
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 28: if (taos == NULL) { 29: fprintf(stderr, "Failed to connect to TDengine server:	

	<pre>%s\n", taos_errstr(NULL)); 30: taos_cleanup(); 31: return -1; 32: }</pre>	
	污染路径: 函数'taos_cleanup'的定义点.	
1668	缺陷发生位置: /source/libs/scalar/src/filter.c	行号: 5558
	步骤描述: 不使用返回值的函数'fltError'应使用'void'说明.	
	代码片段: <pre>5556: int32_t code = 0; 5557: if (pNode == NULL pInfo == NULL) { 5558: fltError("invalid param"); 5559: FLT_ERR_RET(TSDB_CODE_APP_ERROR); 5560: }</pre>	
	污染路径: 函数'fltError'的定义点.	
1669	缺陷发生位置: /source/libs/tdb/test/tdbTest.cpp	行号: 156
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre>154: for (int iData = 1; iData <= nData; iData++) { 155: sprintf(key, "key%d", iData); 156: sprintf(val, "value%d", iData); 157: ret = tdbTblInsert(pDb, key, strlen(key), val, strlen(val), txn); 158: GTEST_ASSERT_EQ(ret, 0);</pre>	
	污染路径: 无	
1670	缺陷发生位置: /source/libs/executor/src/tlinearhash.c	行号: 64
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: <pre>62: int32_t v = bucketId - (1ul << (bits - 1)); 63: if (v >= numOfBuckets) { 64: qError("tlinearhash failed at: %s:%d", __func__, __LINE__);</pre>	

	65: ternno = TSDB_CODE_INTERNAL_ERROR; 66: return -1;	
	污染路径: 函数'qError'的定义点.	
1671	缺陷发生位置: /source/util/src/tcompare.c	行号: 315
	步骤描述: 不使用返回值的函数'DEFAULT_COMP'应使用'void'说明.	
	代码片段: 313: char *realDataRight = POINTER_SHIFT(pRight, CHAR_BYTES); 314: if (leftType == TSDB_DATA_TYPE_BOOL) { 315: DEFAULT_COMP(GET_INT8_VAL(realDataLeft), GET_INT8_VAL(realDataRight)); 316: } else if (leftType == TSDB_DATA_TYPE_DOUBLE) { 317: DEFAULT_DOUBLE_COMP(GET_DOUBLE_VAL(realDataLeft), GET_DOUBLE_VAL(realDataRight));	
	污染路径: 函数'DEFAULT_COMP'的定义点.	
1672	缺陷发生位置: /source/dnode/vnode/src/bse/bseTable.c	行号: 862
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 860: blockWrapperCleanup(&p->blockWrapper); 861: 862: taosMemoryFree(p); 863: } 864:	
	污染路径: 函数'taosMemoryFree'的定义点.	
1673	缺陷发生位置: /source/libs/tss/src/tss.c	行号: 154
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 152: char* p = *(char**)taosArrayGet(paths, i); 153: code = tssDeleteFile(ss, p); 154: taosMemoryFree(p);	

	155: if (code != TSDB_CODE_SUCCESS) { 156: tssWarn("failed to delete file %s, code: %d", p, code);	
	污染路径: 函数'taosMemoryFree'的定义点.	
1674	缺陷发生位置: /source/libs/parser/src/parlInsertSql.c	行号: 599
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 597: code = parseSingleStrParam(pCxt, ppSql, pToken, pVal, tmpTokenBuf, &inputBytes, &final); 598: if (TSDB_CODE_SUCCESS != code final) { 599: taosMemoryFree(tmpTokenBuf); 600: 601: return code;	
	污染路径: 函数'taosMemoryFree'的定义点.	
1675	缺陷发生位置: /source/libs/scalar/src/sclvector.c	行号: 122
	步骤描述: 不使用返回值的函数'sclError'应使用'void'说明.	
	代码片段: 120: int len = taosUcs4ToMbs((TdUcs4 *)varDataVal(inData), varDataLen(inData), tmp, NULL); 121: if (len < 0) { 122: sclError("castConvert taosUcs4ToMbs error 1"); 123: SCL_ERR_JRET(TSDB_CODE_SCALAR_CONVERT_ERROR); 124: }	
	污染路径: 函数'sclError'的定义点.	
1676	缺陷发生位置: /source/util/test/lrucacheTest.cpp	行号: 183
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 181: for (int32_t i = 0; i < numItems; i++) { 182: char key[32]; 183: snprintf(key, sizeof(key), "key_%d", i);	

	184: char value[64]; 185: memset(value, 'A' + (i % 26), itemSize);	
	污染路径: 无	
1677	缺陷发生位置: /source/libs/planner/src/planPhysiCreator.c	行号: 108
	步骤描述: 不使用返回值的函数'TAOS_STRNCAT'应使用'void'说明.	
	代码片段: 106: TAOS_STRNCAT(*ppKey, ".", 2); 107: TAOS_STRNCAT(*ppKey, pCol->tableAlias, TSDB_TABLE_NAME_LEN); 108: TAOS_STRNCAT(*ppKey, ".", 2); 109: TAOS_STRNCAT(*ppKey, pCol->colName, TSDB_COL_NAME_LEN); 110: *cap = TSDB_TABLE_FNAME_LEN + 1 + TSDB_COL_NAME_LEN + 1 + extraBufLen;	
	污染路径: 函数'TAOS_STRNCAT'的定义点.	
1678	缺陷发生位置: /contrib/test/cos/main.c	行号: 761
	步骤描述: 不使用返回值的函数'cos_error_log'应使用'void'说明.	
	代码片段: 759: res = cos_open_file_for_all_read(p, TEST_MULTIPART_FILE, fb); 760: if (res != COSE_OK) { 761: cos_error_log("Open read file fail, filename:%s\n", TEST_MULTIPART_FILE); 762: return; 763: }	
	污染路径: 函数'cos_error_log'的定义点.	
1679	缺陷发生位置: /source/libs/executor/src/executor.c	行号: 142
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 140: int32_t code = pAPI->tsdReader.tsdSetReaderTaskId(pScanInfo->base.dataReader,	

	pTaskInfo->id.str); <pre> 141: if (code) { 142: qError("failed to set reader id for executor, code: %s", tstrerror(code)); 143: return code; 144: } </pre>	
	污染路径: 函数'qError'的定义点.	
1680	缺陷发生位置: /source/dnode/mnode/impl/src/mndConsumer.c	行号: 288
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: <pre> 286: 287: if(tSerializeSMqHbRsp(buf, tlen, rsp) <= 0){ 288: rpcFreeCont(buf); 289: return TSDB_CODE_TMQ_INVALID_MSG; 290: } </pre>	
	污染路径: 函数'rpcFreeCont'的定义点.	
1681	缺陷发生位置: /source/client/src/clientMain.c	行号: 132
	步骤描述: 不使用返回值的函数'tscWarn'应使用'void'说明.	
	代码片段: <pre> 130: tz = tzalloc(val); 131: if (tz == NULL) { 132: tscWarn("%s unknown timezone %s change to UTC", __func__, val); 133: tz = tzalloc("UTC"); 134: if (tz == NULL) { </pre>	
	污染路径: 函数'tscWarn'的定义点.	
1682	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 440
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: <pre> 438: snprintf(fenGbk, sizeof(fenGbk), "%c%c", 0xB7, 0xD2); 439: snprintf(zhongguoUtf8, sizeof(zhongguoUtf8), "%c%c%c </pre>	

	%c%c%c", 0xE4, 0xB8, 0xAD, 0xE5, 0x9B, 0xBD); 440: snprintf(guoUtf8, sizeof(guoUtf8), "%c%c%c", 0xE5, 0x9B, 0xBD); 441: snprintf(zhongguoGbk, sizeof(zhongguoGbk), "%c%c%c%c", 0xD6, 0xD0, 0xB9, 0xFA); 442: snprintf(zhongGbk, sizeof(zhongGbk), "%c%c", 0xD6, 0xD0);	
	污染路径: 无	
1683	缺陷发生位置: /source/os/src/osThread.c	行号: 782
	步骤描述: 不使用返回值的函数'OS_PARAM_CHECK'应使用'void'说明.	
	代码片段: 780: 781: int32_t taosThreadSetSchedParam(TdThread thread, int32_t policy, const struct sched_param *param) { 782: OS_PARAM_CHECK(param); 783: int32_t code = pthread_setschedparam(thread, policy, param); 784: if (code) {	
	污染路径: 函数'OS_PARAM_CHECK'的定义点.	
1684	缺陷发生位置: /source/util/src/tsched.c	行号: 230
	步骤描述: 不使用返回值的函数'taosMsleep'应使用'void'说明.	
	代码片段: 228: atomic_store_8(&pSched->stop, 1); 229: 230: taosMsleep(200); 231: 232: for (int32_t i = 0; i < pSched->numOfThreads; ++i) {	
	污染路径: 函数'taosMsleep'的定义点.	
1685	缺陷发生位置: /utils/test/c/blob_test.c	行号: 297
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段:	

	295: taos_free_result(pRes); 296: 297: sprintf(sql, "insert into tb6 using stb1 tags (6, 'tb6_bin1', '\\\\xg') values (now + 4s, '%s')", tmp); 298: pRes = taos_query(taos, sql); 299: taos_free_result(pRes);	
	污染路径: 无	
1686	缺陷发生位置: /source/common/src/ttime.c	行号: 618
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 616: TAOS_RETURN(TSDB_CODE_INVALID_TIMESTAMP); 617: } 618: taosMemoryFree(newColData); 619: } else if (type == TSDB_DATA_TYPE_NCHAR) { 620: newColData = taosMemoryCalloc(1, charLen + TSDB_NCHAR_SIZE);	
	污染路径: 函数'taosMemoryFree'的定义点.	
1687	缺陷发生位置: /source/libs/monitorfw/src/taos_metric_formatter_custom.c	行号: 147
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 145: 146: taosMemoryFreeClear(arr); 147: taosMemoryFreeClear(keyvalue); 148: taosMemoryFreeClear(keyvalues); 149:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
1688	缺陷发生位置: /source/dnode/vnode/src/bse/bseSnapshot.c	行号: 162
	步骤描述: 不使用返回值的函数'bseSnapWriterClose'应使用'void'说明.	
	代码片段: 160: bseError("vgId:%d failed to open table pWriter at line %d since %s", BSE_VGID((SBse *)pBse), lino,	

	161: tsterror(code)); 162: bseSnapWriterClose(&p, 0); 163: } 164: *pWriter = NULL;	
	污染路径: 函数'bseSnapWriterClose'的定义点.	
1689	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqPacket.c	行号: 226
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 224: 225: if (NULL == ttq->current_out_packet) { 226: UNUSED(ttqMuxRemoveOut(ttq)); 227: } 228:	
	污染路径: 函数'UNUSED'的定义点.	
1690	缺陷发生位置: /source/dnode/mnode/impl/src/mndXnode.c	行号: 707
	步骤描述: 不使用返回值的函数'taosMemFree'应使用'void'说明.	
	代码片段: 705: } 706: if (pContStr != NULL) { 707: taosMemFree(pContStr); 708: } 709: if (pJson != NULL) {	
	污染路径: 函数'taosMemFree'的定义点.	